



Продвинутый Git

Модуль 1 - Как Git хранит данные

Урок 1 - Объекты Git blob, tree, commit, tag

Шаг 01:

Миссия 1

Алексей сделал первый коммит в новый проект и задумался: какой объект Git хранит содержимое файла?

Подсказка: Подумайте, какой объект отвечает именно за бинарное содержимое, а не за структуру или историю.

Ответ:

- **blob**
- tree
- commit
- tag

Шаг 02:

Миссия 2

Марина просматривает репозиторий и пытается понять, какой объект Git хранит список файлов и их структуры в папке.

Подсказка: Этот объект похож на каталог файловой системы.

Ответ:

- blob

- **tree**
- commit
- tag

🔗 Шаг 03:

Миссия 3

Дмитрий сделал коммит и удивился: какой объект Git содержит метаданные — автора, дату, ссылку на дерево и сообщение?

Подсказка: Этот объект связывает историю изменений и файловую систему.

Ответ:

- blob
- tree
- **commit**
- tag

🔗 Шаг 04:

Миссия 4

Анна хочет закрепить конкретный коммит как версию релиза. Какой объект Git используется для этого?

Подсказка: Этот объект добавляет человеко-понятную метку к коммиту.

Ответ:

- blob
- tree
- commit
- **tag**

🔗 Шаг 05:

Миссия 5

Иван заметил, что при создании файла создается объект, который не хранит имя файла, а только его содержимое. Какой это объект?

Подсказка: Имя хранится в другом объекте — каталоге.

Ответ:

- **blob**
- tree
- commit
- tag

🔗 Шаг 06:

Миссия 6

Сопоставьте объекты Git с их назначением.

Подсказка: Вспомните: один объект хранит содержимое файлов, другой — структуру папок, третий — историю, а последний — ярлыки.

Ответ:

- **blob** — Хранит содержимое файла
- **tree** — Хранит структуру папок и ссылки на файлы

- **commit** — Хранит ссылку на дерево и метаданные
- **tag** — Делает ссылку на коммит с меткой

🔧 Шаг 07:

Миссия 7

🔗 Сопоставьте разработчиков и действия с объектами Git.

Подсказка: Подумайте, кто работает с файлами, кто с историей, а кто с релизами.

Ответ:

- **Саша добавил новый код** — Создал blob
- **Ира создала папку с файлами** — Создала tree
- **Виктор закоммитил изменения** — Создал commit
- **Марина отметила релиз** — Создала tag

🔧 Шаг 08:

Миссия 8

🔗 Сопоставьте объект Git с примером его содержимого.

Подсказка: Подумайте, что именно хранится внутри: байты файла, список ссылок или метаданные.

Ответ:

- **blob** — Содержимое: `function hello() { return 'Hi'; }`
- **tree** — Список: `main.c` → blob, `lib/` → tree
- **commit** — Сообщение: `'Fix bug #42'`, автор, дата
- **tag** — Метка: `v1.0` с описанием

🔧 Шаг 09:

Миссия 9

🔗 Сопоставьте Git-команду с объектом, который она чаще всего показывает.

Подсказка: `git log` работает не с файлами, а с историей.

Ответ:

- **git cat-file -p** — blob
- **git ls-tree HEAD** — tree
- **git log** — commit
- **git tag -l** — tag

🔧 Шаг 10:

Миссия 10

✂ Сопоставьте задачу программиста с нужным объектом Git.

Подсказка: Подумайте: редактирование текста — это blob, релиз — tag.

Ответ:

- **Хранение текста файла** — blob
- **Фиксация состояния проекта** — commit
- **Создание версии релиза** — tag
- **Структура каталогов** — tree

🔗 Шаг 11:

Миссия 11

📖 Сопоставьте описание с правильным объектом Git.

Подсказка: Обратите внимание на ключевые слова: содержимое, каталог, история, ярлык.

Ответ:

- **Содержит байты файлов без имён** — blob
- **Определяет папку и ссылки на файлы** — tree
- **Фиксирует точку в истории разработки** — commit
- **Позволяет обозначить важный коммит** — tag

🔗 Шаг 12:

Миссия 12

📖 Расположите объекты Git в порядке их создания при коммите нового файла.

Подсказка: Сначала сохраняется содержимое, затем каталог, потом история, и в конце можно добавить метку.

Ответ:

- blob
- tree
- commit
- tag

🔗 Шаг 13:

Миссия 13

📖 Расположите шаги при сохранении файла в Git.

Подсказка: Подумайте: сначала содержимое, потом дерево, потом коммит.

Ответ:

- Сохранение содержимого файла в blob
- Обновление дерева ссылающегося на blob
- Создание commit, который указывает на дерево

🔗 Шаг 14:

Миссия 14

📖 Расположите действия при пометке релиза.

Подсказка: Сначала фиксируется история, потом добавляется ярлык.

Ответ:

- Создание commit с изменениями
- Создание tag, указывающего на commit

🔗 Шаг 15:

Миссия 15

📖 Расположите шаги получения содержимого файла из репозитория.

Подсказка: Двигайтесь от ярлыка или истории до конкретного содержимого.

Ответ:

- Определение нужного commit
- Переход к tree
- Поиск blob в tree
- Извлечение содержимого из blob

🔧 **Шаг 16:**

Миссия 16

🔧 Расположите шаги при изучении репозитория.

Подсказка: Сначала смотри историю, потом дерево, потом файлы.

Ответ:

- Просмотр commit
- Открытие tree
- Чтение blob

🔧 **Шаг 17:**

Миссия 17

🔧 Расположите в порядке важности при восстановлении версии проекта.

Подсказка: Восстановление идёт сверху вниз: история → дерево → файлы.

Ответ:

- commit
- tree
- blob

📖 **Урок 2 - SHA1-хеши и уникальность коммитов**

🔧 **Шаг 01:**

Миссия 1

Антон добавил первый коммит в свой новый проект и заметил длинный идентификатор. Что это такое?

Подсказка: Git использует уникальные хеши, чтобы отличать объекты друг от друга.

Ответ:

- **SHA-1 хеш коммита**
- Имя файла коммита
- Случайное число от Git
- Версия Git на компьютере

🔧 **Шаг 02:**

Миссия 2

Мария спорит с коллегой: может ли хеш двух разных коммитов совпасть?

Подсказка: Теоретически возможно, но вероятность очень мала.

Ответ:

- **Да, но вероятность крайне мала**
- Нет, Git это запрещает

- Да, и это случается каждые пару месяцев
- Хеши всегда одинаковы

🔮 Шаг 03:

Миссия 3

Игорь сделал коммит и увидел строку `a3f5c1e`. Что это?

Подсказка: Git часто показывает только первые символы идентификатора.

Ответ:

- **Сокращённая форма SHA-1 хеша**
- Номер ветки
- Часть имени файла
- Код ошибки Git

🔮 Шаг 04:

Миссия 4

Алексей сделал два одинаковых коммита с разным временем. Хеши совпадут?

Подсказка: Git учитывает не только изменения в файлах, но и метаданные.

Ответ:

- Да, всегда совпадут
- **Нет, время тоже влияет на хеш**
- Зависит от длины сообщения коммита
- Git специально делает разные хеши случайно

🔮 Шаг 05:

Миссия 5

Настя нашла старый проект и хочет проверить, изменились ли коммиты. Что ей достаточно сравнить?

Подсказка: В Git идентификатор сам по себе уже доказывает уникальность.

Ответ:

- **SHA-1 хеши коммитов**
- Только авторов коммитов
- Только даты создания
- Содержание файлов вручную

🔮 Шаг 06:

Миссия 6

Сопоставьте элементы коммита и их влияние на хеш:

Подсказка: Хеш зависит не только от содержимого, но и от метаданных.

Ответ:

- **Сообщение коммита** — Меняется — хеш тоже будет другим
- **Время создания** — Меняется — хеш пересчитывается
- **Содержимое файлов** — Любое изменение даёт новый хеш
- **Автор коммита** — Входит в расчёт хеша

🔗 Шаг 07:

Миссия 7

Свяжите ситуацию с объяснением:

Подсказка: Хеши помогают быстро определить изменения.

Ответ:

- **Разработчик видит новый хеш** — Значит, что-то изменилось в объекте
- **Одинаковые хеши у файлов** — Файлы полностью совпадают
- **Два разных коммита с одним автором и временем** — Если изменения разные — хеши будут разными
- **Сравнение веток по хешам** — Позволяет быстро находить различия

🔗 Шаг 08:

Миссия 8

Сопоставьте тип объекта Git и роль хеша:

Подсказка: Все объекты Git индексируются через хеш.

Ответ:

- **Blob** — Хеш хранит содержимое файла
- **Tree** — Хеш фиксирует структуру каталогов
- **Commit** — Хеш учитывает метаданные и ссылки на родителя
- **Tag** — Хеш связывает имя и объект

🔗 Шаг 09:

Миссия 9

Сопоставьте команду Git и её работу с хешами:

Подсказка: Хеши — это основа для ссылок и поиска.

Ответ:

- **git log** — Показывает список хешей и сообщений
- **git show** — Выводит данные объекта по хешу
- **git checkout** — Переключение на коммит по хешу
- **git cat-file -p** — Просмотр содержимого объекта

🔗 Шаг 10:

Миссия 10

Сопоставьте проблему и роль хеша в её решении:

Подсказка: Git решает вопросы идентификации с помощью хешей.

Ответ:

- **Как убедиться, что файл не изменился** — Сравнить SHA-1 хеши содержимого
- **Как найти родительский коммит** — Использовать хеш родителя
- **Как определить одинаковые файлы в истории** — Их blob-объекты будут иметь один хеш
- **Как проверить подлинность коммита** — Подпись хеша ключом разработчика

🔗 Шаг 11:

Миссия 11

Сопоставьте ситуацию и объяснение уникальности хеша:

Подсказка: Уникальность гарантируется алгоритмом и объёмом данных.

Ответ:

- **Коллизии SHA-1 редки** — Вероятность крайне мала
- **Два разных файла** — С вероятностью 99.9999% у них будут разные хеши
- **Коммиты с одинаковым содержимым и временем** — Хеш совпадёт
- **Изменение одного байта** — Полностью меняет хеш

Шаг 12:

Миссия 12

Расставьте шаги вычисления хеша коммита:

Подсказка: Git берёт содержимое и метаданные, а затем применяет SHA-1.

Ответ:

- Формирование данных: дерево, автор, сообщение
- Передача строки в SHA-1
- Получение 40-символьного хеша
- Использование хеша как идентификатора

Шаг 13:

Миссия 13

Упорядочите действия программиста, чтобы проверить подлинность коммита:

Подсказка: Верификация всегда идёт после извлечения хеша.

Ответ:

- Извлечь хеш коммита
- Получить цифровую подпись
- Проверить подпись ключом автора
- Подтвердить подлинность

Шаг 14:

Миссия 14

Расположите шаги при переключении на конкретный коммит по хешу:

Подсказка: Checkout работает строго по порядку: поиск → подготовка → переключение.

Ответ:

- Ввод команды git checkout
- Поиск коммита по хешу в базе
- Обновление рабочей директории
- Переключение HEAD

Шаг 15:

Миссия 15

Определите порядок работы с SHA-1 при сравнении файлов:

Подсказка: Git всегда сравнивает через хеши, а не по содержимому напрямую.

Ответ:

- Вычисление хеша файла

- Сравнение хешей
- Определение совпадения или различий
- Вывод результата

🔗 Шаг 16:

Миссия 16

Расположите шаги в процессе проверки уникальности коммита:

Подсказка: Git ищет совпадения в базе хешей.

Ответ:

- Вычисление хеша нового коммита
- Сравнение с существующими хешами
- **Если совпадение найдено** — использовать существующий
- **Если нет** — создать новый объект

🔗 Шаг 17:

Миссия 17

Упорядочите шаги хранения коммита в Git:

Подсказка: Всё заканчивается записью нового объекта с хешем.

Ответ:

- Сбор содержимого и метаданных
- Формирование строки для SHA-1
- Вычисление хеша
- Запись объекта в базу данных .git/objects

📖 Урок 3 - Практика просмотр объектов через git cat-file

🔗 Шаг 01:

Миссия 1

Маша работает над своим первым open-source проектом. Она только что закоммитила изменения и хочет понять, как Git хранит сам commit. Какой объект Git нужно посмотреть через git cat-file -r, чтобы увидеть сообщение коммита и ссылки на дерево?

Подсказка: Думай о том объекте, который связывает автора, дату и дерево файлов.

Ответ:

- **commit**
- tree
- blob
- tag

🔗 Шаг 02:

Миссия 2

Сергей анализирует старый репозиторий. Он хочет понять, какой объект в Git хранит содержимое самого файла, а не структуру каталогов. Какой тип объекта он увидит через git cat-file -r для исходного кода?

Подсказка: Подумай: где лежит сам текст файла?

Ответ:

- **blob**
- tree
- commit
- branch

🔗 **Шаг 03:**

Миссия 3

Иван изучает дерево объектов. Он хочет узнать, какой объект Git указывает на файлы и другие папки. Что он увидит в выводе `git cat-file -p` для такого объекта?

Подсказка: Этот объект играет роль папки.

Ответ:

- **tree**
- blob
- commit
- tag

🔗 **Шаг 04:**

Миссия 4

Алина создаёт релиз и видит объект `tag`. Она решает исследовать его через `git cat-file -p`. Что особенного хранит `tag`-объект?

Подсказка: Это не только ссылка на `commit`, но и что-то ещё.

Ответ:

- **Ссылку на `commit` и аннотацию (сообщение тега)**
- Только список файлов
- Только хэш `blob`
- Только имя ветки

🔗 **Шаг 05:**

Миссия 5

Вика случайно удалила ветку и теперь изучает объект `commit` через `git cat-file -p`. Какую информацию она точно увидит внутри `commit`-объекта?

Подсказка: Там хранится не только дерево, но и авторство.

Ответ:

- **Автор коммита**
- **Ссылка на дерево**
- Размер файла
- Текущий HEAD

🔗 **Шаг 06:**

Миссия 6

Денис исследует разные типы объектов Git. Сопоставь тип объекта с его назначением.

Подсказка: Вспомни: `blob` = содержимое, `tree` = структура, `commit` = история, `tag` = подпись.

Ответ:

- **blob** — Хранит содержимое файла
- **tree** — Хранит структуру каталогов и ссылки на blob
- **commit** — Содержит ссылку на дерево и метаданные об изменениях
- **tag** — Аннотированная ссылка на commit

🔮 **Шаг 07:**

Миссия 7

Оля хочет разобраться, какие поля чаще всего встречаются внутри commit-объекта. Сопоставь поле и его назначение.

Подсказка: В commit всегда есть дерево, автор и сообщение.

Ответ:

- **tree** — Ссылка на структуру файлового дерева
- **parent** — Ссылка на предыдущий commit
- **author** — Кто сделал commit
- **message** — Текст описания изменений

🔮 **Шаг 08:**

Миссия 8

Максим исследует git cat-file и хочет понять, какие команды показывают информацию. Сопоставь команду с тем, что она выводит.

Подсказка: -t это тип, -s это размер, -p это содержимое.

Ответ:

- **git cat-file -t** — Тип объекта (commit, tree, blob, tag)
- **git cat-file -s** — Размер объекта
- **git cat-file -p** — Читаемое содержимое объекта

🔮 **Шаг 09:**

Миссия 9

Антон хочет понять различие между объектами. Сопоставь объект с примером его содержимого.

Подсказка: commit содержит автора, blob хранит текст файла.

Ответ:

- **blob** — Текст кода функции на Python
- **tree** — Список файлов и подпапок
- **commit** — Автор, дата и сообщение
- **tag** — Имя версии и описание релиза

🔮 **Шаг 10:**

Миссия 10

Катя запуталась в том, как Git связывает объекты. Сопоставь связь с её описанием.

Подсказка: Commit указывает на дерево, дерево — на blob.

Ответ:

- **Commit → Tree** — Commit хранит ссылку на дерево проекта
- **Tree → Blob** — Tree указывает на файлы

- **Tag → Commit** — Tag хранит ссылку на commit
- **Commit → Parent** — Commit указывает на предыдущий commit

🔗 Шаг 11:

Миссия 11

Саша хочет проверить, какие объекты чаще всего встречаются в проекте. Сопоставь объект и его реальный пример в жизни проекта.

Подсказка: Представь, что ты смотришь через cat-file содержимое.

Ответ:

- **blob** — Файл README.md
- **tree** — Папка src/
- **commit** — История добавления функции
- **tag** — Релиз v1.0

🔗 Шаг 12:

Миссия 12

Павел учится шаг за шагом просматривать commit через git cat-file . Расположи действия в правильном порядке.

Подсказка: Сначала нужен хэш коммита, потом его проверка, затем просмотр.

Ответ:

- Найти хэш коммита с помощью git log
- Посмотреть тип объекта через git cat-file -t
- Посмотреть содержимое объекта через git cat-file -p

🔗 Шаг 13:

Миссия 13

Егор исследует, как устроена цепочка объектов в Git. Расположи объекты в правильном порядке от файла до истории.

Подсказка: Начни с содержимого файла и закончи историей коммитов.

Ответ:

- blob
- tree
- commit
- tag

🔗 Шаг 14:

Миссия 14

Настя хочет понять, как лучше исследовать объект через cat-file. Расположи команды в порядке: тип → размер → содержимое.

Подсказка: -t, затем -s, затем -p.

Ответ:

- git cat-file -t
- git cat-file -s
- git cat-file -p

Шаг 15:

Миссия 15

Дима хочет изучить историю файлов через Git. Расположи шаги в правильном порядке, чтобы проследить путь объекта.

Подсказка: Начни с blob и иди до commit.

Ответ:

- blob
- tree
- commit

Шаг 16:

Миссия 16

Илья разбирается в структуре Git. Расположи элементы commit-объекта в порядке их появления.

Подсказка: Сначала дерево, потом родитель, потом автор и сообщение.

Ответ:

- tree
- parent
- author
- message

Шаг 17:

Миссия 17

Лена готовит тренинг для команды по Git. Ей нужно объяснить процесс изучения объекта с помощью cat-file. Расположи шаги в логической последовательности.

Подсказка: Сначала определить хэш, потом узнать тип, затем вывести содержимое.

Ответ:

- Определить хэш объекта
- Проверить тип объекта
- Вывести содержимое объекта

Урок 4 - Git как key-value store

Шаг 01:

Миссия 1

Антон работает над pet-проектом и заметил, что Git хранит объекты в виде ключ-значение. Какой тип данных Git использует в качестве ключа?

Подсказка: Ключи должны быть уникальными и вычисляться на основе содержимого файла.

Ответ:

- **SHA-1 хэш объекта**
- UUID версии коммита
- Имя файла
- Время последнего коммита

🔮 Шаг 02:

Миссия 2

Марина спорит с коллегами: что именно Git хранит как «значение» в key-value store?

Подсказка: Подумайте про разные типы объектов: blob, tree, commit, tag.

Ответ:

- **Содержимое blob (файлов)**
- **Объекты tree (структура каталогов)**
- **Commit-объекты**
- Только названия файлов

🔮 Шаг 03:

Миссия 3

Денис заметил, что Git хранит blob одинаково, даже если файл скопирован в другой каталог. Почему?

Подсказка: Вспомните, что определяет ключ.

Ответ:

- **Ключ вычисляется только из содержимого файла**
- Git всегда учитывает путь к файлу
- Git добавляет дату создания
- Git различает копии как разные объекты

🔮 Шаг 04:

Миссия 4

Лена экспериментирует: она добавила новый файл и сделала git hash-object. Что Git вернул?

Подсказка: Вспомните, что команда возвращает ключ объекта.

Ответ:

- **SHA-1 хэш blob объекта**
- UUID коммита
- Размер файла в байтах
- Имя файла

🔮 Шаг 05:

Миссия 5

Разработчик Василий обнаружил объект commit. Что он хранит внутри?

Подсказка: Это больше, чем просто ссылка на файлы.

Ответ:

- **Ссылку на tree**
- **Ссылки на родителей**
- **Метаинформацию: автор, дата, сообщение**
- Полный дамп всех файлов

🔮 Шаг 06:

Миссия 6

Оля спрашивает: почему объект tree важен?

Подсказка: Он описывает не содержимое файлов, а их организацию.

Ответ:

- **Он хранит структуру каталогов и ссылки на blob/другие tree**
- Он содержит текст коммита
- Он показывает историю изменений
- Он хранит резервные копии файлов

Шаг 07:

Миссия 7

 Задача: Сопоставьте типы объектов Git с их назначением.

Подсказка: Вспомните: blob = содержимое, tree = структура, commit = история.

Ответ:

- **Blob** — Хранит содержимое файла
- **Tree** — Хранит структуру каталогов
- **Commit** — Хранит ссылку на дерево и метаданные
- **Tag** — Хранит человекочитаемую метку для коммита

Шаг 08:

Миссия 8

 Задача: Сопоставьте команду и её назначение в Git как key-value store.

Подсказка: Одни команды создают объект, другие показывают содержимое или тип.

Ответ:

- **git hash-object** — Создаёт объект и возвращает его хэш
- **git cat-file -p** — Показывает содержимое объекта
- **git cat-file -t** — Показывает тип объекта
- **git write-tree** — Создаёт tree из индекса

Шаг 09:

Миссия 9

 Задача: Сопоставьте реальную ситуацию и объект Git, который в ней используется.

Подсказка: Подумайте: при сохранении файла — blob, при фиксации состояния проекта — commit.

Ответ:

- **Добавили новый файл** — Создан blob
- **Закоммитили изменения** — Создан commit
- **Сохранили структуру папок** — Создан tree
- **Создали метку v1.0** — Создан tag

Шаг 10:

Миссия 10

 Задача: Сопоставьте хэш-ключ и его свойство.

Подсказка: Важна уникальность и неизменность при одинаковом содержимом.

Ответ:

- **SHA-1** — Алгоритм, используемый в Git

- **Ключ** — Зависит от содержимого
- **Коллизия** — Редкое совпадение хэшей
- **Неизменность** — Одинаковое содержимое = одинаковый хэш

🔗 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте элементы хранилища `.git/objects` с их описанием.

Подсказка: Каждое имя папки/файла связано с хэшем объекта.

Ответ:

- **`.git/objects/aa`** — Папка для префикса хэша
- **Файл внутри папки** — Содержит сжатый объект
- **Хэш** — Разделяется на префикс и суффикс
- **Значение** — Реальные данные объекта

🔗 Шаг 12:

Миссия 12

📖 Задача: Сопоставьте действия программиста и то, какой объект Git они затрагивают.

Подсказка: Помните про различие между содержимым, структурой и историей.

Ответ:

- **Изменение содержимого файла** — Blob
- **Организация файлов в каталоге** — Tree
- **Фиксация состояния проекта** — Commit
- **Добавление версии 2.0** — Tag

🔗 Шаг 13:

Миссия 13

Расположите шаги, которые выполняет Git при создании коммита, в правильном порядке.

Подсказка: Сначала файлы попадают в blob, потом строится дерево, и только затем фиксируется commit.

Ответ:

- Создание blob объектов для новых файлов
- Создание tree с указанием на blob
- Создание commit, указывающего на tree
- Добавление метаданных (автор, дата, сообщение)

🔗 Шаг 14:

Миссия 14

Расставьте шаги, которые делает git cat-file для вывода объекта.

Подсказка: Сначала находится объект по хэшу, потом он распаковывается и определяется его тип.

Ответ:

- Поиск объекта по хэшу
- Распаковка содержимого
- Определение типа объекта
- Вывод содержимого

Шаг 15:

Миссия 15

Расположите по шагам процесс вычисления хэша для нового blob.

Подсказка: Git добавляет заголовок к содержимому перед хэшированием.

Ответ:

- Добавление заголовка с типом и длиной
- Объединение заголовка и содержимого
- Вычисление SHA-1 хэша
- Сохранение объекта в .git/objects

Шаг 16:

Миссия 16

Расставьте шаги при работе Git как key-value store от добавления файла до его чтения.

Подсказка: Сначала объект сохраняется, потом можно получить его по ключу.

Ответ:

- git hash-object для файла
- Хэш сохраняется в .git/objects
- git cat-file -р для хэша
- Получение содержимого файла

Шаг 17:

Миссия 17

Расположите шаги построения нового дерева проекта.

Подсказка: Дерево строится из blobs и других trees, а затем коммит указывает на него.

Ответ:

- Создание blob объектов для файлов
- Создание tree из blobs
- Создание root tree
- Создание commit, указывающего на root tree

Модуль 2 - Индекс и staging area

Урок 1 - Рабочая директория, индекс и репозиторий

Шаг 01:

Миссия 1

Маша пишет код для стартапа и заметила: Git хранит файлы в трёх местах. Где именно?

Подсказка: Вспомни: есть место, где файлы редактируются, место для подготовки и место хранения истории.

Ответ:

- Рабочая директория
- Индекс (staging area)
- Репозиторий (commit history)
- Корзина (Recycle Bin)

🔗 Шаг 02:

Миссия 2

Андрей заметил, что после редактирования файла в проекте Git сразу его не учитывает. Где изменения находятся до выполнения `git add` ?

Подсказка: Подумай: перед индексом есть одно особое место — там, где мы редактируем код.

Ответ:

- **В рабочей директории**
- В индексе
- В локальном репозитории

🔗 Шаг 03:

Миссия 3

Оля изменила файл и выполнила `git add` , но ещё не сделала коммит. Где файл теперь находится?

Подсказка: После `git add` файл ждёт своей очереди попасть в историю.

Ответ:

- **В индексе (staging area)**
- В рабочей директории
- Уже в коммите

🔗 Шаг 04:

Миссия 4

Вася сделал `git commit` . Где теперь хранятся его изменения?

Подсказка: После коммита изменения больше не просто подготовлены, а попадают в постоянное место.

Ответ:

- **В локальном репозитории (в истории коммитов)**
- В индексе
- В удалённом репозитории

🔗 Шаг 05:

Миссия 5

Егор изменил файл и хочет, чтобы Git не отслеживал его. Какой способ подойдёт?

Подсказка: Вспомни про особый файл, где мы можем прописать имена для исключения.

Ответ:

- Сделать `git add`
- **Добавить файл в `.gitignore`**
- Удалить Git
- Сделать `git commit` без этого файла

🔗 Шаг 06:

Миссия 6

Настя сделала правки и сразу коммит без `git add` . Что произойдёт?

Подсказка: Подумай: коммит собирает только то, что лежит в `staging area`.

Ответ:

- **Закоммитаются только изменения, уже добавленные в индекс**
- Закоммитаются все изменения из рабочей директории
- Коммит не произойдёт вовсе
- Git удалит изменения

🔗 **Шаг 07:**

Миссия 7

Игорь проверяет статус проекта с помощью `git status`. Что он увидит, если есть изменённые, но не добавленные файлы?

Подсказка: Git всегда показывает разницу между рабочей директорией и индексом.

Ответ:

- **Список изменённых файлов в рабочей директории**
- Список файлов, добавленных в коммит
- **Подсказку: используйте `git add`, чтобы зафиксировать изменения**
- Историю всех коммитов

🔗 **Шаг 08:**

Миссия 8

Влад работает над проектом и хочет убрать файл из индекса, но оставить его в рабочей директории. Какой командой он это сделает?

Подсказка: Здесь используется параметр `--cached`.

Ответ:

- **`git rm --cached file.txt`**
- `git rm file.txt`
- `git reset --hard`
- `git clean -fd`

🔗 **Шаг 09:**

Миссия 9

📖 Сопоставьте команды Git с их описанием.

Подсказка: Подумай: одни команды добавляют, другие убирают, а третьи показывают статус.

Ответ:

- **`git add`** — Добавляет изменения в staging area
- **`git status`** — Показывает состояние рабочей директории и индекса
- **`git commit`** — Создаёт снимок текущего состояния индекса
- **`git rm --cached`** — Удаляет файл из индекса, оставляя его в рабочей директории

🔗 **Шаг 10:**

Миссия 10

📖 Сопоставьте области Git с их назначением.

Подсказка: Одна область для работы, другая — для подготовки, третья — для хранения истории.

Ответ:

- **Рабочая директория** — Место, где программист редактирует файлы
- **Индекс (staging area)** — Место для подготовки изменений перед коммитом
- **Репозиторий** — Хранение всей истории проекта

Шаг 11:

Миссия 11

 Сопоставьте команды с ситуациями.

Подсказка: Внимательно подумай: что нужно, если просто проверить, а что — если зафиксировать.

Ответ:

- **git diff** — Посмотреть различия между рабочей директорией и индексом
- **git log** — Просмотреть историю коммитов
- **git reset HEAD file.txt** — Убрать файл из индекса, оставив в рабочей директории
- **git add .** — Добавить все изменения в индекс

Шаг 12:

Миссия 12

 Сопоставьте понятие с его определением.

Подсказка: Индекс — промежуточное звено. Подумай, что куда попадает.

Ответ:

- **Изменения в рабочей директории** — Файлы, которые только что были отредактированы
- **Изменения в индексе** — Файлы, подготовленные для коммита
- **Изменения в репозитории** — Файлы, которые уже сохранены в истории

Шаг 13:

Миссия 13

 Сопоставьте фразы с действиями программиста.

Подсказка: Подумай, где именно изменения фиксируются: в директории, индексе или истории.

Ответ:

- **«Я редактирую код»** — Рабочая директория
- **«Я готовлю изменения»** — Индекс
- **«Я сохранил изменения навсегда»** — Репозиторий

Шаг 14:

Миссия 14

Расположите шаги работы с Git в правильном порядке.

Подсказка: Сначала редактируем, потом добавляем, затем сохраняем.

Ответ:

- Редактирование файлов
- git add
- git commit

🔗 Шаг 15:

Миссия 15

Расположите области Git по пути изменений.

Подсказка: Подумай: сначала работа, потом подготовка, потом история.

Ответ:

- Рабочая директория
- Индекс
- Репозиторий

🔗 Шаг 16:

Миссия 16

Расположите шаги отката изменений в правильном порядке.

Подсказка: Сначала убираем из индекса, потом отменяем в рабочей директории.

Ответ:

- git reset HEAD file.txt
- Редактирование отменяется (git checkout -- file.txt)

📖 Урок 2 - Практика добавление и удаление файлов

🔗 Шаг 01:

Миссия 1

Ты добавил новый файл notes.txt в проект. Какой командой ты перенесёшь его в staging area?

Подсказка: Вспомни основную команду Git для добавления файла в индекс.

Ответ:

- **git add notes.txt**
- git push notes.txt
- git commit notes.txt
- git stage notes.txt

🔗 Шаг 02:

Миссия 2

Аня случайно создала файл temp.log . Какой командой она исключит его из индекса, если он уже был туда добавлен?

Подсказка: Команда похожа на добавление, но работает в обратную сторону.

Ответ:

- **git reset HEAD temp.log**
- git delete temp.log
- git rm --cached temp.log
- git remove temp.log

🔗 Шаг 03:

Миссия 3

Тебе нужно добавить все изменения в текущей директории в индекс. Какую команду выбрать?

Подсказка: Используй специальный символ для выбора всех файлов.

Ответ:

- **git add .**
- **git add *.***
- **git commit .**
- **git stage all**

Шаг 04:

Миссия 4

Сергей изменил файл config.yml и хочет его удалить из рабочего каталога и из индекса. Какой командой он это сделает?

Подсказка: Удаление файла и из репозитория, и из рабочей директории.

Ответ:

- **git rm config.yml**
- **git reset config.yml**
- **rm config.yml**
- **git delete config.yml**

Шаг 05:

Миссия 5

Маша добавила файл draft.txt в индекс, но теперь хочет убрать его только из индекса, оставив в рабочей директории. Какую команду использовать?

Подсказка: Нужно удалить файл из индекса, но не с диска.

Ответ:

- **git rm --cached draft.txt**
- **git reset draft.txt**
- **rm draft.txt**
- **git remove draft.txt**

Шаг 06:

Миссия 6

Сопоставьте команды Git с их назначением при добавлении или удалении файлов.

Подсказка: Обратите внимание на разницу между --cached и обычным удалением.

Ответ:

- **git add file.txt** — Добавить файл в staging area
- **git rm file.txt** — Удалить файл из индекса и рабочей директории
- **git rm --cached file.txt** — Удалить файл только из индекса
- **git reset HEAD file.txt** — Убрать файл из staging area, оставив его в рабочей директории

Шаг 07:

Миссия 7

Программисты обсуждают разные способы добавления файлов в индекс. Сопоставьте команды с их кратким описанием.

Подсказка: Вспомни, что . и -A означают.

Ответ:

- **git add .** — Добавить все изменения из текущей директории
- **git add -A** — Добавить все изменения в проекте, включая удаления
- **git add file1.txt file2.txt** — Добавить несколько файлов по имени
- **git add *.js** — Добавить все файлы с расширением .js

🔗 Шаг 08:

Миссия 8

Ты тренируешь новичка. Сопоставь команды удаления файлов с правильными сценариями.
Подсказка: Различай удаление из рабочего каталога и из индекса.

Ответ:

- **git rm file.txt** — Файл удаляется из проекта полностью
- **git rm --cached secret.key** — Файл исключается только из индекса
- **rm file.txt** — Файл удаляется только из рабочей директории
- **git reset HEAD unwanted.txt** — Файл убирается из staging area без удаления

🔗 Шаг 09:

Миссия 9

Сопоставь команды с их результатами при работе с индексом и рабочей директорией.
Подсказка: Подумай, где именно остаётся файл после каждой команды.

Ответ:

- **git add image.png** — Файл готов к коммиту
- **git rm --cached image.png** — Файл исключён из индекса, но остаётся на диске
- **git rm image.png** — Файл удалён полностью
- **git reset HEAD image.png** — Файл перестал быть staged, но остался в рабочей директории

🔗 Шаг 10:

Миссия 10

Сопоставь ситуацию и команду, которую нужно использовать.
Подсказка: Представь разные сценарии: добавление, удаление, исключение файла.

Ответ:

- **Нужно добавить новый скрипт script.js в проект** — `git add script.js`
- **Нужно удалить test.tmp из репозитория и локальной папки** — `git rm test.tmp`
- **Нужно убрать из индекса файл keys.json, но оставить его на диске** — `git rm --cached keys.json`
- **Нужно исключить readme.txt из коммита, не удаляя его** — `git reset HEAD readme.txt`

🔗 Шаг 11:

Миссия 11

Сопоставь команды и их краткие описания. Все команды связаны с добавлением или удалением файлов.
Подсказка: Проверь, какие команды изменяют индекс, а какие — файловую систему.

Ответ:

- **git add data.csv** — Файл подготовлен к коммиту
- **rm data.csv** — Файл удалён только из рабочей директории
- **git rm --cached data.csv** — Файл исключён из индекса, но остался локально
- **git rm data.csv** — Файл удалён из индекса и с диска

Шаг 12:

Миссия 12

Маша создала новый файл `notes.txt`. Как добавить его в индекс для следующего коммита?

Подсказка: Подумайте, какая команда переводит файл из рабочей директории в staging area.

Ответ:

- **git add notes.txt**
- `git commit notes.txt`
- `git rm notes.txt`
- `git status notes.txt`

Шаг 13:

Миссия 13

Сергей добавил несколько файлов, но один случайно попал в staging, а остальные нет. Как проверить, какие именно файлы подготовлены к коммиту?

Подсказка: Используйте команду, которая показывает состояние индексированных и неиндексированных файлов.

Ответ:

- **git status**
- `git diff`
- `git log`
- `git show`

Шаг 14:

Миссия 14

Антон добавил файл в staging, но передумал. Как убрать его из индекса, сохранив изменения в рабочей директории?

Подсказка: Команда `reset` позволяет отменить добавление в staging без потери изменений.

Ответ:

- **git reset HEAD filename**
- `git rm filename`
- `git checkout filename`
- `git commit filename`

Урок 3 - Staging и unstaging файлов

Шаг 01:

Миссия 1

Анна случайно изменила файл `index.html` и хочет подготовить его к коммиту. Какой командой она это сделает?

Подсказка: Подумайте, какая команда помещает изменения из рабочей директории в staging area.

Ответ:

- **git add index.html**
- git commit index.html
- git reset index.html
- git rm index.html

Шаг 02:

Миссия 2

Богдан добавил файл style.css в staging area, но вспомнил, что сделал это рано. Какой командой он уберет его оттуда, оставив изменения в рабочей директории?

Подсказка: Эта команда противоположна git add , но не удаляет сам файл.

Ответ:

- **git reset HEAD style.css**
- git rm style.css
- git checkout style.css
- git restore style.css --staged

Шаг 03:

Миссия 3

Катя сделала изменения сразу в трёх файлах и хочет добавить их все одним действием. Какую команду она должна использовать?

Подсказка: Эта команда добавит все изменения в staging area.

Ответ:

- **git add .**
- git commit -am
- git stage all
- git add -A

Шаг 04:

Миссия 4

Разработчик Дима по ошибке добавил лишний файл в staging area. Какой командой он сможет убрать его из индекса в новых версиях Git?

Подсказка: Подумайте, какой современный аналог git reset используется для удаления из staging area.

Ответ:

- **git restore --staged file.txt**
- git reset file.txt
- git clean file.txt
- git checkout file.txt

Шаг 05:

Миссия 5

Ольга случайно добавила бинарный файл `dump.bin` в `staging area`. Что лучше всего сделать, чтобы он туда больше не попадал?

Подсказка: Нужно исключить такие файлы от отслеживания в будущем.

Ответ:

- **Добавить `dump.bin` в `.gitignore`**
- Удалить файл навсегда с диска
- Использовать `git reset --hard`
- Закоммитить файл и потом удалить

🔗 Шаг 06:

Миссия 6

Иван сделал исправления в `main.py`, но не уверен, добавил ли он их в `staging area`. Какой командой он это проверит?

Подсказка: Эта команда показывает различия между рабочей директорией и индексом.

Ответ:

- **`git status`**
- `git log`
- `git show`
- `git diff HEAD`

🔗 Шаг 07:

Миссия 7

Сопоставьте команды Git с их назначением при работе с `staging area`:

Подсказка: Вспомните, какие команды добавляют, а какие убирают файлы из индекса.

Ответ:

- **`git add file.txt`** — Добавляет файл в `staging area`
- **`git reset HEAD file.txt`** — Убирает файл из `staging area`, оставляя изменения
- **`git restore --staged file.txt`** — Современная команда для удаления из `staging area`
- **`git status`** — Показывает текущее состояние файлов в рабочей директории и `staging area`

🔗 Шаг 08:

Миссия 8

Сопоставьте ситуации с правильными действиями в Git:

Подсказка: Подумайте, когда файл нужно добавить, убрать или проверить.

Ответ:

- **Нужно подготовить новый файл к коммиту** — `git add newfile.txt`
- **Случайно добавили лишний файл в `staging`** — `git restore --staged wrong.txt`
- **Хотим убедиться, что нужные файлы попали в индекс** — `git status`
- **Хотим удалить файл полностью и из индекса, и с диска** — `git rm old.txt`

🔗 Шаг 09:

Миссия 9

Сопоставьте команду Git с её кратким описанием:

Подсказка: Обратите внимание на разницу между `git reset` и `git restore`.

Ответ:

- **git reset HEAD file.txt** — Старая команда для удаления файла из staging
- **git restore --staged file.txt** — Новый способ убрать файл из staging
- **git add .** — Добавляет все изменения в staging area
- **git diff --cached** — Показывает различия между staging area и последним коммитом

🔮 Шаг 10:

Миссия 10

Сопоставьте программистов и их действия в Git:

Подсказка: Внимательно подумайте, какие команды нужны в каждой истории.

Ответ:

- **Анна хочет добавить файл readme.md** — `git add readme.md`
- **Богдан случайно закоммитил tmp.log** — `git rm --cached tmp.log`
- **Катя проверяет, что подготовлено к коммиту** — `git status`
- **Дима передумал добавлять script.py в staging** — `git restore --staged script.py`

🔮 Шаг 11:

Миссия 11

Сопоставьте понятия с их определениями:

Подсказка: Различайте рабочую директорию, staging area и репозиторий.

Ответ:

- **Рабочая директория** — Место, где вы редактируете файлы
- **Staging area** — Буфер, куда помещаются изменения перед коммитом
- **Репозиторий** — История всех зафиксированных коммитов
- **Коммит** — Фиксация изменений из staging area в репозиторий

🔮 Шаг 12:

Миссия 12

Сопоставьте команды и их результат:

Подсказка: Учитывайте, что `git rm` удаляет файл полностью, а `git rm --cached` только из индекса.

Ответ:

- **git rm file.txt** — Удаляет файл из staging и из рабочей директории
- **git rm --cached file.txt** — Удаляет файл только из staging area
- **git add file.txt** — Добавляет файл в staging area
- **git diff** — Показывает изменения, которые ещё не в staging area

🔮 Шаг 13:

Миссия 13

Расположите шаги добавления файла к коммиту в правильном порядке:

Подсказка: Сначала создаём файл, затем добавляем в staging, и только потом фиксируем.

Ответ:

- Создать или изменить файл
- Добавить файл в staging area с помощью `git add`
- Проверить статус файлов с `git status`

- Сделать git commit

Шаг 14:

Миссия 14

Расставьте шаги отмены добавления файла в staging area в правильном порядке:

Подсказка: Подумайте, как вы сначала видите проблему, а потом убираете файл из staging.

Ответ:

- Обнаружить лишний файл в staging через git status
- Использовать git restore --staged file.txt
- Проверить результат ещё раз с git status
- Сделать коммит без лишнего файла

Шаг 15:

Миссия 15

Определите порядок действий при подготовке большого количества файлов к коммиту:

Подсказка: Вспомните про команду, которая добавляет все изменения сразу.

Ответ:

- Редактирование множества файлов
- Добавление всех изменений с git add .
- Проверка изменений с git status
- Коммит изменений

Шаг 16:

Миссия 16

Расположите шаги удаления файла из Git, чтобы он не отслеживался:

Подсказка: Нужно сначала убрать из staging, затем при необходимости удалить физически.

Ответ:

- Обнаружить лишний файл
- Удалить его из индекса с git rm --cached file.txt
- Добавить в .gitignore, если не нужно отслеживать
- Сделать коммит изменений

Урок 4 - Проверка staged и unstaged изменений

Шаг 01:

Миссия 1

Аня добавила новые функции в код, но забыла выполнить git add . Как Git классифицирует её изменения?

Подсказка: Подумайте, что попадает в staging area, а что остаётся в рабочей директории.

Ответ:

- **Unstaged изменения**
- Staged изменения
- Committed изменения
- Remote изменения

🔧 Шаг 02:

Миссия 2

Игорь выполнил `git add file.txt`, а потом снова отредактировал этот файл. Какой статус будет у файла?

Подсказка: Один и тот же файл может иметь staged и unstaged изменения одновременно.

Ответ:

- **Часть staged, часть unstaged**
- Только staged
- Только unstaged
- Файл без изменений

🔧 Шаг 03:

Миссия 3

Лена хочет проверить, какие изменения уже подготовлены для коммита. Какую команду использовать?

Подсказка: Сравните команду, которая показывает staged изменения, и команду, которая показывает unstaged.

Ответ:

- **git diff --cached**
- git diff
- git log
- git show

🔧 Шаг 04:

Миссия 4

Андрей заметил, что у него есть изменения в staging и в рабочей директории. Какая команда покажет только unstaged изменения?

Подсказка: Есть два режима у `git diff`: обычный и с флагом `--cached`.

Ответ:

- **git diff**
- git diff --cached
- git status
- git show

🔧 Шаг 05:

Миссия 5

Олег добавил файл в staging, но решил отменить это действие. Какой командой он уберёт файл из staging area, сохранив изменения в рабочей директории?

Подсказка: Здесь помогает опция команды `reset`.

Ответ:

- **git reset HEAD file.txt**
- git checkout file.txt
- git restore file.txt
- git rm file.txt

🔗 Шаг 06:

Миссия 6

Света набрала `git status`. В выводе она видит разделы «Changes to be committed» и «Changes not staged for commit». Что это означает?

Подсказка: Эти две секции показывают staged и unstaged изменения.

Ответ:

- **Подготовленные и неподготовленные изменения**
- Изменения в локальном и удалённом репозитории
- Изменения до и после коммита
- Файлы в разных ветках

🔗 Шаг 07:

Миссия 7

Вася случайно добавил бинарный файл в staging. Какой способ лучше всего использовать, чтобы убрать его из индекса?

Подсказка: Сравните команды `rm` и `reset`.

Ответ:

- **`git reset HEAD binaryfile`**
- `git rm binaryfile`
- `git checkout binaryfile`
- `git diff binaryfile`

🔗 Шаг 08:

Миссия 8

Костя хочет одним взглядом увидеть все staged и unstaged изменения. Какую команду лучше использовать?

Подсказка: Есть команда, которая всегда показывает текущий статус репозитория.

Ответ:

- **`git status`**
- `git diff --cached`
- `git show`
- `git log`

🔗 Шаг 09:

Миссия 9

Сопоставьте команды Git с тем, что они показывают.

Подсказка: Вспомните разницу между `diff`, `diff --cached` и `status`.

Ответ:

- **`git diff`** — Показывает только unstaged изменения
- **`git diff --cached`** — Показывает staged изменения
- **`git status`** — Показывает общее состояние staged и unstaged изменений
- **`git log`** — Показывает историю коммитов

🔗 Шаг 10:

Миссия 10

Сопоставьте команды с их назначением при работе с индексом.

Подсказка: Обратите внимание, какие команды изменяют staging area, а какие только читают информацию.

Ответ:

- **git add file.txt** — Добавляет файл в staging area
- **git reset HEAD file.txt** — Убирает файл из staging area
- **git restore --staged file.txt** — Аналогично убирает файл из staging area
- **git show HEAD** — Показывает содержимое последнего коммита

🔗 Шаг 11:

Миссия 11

Сопоставьте ситуации с правильным инструментом проверки изменений.

Подсказка: Вспомните, что одни команды смотрят на staged, другие на рабочую директорию.

Ответ:

- **Нужно увидеть неподготовленные изменения** — git diff
- **Нужно увидеть подготовленные изменения** — git diff --cached
- **Нужно увидеть все файлы с изменениями** — git status
- **Нужно увидеть результат последнего коммита** — git show HEAD

🔗 Шаг 12:

Миссия 12

Сопоставьте термины с их определениями.

Подсказка: staged — это подготовлено, unstaged — нет.

Ответ:

- **Staging area** — Область подготовленных файлов для коммита
- **Unstaged changes** — Изменения в рабочей директории, не добавленные в staging
- **Commit** — Фиксация изменений из staging area
- **Working directory** — Текущие файлы проекта, с которыми работает разработчик

🔗 Шаг 13:

Миссия 13

Сопоставьте команды Git с их кратким описанием.

Подсказка: Две из них убирают файлы из staging area, но делают это разными способами.

Ответ:

- **git restore --staged** — Убирает файл из staging area
- **git reset** — Убирает файл из staging area или возвращает состояние индекса
- **git add** — Добавляет файл в staging area
- **git commit** — Сохраняет staged изменения в историю

🔗 Шаг 14:

Миссия 14

Расположите команды в порядке их применения, если нужно отменить staging для файла и затем посмотреть текущее состояние.

Подсказка: Сначала уберите из staging, потом проверяйте статус.

Ответ:

- git reset HEAD file.txt
- git status
- git diff
- git add file.txt

Модуль 3 - Коммиты и ссылки

Урок 1 - Создание коммитов и роль HEAD

Шаг 01:

Миссия 1

Мария заметила, что забыла добавить файлы перед коммитом. Какой командой она подготовит их?

Подсказка: Вспомните команду, которая помещает изменения в staging area.

Ответ:

- git commit
- **git add**
- git log
- git branch

Шаг 02:

Миссия 2

Иван сделал коммит, но хочет уточнить, какой идентификатор (hash) у него появился. Какой командой посмотреть историю?

Подсказка: Есть команда, которая выводит список коммитов с их hash.

Ответ:

- git diff
- **git log**
- git status
- git branch -a

Шаг 03:

Миссия 3

Программист Катя хочет узнать, куда указывает HEAD. Что правильнее всего сказать?

Подсказка: HEAD — это не сама ветка, а указатель.

Ответ:

- **HEAD указывает на текущий коммит в активной ветке**
- **HEAD** — это копия удалённого репозитория
- HEAD хранит список всех веток
- HEAD используется только при push

🔧 Шаг 04:

Миссия 4

Разработчик Артём случайно сделал коммит с опечаткой в сообщении. Как он может изменить текст последнего коммита?

Подсказка: Используется команда `commit` с особым флагом.

Ответ:

- **git commit --amend**
- `git reset --hard`
- `git checkout HEAD~1`
- `git push --force`

🔧 Шаг 05:

Миссия 5

Миша проверяет, какие файлы будут добавлены в следующий коммит. Какой командой это можно увидеть?

Подсказка: Эта команда показывает `staged` и `unstaged` изменения.

Ответ:

- `git show`
- **git status**
- `git branch`
- `git log --oneline`

🔧 Шаг 06:

Миссия 6

Елена сделала коммит и хочет сразу отправить его в удалённый репозиторий. Какая команда поможет?

Подсказка: Вспомните команду для отправки ветки на сервер.

Ответ:

- **git push**
- `git commit`
- `git add`
- `git merge`

🔧 Шаг 07:

Миссия 7

Сергей изучает, как работает `HEAD`. Что произойдёт, если выполнить команду `git checkout HEAD~1` ?

Подсказка: Обратите внимание, что это сдвигает указатель на предыдущий коммит.

Ответ:

- **HEAD переместится на предыдущий коммит**
- Будет удалена ветка `main`
- Все изменения будут отправлены на сервер
- Git переключится на новую ветку

🔦 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте команды с их назначением.

Подсказка: Вспомните, какие действия выполняют команды `commit`, `add`, `log`, `status`.

Ответ:

- **git commit** — Фиксирует изменения в репозитории
- **git add** — Подготавливает файлы к коммиту
- **git log** — Показывает историю коммитов
- **git status** — Отображает состояние рабочего каталога

🔦 Шаг 09:

Миссия 9

📖 Задача: Алексей путает термины. Помогите ему сопоставить.

Подсказка: HEAD связан с текущей веткой, а `commit hash` — это уникальный идентификатор.

Ответ:

- **HEAD** — Указатель на текущий коммит
- **Коммит** — Снимок состояния проекта
- **Hash** — Уникальный идентификатор коммита
- **Ветка** — Последовательность коммитов с именем

🔦 Шаг 10:

Миссия 10

📖 Задача: Настя учит команды Git. Сопоставьте команду с примером результата.

Подсказка: Вспомните, где выводится история, а где — `staged` файлы.

Ответ:

- **git log --oneline** — a1b2c3 Добавил модуль авторизации
- **git status** — Файл `index.js` готов к коммиту
- **git show** — Выводит изменения конкретного коммита
- **git branch** — Список доступных веток

🔦 Шаг 11:

Миссия 11

📖 Задача: В команде решили закрепить основы Git. Сопоставьте термин с его определением.

Подсказка: Обратите внимание, что `staging area` — это подготовительная зона.

Ответ:

- **Staging area** — Зона подготовки файлов перед коммитом
- **Working directory** — Текущая рабочая папка с файлами
- **Remote** — Удалённый репозиторий
- **Index** — Другой термин для `staging area`

🔦 Шаг 12:

Миссия 12

📖 Задача: Дмитрий разбирается с ветками. Сопоставьте команды и их действие.

Подсказка: Каждая команда отвечает за определённый аспект работы с ветками.

Ответ:

- **git branch feature** — Создание новой ветки feature
- **git checkout feature** — Переключение на ветку feature
- **git merge feature** — Слияние ветки feature в текущую
- **git branch -d feature** — Удаление ветки feature

Шаг 13:

Миссия 13

Расположите шаги создания нового коммита в правильном порядке.

Подсказка: Сначала изменяются файлы, затем они попадают в staging, потом фиксируются коммитом.

Ответ:

- Изменение файлов в рабочей директории
- Добавление файлов в staging area
- Создание коммита
- Отправка коммита в удалённый репозиторий

Шаг 14:

Миссия 14

Сергей изучает историю проекта. Расположите команды в порядке их использования для изучения истории.

Подсказка: Сначала смотрят краткий log, потом конкретный коммит.

Ответ:

- git log --oneline
- git log
- git show
- git diff HEAD~1

Шаг 15:

Миссия 15

Расположите шаги при исправлении последнего коммита с помощью amend.

Подсказка: Сначала вносятся изменения, затем они добавляются, и только потом используется commit --amend.

Ответ:

- Внесение изменений в файлы
- Добавление файлов в staging area
- Использование git commit --amend
- Проверка истории командой git log

Шаг 16:

Миссия 16

Поставьте шаги переключения HEAD в правильном порядке.

Подсказка: Вспомните: сначала нужно найти hash коммита, потом checkout, потом проверить.

Ответ:

- Просмотр истории через git log
- Выбор нужного hash коммита
- git checkout
- Проверка состояния HEAD

Урок 2 - Ветки и указатели в Git

Шаг 01:

Миссия 1

Марина работает в ветке main , но хочет переключиться в feature-auth . Какой командой это сделать?

Подсказка: Для переключения используется команда checkout или более современная switch.

Ответ:

- **git checkout feature-auth**
- git branch feature-auth
- git switch main
- git init feature-auth

Шаг 02:

Миссия 2

Олег хочет увидеть все доступные ветки в своём проекте. Какой командой он это сделает?

Подсказка: Есть специальная команда, которая показывает список веток.

Ответ:

- **git branch**
- git status
- git log
- git show

Шаг 03:

Миссия 3

Саша создала ветку feature-api и сделала несколько коммитов. Теперь она хочет объединить эту ветку с main . Какой командой это делается?

Подсказка: Для объединения используется команда merge .

Ответ:

- **git merge feature-api**
- git branch -d feature-api
- git push origin feature-api
- git checkout feature-api

Шаг 04:

Миссия 4

Ваня хочет удалить локальную ветку fix-bug , которая уже не нужна. Какой командой это сделать?

Подсказка: Используется команда branch с флагом удаления.

Ответ:

- **git branch -d fix-bug**
- git rm fix-bug
- git delete fix-bug
- git reset fix-bug

 **Шаг 05:**

Миссия 5

Лена хочет быстро узнать, в какой ветке она сейчас работает. Какой командой это проверить?

Подсказка: Это можно увидеть и в git status, и в списке веток.

Ответ:

- **git status**
- git log --oneline
- git merge
- git show HEAD

 **Шаг 06:**

Миссия 6

Дмитрий работает с удалённым репозиторием. Какой командой он создаст ветку и сразу переключится в неё?

Подсказка: Есть удобный флаг для команды checkout.

Ответ:

- **git checkout -b feature-x**
- git branch feature-x
- git init feature-x
- git add feature-x

 **Шаг 07:**

Миссия 7

Катя переключилась на конкретный коммит с помощью git checkout . Как называется такое состояние?

Подсказка: Оно связано с отсоединённым указателем HEAD.

Ответ:

- **Detached HEAD**
- Lost branch
- Safe mode
- Headless mode

 **Шаг 08:**

Миссия 8

 Сопоставьте команду с её действием при работе с ветками.

Подсказка: Вспомните разницу между созданием, удалением, переключением и просмотром веток.

Ответ:

- **git branch** — Вывод списка веток
- **git checkout -b test** — Создание и переключение на новую ветку
- **git branch -d old** — Удаление ветки old
- **git checkout dev** — Переключение на ветку dev

🔧 Шаг 09:

Миссия 9

📖 Сопоставьте термин с правильным объяснением.

Подсказка: Ветки — это указатели, HEAD — это текущая позиция.

Ответ:

- **Ветка** — Указатель на последовательность коммитов
- **HEAD** — Указатель на текущий коммит
- **Origin/main** — Удалённая ветка на сервере
- **Merge** — Операция объединения двух веток

🔧 Шаг 10:

Миссия 10

📖 Иван изучает команды Git. Сопоставьте команду и пример результата.

Подсказка: Подумайте, какая команда выводит список веток, а какая — историю.

Ответ:

- **git branch** — * main dev
- **git log --oneline** — a1b2c3 Fix bug
- **git merge feature** — Объединяет изменения из ветки feature
- **git checkout hotfix** — Переключение на ветку hotfix

🔧 Шаг 11:

Миссия 11

📖 Настя путается в терминах. Помогите ей сопоставить.

Подсказка: У удалённого репозитория всегда есть своё имя (по умолчанию origin).

Ответ:

- **Origin** — Имя удалённого репозитория
- **Upstream** — Удалённый репозиторий, откуда форкнут проект
- **Tracking branch** — Локальная ветка, связанная с удалённой
- **Detached HEAD** — Состояние без привязки к ветке

🔧 Шаг 12:

Миссия 12

📖 Сопоставьте команды с их кратким описанием.

Подсказка: Обратите внимание: reset и merge работают по-разному.

Ответ:

- **git merge dev** — Слияние изменений из ветки dev
- **git reset --hard HEAD~1** — Откат на один коммит назад
- **git rebase main** — Перенос коммитов поверх ветки main

- **git branch -m old new** — Переименование ветки old в new

Шаг 13:

Миссия 13

Расположите шаги удаления ветки после успешного слияния.

Подсказка: Сначала нужно убедиться, что ветка влита, потом переключиться на основную.

Ответ:

- git checkout main
- git merge feature
- git branch -d feature
- git push origin --delete feature

Шаг 14:

Миссия 14

Расположите шаги для работы с удалённой веткой.

Подсказка: Сначала нужно получить изменения, потом создать локальную ветку и связать её с удалённой.

Ответ:

- git fetch
- git checkout -b feature origin/feature
- Внесение изменений
- git push -u origin feature

Урок 3 - Просмотр истории коммитов

Шаг 01:

Миссия 1

Андрей только что сделал несколько коммитов и хочет посмотреть их историю. Какую команду он должен использовать?

Подсказка: Подумайте о самой популярной команде для просмотра истории.

Ответ:

- **git log**
- git status
- git branch
- git diff

Шаг 02:

Миссия 2

Света работает в команде и хочет увидеть историю в одну строку для удобства. Какой параметр ей поможет?

Подсказка: Этот параметр сокращает вывод и делает его компактным.

Ответ:

- **git log --oneline**
- git log --all
- git log --graph

- `git show --oneline`

🔗 Шаг 03:

Миссия 3

Безопасник Дима расследует инцидент и хочет увидеть историю с графом веток. Что он использует?

Подсказка: Ключ добавляет ASCII-граф в консоль.

Ответ:

- **`git log --graph`**
- `git log --oneline`
- `git show --graph`
- `git diff --graph`

🔗 Шаг 04:

Миссия 4

Ирина хочет ограничить вывод истории последними 5 коммитами. Что она должна сделать?

Подсказка: Есть параметр для ограничения количества записей.

Ответ:

- **`git log -n 5`**
- `git log --last 5`
- `git log --limit 5`
- `git show -5`

🔗 Шаг 05:

Миссия 5

Александр хочет посмотреть изменения конкретного коммита по его хэшу. Какую команду выбрать?

Подсказка: Используется команда, которая показывает содержимое коммита.

Ответ:

- **`git show`**
- `git log`
- `git diff`
- `git commit`

🔗 Шаг 06:

Миссия 6

Ольга хочет найти коммиты, где менялся файл README.md. Как это сделать?

Подсказка: Нужно указать имя файла после команды.

Ответ:

- **`git log README.md`**
- `git show README.md`
- `git diff README.md`
- `git commit README.md`

🔧 Шаг 07:

Миссия 7

Программист Миша хочет просмотреть историю с датами и авторами. Какой параметр использовать?

Подсказка: Есть формат для подробного вывода с метаданными.

Ответ:

- **git log --pretty=fuller**
- git log --oneline
- git log --author
- git log --stat

🔧 Шаг 08:

Миссия 8

📖 Задача: сопоставьте команды Git с их результатами.

Подсказка: Каждая команда показывает историю в разном виде.

Ответ:

- **git log --oneline** — История в одну строку
- **git log --graph** — История с ASCII-графом
- **git show** — Подробности конкретного коммита
- **git log -n 3** — Вывод последних 3 коммитов

🔧 Шаг 09:

Миссия 9

Сопоставьте параметры с их описанием.

Подсказка: Вспомните, какой параметр отвечает за граф, какой — за количество.

Ответ:

- **--oneline** — Сокращенный формат истории
- **--graph** — ASCII-граф ветвлений
- **-n 5** — Ограничение числа коммитов
- **--pretty=fuller** — Вывод автора и коммитера

🔧 Шаг 10:

Миссия 10

Свяжите команду с тем, что можно узнать о коммитах.

Подсказка: У каждой команды — свой фокус: хэш, изменения, файлы.

Ответ:

- **git log --stat** — Какие файлы и сколько строк изменились
- **git log --pretty=oneline** — Хэш и сообщение коммита в строку
- **git show** — Все изменения в выбранном коммите
- **git log README.md** — История коммитов для файла README.md

🔧 Шаг 11:

Миссия 11

Сопоставьте флаги Git с их действием.

Подсказка: Вспомните про подробный вывод, автора и изменения.

Ответ:

- **--author** — Фильтрация по имени автора
- **--stat** — Показывает статистику изменений
- **--patch** — Показывает сами изменения (diff)
- **--all** — История для всех веток

🔮 Шаг 12:

Миссия 12

Соотнесите команду с её реальным сценарием.

Подсказка: Представьте задачи: быстро посмотреть историю, изучить файл, найти изменения.

Ответ:

- **git log --oneline** — Быстрое чтение истории перед митингом
- **git log README.md** — Проверка изменений в документации
- **git log --graph --oneline** — Визуализация веток при исследовании репо
- **git show** — Анализ конкретного подозрительного коммита

🔮 Шаг 13:

Миссия 13

Расположите шаги изучения истории коммитов в правильном порядке.

Подсказка: Сначала смотрим общую историю, затем детали.

Ответ:

- `git log --oneline`
- `git log --graph`
- `git show`
- `git log --stat`

🔮 Шаг 14:

Миссия 14

Упорядочите действия программиста, который ищет баг в истории.

Подсказка: Сначала нужен общий обзор, потом детали и статистика.

Ответ:

- Просмотреть `git log --oneline`
- Найти подозрительный хэш
- Использовать `git show`
- Анализировать изменения с `git diff`

🔮 Шаг 15:

Миссия 15

Расставьте команды в порядке от самого краткого к самому подробному выводу истории.

Подсказка: Думайте: сокращенный формат → граф → статистика → полный diff.

Ответ:

- `git log --oneline`
- `git log --graph --oneline`

- `git log --stat`
- `git show`

💡 Шаг 16:

Миссия 16

Поставьте шаги ревью истории перед релизом в правильном порядке.

Подсказка: Сначала быстрый просмотр, потом углубление.

Ответ:

- `git log --oneline`
- `git log --stat`
- `git show`
- `git log --pretty=fuller`

💡 Шаг 17:

Миссия 17

Отсортируйте шаги поиска изменений конкретного файла.

Подсказка: Сначала найдите историю файла, затем углубляйтесь в детали.

Ответ:

- `git log README.md`
- Найти нужный хэш
- `git show`
- Сравнить изменения с `git diff`

📖 Урок 4 - Работа с логом и графами

💡 Шаг 01:

Миссия 1

Андрей открыл консоль и хочет быстро просмотреть историю коммитов с краткой информацией. Какую команду ему лучше использовать?

Подсказка: Подумайте, какая команда выводит список коммитов в одну строку с хешем и сообщением.

Ответ:

- **`git log --oneline`**
- `git status`
- `git show`
- `git commit --amend`

💡 Шаг 02:

Миссия 2

Марина хочет увидеть графическую структуру веток прямо в консоли. Какой параметр она должна добавить к `git log`?

Подсказка: Вспомните параметр, который добавляет древовидное отображение.

Ответ:

- **`--graph`**
- `--tree`

- `--visual`
- `--map`

🔗 Шаг 03:

Миссия 3

Фронтендер Лена хочет ограничить вывод истории только последними 5 коммитами. Как это сделать?

Подсказка: Есть специальный параметр для указания числа выводимых коммитов.

Ответ:

- **`git log -5`**
- `git log --recent=5`
- `git log --show=5`
- `git last 5`

🔗 Шаг 04:

Миссия 4

Андрей открыл консоль и хочет быстро просмотреть историю коммитов с краткой информацией. Какую команду ему лучше использовать?

Подсказка: Подумайте, какая команда выводит список коммитов в одну строку с хешем и сообщением.

Ответ:

- **`git log --oneline`**
- `git status`
- `git show`
- `git commit --amend`

🔗 Шаг 05:

Миссия 5

Марина хочет увидеть графическую структуру веток прямо в консоли. Какой параметр она должна добавить к `git log`?

Подсказка: Вспомните параметр, который добавляет древовидное отображение.

Ответ:

- **`--graph`**
- `--tree`
- `--visual`
- `--map`

🔗 Шаг 06:

Миссия 6

Фронтендер Лена хочет ограничить вывод истории только последними 5 коммитами. Как это сделать?

Подсказка: Есть специальный параметр для указания числа выводимых коммитов.

Ответ:

- **`git log -5`**
- `git log --recent=5`

- `git log --show=5`
- `git last 5`

🔧 Шаг 07:

Миссия 7

Артём хочет видеть не только коммиты, но и изменения файлов в них. Какой параметр он должен использовать?

Подсказка: Есть флаг, который добавляет список изменённых файлов.

Ответ:

- `--stat`
- `--diff`
- `--patch`
- `--files`

🔧 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте команды с их назначением при работе с `git log`.

Подсказка: Обратите внимание: одни параметры сокращают вывод, другие делают его более наглядным.

Ответ:

- `git log --oneline` — Выводит историю в одну строку на коммит.
- `git log --graph` — Добавляет древовидное отображение веток.
- `git log -3` — Ограничивает вывод последними тремя коммитами.
- `git log --stat` — Показывает изменённые файлы и статистику изменений.

🔧 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте ключи `pretty-format` с тем, что они выводят.

Подсказка: Эти символы позволяют настроить вид истории коммитов.

Ответ:

- `%h` — Сокращённый хеш коммита.
- `%an` — Имя автора коммита.
- `%s` — Сообщение коммита.
- `%ad` — Дата коммита.

🔧 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте команды с результатом вывода.

Подсказка: Подумайте, чем отличается полный вывод от сокращённого и от показа конкретного коммита.

Ответ:

- `git log` — Полная история коммитов.
- `git show HEAD` — Подробный вывод только одного последнего коммита.
- `git log --oneline --graph` — Краткая история в одну строку с древовидным отображением.
- `git log --stat` — История коммитов с изменёнными файлами и статистикой.

Шаг 11:

Миссия 11

 Задача: Сопоставьте флаги git log с их назначением.

Подсказка: Иногда важно видеть детали, иногда — только кратко.

Ответ:

- **--decorate** — Добавляет информацию об указателях (ветках, тегах).
- **--patch** — Показывает полный diff по каждому коммиту.
- **--since** — Ограничивает вывод коммитов начиная с определённой даты.
- **--author** — Фильтрует коммиты по автору.

Шаг 12:

Миссия 12

Расположите шаги для анализа истории коммитов в правильном порядке.

Подсказка: Сначала нужно получить список коммитов, затем ограничить или отфильтровать, потом рассмотреть детали.

Ответ:

- Посмотреть список коммитов через git log
- Добавить фильтры (автор, дата, количество)
- Включить визуализацию веток
- Изучить детали отдельных коммитов

Шаг 13:

Миссия 13

Расположите команды в порядке увеличения детализации информации о коммитах.

Подсказка: Начните с самого краткого формата и завершите самым подробным.

Ответ:

- git log --oneline
- git log --oneline --graph
- git log --stat
- git log --patch

Шаг 14:

Миссия 14

Расположите действия разработчика, проверяющего подозрительный коммит.

Подсказка: Нужно идти от общего к частному.

Ответ:

- Просмотреть список коммитов
- Ограничить список по автору
- Найти подозрительный коммит
- Использовать git show для анализа изменений

Шаг 15:

Миссия 15

Расставьте шаги работы с git log для подготовки отчёта по проекту.

Подсказка: Сначала фильтруем по дате и автору, потом собираем визуализацию.

Ответ:

- Использовать `git log --since` для ограничения даты
- Применить фильтрацию по автору
- Добавить визуализацию с `--graph`
- Сохранить вывод в файл

Шаг 16:

Миссия 16

Расставьте команды по сложности восприятия информации.

Подсказка: Чем больше деталей — тем труднее быстро воспринимать вывод.

Ответ:

- `git log --oneline`
- `git log --oneline --graph --decorate`
- `git log --stat --graph`
- `git log --patch --graph`

Шаг 17:

Миссия 17

Расположите действия команды при анализе ветвления в проекте.

Подсказка: Начните с вызова истории, затем добавляйте больше информации для анализа.

Ответ:

- Открыть историю через `git log`
- Добавить `--graph`
- Включить `--decorate` для отображения веток
- Использовать `--oneline` для компактности

Шаг 18:

Миссия 18

Расставьте шаги для того, чтобы наглядно показать историю коммитов новому сотруднику.

Подсказка: Вспомните, как сделать вывод и компактным, и красивым.

Ответ:

- Вызвать `git log --oneline`
- Добавить `--graph`
- Добавить `--decorate`
- Объяснить структуру коммитов

Модуль 4 - Управление ветками

Урок 1 - Создание, удаление и переименование веток

Шаг 01:

Миссия 1

Вы работаете в стартапе и вам нужно создать отдельную ветку для новой фичи. Какой командой это делается?

Подсказка: Вспомните: создание новой ветки делается через `git branch` или сразу с переключением.

Ответ:

- **git checkout -b feature/login**
- `git init feature/login`
- `git commit -b feature/login`
- `git branch --start feature/login`

Шаг 02:

Миссия 2

Разработчица Лена заметила, что случайно создала ветку с опечаткой: `featuer/api` . Какой командой лучше всего её переименовать?

Подсказка: Переименование ветки делается без её пересоздания, команда содержит `-m` .

Ответ:

- **git branch -m featuer/api feature/api**
- `git checkout -b feature/api`
- `git rename featuer/api feature/api`
- `git branch --rename feature/api`

Шаг 03:

Миссия 3

Андрей завершил работу над задачей и хочет удалить локальную ветку `bugfix/ui` . Какой командой это делается?

Подсказка: Вспомните: удаление ветки отличается флагами `-d` или `-D` .

Ответ:

- **git branch -d bugfix/ui**
- `git delete branch bugfix/ui`
- `git remove bugfix/ui`
- `git checkout --delete bugfix/ui`

Шаг 04:

Миссия 4

Вы работаете с коллегой, и он спрашивает: как посмотреть список всех веток в проекте?

Подсказка: Вспомните самую короткую команду.

Ответ:

- **git branch**
- `git list branches`
- `git checkout --list`
- `git show-branches`

Шаг 05:

Миссия 5

Вася хочет переключиться с текущей ветки на `develop` . Какой командой он это сделает?

Подсказка: Раньше использовали `checkout` , но теперь появился более современный вариант.

Ответ:

- **git switch develop**
- git change develop
- git open develop
- git checkout -D develop

Шаг 06:

Миссия 6

Оля случайно удалила локальную ветку и теперь хочет проверить, осталась ли она на удалённом репозитории. Какую команду ей запустить?

Подсказка: Ветка на сервере отображается вместе с префиксом `remotes/`.

Ответ:

- **git branch -r**
- git show-remote
- git list remote
- git checkout remote

Шаг 07:

Миссия 7

 Сопоставьте команды работы с ветками с их назначением.

Подсказка: Внимательно вспомните различия между созданием, переключением и удалением ветки.

Ответ:

- **git switch main** — Переключение на ветку `main`
- **git branch -d hotfix** — Удаление локальной ветки `hotfix`
- **git checkout -b feature/auth** — Создание и переключение на новую ветку `feature/auth`
- **git branch -m old new** — Переименование ветки `old` в `new`

Шаг 08:

Миссия 8

Сопоставьте ситуации с правильными Git-командами.

Подсказка: Подумайте, где нужно создавать ветку, где удалять, а где просто смотреть.

Ответ:

- **Нужно посмотреть список всех локальных веток** — `git branch`
- **Хочется удалить ветку, которая уже слита** — `git branch -d name`
- **Нужно срочно перейти на ветку `develop`** — `git switch develop`
- **Создать ветку и сразу на неё перейти** — `git checkout -b newbranch`

Шаг 09:

Миссия 9

Вы работаете в команде. Сопоставьте действия коллег с командами Git.

Подсказка: Каждое действие связано с ветками: переименование, проверка, удаление и создание.

Ответ:

- **Марина хочет переименовать ветку dev в develop** — `git branch -m dev develop`
- **Сергей проверяет ветки на сервере** — `git branch -r`
- **Аня создаёт новую ветку hotfix/ui** — `git checkout -b hotfix/ui`
- **Игорь удаляет ненужную ветку old** — `git branch -d old`

🔗 Шаг 10:

Миссия 10

Сопоставьте команду Git с её результатом.

Подсказка: Обратите внимание: есть команды для просмотра локальных и удалённых веток.

Ответ:

- **git branch -a** — Показать все ветки: локальные и удалённые
- **git branch** — Показать только локальные ветки
- **git switch -c exp** — Создать и перейти на новую ветку exp
- **git branch -D temp** — Принудительно удалить ветку temp

🔗 Шаг 11:

Миссия 11

Сопоставьте ветку и причину её удаления.

Подсказка: Подумайте: бывают ветки с багфиксами, экспериментами, фичами, которые уже не нужны.

Ответ:

- **hotfix/typo** — Исправление уже попало в main
- **feature/old_ui** — Фича не будет внедряться
- **experiment/cache** — Эксперимент не удался
- **temp/migration** — Временная ветка после миграции больше не нужна

🔗 Шаг 12:

Миссия 12

Сопоставьте команды с их коротким описанием.

Подсказка: Две команды похожи на удаление, но одна более строгая.

Ответ:

- **git branch -d** — Удаление ветки только если она слита
- **git branch -D** — Принудительное удаление ветки
- **git branch -m** — Переименование ветки
- **git switch** — Переключение между ветками

🔗 Шаг 13:

Миссия 13

Расположите шаги работы программиста с новой фичей в правильном порядке.

Подсказка: Сначала создаём ветку, потом работаем, затем сливаем и удаляем.

Ответ:

- Создать ветку feature/api
- Внести изменения в код

- Сделать merge в main
- Удалить ветку feature/api

Шаг 14:

Миссия 14

Вы тестируете экспериментальную функцию. Расположите действия по работе с веткой.

Подсказка: Сначала создаём, потом проверяем, а в конце убираем ненужное.

Ответ:

- Создать ветку experiment/logs
- Закоммитить изменения
- Понять, что эксперимент неудачный
- Удалить ветку experiment/logs

Шаг 15:

Миссия 15

Сергей готовит хотфикс. Расположите действия правильно.

Подсказка: Сначала новая ветка, исправление, merge, затем удаление.

Ответ:

- Создать ветку hotfix/payment
- Исправить баг
- Сделать merge в main
- Удалить ветку hotfix/payment

Шаг 16:

Миссия 16

Вася экспериментирует с UI. Расположите шаги в верном порядке.

Подсказка: Работа с экспериментами всегда включает удаление в конце.

Ответ:

- Создать ветку experiment/ui
- Сделать несколько коммитов
- Решить, что дизайн не подходит
- Удалить ветку experiment/ui

Шаг 17:

Миссия 17

Расположите шаги при переименовании ветки в правильном порядке.

Подсказка: Нужно сначала переключиться, потом переименовать и проверить результат.

Ответ:

- Переключиться на ветку old
- Переименовать её в new
- Посмотреть список веток, чтобы убедиться в изменении

Урок 2 - Переключение и checkout веток

Шаг 01:

Миссия 1

Андрей работает над новой задачей и хочет переключиться на ветку develop . Какую команду ему нужно использовать?

Подсказка: Существует старая команда checkout и более новая — switch .

Ответ:

- **git switch develop**
- git change develop
- git open develop
- git move develop

Шаг 02:

Миссия 2

Оля хочет переключиться на ветку feature/auth , которой пока нет локально, но она есть на сервере. Какую команду использовать?

Подсказка: Нужно не просто переключиться, а ещё и подтянуть ветку с origin.

Ответ:

- **git checkout -b feature/auth origin/feature/auth**
- git switch feature/auth --remote
- git fetch feature/auth
- git clone feature/auth

Шаг 03:

Миссия 3

Вася работает в ветке bugfix/ui , но ему срочно нужно вернуться в main . Какой командой он это сделает?

Подсказка: Используйте самую привычную команду для переключения.

Ответ:

- **git checkout main**
- git move main
- git select main
- git branch main

Шаг 04:

Миссия 4

Программистка Катя случайно написала git checkoutt main . Почему команда не сработала?

Подсказка: Внимательно проверьте синтаксис и название команды.

Ответ:

- **Опечатка в слове checkout**
- Ветка main не существует
- Git не поддерживает checkout
- Нужно было использовать git clone

Шаг 05:

Миссия 5

Разработчик переключился с main на feature/ui , но забыл закоммитить изменения. Что произойдёт при переключении?

Подсказка: Git может перенести изменения, если они не конфликтуют.

Ответ:

- **Изменения перейдут в новую ветку, если нет конфликтов**
- Git автоматически создаст stash
- Все изменения будут потеряны
- Git запретит переключение навсегда

Шаг 06:

Миссия 6

Вы случайно пытаетесь переключиться на несуществующую ветку. Что ответит Git?

Подсказка: Сообщение об ошибке будет говорить, что такой ветки нет.

Ответ:

- **error: pathspec 'branch_name' did not match any file(s) known to git**
- fatal: unknown command
- no branch found, auto-creating...
- git clone required

Шаг 07:

Миссия 7

 Сопоставьте команды переключения с их назначением.

Подсказка: Обратите внимание, что switch и checkout могут быть взаимозаменяемы.

Ответ:

- **git switch main** — Переключение на ветку main
- **git checkout feature/login** — Переключение на ветку feature/login
- **git switch -c test** — Создание и переключение на новую ветку test
- **git checkout -b hotfix/bug** — Создание и переключение на ветку hotfix/bug

Шаг 08:

Миссия 8

Сопоставьте ситуации и команды Git.

Подсказка: Одни команды переключают, другие создают ветку при переключении.

Ответ:

- **Нужно переключиться на ветку develop** — git checkout develop
- **Хочу создать новую ветку exp и перейти на неё** — git switch -c exp
- **Перейти на ветку из origin** — git checkout -b feature origin/feature
- **Просто переключиться на main** — git switch main

Шаг 09:

Миссия 9

Программисты перепутали команды. Помогите сопоставить правильные пары.

Подсказка: Вспомните: checkout старше, switch — новее.

Ответ:

- **git checkout -b demo** — Создать и перейти в ветку demo
- **git switch feature/api** — Перейти в ветку feature/api
- **git switch -c quickfix** — Создать и перейти в quickfix
- **git checkout release** — Переключение на ветку release

🔮 Шаг 10:

Миссия 10

Сопоставьте ветку и команду для переключения.

Подсказка: Подумайте, когда удобнее использовать switch, а когда checkout.

Ответ:

- **Перейти в main** — git switch main
- **Создать ветку hotfix и перейти** — git checkout -b hotfix
- **Перейти на experimental** — git checkout experimental
- **Создать branchX и сразу переключиться** — git switch -c branchX

🔮 Шаг 11:

Миссия 11

Сопоставьте описание с корректной командой.

Подсказка: Все варианты связаны с переключением и созданием веток.

Ответ:

- **Создать ветку test и перейти в неё** — git switch -c test
- **Просто перейти в ветку dev** — git checkout dev
- **Переключиться на release** — git switch release
- **Создать ветку featureX и переключиться** — git checkout -b featureX

🔮 Шаг 12:

Миссия 12

Свяжите действия и команды Git.

Подсказка: Проверьте внимательно, где именно используется -с.

Ответ:

- **Создать branch1 и переключиться** — git switch -c branch1
- **Перейти в main** — git checkout main
- **Создать hotfix1 и сразу перейти** — git checkout -b hotfix1
- **Перейти в dev** — git switch dev

🔮 Шаг 13:

Миссия 13

Расположите шаги при создании новой ветки с помощью switch.

Подсказка: -с используется для создания сразу с переключением.

Ответ:

- Выполнить git switch -c feature/new
- Внести изменения

- Сделать commit
- Вернуться в main

Шаг 14:

Миссия 14

Расположите действия при работе с экспериментальной веткой.

Подсказка: Эксперимент создаётся, проверяется и при необходимости удаляется.

Ответ:

- Создать и переключиться `git checkout -b experiment`
- Сделать изменения
- Понять, что эксперимент не нужен
- Вернуться в main

Шаг 15:

Миссия 15

У вас конфликт при переключении веток. Расставьте шаги действий.

Подсказка: Перед переключением важно сохранить или убрать изменения.

Ответ:

- Закоммитить или сделать `stash`
- Выполнить `git switch другая_ветка`
- Продолжить работу в новой ветке

Шаг 16:

Миссия 16

Расположите шаги при возвращении из feature в main.

Подсказка: Сначала сохраняем работу, потом переключаемся.

Ответ:

- Закоммитить изменения в feature
- Выполнить `git switch main`
- Проверить, что мы в main

Урок 3 - Merge и fast-forward

Шаг 01:

Миссия 1

Разработчик Костя создал новую ветку feature и хочет объединить её с main . При выполнении `git merge feature` Git выполнил fast-forward. Что это значит?

Подсказка: Подумайте, изменился ли граф коммитов или просто сдвинулась ссылка на ветку.

Ответ:

- Ветка main просто переместилась вперёд к последнему коммиту из feature
- Git создал новый merge-коммит
- Коммиты из feature были удалены
- Все изменения переписаны интерактивным ребейзом

🔗 Шаг 02:

Миссия 2

Лена делает merge ветки bugfix в develop . Git не смог выполнить fast-forward и создал merge-коммит. Почему так произошло?

Подсказка: Сравните историю коммитов: у fast-forward должна быть линейная цепочка.

Ответ:

- **Обе ветки имели собственные коммиты, история разошлась**
- Ветка bugfix не содержала изменений
- Git всегда создаёт merge-коммит по умолчанию
- Fast-forward работает только для удалённых репозитория

🔗 Шаг 03:

Миссия 3

Андрей заметил, что после fast-forward merge у него нет merge-коммита. Зачем иногда разработчики специально отключают fast-forward с помощью опции --no-ff ?

Подсказка: Подумайте о том, как лучше видеть историю разработки.

Ответ:

- **Чтобы явно сохранить точку слияния и историю ветки**
- Чтобы уменьшить размер репозитория
- Чтобы отменить изменения из ветки
- Чтобы Git автоматически удалил старую ветку

🔗 Шаг 04:

Миссия 4

Маша объединила ветку analytics в main через fast-forward. Что произойдёт с коммитами из analytics ?

Подсказка: Вспомните: fast-forward — это не новый коммит, а просто перемещение указателя.

Ответ:

- **Они станут частью истории main**
- Они будут удалены
- Они превратятся в один новый коммит
- Git перенесёт их в stash

🔗 Шаг 05:

Миссия 5

Вася выполнил git merge --no-ff feature . Чем это отличается от обычного fast-forward?

Подсказка: Сравните историю: где появляется дополнительный узел?

Ответ:

- **Будет создан merge-коммит даже при возможности fast-forward**
- Изменения не попадут в main
- Коммиты будут переписаны
- Git выполнит rebase вместо merge

🔗 Шаг 06:

Миссия 6

Дима заметил: иногда merge создаёт конфликты, даже если истории не слишком разошлись. Когда именно Git не может выполнить fast-forward?

Подсказка: У fast-forward нет конфликтов, если история строго линейна.

Ответ:

- **Когда обе ветки развивались независимо и имеют разные коммиты**
- Когда в ветке нет ни одного коммита
- Когда ветка удалена на сервере
- Когда пользователь не указывает флаг --ff-only

🔗 Шаг 07:

Миссия 7

Инженер Саша сделал merge ветки UI в main с флагом --ff-only . Что произойдёт, если fast-forward невозможен?

Подсказка: Этот флаг защищает историю от merge-коммитов.

Ответ:

- **Git откажется выполнять merge и выдаст ошибку**
- Git выполнит обычный merge с коммитом
- Git автоматически сделает rebase
- Git проигнорирует ветку UI

🔗 Шаг 08:

Миссия 8

Олег открыл историю репозитория в графическом клиенте. Как визуально выглядит fast-forward merge?

Подсказка: Вспомните, что merge-коммита нет.

Ответ:

- **Обычная линейная цепочка коммитов без развилок**
- Две линии истории, сходящиеся в merge-коммит
- Цепочка с большим количеством rebase-меток
- История обрывается и продолжается с другой ветки

🔗 Шаг 09:

Миссия 9

Тимур сравнивает merge и rebase. Чем fast-forward merge похож на rebase?

Подсказка: Оба метода стараются сохранить линейную историю.

Ответ:

- **Оба дают линейную историю без merge-коммитов**
- Оба всегда создают merge-коммит
- Оба требуют ручного редактирования всех коммитов
- Оба удаляют историю побочной ветки

🔧 Шаг 10:

Миссия 10

Сопоставьте ситуации с правильными действиями Git при merge.

Подсказка: Вспомните: fast-forward возможен только при линейной истории.

Ответ:

- **История линейна, ветка** — продолжение основной — Fast-forward merge без нового коммита
- **В обеих ветках есть разные новые коммиты** — Создаётся merge-коммит
- **Указан флаг --no-ff при линейной истории** — Git создаёт merge-коммит специально
- **Указан флаг --ff-only, но история разошлась** — Git выдаёт ошибку и прерывает merge

🔧 Шаг 11:

Миссия 11

Сопоставьте команды с их эффектами при работе с merge.

Подсказка: Ключевые флаги изменяют поведение Git при объединении.

Ответ:

- **git merge feature** — Обычный merge, fast-forward если возможно
- **git merge --no-ff feature** — Принудительное создание merge-коммита
- **git merge --ff-only feature** — Merge выполнится только если возможен fast-forward
- **git rebase main** — Переписывает коммиты поверх другой ветки

🔧 Шаг 12:

Миссия 12

Сопоставьте термины Git с их определениями.

Подсказка: Обратите внимание на отличие fast-forward и merge-коммита.

Ответ:

- **Fast-forward** — Перемещение указателя ветки вперёд без нового коммита
- **Merge-коммит** — Коммит, у которого два или более родителя
- **Rebase** — Перенос коммитов на другую ветку с изменением их хэшей
- **Конфликт** — Ситуация, когда Git не может автоматически объединить изменения

🔧 Шаг 13:

Миссия 13

Сопоставьте причины и последствия использования --no-ff .

Подсказка: Подумайте, зачем команде нужно явно видеть ветки в истории.

Ответ:

- **Нужно сохранить историю ветки** — Git создаёт merge-коммит даже при линейной истории
- **Хотим видеть, что фича была отдельной веткой** — История содержит явную точку слияния
- **Команда применяет GitFlow** — Обязательное использование merge-коммитов для всех фич
- **Не используем --no-ff** — Ветка исчезает, история линейная

Шаг 14:

Миссия 14

Расположите шаги выполнения fast-forward merge в правильном порядке.

Подсказка: Сначала нужно убедиться в линейности истории.

Ответ:

- Git проверяет, что история линейна
- Указатель целевой ветки перемещается на последний коммит
- История сохраняется без merge-коммита

Шаг 15:

Миссия 15

Расположите шаги при обычном merge с конфликтами.

Подсказка: Обратите внимание: Git создаёт merge-коммит только после разрешения конфликтов.

Ответ:

- Git выполняет попытку объединения изменений
- Git сообщает о конфликте
- Разработчик вручную разрешает конфликт
- Создаётся merge-коммит

Урок 4 - Практика слияние разных веток

Шаг 01:

Миссия 1

Анна создала ветку feature/login и внесла изменения. Она хочет объединить её с main . Что произойдёт при обычном merge, если история линейна?

Подсказка: Подумайте, нужен ли новый merge-коммит в линейной истории.

Ответ:

- **Git выполнит fast-forward, merge-коммит не создаётся**
- Git создаст merge-коммит
- Коммиты будут удалены
- Git откатит ветку main

Шаг 02:

Миссия 2

Игорь пытается объединить ветку hotfix в develop . Git создаёт merge-коммит. Почему fast-forward невозможен?

Подсказка: Обратите внимание на наличие новых коммитов в обеих ветках.

Ответ:

- **Обе ветки имеют разные новые коммиты**
- Ветка hotfix пуста
- Git всегда создаёт merge-коммит по умолчанию
- Git потерял историю main

Шаг 03:

Миссия 3

Лена делает merge ветки feature/payment в main с флагом --no-ff . Что произойдёт?

Подсказка: Этот флаг создаёт merge-коммит даже если fast-forward возможен.

Ответ:

- **Git создаст merge-коммит для сохранения истории ветки**
- Git выполнит fast-forward и не создаст коммит
- Коммиты feature будут удалены
- Git проигнорирует ветку feature

Шаг 04:

Миссия 4

Сергей сливает ветку frontend в develop . После merge появились конфликты. Что нужно сделать сначала?

Подсказка: Git не завершает merge до разрешения конфликтов.

Ответ:

- **Разрешить конфликты вручную**
- Принудительно удалить merge-коммит
- Сделать rebase ветки frontend
- Удалить ветку develop

Шаг 05:

Миссия 5

Оля хочет объединить ветку backend с main и видит, что история не линейна. Что произойдёт при обычном merge?

Подсказка: Обратите внимание, создаёт ли Git merge-коммит при разветвлённой истории.

Ответ:

- **Git создаст merge-коммит**
- Git выполнит fast-forward
- Git удалит ветку backend
- Git проигнорирует изменения main

Шаг 06:

Миссия 6

Максим делает merge ветки feature/ui в main . Он хочет, чтобы история была линейной. Как это сделать?

Подсказка: Rebase помогает перемещать коммиты ветки на другой базовый коммит.

Ответ:

- **Сделать rebase ветки feature/ui на main, потом fast-forward merge**
- Удалить ветку main
- Принудительно сделать merge с конфликтами
- Использовать --no-ff без проверки истории

🔗 Шаг 07:

Миссия 7

Иван использует `git merge --ff-only` для ветки `feature/api`. Что произойдёт, если fast-forward невозможен?

Подсказка: Этот флаг запрещает создание merge-коммита.

Ответ:

- **Git прервет merge с ошибкой**
- Git создаст merge-коммит
- Git выполнит rebase автоматически
- Git проигнорирует ветку `feature/api`

🔗 Шаг 08:

Миссия 8

Света сливает ветку `feature/notifications` с `main` после нескольких коммитов. Какой тип merge создаст Git по умолчанию?

Подсказка: Обратите внимание, есть ли разветвление в истории.

Ответ:

- **Обычный merge с merge-коммитом**
- Fast-forward merge без коммита
- Rebase merge
- Git откатит `main`

🔗 Шаг 09:

Миссия 9

Алексей сливает ветку `feature/settings` с `main` и видит сообщение о конфликте в файле `config.json`. Что нужно сделать?

Подсказка: Git не завершит merge, пока конфликты не разрешены.

Ответ:

- **Открыть файл, разрешить конфликт, затем завершить merge**
- Удалить `main` и попробовать снова
- Пропустить конфликт и продолжить merge
- Сделать rebase без проверки конфликта

🔗 Шаг 10:

Миссия 10

Сопоставьте тип merge с результатом для истории Git.

Подсказка: Обратите внимание на наличие merge-коммита.

Ответ:

- **Fast-forward merge** — Линейная история, merge-коммит не создаётся
- **Обычный merge** — Создаётся merge-коммит, история развилкой
- **Merge с --no-ff** — Создаётся merge-коммит даже при линейной истории
- **Merge с --ff-only** — Merge выполняется только если возможен fast-forward, иначе ошибка

🔗 Шаг 11:

Миссия 11

Сопоставьте команды Git с их эффектами при merge.

Подсказка: Ключевой момент — флаги влияют на создание merge-коммита.

Ответ:

- **git merge feature** — Merge выполняется, fast-forward если возможно
- **git merge --no-ff feature** — Merge-коммит создаётся даже при линейной истории
- **git merge --ff-only feature** — Merge выполнится только если возможен fast-forward
- **git rebase main** — Коммиты ветки перемещаются на верхнюю точку другой ветки

🔗 Шаг 12:

Миссия 12

Сопоставьте возможные конфликты при merge с их причиной.

Подсказка: Конфликт возникает, когда изменения пересекаются в одном файле.

Ответ:

- **Файл изменён в обеих ветках** — Git не может автоматически объединить изменения
- **Добавлены новые файлы без совпадений** — Merge проходит без конфликтов
- **Rebase с конфликтами** — Git требует ручного разрешения конфликтов
- **Fast-forward merge** — Конфликты не возникают

🔗 Шаг 13:

Миссия 13

Сопоставьте действие Git с результатом при работе с ветками.

Подсказка: Обратите внимание, как разные merge-флаги изменяют историю.

Ответ:

- **git merge feature с линейной историей** — Fast-forward merge без нового коммита
- **git merge feature с разветвлённой историей** — Создаётся merge-коммит
- **git merge --no-ff feature** — Создаётся merge-коммит принудительно
- **git merge --ff-only feature при разветвлённой истории** — Git выдаёт ошибку

🔗 Шаг 14:

Миссия 14

Расположите шаги merge с конфликтами в правильном порядке.

Подсказка: Git не завершает merge до разрешения всех конфликтов.

Ответ:

- Git пытается объединить изменения
- Появляется сообщение о конфликте
- Разработчик разрешает конфликт вручную
- Git создаёт merge-коммит после разрешения

🔗 Шаг 15:

Миссия 15

Расположите шаги fast-forward merge в правильном порядке.

Подсказка: Убедитесь, что история линейна перед слиянием.

Ответ:

- Проверка, что история веток линейна
- Указатель целевой ветки перемещается на последний коммит исходной ветки
- История сохраняется без merge-коммита

Модуль 5 - Стратегии работы с Git

Урок 1 - Git-flow обзор и практика

Шаг 01:

Миссия 1

Разработчик Миша впервые заходит в новый проект и видит ветки main и develop . Какая из них в Git-flow считается стабильной и предназначенной только для релизов?

Подсказка: В эту ветку обычно нельзя коммитить напрямую.

Ответ:

- **main**
- develop
- feature/login
- hotfix/typo

Шаг 02:

Миссия 2

Аня создала новую фичу, но не знает, как правильно назвать ветку по Git-flow. Какой префикс рекомендуется использовать?

Подсказка: Этот префикс помогает отличать новые функции от исправлений.

Ответ:

- **feature/**
- bugfix/
- release/
- hotfix/

Шаг 03:

Миссия 3

Тимлид Денис запрещает прямые коммиты в main . Вместо этого он просит всех создавать pull request'ы. Почему?

Подсказка: Это связано с безопасностью релизов.

Ответ:

- **Чтобы каждый релиз проходил код-ревью**
- Потому что pull request выглядит красиво
- Чтобы ускорить разработку
- Потому что Git не позволяет

Шаг 04:

Миссия 4

QA-инженер Пётр нашёл критическую ошибку в продакшене. Какой тип ветки Git-flow он должен создать?

Подсказка: Это не feature и не release.

Ответ:

- **hotfix/**
- feature/
- develop/
- support/

Шаг 05:

Миссия 5

Когда обычно создаётся ветка release/ ?

Подсказка: Это момент, когда все запланированные фичи готовы.

Ответ:

- **После завершения всех feature-веток**
- Перед началом новой задачи
- Сразу после создания репозитория
- Когда настроен CI/CD

Шаг 06:

Миссия 6

Саша смотрит на список веток в репозитории и пытается вспомнить, для чего каждая из них. Помогите ему сопоставить!

Подсказка: Подумай, какие ветки долгоживущие, а какие временные.

Ответ:

- **main** — Хранит только стабильные релизы
- **develop** — Основная ветка для интеграции новых фич
- **feature/UI-redesign** — Разработка новой функциональности
- **release/1.2.0** — Подготовка к релизу
- **hotfix/null-pointer** — Срочное исправление ошибки в продакшене

Шаг 07:

Миссия 7

Программист Коля сомневается, какие команды использовать для работы с ветками. Помогите ему сопоставить команды с действиями.

Подсказка: Одна команда создаёт ветку, другая переключает, третья объединяет.

Ответ:

- **git checkout -b feature/login** — Создание и переход в новую ветку
- **git merge feature/login** — Объединение изменений
- **git checkout develop** — Переключение ветки
- **git branch -d feature/login** — Удаление локальной ветки

Шаг 08:

Миссия 8

DevOps-инженер Лена настраивает CI и хочет знать, какие ветки триггерят какие процессы. Сопоставь ветку и соответствующий пайплайн.

Подсказка: Продакшен — только из main.

Ответ:

- **main** — Deploy в продакшен
- **develop** — Сборка и тесты в staging
- **feature/auth-2fa** — Запуск unit-тестов
- **hotfix/config-error** — Немедленный deploy в продакшен

Шаг 09:

Миссия 9

UX-дизайнер прислал Игорю список задач, и он пытается решить, какие из них — новые фичи, а какие — hotfix. Помоги ему!

Подсказка: Если что-то «горит» — это явно hotfix.

Ответ:

- **feature/add-dark-mode** — Добавить тёмную тему
- **hotfix/crash-on-startup** — Приложение падает при запуске
- **feature/share-button** — Добавить кнопку «Поделиться»
- **hotfix/payment-timeout** — Ошибки при оплате

Шаг 10:

Миссия 10

Расставь шаги создания новой фичи по Git-flow в правильном порядке:

Подсказка: Начало — от develop, конец — с merge обратно.

Ответ:

- Создать ветку feature от develop
- Написать код и сделать коммиты
- Открыть pull request в develop
- Смерджить после ревью

Шаг 11:

Миссия 11

Как выглядит типичный цикл релиза в Git-flow?

Подсказка: Релизы не появляются из воздуха.

Ответ:

- Смерджить release в main
- Создать ветку release от develop
- Пофиксить баги и обновить версию
- Смерджить release обратно в develop

Шаг 12:

Миссия 12

У тебя случился hotfix! Быстро восстанови порядок действий:

Подсказка: Всё начинается с ветки main.

Ответ:

- Создать ветку hotfix от main
- Исправить ошибку
- Смерджить в main
- Смерджить в develop

 Шаг 13:

Миссия 13

Порядок действий для code review в Git-flow:

Подсказка: Без ревью — ни шагу!

Ответ:

- Создать pull request
- Получить комментарии от команды
- Внести исправления
- Смерджить после одобрения

 Шаг 14:

Миссия 14

Расположи этапы интеграции CI/CD с Git-flow:

Подсказка: Билд должен запускаться до деплоя.

Ответ:

- Билд и юнит-тесты для feature
- Автотесты для develop
- Deploy из main

 Шаг 15:

Миссия 15

Синхронизация с remote: расставь шаги правильно.

Подсказка: Сначала заведи, потом пушь.

Ответ:

- git fetch
- git rebase origin/develop
- git push

 Шаг 16:

Миссия 16

Финальный бой! Как выглядит полный путь задачи от идеи до продакшена?

Подсказка: От ветки до релиза.

Ответ:

- Создание feature-ветки
- Разработка и тесты
- Merge в develop
- Создание release
- Merge в main

Урок 2 - Trunk-based development

Шаг 01:

Миссия 1

Дима впервые подключился к команде, где используют trunk-based development. Какой главный принцип этой стратегии?

Подсказка: Представь, что есть только одна главная ветка, как единственный вход в метро.

Ответ:

- **Частые небольшие коммиты прямо в основную ветку**
- Работа только в локальных ветках без синхронизации
- Создание долгоживущих feature-веток
- Запрещено коммитить чаще одного раза в неделю

Шаг 02:

Миссия 2

Таня пытается понять, почему команда запрещает ей держать ветку больше двух дней. Какова причина?

Подсказка: Чем дольше ты живёшь в параллельной реальности, тем сложнее вернуться в основную.

Ответ:

- **Чтобы избежать сложных конфликтов при слиянии**
- Чтобы уменьшить размер репозитория
- Чтобы Git быстрее запускался
- Чтобы казаться более занятым

Шаг 03:

Миссия 3

Разработчик Саша делает 10 коммитов подряд. Что важно соблюдать при trunk-based development?

Подсказка: Маленькие шаги, но каждый должен быть стабильным.

Ответ:

- **Каждый коммит должен быть рабочим и проходить тесты**
- Каждый коммит должен содержать минимум 500 строк кода
- Коммиты должны быть строго между 12:00 и 13:00
- Название коммита обязано содержать шутку

Шаг 04:

Миссия 4

Лена добавила экспериментальную фичу, но не уверена, можно ли выкатывать её сразу в trunk. Что ей использовать?

Подсказка: Иногда нужно спрятать фичу под «одеяло».

Ответ:

- **Feature flags (фича-флаги)**
- Хеширование коммитов
- git revert

- Очистка рабочего каталога

🔧 Шаг 05:

Миссия 5

В команде появился новичок, который всегда делает `git push --force` прямо в `trunk`. Что с ним случится?

Подсказка: Люди помнят ошибки не хуже, чем Git — историю коммитов.

Ответ:

- Он перепишет историю и вызовет гнев всей команды
- Git автоматически заблокирует его аккаунт
- `Trunk` удалится сам собой
- Все коммиты превратятся в эмодзи

🔧 Шаг 06:

Миссия 6

Стас внедряет автоматическое CI для `trunk`. Какова цель?

Подсказка: Лучше робот поругает, чем клиент заметит баг.

Ответ:

- Проверять каждый коммит перед попаданием в `trunk`
- Автоматически создавать ветки для каждого коммита
- Переименовывать коммиты по шаблону
- Удалять `trunk` раз в неделю

🔧 Шаг 07:

Миссия 7

Сопоставь термин с его объяснением.

Подсказка: Представь, что это роли в команде супергероев-разработчиков.

Ответ:

- **Trunk** — Главная стабильная ветка проекта.
- **Feature branch** — Краткоживущая ветка для небольшой задачи.
- **Continuous Integration** — Автоматическая проверка кода при слиянии.
- **Code Review** — Проверка кода коллегой перед добавлением в `trunk`.

🔧 Шаг 08:

Миссия 8

Свяжи проблему с подходящим решением в `trunk-based development`.

Подсказка: Если конфликт — ищи мирного посредника.

Ответ:

- Долгие конфликты при `merge` — Частые короткие ветки.
- Нестабильный `trunk` — Обязательное прохождение CI.
- Неожиданные баги в проде — Feature flags.
- Медленный код-ревью — Маленькие инкрементальные коммиты.

🔮 Шаг 09:

Миссия 9

Определи, кто в команде нарушает процесс.

Подсказка: Один из них явно живёт ещё в эпохе SVN.

Ответ:

- **Андрей делает 5 коммитов в день** — Следует практике trunk-based.
- **Катя держит ветку 3 недели** — Риск накопленных конфликтов.
- **Олег пишет тесты перед merge** — Повышает стабильность.
- **Марина пушит без review в прод** — Опасное нарушение процесса.

🔮 Шаг 10:

Миссия 10

Сопоставь Git-команду с её назначением.

Подсказка: Представь, что это заклинания программиста.

Ответ:

- **git pull** — Получить изменения из trunk.
- **git rebase** — Переписать историю относительно trunk.
- **git stash** — Временно спрятать изменения.
- **git revert** — Создать обратный коммит.

🔮 Шаг 11:

Миссия 11

Сопоставь тип участника команды с его поведением.

Подсказка: Определи, кто джедай, а кто падаван.

Ответ:

- **Engineeer-спринтер** — Делает частые мелкие коммиты.
- **Архиватор** — Сохраняет изменения раз в неделю.
- **Хакер-недоверчивый** — Никогда не пушит без review.
- **Экспериментатор** — Активно использует feature flags.

🔮 Шаг 12:

Миссия 12

Расположи действия по правильному workflow trunk-based разработки.

Подсказка: Представь, что это кулинарный рецепт, только вместо хлеба — trunk.

Ответ:

- Создать короткую ветку от trunk
- Сделать небольшие изменения
- Пройти тесты в CI
- Слить обратно в trunk

🔮 Шаг 13:

Миссия 13

Упорядочи шаги code review в trunk-based процессе.

Подсказка: Сначала сделай — потом попроси совета.

Ответ:

- Сделать коммит
- Создать pull request
- Получить review
- Смёржить в trunk

🔮 Шаг 14:

Миссия 14

Расположи приоритеты при trunk-based development.

Подсказка: Без стабильности — никакого кайфа.

Ответ:

- Стабильный trunk
- Быстрая поставка кода
- Короткие ветки
- Обязательные тесты

🔮 Шаг 15:

Миссия 15

Как Катя должна вывести фичу с помощью фича-флага?

Подсказка: Сначала спрячь, потом покажи.

Ответ:

- Добавить код с флагом
- Выкатить в trunk
- Включить флаг для тестирования
- Включить флаг для всех

🔮 Шаг 16:

Миссия 16

Как предотвратить падение trunk?

Подсказка: Подумай как охранник в ночном клубе — кого впустить, кого нет.

Ответ:

- Запустить CI
- Проверить тесты локально
- Сделать коммит
- Смёржить в trunk

🔮 Шаг 17:

Миссия 17

Как действовать при конфликте в trunk?

Подсказка: Не паникуй — всё решаемо.

Ответ:

- Сделать git pull
- Разрешить конфликт

- Протестировать изменения
- Сделать push

Урок 3 - Практика feature branch и merge в main

Шаг 01:

Миссия 1

Аня начала разработку новой кнопки и создала ветку feature/button . Какой правильный способ переключиться в неё?

Подсказка: Не забывай про команду для смены веток!

Ответ:

- **git checkout feature/button**
- git new feature/button
- git switch main feature/button
- git start feature/button

Шаг 02:

Миссия 2

Борис создал ветку feature/login и внёс изменения. Как ему добавить все изменения в коммит?

Подсказка: Можно одной командой добавить всё сразу.

Ответ:

- **git add .**
- git include all
- git push .
- git commit all

Шаг 03:

Миссия 3

Катя закончила работу над веткой feature/search . Какой командой она отправит её на удалённый сервер впервые?

Подсказка: Не забудь указать origin и имя ветки.

Ответ:

- **git push -u origin feature/search**
- git upload feature/search
- git remote feature/search
- git publish origin

Шаг 04:

Миссия 4

Дима хочет убедиться, что его ветка feature/api основана на свежей версии main . Что ему нужно сделать?

Подсказка: Нужно подтянуть изменения перед слиянием.

Ответ:

- **git pull origin main**
- git upload origin main

- `git refresh main`
- `git reset main`

🔧 Шаг 05:

Миссия 5

Саша в ветке `feature/ui` решил объединить изменения с `main`. Какой командой он выполнит слияние?

Подсказка: Сначала нужно быть в нужной ветке.

Ответ:

- **`git merge main`**
- `git merge feature/ui`
- `git connect main`
- `git combine main`

🔧 Шаг 06:

Миссия 6

После слияния в ветке появился конфликт. Что делать в первую очередь?

Подсказка: Git не разрешит коммит, пока не устранятся конфликты.

Ответ:

- **Открыть конфликтующие файлы и вручную исправить участки**
- Удалить все изменения
- Сразу выполнить `git push`
- Отключить Git на время

🔧 Шаг 07:

Миссия 7

🧠 Лена запуталась в командах и их назначениях. Помогите ей сопоставить!

Команды:

Ответ:

- `git branch`
- `git checkout`
- `git merge`
- `git status`

Назначения:

1. Просмотр текущего состояния репозитория
2. Создание или просмотр веток
3. Переключение на другую ветку
4. Объединение изменений из одной ветки в другую

Подсказка: Представь, что это как навигация по комнатам: статус — оглядеться, чек-аут — войти, merge — соединить комнаты!

- **`git branch`** — Создание или просмотр веток
- **`git checkout`** — Переключение на другую ветку
- **`git merge`** — Объединение изменений из одной ветки в другую
- **`git status`** — Просмотр текущего состояния репозитория

🔧 Шаг 08:

Миссия 8

🕒 Тимур забывает, какие ветки зачем. Помоги ему соотнести!

Ответ:

- main
- feature/chat
- hotfix/login-bug
- release/v2.0

1. Основная стабильная ветка
2. Ветка с новой функциональностью
3. Ветка для подготовки версии к релизу
4. Ветка для срочного исправления ошибок

Подсказка: Если горит — hotfix, если улучшение — feature!

- **main** — Основная стабильная ветка
- **feature/chat** — Ветка с новой функциональностью
- **hotfix/login-bug** — Ветка для срочного исправления ошибок
- **release/v2.0** — Ветка для подготовки версии к релизу

🔧 Шаг 09:

Миссия 9

Надя увидела в логах сообщения о merge и rebase. Соедини термины и описания!

Ответ:

- Merge commit
- Fast-forward merge
- Rebase
- Conflict

1. Слияние с созданием отдельного коммита
2. Переписывание истории для последовательности
3. Слияние без нового коммита, если нет разветвлений
4. Ситуация, когда Git не может объединить автоматически

Подсказка: Fast-forward — как перемотка вперёд, без ответвлений.

- **Merge commit** — Слияние с созданием отдельного коммита
- **Fast-forward merge** — Слияние без нового коммита, если нет разветвлений
- **Rebase** — Переписывание истории для последовательности
- **Conflict** — Ситуация, когда Git не может объединить автоматически

🔧 Шаг 10:

Миссия 10

🕒 DevOps-Света обучает стажёров. Соотнеси команды с их результатом!

Ответ:

- git pull --rebase
- git fetch
- git push

- `git log --oneline`

1. Получение изменений без слияния
2. Отправка локальных коммитов в удалённый репозиторий
3. Обновление и переписывание истории
4. Просмотр истории коммитов в кратком виде

Подсказка: `fetch` ничего не меняет в рабочем каталоге, только обновляет ссылки.

- **`git pull --rebase`** — Обновление и переписывание истории
- **`git fetch`** — Получение изменений без слияния
- **`git push`** — Отправка локальных коммитов в удалённый репозиторий
- **`git log --oneline`** — Просмотр истории коммитов в кратком виде

Шаг 11:

Миссия 11

🔗 Команда в панике: «Какой инструмент для чего?» Соотнеси!

Ответ:

- Pull Request
 - Code Review
 - Merge Request
 - Approval
1. Процесс проверки изменений
 2. Запрос на объединение ветки
 3. Аналог Pull Request в GitLab
 4. Подтверждение готовности к слиянию

Подсказка: GitHub = Pull Request, GitLab = Merge Request.

- **Pull Request** — Запрос на объединение ветки
- **Code Review** — Процесс проверки изменений
- **Merge Request** — Аналог Pull Request в GitLab
- **Approval** — Подтверждение готовности к слиянию

Шаг 12:

Миссия 12

📦 Расставь шаги работы Кирилла с новой фичей по порядку:

1. Создать ветку `feature/payment`
2. Закоммитить изменения
3. Переключиться на `main`
4. Сделать `merge feature/payment` в `main`

Подсказка: Сначала создаём, потом работаем, потом объединяем!

Ответ:

- Создать ветку `feature/payment`
- Закоммитить изменения
- Переключиться на `main`
- Сделать `merge feature/payment` в `main`

🔧 Шаг 13:

Миссия 13

🔧 Расставь этапы отправки Pull Request в правильном порядке:

1. Открыть Pull Request на GitHub
2. Сделать push ветки
3. Получить review от команды
4. Смержить в main

Подсказка: PR не появится сам — его нужно открыть.

Ответ:

- Сделать push ветки
- Открыть Pull Request на GitHub
- Получить review от команды
- Смержить в main

🔧 Шаг 14:

Миссия 14

⌘ Конфликт! Как Петя должен действовать при конфликте в merge?

1. Открыть конфликтующие файлы
2. Решить конфликт вручную
3. Добавить файлы через git add
4. Сделать commit

Подсказка: Git ждёт, пока ты сам решишь, чей код оставить.

Ответ:

- Открыть конфликтующие файлы
- Решить конфликт вручную
- Добавить файлы через git add
- Сделать commit

🔧 Шаг 15:

Миссия 15

🔧 Ты решил удалить старую ветку. Какой порядок действий?

1. Переключиться на main
2. Удалить ветку локально
3. Удалить ветку на сервере
4. Убедиться, что merge уже выполнен

Подсказка: Сначала убедись, что ветка не нужна.

Ответ:

- Переключиться на main
- Убедиться, что merge уже выполнен
- Удалить ветку локально
- Удалить ветку на сервере

🔧 Шаг 16:

Миссия 16

✂ Как Лиза обновляет свою ветку через rebase?

1. Переключиться на feature/ui
2. Выполнить git fetch
3. Запустить git rebase origin/main
4. Решить конфликты и продолжить rebase

Подсказка: rebase — переписывание истории, будь осторожен!

Ответ:

- Выполнить git fetch
- Переключиться на feature/ui
- Запустить git rebase origin/main
- Решить конфликты и продолжить rebase

🔧 Шаг 17:

Миссия 17

🏁 Финал! Как команда выпускает новую версию?

1. Создать ветку release
2. Протестировать все изменения
3. Смержить release в main
4. Поставить тег версии

Подсказка: Релиз — это почти праздник, но порядок всё равно нужен!

Ответ:

- Создать ветку release
- Протестировать все изменения
- Смержить release в main
- Поставить тег версии

📖 Урок 4 - Workflow для командной разработки

🔧 Шаг 01:

Миссия 1

Фронтендер Саша по ошибке закоммитил огромный лог-файл в репозиторий. Как ему правильно исправить ситуацию, не ломая историю коллег?

Подсказка: Подумай, что безопаснее: изменить историю или отменить изменения новым коммитом?

Ответ:

- Создать commit с git revert и удалить файл
- Сделать git reset --hard и переслать коллегам ZIP-архив
- Использовать git rebase -i и переписать всю историю
- Удалить файл напрямую с GitHub через веб-интерфейс

🔧 Шаг 02:

Миссия 2

Бэкендер Ира создала ветку `feature/login`, но хочет разместить её на сервере, чтобы тимлид сделал ревью. Какой командой ей воспользоваться?

Подсказка: Команда должна отправить новую ветку на удалённый репозиторий.

Ответ:

- `git commit --push origin`
- **`git push -u origin feature/login`**
- `git merge origin/feature/login`
- `git pull feature/login`

Шаг 03:

Миссия 3

DevOps Сева замечает, что команда постоянно делает `push` прямо в `main`. Что он должен предложить, чтобы улучшить процесс?

Подсказка: Подумай, какое правило помогает держать `main` стабильным.

Ответ:

- **Запретить прямые `push` и использовать `pull request`**
- Перейти на SVN, там проще
- Удалить ветку `main` и работать в одной общей ветке
- Каждый день пересоздавать репозиторий

Шаг 04:

Миссия 4

Тимлид Лена просит не коммитить напрямую, а создавать Merge Request / Pull Request. Зачем?

Подсказка: Вспомни, как работает командная проверка кода.

Ответ:

- Чтобы GitHub начислял больше звёзд
- **Чтобы другие смогли посмотреть изменения перед слиянием**
- Чтобы всё выглядело модно
- Чтобы удалить историю коммитов

Шаг 05:

Миссия 5

Безопасник Гриша заметил в PR секретный API-ключ. Что он должен сделать?

Подсказка: Нельзя просто замаскировать — ключ должен быть недействительным.

Ответ:

- **Отозвать ключ и попросить удалить его из истории**
- Скрыть ключ в комментариях
- Переименовать переменную и оставить ключ
- Ничего не делать, никто не заметит

Шаг 06:

Миссия 6

Разработчик Антон создал pull request, но билд упал. Какое его следующее действие?

Подсказка: PR с красным билдом вряд ли примут.

Ответ:

- Удалить PR и сделать вид, что ничего не было
- Попросить тимлида смержить «на глаз»
- **Исправить код и обновить PR**
- Создать второй PR поверх первого

🔗 Шаг 07:

Миссия 7

Сопоставь виды веток с их назначением в Git-flow.

Подсказка: Одна ветка — для релизов, другая — для исправлений в продакшене.

Ответ:

- **develop** — Основная ветка разработки
- **feature/*** — Разработка новой функциональности
- **hotfix/*** — Экстренное исправление в продакшене
- **release/*** — Подготовка стабильной версии перед релизом

🔗 Шаг 08:

Миссия 8

Настя получила код-ревью. Соотнеси тип комментария с правильной реакцией.

Подсказка: Некоторое можно проигнорировать, но лучше обсудить.

Ответ:

- **"nit: лучше назвать переменную понятнее"** — Переименовать переменную
- **"bug: при null всё ломается"** — Исправить логику
- **"optional: можно вынести в отдельную функцию"** — Обсудить с ревьюером, нужно ли менять
- **"typo: опечатка в слове 'success'"** — Исправить текст

🔗 Шаг 09:

Миссия 9

Сопоставь команды с их результатом при работе с удалёнными ветками.

Подсказка: Одна команда забирает, другая — отправляет.

Ответ:

- **git fetch** — Загружает изменения без слияния
- **git pull** — Загружает и сливает изменения
- **git push** — Отправляет локальные коммиты
- **git clone** — Создаёт копию репозитория

🔗 Шаг 10:

Миссия 10

Сопоставь стратегию merge/rebase с её характеристикой.

Подсказка: Один оставляет историю «как есть», другой переписывает.

Ответ:

- **Merge** — Сохраняет историю с ветвлениями
- **Rebase** — Переписывает историю на прямую линию

- **Squash merge** — Объединяет все коммиты в один
- **Fast-forward** — Просто двигает указатель без коммита слияния

🔗 Шаг 11:

Миссия 11

Тимлид просит писать информативные сообщения коммитов. Сопоставь плохой и хороший стиль.

Подсказка: В одном понятно, что сделано, в другом — нет.

Ответ:

- **"fix"** — Плохой — непонятно, что исправлено
- **"Fix crash when user has no avatar"** — Хороший — объясняет причину и эффект
- **"update code"** — Плохой — слишком абстрактно
- **"Add validation for empty email field"** — Хороший — описывает действие

🔗 Шаг 12:

Миссия 12

Сопоставь проблему с соответствующей Git-командой.

Подсказка: Вспомни, что делает stash.

Ответ:

- **Нужно временно сохранить изменения без коммита** — `git stash`
- **Хочу посмотреть историю коммитов** — `git log`
- **Случайно удалил файл и хочу отменить** — `git checkout -- файл`
- **Нужно отменить локальные изменения целиком** — `git reset --hard`

🔗 Шаг 13:

Миссия 13

Расположи шаги создания feature-ветки по Git-flow.

Подсказка: Сначала обновляем develop.

Ответ:

- `git checkout develop`
- `git pull origin develop`
- `git checkout -b feature/new-feature`
- `git push -u origin feature/new-feature`

🔗 Шаг 14:

Миссия 14

Упорядочи процесс прохождения Pull Request.

Подсказка: После ревью обычно происходят правки.

Ответ:

- Создание PR
- Получение ревью
- Внесение изменений по комментариям
- Слияние в основную ветку

Шаг 15:

Миссия 15

DevOps внедряет CI/CD. Упорядочь стадии пайплайна.

Подсказка: Сначала тесты, потом выкладка.

Ответ:

- Сборка
- Тестирование
- Деплой на staging
- Деплой на production

Шаг 16:

Миссия 16

Упорядочь действия при конфликте слияния.

Подсказка: Надо сначала понять, где конфликт.

Ответ:

- Открыть конфликтный файл
- Выбрать правильный вариант кода
- Убрать маркеры <<<<<< >>>>>>
- Создать commit с решением конфликта

Шаг 17:

Миссия 17

Упорядочь шаги подготовки релиза по Git-flow.

Подсказка: Release начинается из develop.

Ответ:

- git checkout develop
- git checkout -b release/1.2.0
- Тестирование и исправления в release
- Слияние в main и создание тега

Модуль 6 - Управление коммитами

Урок 1 - Cherry-pick выборочные коммиты

Шаг 01:

Миссия 1

Аня случайно сделала коммит прямо в основную ветку main . Какой командой она может перенести этот коммит в новую ветку, не изменяя историю main ?

Подсказка: Эта команда позволяет "забрать" конкретный коммит и применить его поверх другой ветки.

Ответ:

- **git cherry-pick**
- git merge
- git reset --hard

- `git stash apply`

🔗 Шаг 02:

Миссия 2

Борис работает над экспериментальной веткой. Он нашёл коммит в `develop`, который хотел бы использовать. Что нужно сделать перед выполнением `git cherry-pick`?

Подсказка: Сначала надо оказаться в той ветке, куда хочешь перенести изменения.

Ответ:

- **Переключиться в нужную ветку с помощью `git checkout`**
- Удалить ветку `develop`
- Запустить `git fetch --all`
- Сделать `git reset HEAD~1`

🔗 Шаг 03:

Миссия 3

Катя сделала несколько коммитов подряд, но хочет перенести только один из середины. Подойдёт ли для этого `cherry-pick`?

Подсказка: Эта команда не обязательно берёт только последний коммит.

Ответ:

- **Да, можно указать конкретный `hash` любого коммита**
- Нет, `cherry-pick` работает только с последним коммитом

🔗 Шаг 04:

Миссия 4

Дима случайно сделал `cherry-pick` не того коммита. Как он может отменить его, если ещё не сделал `push`?

Подсказка: Эта команда откатывает последний коммит, сохранив изменения в истории.

Ответ:

- **`git revert HEAD`**
- `git rm HEAD`
- `git clean -f`
- `git cherry-pick --abort`

🔗 Шаг 05:

Миссия 5

Во время `cherry-pick` Наташа получила конфликт. Что нужно сделать?

Подсказка: Команда не завершится сама — нужно вручную помочь Git.

Ответ:

- **Разрешить конфликт вручную и выполнить `git cherry-pick --continue`**
- Сразу сделать `git push --force`
- Удалить репозиторий и клонировать заново
- Выполнить `git clean -dfx`

🔗 Шаг 06:

Миссия 6

Саша хочет перенести сразу несколько коммитов подряд с помощью cherry-pick. Как это сделать одной командой?

Подсказка: Можно указать диапазон, но важно помнить порядок.

Ответ:

- **git cherry-pick A..B**
- git cherry-pick A-B
- git cherry-pick A+B
- git cherry-pick A>B

🔗 Шаг 07:

Миссия 7

🖥️ Вася заметил разные сценарии использования cherry-pick. Сопоставьте ситуацию с действием.

Подсказка: Одно дело — забрать один коммит, другое — несколько подряд.

Ответ:

- **Перенести один коммит** — git cherry-pick
- **Перенести диапазон коммитов** — git cherry-pick A..B
- **Отменить неудачный cherry-pick** — git revert HEAD
- **Прервать конфликтующий cherry-pick** — git cherry-pick --abort

🔗 Шаг 08:

Миссия 8

👂 Оля хочет различать похожие команды. Сопоставьте действие с правильной командой.

Подсказка: reset изменяет историю, а revert добавляет новый коммит.

Ответ:

- **Откатить commit с сохранением истории** — git revert
- **Откатить commit без сохранения истории** — git reset --hard
- **Откатить cherry-pick с конфликтом** — git cherry-pick --abort
- **Продолжить cherry-pick после конфликта** — git cherry-pick --continue

🔗 Шаг 09:

Миссия 9

✳️ Команда DevOps использует cherry-pick для hotfix'ов. Сопоставьте описание с результатом.

Подсказка: В продакшен обычно уносят минимальные изменения.

Ответ:

- **Нужно срочно применить исправление из ветки develop в main** — git cherry-pick в main
- **Хотим перенести два крайних коммита** — git cherry-pick HEAD~1 HEAD
- **Хотим забрать всё кроме последнего коммита** — git cherry-pick A..HEAD~1
- **Случайно перенесли не тот коммит** — git revert HEAD

🔗 Шаг 10:

Миссия 10

👤 Женья проводит код-ревью. Сопоставьте ошибку новичка и возможное последствие.

Подсказка: Некоторые команды переписывают историю навсегда.

Ответ:

- **Сделал cherry-pick в неправильную ветку** — Придётся отменять через revert
- **Сделал cherry-pick и сразу push --force** — Коллеги потеряли историю
- **Попытался cherry-pick конфликтный коммит** — Получил ручное разрешение конфликтов
- **Забыл, в какой ветке находится** — Перенёс изменения не туда

🔗 Шаг 11:

Миссия 11

🤖 В команде хотят автоматизировать cherry-pick. Сопоставьте задачу с инструментом.

Подсказка: Git не единственный игрок — есть CI/CD.

Ответ:

- **Автоматически cherry-pick в релизную ветку** — GitHub Actions workflow
- **Запросить ручное подтверждение перед переносом** — Protected branch rules
- **Избежать конфликтов заранее** — Pre-merge tests
- **Легко выбрать коммит визуально** — Git GUI

🔗 Шаг 12:

Миссия 12

Расставьте шаги cherry-pick в правильном порядке:

Подсказка: Сначала подготовка, потом перенос.

Ответ:

- Переключиться в нужную ветку
- Выполнить git cherry-pick
- Разрешить возможные конфликты
- Сделать push в удалённый репозиторий

🔗 Шаг 13:

Миссия 13

📋 План действий при неудачном cherry-pick:

Подсказка: Не паникуй — есть аварийный выход.

Ответ:

- Оценить конфликт
- Попробовать решить вручную
- **Если не получилось** — abort
- Попробовать другой коммит

🔗 Шаг 14:

Миссия 14

🔧 DevOps переносит hotfix в продакшен. Выберите порядок:

Подсказка: Сначала тесты, потом прод.

Ответ:

- Сделать cherry-pick в staging
- Запустить тесты
- Сделать cherry-pick в production
- Сообщить команде о деплое

Шаг 15:

Миссия 15

🕒 Безопасник Лёша переносит патч безопасности. Выберите порядок:

Подсказка: Документация — тоже важно!

Ответ:

- Найти commit с фиксом
- Сделать cherry-pick
- Проверить работоспособность системы
- Записать изменения в журнал

Шаг 16:

Миссия 16

🔗 Перенос фичи из экспериментальной ветки:

Подсказка: Иногда cherry-pick — лучше merge.

Ответ:

- Оценить нужные коммиты
- Сделать cherry-pick по очереди
- Проверить совместимость
- Обновить документацию

Шаг 17:

Миссия 17

🏁 Финальный квест: автоматизированный cherry-pick в CI/CD

Подсказка: Важно предусмотреть откат.

Ответ:

- Запустить pipeline
- Выполнить cherry-pick
- Проверить статус
- **Если ошибка** — revert автоматически

Урок 2 - Revert откат коммита без потери истории

Шаг 01:

Миссия 1

Твой коллега удалил важный файл, но уже отправил коммит в общий репозиторий. Что делать, чтобы вернуть файл, не переписывая историю?

Подсказка: Вспомни команду, которая создаёт новый коммит, отменяющий изменения предыдущего.

Ответ:

- **git revert**
- git reset --hard
- rm -rf .git
- git undo

 Шаг 02:

Миссия 2

Ты случайно закоммитил пароль в репозиторий. Как безопасно удалить его из последнего коммита, сохранив историю?

Подсказка: Не нужно ломать историю ради одной оплошности — можно «откатить» только изменение.

Ответ:

- **git revert HEAD**
- git push --force
- git erase secret
- git delete-history

 Шаг 03:

Миссия 3

Катя добавила в проект 200 строк дебажного вывода. Все лог-файлы теперь красным горят в ревью. Что ей сделать?

Подсказка: Она не хочет терять авторство, но хочет отменить изменения.

Ответ:

- **Сделать git revert на коммит с логами**
- Переписать историю через rebase
- Удалить репозиторий и начать заново
- Скрыть файлы в .gitignore

 Шаг 04:

Миссия 4

Как Git ведёт себя при выполнении revert?

Подсказка: Историю он не стирает, но... делает кое-что ещё.

Ответ:

- **Создаёт новый коммит с обратными изменениями**
- Удаляет старый коммит из истории
- Перезаписывает историю с нуля
- Останавливает все процессы и просит подумать

 Шаг 05:

Миссия 5

Чем отличается git revert от git reset --hard?

Подсказка: Один действует как "откат с сохранением", другой — как "машина времени".

Ответ:

- **revert создаёт новый коммит, reset удаляет историю**
- reset безопаснее revert
- revert работает только в Windows
- **reset можно отменить revert** — нет

🔮 Шаг 06:

Миссия 6

Можно ли сделать revert для нескольких коммитов подряд?

Подсказка: Иногда нужно откатить целую серию экспериментов.

Ответ:

- **Да, перечислив коммиты один за другим**
- Нет, revert работает только один раз
- Только через reset --hard
- Только если включён режим "pro mode"

🔮 Шаг 07:

Миссия 7

Паша изучает команды, связанные с revert. Помоги сопоставить их с назначением:

Подсказка: Обрати внимание на режимы и параметры.

Ответ:

- **git revert --no-commit** — Создать изменения, но не добавлять коммит автоматически.
- **git revert --edit** — Открыть редактор для сообщения коммита.
- **git revert --quit** — Прервать процесс revert при конфликте.
- **git revert --continue** — Завершить revert после исправления конфликтов.

🔮 Шаг 08:

Миссия 8

Соотнеси ситуацию и правильное действие:

Подсказка: Думай как тимлид — береги историю.

Ответ:

- **Нужно отменить один конкретный коммит в середине истории** — git revert
- **Нужно полностью переписать историю локально** — git reset --hard
- **Нужно отменить последний коммит, но сохранить изменения в рабочей директории** — git reset --soft HEAD~1
- **Нужно отменить изменения, но оставить след для ревьюеров** — git revert HEAD

🔮 Шаг 09:

Миссия 9

Сопоставь действие и возможный риск:

Подсказка: Revert безопасен, reset — не всегда.

Ответ:

- **git revert** — Может вызвать конфликты, если изменения перекрываются.
- **git reset --hard** — Удаляет локальные изменения без возможности восстановления.

- **git push --force** — Может нарушить историю у других разработчиков.
- **git revert --no-edit** — Создаёт коммит с дефолтным сообщением.

🔗 Шаг 10:

Миссия 10

Какие команды используются при решении конфликтов во время revert?

Подсказка: Как будто mini-rebase.

Ответ:

- **git revert --continue** — Продолжить после решения конфликтов.
- **git revert --abort** — Отменить revert и вернуть всё как было.
- **git status** — Проверить состояние файлов после конфликта.
- **git add** — Пометить файл как решённый.

🔗 Шаг 11:

Миссия 11

Связка сценарий — решение:

Подсказка: Думай прагматично.

Ответ:

- **Откатить несколько коммитов подряд** — `git revert A B C`
- **Откатить коммит в feature-ветке без пуша** — `git revert && git log`
- **Откатить merge-коммит** — `git revert -m 1`
- **Откатить коммит и сразу запустить** — `git revert && git push`

🔗 Шаг 12:

Миссия 12

Какие инструменты помогают при revert в GUI-клиентах Git?

Подсказка: Представь себя в SourceTree или GitKraken.

Ответ:

- **История коммитов** — Позволяет выбрать нужный hash.
- **Контекстное меню "Revert"** — Создаёт обратный коммит автоматически.
- **Встроенный diff** — Показывает, что изменится после revert.
- **Pull/Push панель** — Помогает сразу отправить результат в удалённый репозиторий.

🔗 Шаг 13:

Миссия 13

Расположи шаги выполнения revert с конфликтами:

Подсказка: Это почти как мини-merge.

Ответ:

- Выполнить `git revert`
- Исправить конфликты вручную
- `git add` для пометки файлов
- `git revert --continue`

Шаг 14:

Миссия 14

В каком порядке безопаснее отменять серию ошибочных коммитов?

Подсказка: От старого к новому или наоборот?

Ответ:

- Определить список коммитов
- Применить revert с последнего к первому
- Проверить состояние проекта
- Запустить результат

Шаг 15:

Миссия 15

Как откатить merge-коммит?

Подсказка: Нужно указать сторону родителя.

Ответ:

- Найти hash merge-коммита
- Выполнить `git revert -m 1`
- Разрешить возможные конфликты
- Сделать push

Шаг 16:

Миссия 16

Ты сделал revert по ошибке! Как отменить сам revert?

Подсказка: Иногда приходится откатывать откат.

Ответ:

- Найти hash коммита с revert
- Сделать `git revert` еще раз
- Проверить результат
- Отправить изменения

Шаг 17:

Миссия 17

Команда из пяти человек сломала прод. Как действовать?

Подсказка: Не паникуйте — действуйте по плану.

Ответ:

- Найти проблемный коммит
- Сделать revert
- Проверить на тестовом окружении
- Запустить фикс

Урок 3 - Reset мягкий, смешанный, жёсткий

Шаг 01:

Миссия 1

Маша случайно закоммитила файл с паролями. Какой режим `git reset` лучше использовать, чтобы отменить последний коммит, но сохранить изменения в рабочей директории?

Подсказка: Этот режим просто убирает коммит, но не трогает файлы на диске.

Ответ:

- **`git reset --soft HEAD~1`**
- `git reset --hard HEAD~1`
- `git checkout .`
- `git revert HEAD`

Шаг 02:

Миссия 2

Петя случайно закоммитил временные файлы и хочет убрать их из индекса, но оставить на диске. Какой режим выбрать?

Подсказка: Этот режим затрагивает только индекс (staging area).

Ответ:

- `git reset --soft`
- **`git reset --mixed`**
- `git reset --hard`
- `git clean -fd`

Шаг 03:

Миссия 3

Инженер по безопасности Лена хочет полностью откатить ветку до предыдущего состояния, удалив все изменения в файлах. Какой режим ей нужен?

Подсказка: Этот режим не щадит ни индекс, ни рабочую директорию 🗑

Ответ:

- `git reset --soft HEAD~1`
- `git reset --mixed HEAD~1`
- **`git reset --hard HEAD~1`**
- `git restore .`

Шаг 04:

Миссия 4

Саша сделал три коммита подряд, но понял, что первый был ошибочным. Какой командой можно убрать только самый первый из них?

Подсказка: Можно указать точную ссылку на нужный коммит.

Ответ:

- **`git reset --soft ^`**
- `git reset --hard HEAD`
- `git revert HEAD~2`
- `git reset --mixed HEAD~3`

Шаг 05:

Миссия 5

Аркадий сделал `git reset --soft HEAD~1`, но теперь не знает, где оказались его изменения. Где они?

Подсказка: Этот режим только убирает коммит, но не трогает staging или рабочие файлы.

Ответ:

- Навсегда удалены
- **В индексе (staged)**
- В рабочей директории, но не в индексе
- В stash

🔮 Шаг 06:

Миссия 6

Федя использовал `git reset --mixed`. Что изменилось?

Подсказка: Файлы не удаляются, но staging сбрасывается.

Ответ:

- Удалены все файлы в рабочей директории
- **Индекс очищен, но рабочие файлы остались**
- Создан новый коммит
- Все изменения отправлены на сервер

🔮 Шаг 07:

Миссия 7

Сопоставьте режимы `git reset` с их описанием.

Подсказка: Подумайте, какой из них самый "жесткий".

Ответ:

- **--soft** — Удаляет коммит, но сохраняет индекс и рабочие файлы.
- **--mixed** — Сохраняет рабочие файлы, но очищает индекс.
- **--hard** — Сбрасывает всё: историю, индекс и рабочие файлы.

🔮 Шаг 08:

Миссия 8

Какой режим использует каждый персонаж?

Подсказка: Если человек осторожный — он не трогает файлы!

Ответ:

- **Осторожный Вася** — `--soft`
- **Немного рисковый Игорь** — `--mixed`
- **Бесстрашная Катя** — `--hard`

🔮 Шаг 09:

Миссия 9

Сопоставьте команды с последствиями.

Подсказка: Внимание на то, остаются ли файлы на диске.

Ответ:

- **`git reset --soft HEAD~1`** — Коммит удалён, файлы на месте и в индексе.
- **`git reset --mixed HEAD~1`** — Коммит удалён, файлы только в рабочей директории.

- **git reset --hard HEAD~1** — Удалено всё: и коммит, и изменения.

🔗 Шаг 10:

Миссия 10

Женя изучает HEAD . Сопоставьте ссылки.

Подсказка: Каждая стрелка — это сдвиг назад.

Ответ:

- **HEAD** — Текущий коммит.
- **HEAD~1** — Один коммит назад.
- **HEAD~2** — Два коммита назад.

🔗 Шаг 11:

Миссия 11

Какой режим выбрать для задачи?

Подсказка: Обрати внимание на исход: удалить или оставить.

Ответ:

- **Хочу просто убрать коммит** — --soft
- **Хочу убрать коммит и staging** — --mixed
- **Хочу стереть всё подчистую** — --hard

🔗 Шаг 12:

Миссия 12

Сопоставь команду и её эффект на файлы.

Подсказка: Где изменения остаются — там и безопасность.

Ответ:

- **Сбросить только HEAD** — --soft
- **Сбросить HEAD и индекс** — --mixed
- **Сбросить всё: HEAD, индекс, файлы** — --hard

🔗 Шаг 13:

Миссия 13

Расположи режимы git reset от самого безопасного к самому опасному.

Подсказка: Вспомни, кто ничего не трогает, а кто всё сносит.

Ответ:

- --soft
- --mixed
- --hard

🔗 Шаг 14:

Миссия 14

Ты devops. У тебя panic-mode! Упорядочи шаги восстановления после случайного --hard .

Подсказка: Сначала проверь reflog!

Ответ:

- Проверить историю в reflog
- Найти нужный коммит
- Сделать reset на него

 Шаг 15:

Миссия 15

Расставь команды для подготовки к эксперименту с reset.

Подсказка: Всегда делай копию!

Ответ:

- Создать новую ветку
- Выполнить reset
- Проверить результат

 Шаг 16:

Миссия 16

Последовательность действий после неожиданного конфликта.

Подсказка: Не паникуй — анализируй.

Ответ:

- Сохранить изменения в stash
- Сделать reset
- Вернуть stash обратно

 Шаг 17:

Миссия 17

Ты преподаёшь Git. Как бы ты показал студентам различия reset?

Подсказка: От простого к разрушительному.

Ответ:

- --soft на HEAD~1
- --mixed на HEAD~1
- --hard на HEAD~1

 Урок 4 - Практика восстановление после reset

 Шаг 01:

Миссия 1

Аня случайно сделала git reset --hard HEAD~1 и потеряла последний коммит. Что ей нужно сделать, чтобы попробовать его найти?

Подсказка: История сохраняется глубже, чем кажется. Особенно если заглянуть в специальный журнал.

Ответ:

- **Выполнить git reflog и найти хэш потерянного коммита**
- Снова сделать reset, но на два коммита назад
- Удалить репозиторий и клонировать заново

- **Ничего нельзя сделать** — коммит ушёл навсегда

💡 Шаг 02:

Миссия 2

Борис сделал `git reset --soft HEAD~1` . Что произошло с изменениями из последнего коммита?

Подсказка: Представь, что коммит исчез, но изменения всё ещё лежат у двери и ждут.

Ответ:

- **Они переместились в `staged (index)`**
- Они удалились навсегда
- Они ушли в `stash`
- Они превратились в новый коммит автоматически

💡 Шаг 03:

Миссия 3

Катя сделала `git reset --mixed HEAD~2` . Что стало с файлами?

Подсказка: Представь, что коммиты растворились, но изменения можно потрогать руками.

Ответ:

- **Они остались в рабочей директории, но вышли из `staged`**
- Они отправились в удалённый репозиторий
- Они были заархивированы в `stash` автоматически
- Они остались закоммиченными

💡 Шаг 04:

Миссия 4

Сергей в панике делает `git revert` вместо `reset` . Что он получает?

Подсказка: Этот способ не удаляет прошлое, а добавляет новый поворот в историю.

Ответ:

- **Создаётся новый коммит, отменяющий изменения**
- История полностью переписывается
- Удаляются все ветки
- Git блокирует пользователя на 24 часа

💡 Шаг 05:

Миссия 5

Какой командой лучше всего восстановить коммит после `git reset --hard` , если его хэш найден в `reflog` ?

Подсказка: Если знаешь координаты, можно прыгнуть прямо туда.

Ответ:

- **`git checkout`**
- `git clean -fd`
- `git init --recover`
- `git blame`

🔮 Шаг 06:

Миссия 6

Лена случайно удалила локальную ветку с коммитами. Она помнит только её имя. Что ей поможет?

Подсказка: Следы остаются даже после удаления — если посмотреть в журнал перемещений.

Ответ:

- **git reflog show**
- git stash drop
- git bisect
- git prune

🔮 Шаг 07:

Миссия 7

Свяжи тип reset с его эффектом

Подсказка: Представь три уровня удара: лёгкий, средний и уничтожительный.

Ответ:

- **--soft** — Сохраняет изменения в staged
- **--mixed** — Оставляет изменения в рабочей директории, но убирает из staged
- **--hard** — Удаляет изменения полностью

🔮 Шаг 08:

Миссия 8

Сопоставь команду восстановления с её описанием

Подсказка: Одна команда прыгает в прошлое, другая создаёт новую ветку из него.

Ответ:

- **git checkout** — Переход в состояние конкретного коммита
- **git branch rescue** — Создание новой ветки на основе коммита
- **git reset --hard** — Полный откат рабочей директории к коммиту

🔮 Шаг 09:

Миссия 9

Свяжи команду reflog с её назначением

Подсказка: reflog — это как журнал охранника: записывает все перемещения.

Ответ:

- **git reflog** — Показать весь журнал действий HEAD
- **git reflog show feature** — Показать историю конкретной ветки
- **git reflog expire** — Очистить старые записи журнала

🔮 Шаг 10:

Миссия 10

Сопоставь проблему с корректным действием

Подсказка: У каждого фейла — своё лекарство.

Ответ:

- **Случайный reset --hard** — Восстановление через git reflog
- **Удалённый коммит без push** — Поиск в локальном reflog
- **Reset чужой ветки** — Синхронизация через fetch и restore

 Шаг 11:

Миссия 11

Расставь действия при восстановлении коммита через reflog

Подсказка: Сначала найти, потом прыгнуть, потом сохранить.

Ответ:

- git reflog
- Найти нужный хэш
- git checkout
- git branch rescue

 Шаг 12:

Миссия 12

Восстановление случайно удалённой ветки

Подсказка: Не паникуй — следы остались.

Ответ:

- git reflog show
- Найти последний хэш HEAD
- git branch
- git checkout

 Шаг 13:

Миссия 13

Восстановление после reset --hard, когда изменения нужны обратно

Подсказка: История — не мусорка, она всё помнит.

Ответ:

- git reflog
- Скопировать нужный хэш
- git reset --hard
- Проверить рабочие файлы

 Шаг 14:

Миссия 14

Правильный порядок при soft reset для редактирования коммита

Подсказка: Разбей, исправь, собери снова.

Ответ:

- git reset --soft HEAD~1
- Изменить файлы
- git add .
- git commit --amend

Шаг 15:

Миссия 15

Порядок действий при комбинировании reset и stash

Подсказка: Иногда лучше отложить, а потом вернуться.

Ответ:

- git stash push
- git reset --hard HEAD~1
- git stash pop
- Проверить конфликты

Шаг 16:

Миссия 16

Порядок действий при создании временной ветки для эксперимента после восстановления

Подсказка: Всегда лучше работать не на главной дороге.

Ответ:

- git checkout
- git branch experiment
- git checkout experiment
- Начать изменения

Модуль 7 - Rebase и переписывание истории

Урок 1 - Merge и rebase различия

Шаг 01:

Миссия 1

Аня работает над фичей и хочет влить свои изменения в основную ветку. Что делает git merge по сравнению с git rebase ?

Подсказка: Подумай, одно ли это и то же — создать новый коммит или переписать историю?

Ответ:

- **merge создаёт новый коммит слияния, rebase переписывает историю**
- merge удаляет старые коммиты
- rebase всегда быстрее merge
- merge и rebase полностью одинаковы

Шаг 02:

Миссия 2

Борис заметил, что после git rebase изменились хэши коммитов. Почему так произошло?

Подсказка: Представь, что коммиты как страницы дневника: если ты перепишешь их заново, они будут другими.

Ответ:

- **rebase создаёт новые коммиты на новой базе**
- rebase удаляет автора коммитов
- rebase случайно ломает репозиторий

- хэши меняются из-за merge-конфликтов

🔮 Шаг 03:

Миссия 3

Саша хочет сохранить читаемую историю без лишних «Merge branch...». Что ему выбрать?

Подсказка: Один из инструментов «вычищает» историю, другой — показывает все ответвления.

Ответ:

- **Использовать rebase**
- Всегда использовать merge
- Удалить историю вручную
- Создать новую ветку заново

🔮 Шаг 04:

Миссия 4

В команде Пети принято не изменять общую историю. Какой инструмент безопаснее?

Подсказка: Переписывание чужих коммитов — как редактирование чужого дневника. Не все это любят.

Ответ:

- **git merge**
- git rebase --force
- git reset --hard
- git reinit

🔮 Шаг 05:

Миссия 5

Юля случайно сделала rebase вместо merge на публичной ветке. Что теперь?

Подсказка: Иногда помогает git push --force-with-lease, но это как хирургическая операция — не для всех случаев.

Ответ:

- **Предупредить команду и аккуратно force-pushнуть**
- Сразу удалить репозиторий
- Сделать новый merge поверх rebase
- Закрыть ноутбук и уйти в отпуск

🔮 Шаг 06:

Миссия 6

Стас хочет объединить ветку feature с main, но сохранить точное время всех коммитов. Что выбрать?

Подсказка: Один инструмент сохраняет оригинальный порядок и даты.

Ответ:

- **git merge**
- git rebase
- git squash
- git cherry-pick

🔗 Шаг 07:

Миссия 7

Сопоставь команду и её эффект при rebase.

Подсказка: Представь, что ты редактируешь историю как в текстовом редакторе.

Ответ:

- **git rebase -i** — Интерактивное редактирование коммитов
- **pick** — Оставить коммит как есть
- **squash** — Объединить коммиты
- **reword** — Изменить сообщение коммита

🔗 Шаг 08:

Миссия 8

Петя видит конфликт. Сопоставь действие и правильное решение.

Подсказка: После редактирования файлов не забудь сказать Git'у, что всё готово.

Ответ:

- **Исправить конфликт вручную** — Отредактировать файлы
- **Подтвердить решение** — git add .
- **Продолжить rebase** — git rebase --continue
- **Отменить rebase** — git rebase --abort

🔗 Шаг 09:

Миссия 9

Какая команда что делает при merge?

Подсказка: Иногда Git всё делает за тебя, но иногда нужен ручной merge.

Ответ:

- **git merge feature** — Попытка автоматического слияния
- **Конфликт** — Требуется ручное вмешательство
- **git commit после merge** — Создание merge-коммита
- **git log --graph** — Просмотр дерева слияний

🔗 Шаг 10:

Миссия 10

Сопоставь риск и инструмент.

Подсказка: Не всё, что красиво, безопасно.

Ответ:

- **Изменение публичной истории** — Опасность rebase на общей ветке
- **Засорённый лог** — Слишком много merge-коммитов
- **Нет контекста ветвлений** — Чрезмерное использование rebase
- **Дублирование коммитов** — Неправильный rebase

🔗 Шаг 11:

Миссия 11

Определи, что произойдёт.

Подсказка: Rebase двигает коммиты, как кубики.

Ответ:

- **git rebase main** — Перенос feature на вершину main
- **git merge main** — Слияние main в текущую ветку
- **git rebase --skip** — Пропуск проблемного коммита
- **git merge --no-ff** — Создание merge-коммита даже при fast-forward

Шаг 12:

Миссия 12

Расставь шаги rebase в правильном порядке.

Подсказка: Сначала переключение, потом перенос.

Ответ:

- Переключиться на feature
- Выполнить git rebase main
- Исправить конфликты
- Продолжить rebase

Шаг 13:

Миссия 13

Упорядочи процесс merge.

Подсказка: Не забудь проверить результат!

Ответ:

- Переключиться на main
- git merge feature
- Разрешить конфликты при необходимости
- Запустить тесты

Шаг 14:

Миссия 14

Как отменить неудачный rebase?

Подсказка: Ищи кнопку «Стоп».

Ответ:

- Остановить процесс
- Использовать git rebase --abort
- Вернуться к исходной ветке
- Продолжить работу с чистой историей

Шаг 15:

Миссия 15

Как создать идеально чистую историю коммитов?

Подсказка: Сначала черновик, потом редакция.

Ответ:

- Сделать черновые коммиты
- Использовать git rebase -i

- Объединить лишние коммиты
- Изменить сообщения при необходимости

🔧 Шаг 16:

Миссия 16

Закрепим: что делать после rebase перед пушем?

Подсказка: Историю уже изменили — надо предупредить удалённый репозиторий.

Ответ:

- Проверить историю
- Запустить тесты
- Сделать push --force-with-lease
- Сообщить команде

📖 Урок 2 - Interactive rebase squash и fixup

🔧 Шаг 01:

Миссия 1

Ты только что сделал три коммита подряд с сообщениями «fix», «oops» и «final». Тимлид наблюдает и тяжело вздыхает. Что лучше всего сделать перед пушем?

Подсказка: Можно сделать так, будто твои импульсивные коммиты никогда не существовали.

Ответ:

- **Использовать interactive rebase и объединить коммиты через squash**
- Переименовать ветку, чтобы тимлид не нашёл
- Сделать ещё один коммит с текстом «реально финальный»
- Скрыть ноутбук под стол

🔧 Шаг 02:

Миссия 2

Разработчик Ваня хочет оставить только одно хорошее сообщение коммита, а остальные автоматически слить без редактирования текста. Что ему выбрать?

Подсказка: Один режим объединяет с возможностью редактирования, другой — автоматически.

Ответ:

- **Использовать fixup**
- Использовать squash с ручным редактированием
- Сделать git reset --hard
- Отправить тикет в поддержку Git

🔧 Шаг 03:

Миссия 3

Лена случайно закоммитила в репозиторий файл с паролями. Что безопаснее?

Подсказка: Просто удалить файл новым коммитом недостаточно — он останется в истории.

Ответ:

- **Переписать историю через interactive rebase и удалить файл из старого коммита**
- Сделать rm passwords.txt и закоммитить
- Переименовать файл в passwords_old.txt

- Надеяться, что никто не заметит

🔮 Шаг 04:

Миссия 4

Тебе нужно изменить сообщение третьего коммита назад. Что выбираешь?

Подсказка: Нужен режим, позволяющий редактировать сообщение.

Ответ:

- **reword в interactive rebase**
- fixup без редактирования
- merge --no-ff
- git pull --force

🔮 Шаг 05:

Миссия 5

Ты объединяешь два коммита через squash, но хочешь написать новое сообщение вместо склеенного. Что произойдёт?

Подсказка: Один режим открывает редактор, другой — нет.

Ответ:

- **Редактор откроется, и можно написать новое сообщение**
- Сообщение удалится полностью
- Git сам выберет лучшее сообщение
- Всё сломается и потребуются reset

🔮 Шаг 06:

Миссия 6

Сопоставь действие interactive rebase с его результатом.

Подсказка: Представь, что каждый режим — это кнопка с конкретной функцией.

Ответ:

- **pick** — Оставить коммит без изменений
- **reword** — Изменить сообщение коммита
- **squash** — Объединить с предыдущим, редактируя сообщение
- **fixup** — Объединить с предыдущим без редактирования

🔮 Шаг 07:

Миссия 7

Саша переписывает историю. Сопоставь ситуацию с правильным действием.

Подсказка: Один случай — когда сообщение плохое, другой — когда само содержимое нужно исправить.

Ответ:

- **Не понравилось сообщение, но код норм** — reword
- **Нужно объединить два мелких коммита** — squash
- **Исправил опечатку сразу после большого коммита** — fixup
- **Коммит полностью устраивает** — pick

🔗 Шаг 08:

Миссия 8

В dojo программиста столкнулись два джедая Git. Сопоставь их заявления с термином.

Подсказка: Один хочет порядок, другой — тишины.

Ответ:

- **«Я хочу объединить всё, но лично переписать историю своими словами»** — squash
- **«Я не хочу, чтобы меня вообще замечали в истории»** — fixup
- **«Я говорю, как есть — меня не меняют»** — pick
- **«Я скажу то же самое, только красивее»** — reword

🔗 Шаг 09:

Миссия 9

Ты проводишь код-ревью и видишь странные коммиты. Сопоставь комментарий ревьюера с нужным действием.

Подсказка: Представь, что ревьюер — это голос совести.

Ответ:

- **«Почему этот коммит существует отдельно?»** — squash
- **«Сообщение коммита непонятное»** — reword
- **«Коммит можно просто приклеить к предыдущему»** — fixup
- **«Этот коммит норм, не трогай»** — pick

🔗 Шаг 10:

Миссия 10

Ты наставник. Ученики дают описания, ты — термины.

Подсказка: Один режим объединяет, но даёт слово автору. Другой — действует молча.

Ответ:

- **Объединить и написать новое сообщение** — squash
- **Объединить и не давать писать ничего** — fixup
- **Просто оставить как есть** — pick
- **Изменить фразу, но не код** — reword

🔗 Шаг 11:

Миссия 11

Расставь шаги interactive rebase в правильном порядке.

Подсказка: Сначала вход, потом редактирование, потом сохранение и выход.

Ответ:

- Открыть список коммитов через `git rebase -i HEAD~N`
- Выбрать действия для каждого коммита (pick, squash и т.д.)
- Сохранить и выйти из редактора
- Дождаться выполнения и проверить результат

🔗 Шаг 12:

Миссия 12

Отсортируй команды по степени вмешательства в коммит.

Подсказка: От самого щадящего к самому радикальному.

Ответ:

- pick
- reword
- fixup
- squash

Шаг 13:

Миссия 13

Ты хочешь объединить три коммита в один. Расставь действия.

Подсказка: Сначала цель, потом присоединения.

Ответ:

- pick основной коммит
- squash второй коммит
- fixup третий коммит
- Сохранить и закрыть редактор

Шаг 14:

Миссия 14

Упорядочи команды по тому, открывают ли редактор.

Подсказка: Сначала те, кто молчат.

Ответ:

- fixup
- pick
- reword
- squash

Шаг 15:

Миссия 15

Расставь шаги после конфликта во время rebase.

Подсказка: Сначала исправь, потом добавь, потом продолжай.

Ответ:

- Исправить конфликт вручную
- git add .
- git rebase --continue
- Проверить итог

Шаг 16:

Миссия 16

Тимлид дал тебе задание: переписать историю красиво. Расставь инструменты по уровню автоматизации.

Подсказка: Вручную → полуавтомат → полностью автоматически.

Ответ:

- reword

- squash
- fixup
- pick

🔧 Шаг 17:

Миссия 17

Финальный босс. Представь, что ты делаешь серию коммитов, и хочешь выглядеть идеально. Расставь этапы.

Подсказка: Сначала наделай ошибок, потом наведи порядок, потом гордись.

Ответ:

- Сделать несколько черновых коммитов
- Запустить interactive rebase с squash/fixup
- Переписать сообщения красиво
- Отправить ветку с идеальной историей

📖 Урок 3 - Изменение сообщений коммитов

🔧 Шаг 01:

Миссия 1

Маша уже отправила коммит с опечаткой в сообщение на удалённый репозиторий. Можно ли безопасно исправить сообщение?

Подсказка: Вспомни про команду, которая переписывает историю на сервере.

Ответ:

- Да, можно просто сделать `git commit --amend` и пушнуть без флага
- **Да, но нужно использовать `git push --force-with-lease`**
- Нет, сообщение теперь навсегда в истории
- Достаточно написать исправление в комментарии на GitHub

🔧 Шаг 02:

Миссия 2

QA-инженер Лена хочет изменить сообщение не последнего, а предпоследнего коммита. Что ей нужно использовать?

Подсказка: Здесь одной команды `amend` уже не хватит.

Ответ:

- **`git rebase -i HEAD~2`**
- `git commit --edit=2`
- `git cherry-pick --replace`
- `git reset --hard`

🔧 Шаг 03:

Миссия 3

Тимлид Сева использовал `git rebase -i`, но увидел строки `pick` рядом с коммитами. Что ему нужно заменить на `reword`, чтобы изменить только сообщение?

Подсказка: Выбирается только тот коммит, текст которого он хочет переписать.

Ответ:

- **Только строку с нужным коммитом**
- Все строки в файле
- Первую строку
- Последнюю строку

🔗 Шаг 04:

Миссия 4

Фронтендер Саша практикуется в rebase. Сопоставь команды с их назначением:

Подсказка: Одна из них только меняет сообщение, не трогая содержимое коммита.

Ответ:

- **reword** — Изменить только сообщение коммита.
- **edit** — Остановиться на коммите для изменения содержимого.
- **squash** — Объединить коммиты в один.
- **drop** — Удалить коммит из истории.

🔗 Шаг 05:

Миссия 5

Бэкендер Никита экспериментирует с историей. Соотнеси флаги с их действием:

Подсказка: Один из флагов защищает от перезаписи чужих изменений.

Ответ:

- **--force-with-lease** — Переписать историю, но проверить, что никто не добавил новые коммиты.
- **--force** — Принудительно переписать историю без проверки.
- **--no-edit** — Не открывать редактор при amend.
- **--root** — Применить rebase начиная с самого первого коммита.

🔗 Шаг 06:

Миссия 6

DevOps Вика нашла в истории три коммита с одинаковыми сообщениями. Соедини тип проблемы с решением:

Подсказка: Подумай, что делать с дубликатами.

Ответ:

- **Повторяющиеся коммиты** — Использовать squash во время rebase.
- **Коммиты без описания** — Применить reword.
- **Лишний коммит** — Использовать drop.
- **Неверный автор** — Применить amend с --author.

🔗 Шаг 07:

Миссия 7

Пентестер Гриша любит чистоту в логах. Сопоставь плохие сообщения с улучшенными версиями:

Подсказка: Хорошее сообщение объясняет почему , а не только что .

Ответ:

- **Fix** — Исправляет падение при пустом вводе.
- **Update** — Обновляет зависимости до последней версии.
- **Work** — Реализует обработку ошибок токена.
- **Bug** — Устраняет утечку памяти в кеше.

🔗 **Шаг 08:**

Миссия 8

Фуллстек-разработчик Паша учится писать атомарные коммиты. Соотнеси правило с его смыслом:

Подсказка: Атомарность = одна логическая единица.

Ответ:

- **Один коммит** — одна задача — Не смешивай рефакторинг и новую фичу.
- **Понятный заголовок** — До 50 символов, объясняющих суть.
- **Описание тела** — Поясняет причину изменений.
- **Проверяемость** — После коммита всё должно собираться.

🔗 **Шаг 09:**

Миссия 9

Безопасник Рома просматривает историю. Сопоставь действие с риском:

Подсказка: Переписывание истории может повлиять на других.

Ответ:

- **Использовать --force** — Можно затереть чужие коммиты.
- **Использовать --force-with-lease** — Переписать только если никто не обновлял ветку.
- **Изменять историю в feature-ветке** — Обычно безопасно.
- **Изменять историю main** — Опасно, особенно в командной работе.

🔗 **Шаг 10:**

Миссия 10

У стажёра Жени в истории три коммита с плохими сообщениями. Расставь действия по исправлению в правильном порядке:

Подсказка: Сначала открываем интерактивный rebase.

Ответ:

- Открыть rebase: `git rebase -i HEAD~3`
- Заменить pick на reword для нужных коммитов
- Сохранить и изменить сообщения
- Продолжить rebase: `git rebase --continue`

🔗 **Шаг 11:**

Миссия 11

Бэкендер Толя хочет объединить два коммита в один с новым сообщением. Упорядочи шаги:

Подсказка: Используй squash.

Ответ:

- `git rebase -i HEAD~2`

- Заменить второй pick на squash
- Сохранить файл
- Ввести новое сообщение коммита

Шаг 12:

Миссия 12

Аналитик Лиза изменила сообщение коммита, но забыла отправить на сервер. Расположи шаги отправки безопасно:

Подсказка: Обычный push не сработает.

Ответ:

- git push --force-with-lease
- git commit --amend
- Проверить git log
- Сохранить новое сообщение

Шаг 13:

Миссия 13

Архитектор Денис нашёл ошибку в авторстве первого коммита. Что он сделает?

Подсказка: Придётся переписать всё с начала.

Ответ:

- git rebase -i --root
- Заменить pick на edit в нужном коммите
- Исправить автора через --author
- Продолжить rebase

Шаг 14:

Миссия 14

Тимлид Катя хочет, чтобы вся команда переписала сообщения коммитов перед релизом. Расставь общие шаги:

Подсказка: Главное — не потерять чужую работу.

Ответ:

- Сделать pull последней версии
- Открыть rebase нужной глубины
- Исправить сообщения через reword
- git push --force-with-lease

Урок 4 - Практика чистая история

Шаг 01:

Миссия 1

Младший разработчик Лена случайно создала 5 коммитов с сообщениями вида "fix". Она хочет объединить их в один аккуратный коммит перед отправкой. Что ей следует использовать?

Подсказка: Это делают прямо в процессе rebase.

Ответ:

- git rebase -i и команда squash

- `git merge --no-ff`
- `git stash pop`
- `git cherry-pick`

🔗 Шаг 02:

Миссия 2

DevOps Пётр хочет переписать историю, но боится потерять данные. Какой совет ему даст опытный коллега?

Подсказка: Перед опасными действиями всегда нужно сделать копию.

Ответ:

- **Создай временную ветку перед rebase**
- Просто надейся на `git reflog`
- Отключи автосохранение в IDE
- Используй `git pull --force`

🔗 Шаг 03:

Миссия 3

Бэкендер Костя запускает interactive rebase и видит команды pick, reword, drop. Какую использовать, чтобы изменить текст сообщения коммита?

Подсказка: "Переписать слова" — почти ответ.

Ответ:

- **reword**
- pick
- squash
- fixup

🔗 Шаг 04:

Миссия 4

Фронтендер Даша сделала rebase и получила конфликт. Она в панике. Как правильно продолжить?

Подсказка: Git не двигается дальше, пока конфликт не решён.

Ответ:

- **Исправить конфликт и выполнить `git rebase --continue`**
- Удалить конфликтный файл
- Сделать `git rebase --abort` и надеяться
- Сделать `git push --force`

🔗 Шаг 05:

Миссия 5

Тимлид Вика сказала: «После rebase не забывайте сделать ... , иначе всех отрубим от репозитория!». Что она имела в виду?

Подсказка: Когда история меняется — нужно обновить remote.

Ответ:

- **`git push --force-with-lease`**
- `git commit --amend`

- `git revert HEAD`
- `git pull --rebase`

🔧 Шаг 06:

Миссия 6

Безопасник Артём нашёл в истории коммит с паролем. Какой самый корректный способ удалить его для всех?

Подсказка: Это больше, чем один `rebase` — потребуется переписать историю целиком.

Ответ:

- **Использовать `git filter-repo` или `BFG Repo-Cleaner`**
- Просто удалить файл и закоммитить
- Сделать `git reset --hard`
- Переименовать ветку

🔧 Шаг 07:

Миссия 7

Сопоставьте команды `interactive rebase` с их назначением:

Подсказка: Подумайте: `pick` — взять, `squash` — сжать.

Ответ:

- **`pick`** — Оставить коммит как есть
- **`squash`** — Объединить коммит с предыдущим с сохранением сообщений
- **`fixup`** — Объединить без сохранения сообщения
- **`drop`** — Удалить коммит из истории

🔧 Шаг 08:

Миссия 8

DevOps команда использует разные стратегии. Сопоставьте сценарий и нужный инструмент:

Подсказка: Если история публичная — осторожнее.

Ответ:

- **Переписать локальные черновики** — `git rebase -i`
- **Переместить ветку поверх `main`** — `git rebase origin/main`
- **Слить историю без переписывания** — `git merge`
- **Удалить секреты из всех коммитов** — `git filter-repo`

🔧 Шаг 09:

Миссия 9

Сопоставьте тип конфликта и способ решения:

Подсказка: Иногда нужно руками, иногда — команда помогает.

Ответ:

- **Текстовый конфликт в коде** — Вручную исправить файл
- **Файл был удалён в одной ветке** — Выбрать оставить или удалить
- **Исполняемый бит изменён** — Настроить `chmod` и продолжить
- **Бинарный конфликт** — Выбрать один из вариантов

🔗 Шаг 10:

Миссия 10

Программист Илья хочет улучшить сообщения коммитов. Сопоставьте стиль и его качество:

Подсказка: Чем конкретнее — тем лучше.

Ответ:

- **"fix"** — Слишком абстрактно
- **"Remove debug logs from auth"** — Хороший уровень детализации
- **"Update"** — Бесполезно
- **"Refactor login validation"** — Ясно, что изменено

🔗 Шаг 11:

Миссия 11

Сопоставьте инструмент и риск его использования:

Подсказка: Force — всегда опасно.

Ответ:

- **git push --force** — Может перезаписать чужие коммиты
- **git rebase -i** — Изменяет историю локально
- **git merge** — Сохраняет все коммиты
- **git reset --hard** — Удаляет несохранённые изменения

🔗 Шаг 12:

Миссия 12

Сопоставьте команду и её описание:

Подсказка: reset и revert — не одно и то же!

Ответ:

- **git reset --soft** — Перемещает HEAD, но сохраняет staged
- **git reset --hard** — Удаляет все изменения в рабочей директории
- **git revert** — Создаёт обратный коммит
- **git reflog** — Позволяет восстановить утерянные коммиты

🔗 Шаг 13:

Миссия 13

Старший разработчик Саша объясняет новичку, как сделать tidy rebase. Расставьте шаги:

Подсказка: Сначала подготовь ветку, потом rebase, потом cleanup.

Ответ:

- Создать резервную ветку
- Запустить git rebase -i HEAD~5
- Сгруппировать коммиты с помощью squash
- Сделать push --force-with-lease

🔗 Шаг 14:

Миссия 14

DevOps-команда чинит историю. Расставьте порядок восстановления после ошибки в rebase:

Подсказка: Вспомни про reflog.

Ответ:

- Открыть git reflog
- Найти нужный хеш
- Выполнить git reset --hard
- Проверить состояние репозитория

 **Шаг 15:**

Миссия 15

Инженер Антон хочет применить изменения из другой ветки вручную. Расположите шаги cherry-pick:

Подсказка: Сначала цель, потом команда.

Ответ:

- Переключиться на нужную ветку
- Найти хеш коммита
- Сделать git cherry-pick
- Разрешить возможные конфликты

 **Шаг 16:**

Миссия 16

Сопоставьте порядок действий при полном очищении истории от секретов:

Подсказка: Сначала инструмент, потом валидация, потом публикация.

Ответ:

- Запустить git filter-repo с нужными правилами
- Проверить, что история очищена
- Сделать force push всех веток
- Уведомить всю команду об изменениях

 **Шаг 17:**

Миссия 17

Финальное испытание! Собери идеальный pull request с чистой историей:

Подсказка: Трижды подумай перед тем как отправлять force push.

Ответ:

- Сделать интерактивный rebase и очистить историю
- Проверить тесты и линтеры
- Пушнуть изменения с --force-with-lease
- Создать pull request с понятным описанием

Модуль 8 - Squash и оптимизация истории

Урок 1 - Объединение нескольких коммитов

 **Шаг 01:**

Миссия 1

Разработчик Коля сделал 5 мелких коммитов подряд с опечатками в сообщениях. Что ему поможет объединить их в один аккуратный коммит?

Подсказка: Эта команда позволяет объединять историю прямо из интерактивного режима.

Ответ:

- **git rebase -i HEAD~5**
- git merge --no-ff
- git reset --soft HEAD~5
- git cherry-pick master

🔧 Шаг 02:

Миссия 2

Алиса в спешке сделала два коммита подряд: сначала исправила баг, потом добавила запятую в README. Что логичнее — объединить эти коммиты или оставить?

Подсказка: Подумай про смысловую единицу изменений.

Ответ:

- **Оставить оба** — любой коммит важен
- **Объединить, если они относятся к одному улучшению**
- Удалить оба и переписать с нуля
- Спрятать коммиты в stash

🔧 Шаг 03:

Миссия 3

Петя сделал 3 коммита, но забыл добавить один файл. Какой флаг при git commit поможет добавить его к предыдущему коммиту?

Подсказка: Эта опция позволяет "дописать" коммит.

Ответ:

- **git commit --amend**
- git commit --append
- git commit --fix
- git commit --continue

🔧 Шаг 04:

Миссия 4

Лена провела git rebase -i, но случайно выбрала "edit" вместо "squash". Что произойдёт?

Подсказка: Эта опция делает паузу в процессе.

Ответ:

- **Rebase остановится, ожидая изменений вручную**
- Все коммиты удалятся
- Создастся merge-коммит
- Git автоматически продолжит процесс

🔧 Шаг 05:

Миссия 5

Сережа сделал squash, но в результате получилось слишком длинное сообщение коммита. Что стоит сделать?

Подсказка: Никто не любит полотна из 20 строк в истории.

Ответ:

- **Оставить только главное и удалить лишнее**
- Вставить все commit-hash по очереди
- Добавить ссылки на мемы
- **Ничего не менять** — так исторично

🔮 Шаг 06:

Миссия 6

Юра случайно сделал squash уже запущенных коммитов. Что произойдёт при git push?

Подсказка: История изменилась локально.

Ответ:

- **Git попросит использовать --force**
- Репозиторий взорвётся
- Push просто отменится
- Git создаст merge-коммит

🔮 Шаг 07:

Миссия 7

Сопоставьте команды с их назначением при squash.

Подсказка: Выбирай по логике — что звучит как "слияние", а что как "продолжение".

Ответ:

- **pick** — Использовать коммит в текущем виде
- **squash** — Объединить с предыдущим коммитом
- **fixup** — Присоединить без сообщения
- **reword** — Изменить сообщение коммита

🔮 Шаг 08:

Миссия 8

Сопоставьте ситуации с правильным действием.

Подсказка: Если хочешь тишины — используй fixup.

Ответ:

- **Исправил опечатку** — fixup
- **Хочу переписать сообщение** — reword
- **Оставить как есть** — pick
- **Объединить и оставить сообщения** — squash

🔮 Шаг 09:

Миссия 9

Сопоставьте стратегию с последствиями для истории.

Подсказка: Одни команды "стирают следы", другие — оставляют.

Ответ:

- **reset --hard** — Полностью перезаписать историю
- **rebase** — Перестроить коммиты по-новому

- **merge** — Объединить, сохранив обе ветки
- **cherry-pick** — Вытащить отдельный коммит

🔧 Шаг 10:

Миссия 10

Сопоставьте команды с их рисками.

Подсказка: Подумай, что может испортить историю коллегам.

Ответ:

- **push --force** — Может затереть чужую работу
- **rebase public** — Может нарушить общую историю
- **fixup** — Теряется сообщение коммита
- **merge** — Оставляет лишние коммиты

🔧 Шаг 11:

Миссия 11

Сопоставьте формат сообщения коммита с его назначением.

Подсказка: Представь, что ты читаешь историю через год.

Ответ:

- **feat: add login** — Новая функциональность
- **fix: auth bug** — Исправление ошибки
- **docs: update README** — Изменения документации
- **refactor: cleanup** — Переписывание без изменения логики

🔧 Шаг 12:

Миссия 12

Расположите шаги squash через rebase -i:

Подсказка: Сначала подготовимся, потом редактируем.

Ответ:

- Открыть интерактивное окно rebase
- Изменить pick на squash
- Сохранить и закрыть редактор
- Обновить сообщение коммита

🔧 Шаг 13:

Миссия 13

Расположите действия при ошибке после squash:

Подсказка: Git всегда подсказывает, как продолжить.

Ответ:

- Исправить файлы
- `git add .`
- `git rebase --continue`
- Проверить результат

Шаг 14:

Миссия 14

Расположите этапы подготовки к push --force:

Подсказка: Никогда не форсируй, не предупредив.

Ответ:

- Созвониться с командой
- Проверить локальную историю
- Сделать backup-branch
- Выполнить push --force

Шаг 15:

Миссия 15

Расположите шаги безопасного squash в feature-ветке:

Подсказка: Не забудь про pull перед началом!

Ответ:

- git pull --rebase
- git rebase -i HEAD~N
- Изменить pick на squash
- git push --force-with-lease

Шаг 16:

Миссия 16

Расположите порядок анализа истории перед squash:

Подсказка: Сначала смотри, потом действуй.

Ответ:

- git log --oneline
- Определить лишние коммиты
- Выбрать диапазон для squash
- Открыть rebase -i

Шаг 17:

Миссия 17

Расположите команды для полного переписывания истории ветки:

Подсказка: Этот путь не для слабых...

Ответ:

- git checkout --orphan new-branch
- Добавить файлы и сделать коммит
- git branch -D old-branch
- git branch -m new-name

Урок 2 - Fixup и autosquash

Шаг 01:

Миссия 1

Аня закоммитила опечатку в README и сразу сделала ещё один коммит "Fix typo". Какой тип коммита лучше использовать, чтобы автоматически объединить его при squash?

Подсказка: Вспомни команду, начинающуюся с `git commit --fixup`.

Ответ:

- **git commit --fixup**
- `git commit --amend`
- `git commit --cleanup`
- `git commit --patch`

🔮 Шаг 02:

Миссия 2

Борис написал огромный коммит, а потом понял, что одно изменение надо выделить в отдельный. Что поможет автоматически подсунуть правку к нужному коммиту позже?

Подсказка: Эта команда не меняет историю сразу — только помечает.

Ответ:

- **git commit --fixup**
- `git rebase --merge`
- `git revert`
- `git stash`

🔮 Шаг 03:

Миссия 3

Катя использовала `git commit --fixup`, но никаких изменений в истории не произошло. Что ей нужно сделать, чтобы коммиты объединились?

Подсказка: Здесь участвует ключ `--autosquash`.

Ответ:

- **git rebase -i --autosquash**
- `git merge --autosquash`
- `git rebase --continue`
- `git cherry-pick`

🔮 Шаг 04:

Миссия 4

Игорь случайно сделал `fixup` не к тому коммиту. Как ему отменить этот `fixup`-коммит, не трогая остальные?

Подсказка: Можно просто убрать последний коммит.

Ответ:

- **git reset --soft HEAD~1**
- `git rebase --abort`
- `git commit --drop`
- `git prune`

🔮 Шаг 05:

Миссия 5

После `fixup`- и `autosquash`-операций коммиты исчезли из логов. Стоит ли паниковать?

Подсказка: История меняется, но содержимое остаётся.

Ответ:

- **Нет, они были объединены в другие коммиты**
- Да, всё удалилось навсегда
- Только если использовался `--hard` режим
- Да, `git` автоматически чистит историю

🔧 Шаг 06:

Миссия 6

Саша хочет сделать сразу несколько `fixup`-коммитов к разным частям истории. Что нужно помнить при их применении?

Подсказка: Они применяются не сразу, а позже.

Ответ:

- **Все `fixup` объединяются только при последующем `rebase --autosquash`**
- `Fixup` применяется сразу при коммите
- Только последний `fixup` учитывается
- `Fixup` работает только с `merge`

🔧 Шаг 07:

Миссия 7

Сопоставь команду с её назначением:

Подсказка: Представь, что каждая команда — инструмент в ящике мастера.

Ответ:

- **`git commit --fixup`** — Создаёт коммит, помеченный для объединения с предыдущим
- **`git rebase -i`** — Открывает список коммитов для ручного редактирования
- **`git rebase --autosquash`** — Автоматически переставляет `fixup`-коммиты рядом с целевыми
- **`git reset --hard`** — Полностью откатывает рабочее дерево

🔧 Шаг 08:

Миссия 8

Арсений объясняет другу разницу между типами коммитов. Помоги сопоставить:

Подсказка: Один тип правит прошлое, другой просто дописывает сверху.

Ответ:

- **Обычный `commit`** — Добавляет новый элемент в историю
- **`Fixup commit`** — Помечает изменения для слияния с предыдущим
- **`Amend commit`** — Перезаписывает последний коммит
- **`Revert commit`** — Создаёт противоположное изменение

🔧 Шаг 09:

Миссия 9

Команда безопасников нашла баг. Сопоставь действие с целью:

Подсказка: Один путь — быстрое исправление, другой — чистая история.

Ответ:

- **`git commit --fixup`** — Создать исправление без изменения истории сразу

- **git rebase -i --autosquash** — Объединить исправление с родительским коммитом
- **git push --force** — Перезаписать удалённую историю
- **git reflog** — Найти пропавший коммит

🔮 Шаг 10:

Миссия 10

Подбери описания к сообщениям rebase:

Подсказка: pick — взять, fixup — объединить.

Ответ:

- **pick 1a2b3c** — Оставить коммит как есть
- **fixup 4d5e6f** — Объединить этот коммит без комментария
- **squash 7g8h9i** — Объединить с возможностью редактирования сообщения
- **exec make test** — Запустить команду во время rebase

🔮 Шаг 11:

Миссия 11

У каждого fixup есть цель. Найди пары:

Подсказка: Hash — как адрес дома.

Ответ:

- **git commit --fixup=abc123** — Пометить изменение для коммита abc123
- **git commit --fixup=def456** — Привязать правку к коммиту def456
- **git commit --fixup=xyz789** — Записать исправление для xyz789
- **git commit --fixup=112233** — Добавить правку к коммиту 112233

🔮 Шаг 12:

Миссия 12

Саша хочет аккуратно объединить fixup-коммит. Расставь шаги:

Подсказка: Сначала пометка, потом объединение.

Ответ:

- Сделать git commit --fixup
- Открыть git rebase -i --autosquash
- Просмотреть переставленные строки
- Сохранить и завершить rebase

🔮 Шаг 13:

Миссия 13

Утро DevOps-инженера. Он увидел бага и решил починить. Упорядочи действия:

Подсказка: Чиним сначала, украшаем позже.

Ответ:

- Написать исправление и сделать commit --fixup
- Запустить rebase -i --autosquash
- Проверить, что всё собрано
- Запустить изменения

Шаг 14:

Миссия 14

Марина готовит pull request. Ей нужно сгруппировать мелкие правки. Что сначала?

Подсказка: Fixup до rebase.

Ответ:

- Создать несколько fixup-коммитов
- Запустить rebase с autosquash
- Убедиться в чистоте истории
- Отправить pull request

Шаг 15:

Миссия 15

Тимлид хочет идеальный лог. Восстанови путь:

Подсказка: Вспомни про squash и проверку.

Ответ:

- Сделать commit --fixup к каждому мелкому коммиту
- Выполнить rebase -i --autosquash
- Проверить дерево коммитов
- Отправить финальный push

Шаг 16:

Миссия 16

Хакер Вася нашёл уязвимость и хочет тихо её вставить. Как аккуратно?

Подсказка: Всё по правилам fixup.

Ответ:

- Сделать commit --fixup к коммиту, где баг
- Выполнить rebase --autosquash
- Проверить, что сообщение не изменилось
- Продолжить работу

Шаг 17:

Миссия 17

Ты — гуру Git. Заверши идеальный процесс:

Подсказка: Представь, что каждый шаг — как чистка истории.

Ответ:

- Сделать несколько commit --fixup
- Открыть rebase -i --autosquash
- Просмотреть порядок и сохранить
- Запустить с --force-with-lease

Урок 3 - Практика очистки истории feature ветки

Шаг 01:

Миссия 1

Разработчик Лена поспешила и сделала 10 коммитов подряд с сообщениями вроде "fix", "again", "tmp". Сейчас она хочет подготовить feature-ветку к ревью. Что ей стоит сделать в первую очередь?

Подсказка: Представь, что ты убираешь комнату перед гостями — сначала нужно собрать мусор в один пакет.

Ответ:

- **Использовать squash, чтобы объединить мелкие коммиты**
- Удалить ветку и начать заново
- Сделать ещё один коммит с "final fix"
- Сделать git pull origin main

Шаг 02:

Миссия 2

Тимур сделал интерактивный калькулятор, но каждый раз забывал добавить файлы в коммит и делал git commit --amend. Что произойдёт, если он продолжит пушить такую историю в общий репозиторий?

Подсказка: Представь, что ты подменяешь историю в дневнике, пока учитель читает его вслух.

Ответ:

- **Коллеги получают конфликт истории при git pull**
- Git автоматически исправит всё сам
- Проект станет работать быстрее
- **Ничего** — amend не влияет на пуш

Шаг 03:

Миссия 3

Безопасник Игорь перед слиянием ветки хочет убедиться, что в истории не осталось секретных ключей. Что ему поможет?

Подсказка: Иногда надо не просто спрятать, а полностью стереть запись.

Ответ:

- **git rebase -i с удалением соответствующего коммита**
- git stash apply
- git revert
- git tag

Шаг 04:

Миссия 4

Саша после трёх чашек кофе случайно сделал два одинаковых коммита подряд. Как лучше всего их объединить?

Подсказка: Два одинаковых бутерброда можно сложить вместе.

Ответ:

- **git rebase -i и squash одного в другой**
- git merge --no-ff
- git restore
- git commit --allow-empty

🔧 Шаг 05:

Миссия 5

Фронтендер Костя делает rebase своей ветки на main, но коммиты смешались с чужими. Как этого избежать?

Подсказка: Прежде чем двигать мебель, убедись, что она твоя.

Ответ:

- **Оставить только свои коммиты при интерактивном rebase**
- Сначала сделать `git revert` всех чужих
- Скопировать репозиторий заново
- Выключить интернет

🔧 Шаг 06:

Миссия 6

QA Аня нашла, что один коммит ломает сборку. Как можно временно убрать его, не теряя историю?

Подсказка: Иногда проще накрыть испорченное блюдо крышкой, чем выкинуть.

Ответ:

- **Использовать `git revert`**
- `git reset --hard`
- Удалить ветку
- `git cherry-pick --abort`

🔧 Шаг 07:

Миссия 7

Сопоставь команды Git с их назначением при очистке истории.

Подсказка: Одни команды объединяют, другие отменяют, третьи переписывают.

Ответ:

- **`git squash`** — Объединяет несколько коммитов в один при rebase
- **`git revert`** — Создаёт новый коммит, отменяющий предыдущий
- **`git rebase -i`** — Позволяет вручную переписать историю коммитов
- **`git reset --soft`** — Переносит HEAD, оставляя изменения в индексе

🔧 Шаг 08:

Миссия 8

🧑💻 Разработчики спорят, как лучше чистить историю. Соотнеси их высказывания с инструментом.

Подсказка: Кто-то любит переписывать аккуратно, кто-то — просто прикрывать ошибки.

Ответ:

- **"Я хочу полностью удалить следы неудачного коммита"** — `git rebase -i` с drop
- **"Я не скрываю ошибки, я их документирую"** — `git revert`
- **"Хочу объединить хаос в один аккуратный коммит"** — `squash`
- **"Мне нужно перенести изменения, но не трогать рабочую директорию"** — `git reset --soft`

🔧 Шаг 09:

Миссия 9

🛡️ Безопасник Вова проверяет историю на секреты. Сопоставь проблему и решение.

Подсказка: Не всё лечится revert'ом.

Ответ:

- **В историю попал пароль** — git filter-repo или BFG для удаления из всей истории
- **Случайно закоммитили node_modules** — git rebase -i и drop коммита
- **Ошибка в коммите, но хочется оставить след** — git revert
- **Нужно объединить коммиты "tmp1", "tmp2", "tmp3"** — squash

🔧 Шаг 10:

Миссия 10

⚙️ Саша делал rebase и увидел список операций. Соотнеси флаг с его действием.

Подсказка: pick — взять, drop — выкинуть.

Ответ:

- **pick** — Оставить коммит как есть
- **reword** — Изменить сообщение коммита
- **edit** — Остановиться для правок содержимого
- **fixup** — Слить коммит без сохранения сообщения

🔧 Шаг 11:

Миссия 11

🔧 Укажи, какое действие безопаснее.

Подсказка: Разница между "переписать" и "добавить ещё одну запись".

Ответ:

- **git revert** — Безопасно для общего репозитория
- **git reset --hard** — Опасно при совместной работе
- **git commit --amend** — Перезаписывает последний коммит
- **git rebase** — Меняет историю начиная с выбранной точки

🔧 Шаг 12:

Миссия 12

📋 Упорядочьте шаги интерактивного squash перед слиянием в main.

Подсказка: Сначала обнови базу, потом объединяй, потом пушь.

Ответ:

- git fetch origin main
- git rebase -i origin/main
- Отметить лишние коммиты как squash или fixup
- git push --force-with-lease

🔧 Шаг 13:

Миссия 13

🗑️ Упорядочьте действия при удалении секретов из истории.

Подсказка: Лечить прошлое сложнее настоящего.

Ответ:

- Определить коммиты с секретами
- Использовать BFG или filter-repo
- Переписать историю
- Принудительно запустить изменения

🔮 Шаг 14:

Миссия 14

Разработчик хочет преобразовать историю вида "WIP1 → WIP2 → final" в один понятный коммит. Расставь шаги.

Подсказка: Сначала подготовка, потом уборка.

Ответ:

- git rebase -i HEAD~3
- Отметить WIP1 и WIP2 как squash
- Сохранить сообщение финального коммита
- git push --force-with-lease

🔮 Шаг 15:

Миссия 15

⌘ Вы устраняете конфликт после rebase. Что нужно сделать в правильном порядке?

Подсказка: Почини → добавь → продолжи.

Ответ:

- Исправить конфликт в файлах
- git add исправленные файлы
- git rebase --continue
- Повторить при необходимости

🔮 Шаг 16:

Миссия 16

История слишком тяжёлая из-за больших файлов. Расставь шаги оптимизации.

Подсказка: Найти → вырезать → переписать → запустить.

Ответ:

- Найти большие файлы через git rev-list --objects
- Удалить их через filter-repo
- Переписать историю
- git push --force

🔮 Шаг 17:

Миссия 17

🕒 Соберите финальный сценарий подготовки feature-ветки к ревью.

Подсказка: Обновись → очисти → протестируй → запусти.

Ответ:

- git fetch origin main

- `git rebase -i origin/main` с squash
- Запустить тесты
- `git push --force-with-lease`

Урок 4 - Поддержание линейной истории

Шаг 01:

Миссия 1

Разработчик Паша сделал 10 коммитов подряд с текстами "fix", "again", "final_fix". Тимлид попросил привести историю в порядок с помощью squash. Что Паше нужно сделать?

Подсказка: Вспомни про интерактивный режим, который позволяет объединять коммиты.

Ответ:

- **Использовать `git rebase -i` и объединить коммиты**
- Сделать `git push --force` сразу на мастер
- Удалить `.git` и начать заново
- Просто переименовать последний коммит

Шаг 02:

Миссия 2

Аня сделала squash перед push, но теперь не понимает, почему хеш коммита стал другим. Что произошло?

Подсказка: Подумай, от чего зависит идентификатор коммита.

Ответ:

- **Squash создал новый коммит с новым содержимым**
- Git сломался и сгенерировал случайный ID
- Хеш меняется только если поменялось имя автора
- Это визуальный баг в терминале

Шаг 03:

Миссия 3

Сисадмин Лёша случайно сделал squash уже опубликованных коммитов в общем репозитории. Что ему нужно сделать, чтобы не разозлить всю команду?

Подсказка: Иногда честность и координация важнее технических трюков.

Ответ:

- **Сообщить команде и попросить всех сделать `git pull --rebase`**
- Сделать `git push --force` втихаря и надеяться
- Удалить репозиторий на сервере
- **Игнорировать проблему** — само рассосётся

Шаг 04:

Миссия 4

Фронтендер Маша делает squash, но видит опцию "fixup". Чем fixup отличается от squash?

Подсказка: В одном из них сообщение коммита сохраняется, в другом — нет.

Ответ:

- **fixup объединяет коммит без сохранения сообщения**

- `fixup` удаляет все изменения
- `fixup` создаёт `merge commit`
- `fixup` работает только по пятницам

🔗 Шаг 05:

Миссия 5

Бэкендер Игорь хочет поддерживать линейную историю без `merge`-коммитов. Какую стратегию стоит использовать при `pull`?

Подсказка: Обрати внимание на параметр, который делает историю ровной, как маршрут по линейке.

Ответ:

- **`git pull --rebase`**
- `git pull --merge`
- `git pull --no-ff`
- `git pull --linear=true`

🔗 Шаг 06:

Миссия 6

Разработчик Саша сделал `squash`, но обнаружил, что забыл включить важный файл. Что делать?

Подсказка: Иногда можно поправить историю ещё раз, если она не улетела в удалённый репозиторий.

Ответ:

- **Использовать `git commit --amend` или повторить `rebase -i`**
- Сделать новый коммит `"forgot_file"`
- Откатить всё через `git reset --hard HEAD~100`
- Отправить файл по почте тимлиду

🔗 Шаг 07:

Миссия 7

Сопоставьте команды с их назначением при работе с линейной историей.

Подсказка: Представь, что ты дрессируешь историю коммитов: каждая команда делает её ровнее.

Ответ:

- **`git rebase -i`** — Открывает список коммитов для интерактивного редактирования
- **`git pull --rebase`** — Подтягивает изменения без `merge`-коммита
- **`git commit --amend`** — Изменяет последний коммит
- **`git push --force-with-lease`** — Перезаписывает удалённую историю с проверкой конфликтов

🔗 Шаг 08:

Миссия 8

Сопоставьте тип коммита с его описанием.

Подсказка: Представь, что ты сортируешь письма по назначению.

Ответ:

- **`feature`** — Добавление новой функциональности

- **fix** — Исправление ошибки
- **refactor** — Изменение кода без изменения поведения
- **docs** — Изменение документации

🔗 Шаг 09:

Миссия 9

Сопоставьте проблемы с историей Git с их последствиями.

Подсказка: Вспомни, что будет, если все будут коммитить "вразнобой".

Ответ:

- **Много мелких коммитов "WIP"** — История становится шумной и нечитаемой
- **Отсутствие squash** — Сложно понять цель изменений
- **Частые merge-коммиты** — История превращается в лабиринт
- **Отсутствие описательных сообщений** — Невозможно понять, что произошло

🔗 Шаг 10:

Миссия 10

Сопоставьте действия с их последствиями при работе в команде.

Подсказка: Представь, что Git — это совместная кухня.

Ответ:

- **git push --force без предупреждения** — Коллеги получают конфликты и слёзы
- **Чистая история через squash** — Ревьюеры счастливы
- **Подписанные коммиты** — Повышенная безопасность
- **Оставление "мусорных" веток** — Захламление репозитория

🔗 Шаг 11:

Миссия 11

Сопоставьте команды rebase-редактора с их значением.

Подсказка: Эти слова появляются в интерактивном режиме.

Ответ:

- **pick** — Оставить коммит как есть
- **reword** — Изменить сообщение коммита
- **drop** — Удалить коммит
- **exec** — Выполнить команду после применения коммита

🔗 Шаг 12:

Миссия 12

Сопоставьте причины использования squash.

Подсказка: Вспомни, чем хорош один аккуратный коммит вместо десяти хаотичных.

Ответ:

- **Упростить ревью** — Ревьюер видит одно логичное изменение
- **Скрыть мусорные изменения** — В историю попадает только финальный результат
- **Поддержать линейность** — История выглядит как прямая линия
- **Снизить количество конфликтов** — Меньше мест для пересечений

🔗 Шаг 13:

Миссия 13

Расставь шаги интерактивного rebase для squash.

Подсказка: Вспомни: сначала запускаем процесс, потом редактируем, затем сохраняем.

Ответ:

- Запустить `git rebase -i HEAD~3`
- Поменять pick на squash
- Сохранить и выйти из редактора
- Push с флагом `--force-with-lease`

🔗 Шаг 14:

Миссия 14

Упорядочи действия при исправлении последнего коммита.

Подсказка: Не забудь проверить изменения перед публикацией.

Ответ:

- Изменить файл
- `git add .`
- `git commit --amend`
- `git push --force-with-lease`

🔗 Шаг 15:

Миссия 15

Отсортируй шаги для поддержания линейной истории при pull.

Подсказка: Вспомни про rebase вместо merge.

Ответ:

- `git fetch`
- `git rebase origin/main`
- Разрешить конфликты (если есть)
- Продолжить rebase

🔗 Шаг 16:

Миссия 16

Расположи шаги revert на опубликованной истории.

Подсказка: Помни: нельзя переписывать историю, которую уже увидели другие.

Ответ:

- Найти хеш ненужного коммита
- `git revert`
- Проверить результат
- `git push`

🔗 Шаг 17:

Миссия 17

Расставь шаги подготовки ветки перед merge в основной репозиторий.

Подсказка: Ветка должна быть аккуратной, как комната перед приходом гостей.

Ответ:

- Сделать `git pull --rebase origin/main`
- Провести squash локальных коммитов
- Прогнать тесты
- Создать merge request

Модуль 9 - Восстановление истории

Урок 1 - Reflog и его возможности

Шаг 01:

Миссия 1

Алиса случайно удалила нужный коммит. Какой командой она может посмотреть все недавние перемещения HEAD, включая те, что не видны в обычной истории?

Подсказка: Эта команда показывает даже те шаги, которые не попали в `git log`.

Ответ:

- **git reflog**
- `git history`
- `git branch -av`
- `git show-all`

Шаг 02:

Миссия 2

Боб сделал `git reset --hard` и потерял изменения. Что он может сделать, чтобы вернуть состояние до `reset`?

Подсказка: В reflog хранится ссылка на состояние перед `reset`.

Ответ:

- **Найти нужный hash в git reflog и сделать `git reset --hard`**
- Выполнить `git revert reset`
- Удалить репозиторий и клонировать заново
- Сделать `git merge` потерянной ветки

Шаг 03:

Миссия 3

Марина переключилась на неправильную ветку и сделала несколько коммитов не туда. Что она может сделать, чтобы переместить эти коммиты на другую ветку?

Подсказка: История переключений тоже сохраняется.

Ответ:

- **Создать новую ветку от нужного состояния из reflog**
- Удалить коммиты вручную в `.git` папке
- Выполнить `git rewind`
- Использовать `git pull --rescue`

🔧 Шаг 04:

Миссия 4

Денис пытается понять, почему у него появился лишний merge-коммит. Где он может увидеть факт переключения веток и автоматического merge?

Подсказка: Эта информация не всегда видна в обычном git log.

Ответ:

- **Посмотреть git reflog**
- Открыть .git/config
- Использовать git blame
- Посмотреть git stash list

🔧 Шаг 05:

Миссия 5

Катя выполнила git commit --amend, но теперь хочет вернуть прежнюю версию коммита. Что ей поможет?

Подсказка: Amend тоже записывается в журнал.

Ответ:

- **Посмотреть прежний hash через reflog и сделать reset**
- Выполнить git amend --undo
- Переименовать коммит обратно
- Сделать git undo last

🔧 Шаг 06:

Миссия 6

Егор хочет посмотреть reflog только для конкретной ветки. Как это сделать?

Подсказка: Можно указать имя ветки.

Ответ:

- **git reflog feature-branch**
- git reflog --only=feature-branch
- git reflog branch=feature-branch
- git branch reflog feature-branch

🔧 Шаг 07:

Миссия 7

Сопоставьте команду с её эффектом при работе с reflog.

Подсказка: Подумайте, что влияет на чистку записей.

Ответ:

- **git reflog expire** — Удаляет устаревшие записи по времени
- **git reflog delete** — Удаляет конкретную запись вручную
- **git gc** — Собирает мусор и очищает reflog
- **git show HEAD@{1}** — Показывает состояние HEAD в прошлом

🔧 Шаг 08:

Миссия 8

Свяжите запись в reflog с её значением.

Подсказка: HEAD@{n} — это позиция в истории.

Ответ:

- **HEAD@{0}** — Текущее состояние HEAD
- **HEAD@{2}** — Состояние два шага назад
- **HEAD@{5}** — Состояние пять переключений назад
- **HEAD@{1}** — Состояние перед последним действием

🔮 Шаг 09:

Миссия 9

Соотнесите ситуацию с решением на основе reflog.

Подсказка: Вспомни, что reflog — твоя машина времени.

Ответ:

- **Случайно удалена ветка** — Найти её последний hash в reflog и создать заново
- **Сломал проект после reset** — Вернуться на предыдущий hash через reset
- **Не помню, когда переключился на ветку** — Посмотреть reflog на переключения
- **Хочу увидеть историю amend** — Проследить прошлые HEAD записи

🔮 Шаг 10:

Миссия 10

Сопоставьте тип записи в reflog с её примером.

Подсказка: Запись всегда содержит действие.

Ответ:

- **commit** — commit: added login feature
- **checkout** — checkout: moving from main to dev
- **reset** — reset: moving to abc123
- **merge** — merge dev: Fast-forward

🔮 Шаг 11:

Миссия 11

Определи, кто что сделал с помощью reflog.

Подсказка: Смотри на действия.

Ответ:

- **Антон** — commit: fixed bug
- **Вика** — checkout: moving from test to main
- **Игорь** — reset: moving to previous commit
- **Света** — merge feature/login

🔮 Шаг 12:

Миссия 12

Сопоставьте команду с её назначением при восстановлении через reflog.

Подсказка: Думай как спасатель коммитов.

Ответ:

- **git branch rescue** — Создание новой ветки из старого состояния

- **git reset --hard** — Возврат к точному состоянию
- **git checkout** — Просмотр старого состояния без изменения ветки
- **git reflog** — Поиск нужного hash

🔗 Шаг 13:

Миссия 13

Расположи шаги восстановления коммита через reflog.

Подсказка: Сначала ищем, потом действуем.

Ответ:

- Открыть git reflog
- Найти нужный hash состояния
- Выполнить git reset --hard
- Проверить проект

🔗 Шаг 14:

Миссия 14

Упорядочи действия при переносе коммитов в другую ветку.

Подсказка: Не забудь создать ветку перед reset.

Ответ:

- Открыть git reflog и скопировать hash нужного состояния
- Создать новую ветку от hash
- Переключиться на неё
- Продолжить работу

🔗 Шаг 15:

Миссия 15

Расставь приоритеты при очистке reflog.

Подсказка: Не всё нужно удалять сразу.

Ответ:

- Проверить важные записи
- Удалить только старые через git reflog expire
- Выполнить git gc при необходимости
- Проверить, не исчезло ли нужное

🔗 Шаг 16:

Миссия 16

Выстрой процесс расследования «кто сломал билд» через reflog.

Подсказка: Следы всегда остаются.

Ответ:

- Открыть git reflog
- Найти подозрительный reset или merge
- Показать состояние через git show
- Связаться с автором

🔮 Шаг 17:

Миссия 17

Восстанови приключения HEAD по шагам.

Подсказка: Вспомни, что HEAD путешественник.

Ответ:

- checkout feature/login
- commit new feature
- reset --hard HEAD~1
- checkout main

📖 Урок 2 - Восстановление удалённых коммитов

🔮 Шаг 01:

Миссия 1

Твой коллега Стас случайно удалил последний коммит командой `git reset --hard HEAD~1`. Что ему стоит сделать в первую очередь, чтобы попытаться его вернуть?

Подсказка: В Git почти ничего не исчезает безвозвратно. Вспомни про журнал ссылок.

Ответ:

- **Открыть reflog командой `git reflog`**
- Создать новый репозиторий
- Закоммитить ещё раз те же файлы
- Позвонить в поддержку GitHub

🔮 Шаг 02:

Миссия 2

Разработчица Аня испугалась и сразу сделала `git reset --hard`, забыв записать SHA нужного коммита. Где она может его найти?

Подсказка: Есть команда, которая показывает ВСЕ перемещения указателя HEAD.

Ответ:

- `git log --oneline`
- **`git reflog`**
- `git stash list`
- `git fsck --lost-found`

🔮 Шаг 03:

Миссия 3

Тимлид Дима хочет вернуть удалённый коммит из reflog. Какой командой он может создать новую ветку с этого коммита?

Подсказка: Ветка — это всего лишь указатель на нужный SHA.

Ответ:

- **`git branch rescue`**
- `git new-branch`
- `git undo`
- `git save-branch`

🔧 Шаг 04:

Миссия 4

Младший разработчик Паша нашёл нужный коммит в `git reflog`, но боится, что он пропадёт. Что лучше сделать?

Подсказка: Временные ссылки живут недолго. Лучше превратить их в постоянные.

Ответ:

- **Создать ветку или тег на этот SHA**
- Скопировать SHA в текстовый файл и закрыть репозиторий
- Сделать push в origin master
- Удалить репозиторий и клонировать заново

🔧 Шаг 05:

Миссия 5

Сисадмин Катя случайно сделала `git reset --hard origin/main` и потеряла локальные коммиты.

Какой командой ей проще всего их восстановить, если она помнит ключевое слово из сообщения коммита?

Подсказка: В Git можно искать по сообщениям коммитов.

Ответ:

- `git find "слово"`
- **`git log --grep="слово"`**
- `git search "слово"`
- `git lookup "слово"`

🔧 Шаг 06:

Миссия 6

Фронтендер Олег случайно потерял коммит, но он до этого успел отправить его в удалённую ветку `backup`. Как проще всего вернуть его локально?

Подсказка: Если что-то есть в удалённом репозитории — его можно стянуть обратно.

Ответ:

- **`git fetch && git checkout backup`**
- `git pull --recover`
- `git stash pop backup`
- `git restore backup`

🔧 Шаг 07:

Миссия 7

Соедини команды с их назначением. Представь, что ты DevOps, который восстанавливает историю после ночного сбоя.

Подсказка: Каждая команда — инструмент спасения, но у каждого свой стиль.

Ответ:

- **`git reflog`** — Показать все перемещения HEAD.
- **`git fsck --lost-found`** — Найти потерянные объекты Git.
- **`git branch revive`** — Создать ветку на удалённый SHA.
- **`git cherry-pick`** — Применить изменения коммита поверх текущей ветки.

🔗 Шаг 08:

Миссия 8

Fullstack-разработчик Артём сравнивает разные способы восстановления коммитов. Соотнеси метод с кратким описанием.

Подсказка: Не все методы одинаково сохраняют историю.

Ответ:

- **Reset** — Перемещает указатель и может потерять изменения.
- **Revert** — Создаёт новый коммит, отменяющий старый.
- **Cherry-pick** — Копирует конкретный коммит в текущую ветку.
- **Reflog** — Показывает историю всех перемещений.

🔗 Шаг 09:

Миссия 9

Тебе нужно помочь коллеге выбрать стратегию восстановления. Соотнеси ситуацию и решение.

Подсказка: Иногда проще закоммитить заново, иногда — вернуть старое.

Ответ:

- **Удалённый коммит есть в origin** — Выполнить fetch и checkout.
- **Коммит потерян после reset** — Посмотреть в reflog.
- **Нужно отменить чужой коммит в main** — Сделать revert.
- **Хочется применить один изменённый файл** — Использовать cherry-pick.

🔗 Шаг 10:

Миссия 10

Команды Git любят путать новичков. Помоги стажёру понять, что что делает.

Подсказка: reset и revert — как Ctrl+Z и новая версия документа.

Ответ:

- **git reset --hard** — Полностью откатывает рабочее состояние.
- **git revert** — Создаёт обратный коммит.
- **git commit --amend** — Изменяет последний коммит.
- **git restore .** — Отменяет изменения в файлах без коммита.

🔗 Шаг 11:

Миссия 11

Сопоставь тип ошибки с решением, будто ты ментор на собеседовании.

Подсказка: Забыл SHA? Не беда. Git помнит.

Ответ:

- **Случайный reset** — Использовать reflog.
- **Испорчен последний коммит** — Применить amend.
- **Удалённый merge оказался неправильным** — Выполнить revert.
- **Нужен только один файл из старого коммита** — Воспользоваться checkout -- file.

🔗 Шаг 12:

Миссия 12

Расставь шаги восстановления удалённого коммита через reflog.

Подсказка: Сначала ищем, потом возвращаем.

Ответ:

- Открыть reflog
- Найти нужный SHA
- Создать ветку на SHA
- Перейти в новую ветку

🔮 Шаг 13:

Миссия 13

Расставь шаги восстановления удалённого коммита через reflog.

Подсказка: Сначала ищем, потом возвращаем.

Ответ:

- Открыть reflog
- Найти нужный SHA
- Создать ветку на SHA
- Перейти в новую ветку

🔮 Шаг 14:

Миссия 14

Ты работаешь ночным сисадмином. Как восстановить случайно удалённый merge-коммит?

Подсказка: Не всё нужно переназначать — иногда можно просто повторить.

Ответ:

- Посмотреть merge-коммит в reflog
- Найти SHA merge-коммита
- Выполнить `git cherry-pick -m 1`
- Проверить историю

🔮 Шаг 15:

Миссия 15

Тебе нужно восстановить коммит, который был в удалённой ветке. Как действовать?

Подсказка: Иногда всё начинается с fetch.

Ответ:

- Сделать `git fetch`
- Посмотреть ветку в `remotes/`
- Создать локальную ветку от неё
- Переключиться на неё

🔮 Шаг 16:

Миссия 16

Тебе нужно отменить *последние два* коммита, но сохранить их в истории для отката. Какой порядок действий?

Подсказка: `reset` уничтожит, `revert` — сохранит.

Ответ:

- Выполнить `git revert HEAD~1`

- Выполнить `git revert HEAD~2`
- Убедиться в истории
- Сделать `push`

Шаг 17:

Миссия 17

Восстановление файла из старого коммита. Расставь шаги правильно.

Подсказка: Не обязательно возвращать весь коммит — можно взять один файл.

Ответ:

- Открыть `git log` и взять SHA
- Выполнить `git checkout -- file.txt`
- Проверить, что файл вернулся
- Закоммитить изменения

Шаг 18:

Миссия 18

Представь: ты — безопасник, который должен восстановить удалённую ветку, чтобы найти подозрительный код. Какой порядок действий?

Подсказка: Ветка — это всего лишь ссылка на SHA. Найди её и создай заново.

Ответ:

- Открыть `reflog`
- Найти SHA старой ветки
- Создать ветку `git branch audit`
- Сделать `checkout audit`

Урок 3 - Reset и checkout для отката

Шаг 01:

Миссия 1

Сисадмин Петя случайно закоммитил пароль от прод-сервера в репозиторий. Какой командой он может быстро откатить последний коммит, сохранив изменения в рабочей директории?

Подсказка: Петя не хочет потерять файлы — только отменить фиксацию.

Ответ:

- `git reset --soft HEAD~1`
- `git checkout -- .`
- `git reset --hard HEAD~1`
- `git revert HEAD`

Шаг 02:

Миссия 2

Разработчица Оля случайно удалила файл и хочет восстановить его из последнего коммита, не трогая другие изменения. Что ей поможет?

Подсказка: Ей нужно вернуть только один файл.

Ответ:

- `git reset --hard`

- **git checkout HEAD -- filename**
- git merge HEAD
- git revert filename

🔗 Шаг 03:

Миссия 3

Backend-разработчик Саша хочет вернуться к состоянию репозитория двух коммитов назад, полностью потеряв текущие изменения. Какой способ самый радикальный?

Подсказка: Саша не боится — он сделал бэкап на флешку.

Ответ:

- **git reset --hard HEAD~2**
- git checkout HEAD~2
- git revert HEAD~2
- git stash apply

🔗 Шаг 04:

Миссия 4

DevOps-инженер Игорь сломал конфиг и хочет посмотреть, как он выглядел в прошлом коммите, но не менять текущие файлы. Что использовать?

Подсказка: Нужно только заглянуть, а не переписать.

Ответ:

- git reset --mixed
- git clean -fd
- **git checkout -- path/to/file**
- git revert

🔗 Шаг 05:

Миссия 5

Фуллстек Костя случайно сделал коммит не в ту ветку. Он хочет перенести изменения в правильную. Что сделать сначала?

Подсказка: Сначала — отделить коммит, потом — забрать его в нужное место.

Ответ:

- **git reset HEAD~1**
- git revert HEAD~1
- git stash drop
- git merge HEAD~1

🔗 Шаг 06:

Миссия 6

Сопоставьте команду и её эффект при откате:

Подсказка: Одни команды сохраняют файлы, другие стирают беспощадно.

Ответ:

- **git reset --soft** — Откат коммита, но файлы остаются в индексе
- **git reset --hard** — Полный откат коммита и удаление изменений из рабочей директории
- **git checkout** — Перемещение в состояние коммита в режиме просмотра

- **git revert** — Создание обратного коммита, отменяющего изменения

🔮 Шаг 07:

Миссия 7

Алиса экспериментировала с конфигами. Подберите действие к её цели.

Подсказка: Она не знает, разрушать или только посмотреть.

Ответ:

- **Посмотреть старую версию файла** — `git checkout HEAD~1 -- config.yaml`
- **Откатить коммит, но оставить изменения** — `git reset --soft HEAD~1`
- **Полностью вернуть всё назад** — `git reset --hard HEAD~1`
- **Создать новый коммит-откат** — `git revert HEAD`

🔮 Шаг 08:

Миссия 8

Что делает каждая часть команды? `git reset --mixed HEAD~2`

Подсказка: Состоит из трёх элементов.

Ответ:

- **reset** — Перемещает указатель ветки
- **--mixed** — Сбрасывает индекс, но сохраняет файлы
- **HEAD~2** — Ссылка на состояние два коммита назад

🔮 Шаг 09:

Миссия 9

Фронтендер Миша любит порядок. Сопоставьте проблему с подходящим откатом.

Подсказка: Важно не перепутать мягкое и жёсткое удаление.

Ответ:

- **Коммит случайно в не ту ветку** — `git reset HEAD~1`
- **Файлы полностью сломали всё** — `git reset --hard HEAD~1`
- **Нужно отменить, но оставить след в истории** — `git revert HEAD`
- **Хочу посмотреть, как было раньше** — `git checkout HEAD~1`

🔮 Шаг 10:

Миссия 10

У Яны в проекте хаос. Подберите команду к описанию.

Подсказка: Разные ситуации — разные уровни паники.

Ответ:

- **Паника уровня 1: хочу посмотреть старую версию** — `git checkout -- file`
- **Паника уровня 2: отменить последний коммит, но оставить изменения** — `git reset --soft HEAD~1`
- **Паника уровня 3: всё сломано, стираем всё** — `git reset --hard HEAD~1`
- **Паника уровня 4: нужно отменить, но оставить историю чистой** — `git revert HEAD`

🔮 Шаг 11:

Миссия 11

Укажите, что относится к reset, а что к checkout.

Подсказка: Один меняет историю, другой — просто перемещает голову.

Ответ:

- **reset** — Меняет указатель ветки и индекс
- **checkout** — Переключает файлы и HEAD в другое состояние

🔗 Шаг 12:

Миссия 12

Расположите шаги по восстановлению случайно удалённого файла.

Подсказка: Сначала — диагностика, потом — действие.

Ответ:

- `git log -- filename`
- Найти нужный commit hash
- `git checkout -- filename`
- Проверить, что файл восстановлен

🔗 Шаг 13:

Миссия 13

Как правильно перенести случайный коммит в другую ветку?

Подсказка: Думай как хирург — отделить, переместить, зафиксировать.

Ответ:

- `git reset HEAD~1`
- `git switch правильная_ветка`
- `git cherry-pick`

🔗 Шаг 14:

Миссия 14

Шаги безопасного отката через revert.

Подсказка: Revert — не убийство, а аннулирование.

Ответ:

- Определить commit hash
- `git revert`
- Проверить результат

🔗 Шаг 15:

Миссия 15

Как вернуться к версии файла из прошлого без потери текущих данных?

Подсказка: Checkout умеет точно доставать файлы.

Ответ:

- `git log -- filename`
- Скопировать hash
- `git checkout -- filename`

🔧 Шаг 16:

Миссия 16

Порядок полного сброса до origin/main.

Подсказка: Сначала обновление, потом синхронизация.

Ответ:

- `git fetch origin`
- `git reset --hard origin/main`

🔧 Шаг 17:

Миссия 17

Шаги проверки перед опасным `reset --hard`.

Подсказка: Не прыгай в пропасть, не посмотрев вниз.

Ответ:

- `git status`
- `git log --oneline`
- `git reset --hard HEAD~1`

📖 Урок 4 - Практика вернуть удалённую ветку

🔧 Шаг 01:

Миссия 1

Даша случайно удалила ветку `feature/login` после успешного слияния. Паника! Она хочет проверить, остался ли коммит в истории. Какой командой начать поиск?

Подсказка: Вспомни про команды, которые показывают историю всех ссылок, даже удалённых.

Ответ:

- **`git reflog`**
- `git reset --hard`
- `git rebase`
- `git clone`

🔧 Шаг 02:

Миссия 2

Саша нашёл нужный хеш коммита в `git reflog`. Как восстановить ветку с именем `bugfix/api` на этот коммит?

Подсказка: Представь, будто ты просто создаёшь новую ветку на старом коммите.

Ответ:

- **`git branch bugfix/api`**
- `git pull origin bugfix/api`
- `git revert`
- `git stash apply`

🔧 Шаг 03:

Миссия 3

Инженер по безопасности Женя случайно удалил ветку прямо на удалённом репозитории: `git push origin --delete feature/auth` . Как восстановить её на удалённом сервере после локального восстановления?

Подсказка: Локальная ветка уже создана. Нужно просто отправить её обратно.

Ответ:

- **git push origin feature/auth**
- `git fetch --all`
- `git reset origin/feature/auth`
- `git init --bare`

🔗 Шаг 04:

Миссия 4

Лена увидела, что нужный коммит был перемещён в detached HEAD . Что безопаснее сделать перед созданием ветки?

Подсказка: Лучше не работать в подвешенном состоянии.

Ответ:

- **Создать новую ветку из текущего состояния**
- **Сразу сделать push** — оно само создастся
- Удалить коммит, чтобы не мешался
- Перейти на main и забыть про всё

🔗 Шаг 05:

Миссия 5

Олег обнаружил, что удалённая ветка была уже с мержена. Зачем вообще её восстанавливать?

Подсказка: Даже если код уже в main, сама ветка могла быть полезна.

Ответ:

- **Для продолжения работы на основе её истории**
- Чтобы украсить список веток
- Потому что удалённые ветки платные
- Чтобы случайно что-то сломать

🔗 Шаг 06:

Миссия 6

Программист Тимур решил восстановить ветку, но не помнит её точное имя. Что ему поможет?

Подсказка: История знает всё.

Ответ:

- **Просмотр сообщений в git reflog**
- Удаление всего репозитория
- Запуск `git merge --abort`
- Открытие `.gitignore` и поиск там

🔗 Шаг 07:

Миссия 7

Сопоставьте команду с её назначением при восстановлении удалённой ветки.

Подсказка: Представь, что ты спасатель, и каждая команда — твой инструмент.

Ответ:

- **git reflog** — Показывает историю всех перемещений, включая удалённые ветки
- **git branch** — Создаёт новую ветку на нужном коммите
- **git push origin** — Загружает локальную ветку на удалённый репозиторий
- **git checkout** — Переключается на восстановленную ветку

🔗 Шаг 08:

Миссия 8

Саша использует разные инструменты поиска коммитов. Сопоставьте источник с тем, что он показывает.

Подсказка: Один показывает только историю ветки, другой — всё подряд.

Ответ:

- **git log** — Показывает историю текущей ветки
- **git reflog** — Показывает все перемещения HEAD
- **.git/refs** — Содержит файлы с указателями на ветки
- **GUI-инструменты (например, SourceTree)** — Графическое отображение истории коммитов

🔗 Шаг 09:

Миссия 9

Артём работает в команде и хочет предупредить коллег о восстановлении ветки. Соотнеси действие с правильным каналом.

Подсказка: Некоторые вещи лучше сказать вслух, некоторые — командой.

Ответ:

- **Создание ветки** — git branch feature/restore
- **Публикация ветки** — git push origin feature/restore
- **Написать коллегам** — Сообщение в Slack
- **Добавить описание восстановления** — Сообщение в pull request

🔗 Шаг 10:

Миссия 10

Костя восстановил ветку, но теперь сомневается, стоит ли её пушить. Сопоставьте ситуацию и правильное действие.

Подсказка: Иногда лучше оставить всё локально.

Ответ:

- **Ветка нужна только тебе** — Оставить локально
- **Надо возобновить совместную работу** — Сделать git push origin
- **Коммит устарел** — Создать новую ветку и cherry-pick
- **Ветка содержит важную документацию** — Пушнуть с пометкой в описании

🔗 Шаг 11:

Миссия 11

Сопоставь действие с риском при восстановлении ветки прямо в продакшн.

Подсказка: Герой не тот, кто всё чинит, а тот, кто не ломает.

Ответ:

- **Восстановить ветку без проверки** — Может привести к конфликтам
- **Проверить в отдельном окружении** — Минимизирует риск поломки
- **Не сообщать команде** — Коллеги будут в шоке
- **Создать PR перед пушем** — Позволяет получить ревью

🔮 Шаг 12:

Миссия 12

Расставь шаги восстановления удалённой ветки в правильном порядке.

Подсказка: Представь, что ты следопыт: сначала находишь след, потом идёшь по нему.

Ответ:

- Выполнить `git reflog` и найти нужный коммит
- Создать ветку: `git branch`
- Переключиться: `git checkout`
- Опубликовать: `git push origin`

🔮 Шаг 13:

Миссия 13

Восстановление ветки после `force-push` выглядит так...

Подсказка: Не забывай: `force-push` — как портал в другую вселенную.

Ответ:

- Найти старый хеш через `git reflog origin/main`
- Создать временную ветку на этот коммит
- Сравнить изменения с текущей веткой
- **Решить, что оставить** — `merge`, `cherry-pick` или новый `push`

🔮 Шаг 14:

Миссия 14

Участник команды случайно удалил ветку на сервере. Разложи действия ПРАВИЛЬНОГО DevOps.

Подсказка: Не всё решает одна команда.

Ответ:

- Проверить локальные копии у коллег
- Найти последний коммит ветки через `reflog`
- Создать ветку заново
- Синхронизировать с `origin`

🔮 Шаг 15:

Миссия 15

Что делать, если ветка удалена, но ты помнишь только название файла, который в ней был изменён?

Подсказка: Git тоже умеет искать по содержимому.

Ответ:

- Использовать `git log --all --`
- Найти коммит, где менялся файл

- Создать ветку на этот хеш
- Переименовать ветку при необходимости

Шаг 16:

Миссия 16

Ты спасатель веток. Расставь действия при полном восстановлении потерянной истории.

Подсказка: Думай как архивариус.

Ответ:

- Сделать бэкап текущего репозитория
- Изучить reflog и refs вручную
- Восстановить ветку на нужный хеш
- Задokumentировать шаги

Шаг 17:

Миссия 17

Финал! Как обезопасить себя от удаления веток в будущем?

Подсказка: Лучший спасатель — профилактика.

Ответ:

- Включить защиту веток в репозитории
- Добавить alias для безопасного удаления
- Настроить уведомления о удалениях
- Обучить команду правилам удаления

Модуль 10 - Работа с удалёнными репозиториями

Урок 1 - Push, pull, fetch — различия

Шаг 01:

Миссия 1

Андрей впервые настроил удалённый репозиторий origin и написал код, но забыл, как отправить изменения. Какую команду он должен использовать?

Подсказка: Он хочет отправить свои локальные коммиты на сервер.

Ответ:

- **git push**
- git pull
- git fetch
- git clone

Шаг 02:

Миссия 2

Бэкендер Маша сидит в поезде без интернета и делает коммиты. Что произойдёт, если она выполнит git push ?

Подсказка: Удалённый сервер недоступен.

Ответ:

- Коммиты загрузятся в удалённый репозиторий

- Команда удалит все локальные коммиты
- **Git выдаст ошибку подключения**
- Создаст новый удалённый репозиторий

🔗 Шаг 03:

Миссия 3

Фронтендер Кирилл сделал `git fetch`, и теперь он в замешательстве — он не видит изменений в своём файле. Почему?

Подсказка: `Fetch` не изменяет рабочую директорию.

Ответ:

- **Fetch только загружает данные, но не сливает их в текущую ветку**
- `Fetch` удаляет изменения
- `Fetch` автоматически выполняет `push`
- `Fetch` обновляет только `stash`

🔗 Шаг 04:

Миссия 4

DevOps-инженер Олег пишет: «Я сделал `git pull`, и у меня появились конфликты!». Что произошло?

Подсказка: `Pull` включает в себя сразу две команды.

Ответ:

- `Pull` только скачал изменения
- **`Pull` скачал и попытался автоматически слить изменения**
- `Pull` выполнил `reset`
- `Pull` удалил локальные ветки

🔗 Шаг 05:

Миссия 5

Аня работает в команде с медленным интернетом. Она хочет заранее скачать изменения коллег, но не применять их. Что выбрать?

Подсказка: Команда не должна менять рабочее состояние файлов.

Ответ:

- `git pull`
- **`git fetch`**
- `git commit`
- `git init`

🔗 Шаг 06:

Миссия 6

Безопасник Стас проверяет чужой репозиторий. Он не хочет вносить изменения, а только посмотреть удалённые ветки. Его выбор?

Подсказка: Команда должна только обновить ссылки на удалённые ветки.

Ответ:

- **`git fetch --all`**
- `git pull origin dev`

- git merge
- git rebase

🔧 Шаг 07:

Миссия 7

Сопоставьте команду с её действием в Git.

Подсказка: Вспомните: push — наружу, pull — внутрь.

Ответ:

- **git push** — Отправка локальных коммитов в удалённый репозиторий
- **git pull** — Загрузка и автоматическое слияние удалённых изменений
- **git fetch** — Загрузка удалённых изменений без объединения
- **git clone** — Создание локальной копии удалённого репозитория

🔧 Шаг 08:

Миссия 8

Фуллстек-разработчик Лена объясняет новичку отличия push и pull. Сопоставьте ситуацию с подходящей командой.

Подсказка: Что идёт от тебя — а что к тебе?

Ответ:

- **Ты закончил фичу и хочешь поделиться с командой** — git push
- **Ты пришёл утром и хочешь обновить свой код** — git pull
- **Ты хочешь забрать изменения, но не смешивать их** — git fetch
- **Ты хочешь создать копию удалённого репозитория** — git clone

🔧 Шаг 09:

Миссия 9

Сисадмин Паша проверяет, кто куда тянет данные. Соотнесите описание с командой.

Подсказка: Представьте движение данных как стрелки.

Ответ:

- **Выгрузка данных наружу** — git push
- **Забор данных внутрь** — git pull
- **Проверка удалённых изменений без внедрения** — git fetch
- **Инициализация связи с удалённым репозиторием** — git remote add

🔧 Шаг 10:

Миссия 10

DevOps Ярослав увидел странные команды в логе. Сопоставьте команду и её скрытый смысл.

Подсказка: Смотри на последствия.

Ответ:

- **git fetch origin main** — Обновление ссылки на ветку main без слияния
- **git pull origin main** — Получение и объединение ветки main
- **git push origin feature** — Отправка фичи на сервер
- **git push --force** — Перезапись истории на удалёнке

Шаг 11:

Миссия 11

Команда устроила батл: кто лучше объяснит Git с помощью метафор. Соотнесите метафору с командой.

Подсказка: Спорили до утра.

Ответ:

- «**Push** — как отправить письмо на почту» — `git push`
- «**Pull** — как получить посылку домой» — `git pull`
- «**Fetch** — как узнать, есть ли посылка, но не забирать» — `git fetch`
- «**Clone** — как получить полный архив переписки» — `git clone`

Шаг 12:

Миссия 12

Паша изучает риски. Соотнесите действие с его потенциальной опасностью.

Подсказка: Самая опасная команда — та, что переписывает историю.

Ответ:

- **git push** — Может вызвать конфликт у коллег
- **git pull** — Может автоматически изменить текущий код
- **git fetch** — Не вносит изменения в файлы
- **git push --force** — Может уничтожить чужую работу

Шаг 13:

Миссия 13

В каком порядке новичок Коля должен действовать, чтобы отправить код в GitHub?

Подсказка: Репозиторий не знает адрес сам по себе.

Ответ:

- `git remote add origin ...`
- `git add .`
- `git commit -m "init"`
- `git push origin main`

Шаг 14:

Миссия 14

Старший разработчик дал Коле задачу правильно обновить проект. Расставь шаги pull-процесса.

Подсказка: Pull = fetch + merge.

Ответ:

- `git fetch origin main`
- `git merge origin/main`
- Разрешение конфликтов (если есть)
- Тестирование результата

Шаг 15:

Миссия 15

Андрей восстанавливает удалённый репозиторий после сбоя. В каком порядке нужно действовать?

Подсказка: Вначале — копия, потом — работа.

Ответ:

- git clone ...
- git checkout нужную ветку
- git pull origin ветка
- git push origin ветка

Шаг 16:

Миссия 16

Фронтендер решил создать новую ветку и отправить её коллегам. Упорядочь команды.

Подсказка: Сначала создаём, потом отправляем.

Ответ:

- git checkout -b feature/ui
- git add .
- git commit -m "UI start"
- git push origin feature/ui

Шаг 17:

Миссия 17

DevOps автоматизирует CI. Он хочет регулярно синхронизировать локальный код с удалённым. Расставь шаги скрипта.

Подсказка: Не стоит сразу вливать всё — сначала посмотри.

Ответ:

- git fetch origin main
- git merge origin/main
- Запуск тестов
- Уведомление о результате

Урок 2 - Upstream и tracking ветки

Шаг 01:

Миссия 1

Алиса впервые отправляет изменения на GitHub и видит сообщение: no upstream branch . Что это значит?

Подсказка: Локальная ветка не знает, куда отправлять свои коммиты.

Ответ:

- **Ветка не привязана к удалённой ветке**
- GitHub временно недоступен
- Нужно обновить токен доступа
- Локальный репозиторий пуст

Шаг 02:

Миссия 2

Боб пишет: «Я сделал git pull , но изменений нет!» Какой вариант объяснения наиболее вероятен?

Подсказка: Проверь, с какой веткой связан pull.

Ответ:

- **Текущая локальная ветка не отслеживает нужную удалённую**
- GitHub перестал поддерживать pull
- Нужно перезапустить терминал
- Сначала нужно сделать merge вручную

🔮 Шаг 03:

Миссия 3

Катя создала ветку feature/login и хочет сразу привязать её к удалённой. Какой командой она может сделать push с установкой upstream?

Подсказка: Можно сделать всё одной командой.

Ответ:

- **git push -u origin feature/login**
- git connect origin feature/login
- git init --upstream
- git bind feature/login

🔮 Шаг 04:

Миссия 4

Сергей случайно сделал push без -u . Как ему теперь привязать локальную ветку к удалённой?

Подсказка: Есть команда, позволяющая переопределить upstream.

Ответ:

- **git branch --set-upstream-to=origin/main**
- git link origin main
- git push --repair
- git assign upstream

🔮 Шаг 05:

Миссия 5

Маша видит в выводе git status : Your branch is ahead of 'origin/main' by 2 commits . Что это значит?

Подсказка: Локально — впереди, удалённо — ещё нет.

Ответ:

- **Есть локальные коммиты, которые ещё не отправлены на сервер**
- Удалённая ветка новее локальной
- Где-то конфликт с другим репозиторием
- Git не понимает, куда коммитить

🔮 Шаг 06:

Миссия 6

Сопоставьте команды Git с их назначением.

Подсказка: Думай, что происходит с привязкой ветки.

Ответ:

- **git branch --set-upstream-to** — Привязка существующей ветки к удалённой
- **git push -u** — Отправка и одновременная привязка ветки
- **git remote -v** — Просмотр списка удалённых репозиториев
- **git branch -vv** — Просмотр привязок локальных веток

🔮 Шаг 07:

Миссия 7

Сопоставьте термины с их определениями.

Подсказка: Вспомни, кто кого отслеживает.

Ответ:

- **Upstream** — Удалённая ветка, с которой синхронизируется локальная
- **Tracking branch** — Локальная ветка, знающая свой upstream
- **Origin** — Имя удалённого репозитория по умолчанию
- **Fetch** — Получение изменений без слияния

🔮 Шаг 08:

Миссия 8

Дан ситуации — выбери, что нужно сделать.

Подсказка: Представь, что ты тимлид.

Ответ:

- **Нужно сменить upstream на origin/dev** — `git branch --set-upstream-to=origin/dev`
- **Хочешь узнать, какая ветка отслеживается** — `git status`
- **Хочешь увидеть все ветки и upstream** — `git branch -vv`
- **Удалённая ветка больше не нужна** — `git push origin --delete`

🔮 Шаг 09:

Миссия 9

Сопоставьте описания ошибок с причинами.

Подсказка: Всё крутится вокруг неправильной привязки.

Ответ:

- **git pull ничего не скачивает** — Локальная ветка не отслеживает удалённую
- **git push требует указать репозиторий** — Нет установленного upstream
- **Конфликты после pull** — Изменения в локальной и удалённой ветках расходятся
- **git fetch не видит новый branch** — Удалённый ещё не синхронизирован

🔮 Шаг 10:

Миссия 10

Что говорит Git? Сопоставь сообщение с его смыслом.

Подсказка: Переведи на человеческий язык.

Ответ:

- **Your branch is behind 'origin/main'** — Есть новые коммиты на сервере
- **No upstream configured** — Git не знает, куда отправлять изменения
- **Everything up-to-date** — Локальная и удалённая ветки синхронизированы

- **Diverged** — История разошлась — нужен merge

🔮 Шаг 11:

Миссия 11

Сервер сообщает, что ветки разошлись. Расположи шаги для исправления.

Подсказка: Всегда начинай с получения удалённых изменений.

Ответ:

- git fetch
- git merge origin/main
- Исправление конфликтов
- git push

🔮 Шаг 12:

Миссия 12

Тебе нужно полностью сменить upstream. Расставь команды.

Подсказка: Сначала убери старое, потом добавь новое.

Ответ:

- git branch --unset-upstream
- git branch --set-upstream-to=origin/dev
- git pull
- git push

🔮 Шаг 13:

Миссия 13

Как правильно восстановить удалённую ветку, если локальная ещё жива?

Подсказка: Нужно просто отправить её обратно.

Ответ:

- git checkout feature/docs
- git push -u origin feature/docs
- git pull
- Проверка на GitHub

🔮 Шаг 14:

Миссия 14

Как проверить, чем локальная ветка отличается от upstream?

Подсказка: Нужно сравнить с origin.

Ответ:

- git fetch
- git log HEAD..origin/main
- git status
- git pull

🔮 Шаг 15:

Миссия 15

Удали старую ветку и создай новую с тем же именем, но с другим upstream.

Подсказка: Не перепутай локальное и удалённое.

Ответ:

- `git push origin --delete feature/old`
- `git branch -D feature/old`
- `git checkout -b feature/old`
- `git push -u origin feature/old`

Шаг 16:

Миссия 16

Финальный босс! Полная настройка нового проекта с нуля.

Подсказка: Представь, что ты DevOps.

Ответ:

- `git init`
- `git remote add origin [https://example.com/repo.git`
- `git](https://example.com/repo.git%22},{%22text%22:%22git) checkout -b main`
- `git push -u origin main`

Урок 3 - Практика синхронизация веток

Шаг 01:

Миссия 1

Андрей сделал коммит и хочет отправить его в удалённую ветку main . Какую команду он использует?

Подсказка: Отправка — это push.

Ответ:

- **`git push origin main`**
- `git upload main`
- `git send origin main`
- `git remote push main`

Шаг 02:

Миссия 2

Катя клонировала репозиторий и хочет узнать, какие удалённые адреса к нему подключены. Что ей сделать?

Подсказка: Команда с параметром -v покажет URL-ы.

Ответ:

- **`git remote -v`**
- `git log --remote`
- `git show remote`
- `git fetch --list`

Шаг 03:

Миссия 3

Марина хочет скачать изменения из удалённой ветки, но не сливать их автоматически. Что ей нужно?

Подсказка: Это не pull.

Ответ:

- **git fetch**
- git pull
- git merge origin
- git download

🔧 Шаг 04:

Миссия 4

Витя сделал git pull и увидел сообщение о конфликте. Что ему стоит сделать первым делом?

Подсказка: Не спеши коммитить — сначала порядок.

Ответ:

- **Открыть файлы с конфликтами и вручную их исправить**
- Сразу сделать git push
- Удалить репозиторий и начать заново
- Сделать git fetch --force

🔧 Шаг 05:

Миссия 5

Сопоставьте команду Git с её назначением.

Подсказка: Вспомни: pull = fetch + merge.

Ответ:

- **git fetch** — Загружает изменения, но не сливает их
- **git pull** — Загружает и сразу сливает изменения
- **git push** — Отправляет локальные коммиты на сервер
- **git remote -v** — Показывает ссылки привязанных репозитория

🔧 Шаг 06:

Миссия 6

Ты работаешь в стартапе. Соотнеси задачу с правильной командой.

Подсказка: Один из вариантов связан с удалением.

Ответ:

- **Удалить связь с origin** — git remote remove origin
- **Переименовать удалённый репозиторий** — git remote rename
- **Посмотреть все ветки на удалённом сервере** — git branch -r
- **Сравнить локальную и удалённую ветку** — git diff origin/main

🔧 Шаг 07:

Миссия 7

На хакатоне ребята спорят, что делает каждая часть команды. Разберёмся?

Подсказка: Обрати внимание на порядок: remote → имя → действие.

Ответ:

- **git remote add** — Создаёт новый удалённый репозиторий
- **git remote set-url** — Меняет URL существующего удалённого
- **git remote prune** — Удаляет устаревшие ссылки на ветки
- **git remote show** — Показывает подробности по удалённому репозиторию

🔗 Шаг 08:

Миссия 8

Соедини ситуацию и действие.

Подсказка: Когда сомневаешься — сначала смотри, а потом делай.

Ответ:

- **Нужно увидеть лог удалённой ветки** — `git log origin/main`
- **Получить все обновления без merge** — `git fetch origin`
- **Загрузить изменения и влить их** — `git pull origin main`
- **Записать изменения на сервер** — `git push origin main`

🔗 Шаг 09:

Миссия 9

Сопоставь проблему и решение.

Подсказка: Иногда проще сменить URL, чем удалять.

Ответ:

- **Неправильный URL у origin** — `git remote set-url origin NEW`
- **Лишний удалённый репозиторий** — `git remote remove origin`
- **Хочу видеть все удалённые ветки** — `git branch -r`
- **Сомневаюсь, с кем связан origin** — `git remote -v`

🔗 Шаг 10:

Миссия 10

Расставь шаги: как правильно отправить новую ветку на сервер?

Подсказка: Сначала создаём, потом переключаемся.

Ответ:

- Создать ветку: `git branch feature`
- Переключиться: `git checkout feature`
- Сделать коммиты
- Отправить: `git push -u origin feature`

🔗 Шаг 11:

Миссия 11

Синхронизация перед публикацией релиза. Расставь команды в правильном порядке.

Подсказка: Получи изменения → слей → отправь.

Ответ:

- `git fetch origin`
- `git merge origin/main`
- `git push origin main`

- Запустить тесты

🔧 Шаг 12:

Миссия 12

Тебе дали чужой репозиторий. Что делать?

Подсказка: Не начнёшь — не закончишь.

Ответ:

- `git clone URL`
- `cd repo`
- `git fetch origin`
- `git switch -c dev`

🔧 Шаг 13:

Миссия 13

Отработка конфликта: что за чем?

Подсказка: Не пушь, пока не починишь.

Ответ:

- `git pull origin main`
- Открыть конфликтные файлы
- Исправить и `git add`
- `git commit`

🔧 Шаг 14:

Миссия 14

Публикация большого обновления. Расставь шаги.

Подсказка: Сделал → проверил → отправил.

Ответ:

- Сделать коммиты
- `git fetch origin`
- `git merge origin/main`
- `git push origin main`

🔧 Шаг 15:

Миссия 15

Финальная миссия: как правильно восстановить ветку, если её удалили на сервере?

Подсказка: Сначала проверь, есть ли она в локальной истории.

Ответ:

- `git reflog`
- Найти нужный `commit-hash`
- `git branch recovered`
- `git push origin recovered`

Урок 4 - Работа с несколькими remotes

Шаг 01:

Миссия 1

Борис добавил remote с именем origin и backup . Как посмотреть список всех remotes?

Подсказка: Эта команда работает даже без подключения к интернету.

Ответ:

- **git remote -v**
- git show remotes
- git list remotes
- git status remotes

Шаг 02:

Миссия 2

Катя заметила, что случайно назвала remote origн вместо origin . Как правильно переименовать remote?

Подсказка: Переименование делается одной командой, без повторного добавления URL.

Ответ:

- **git remote rename origн origin**
- git remote mv origн origin
- git rename remote origн origin
- git remote update origн=origin

Шаг 03:

Миссия 3

Игорь хочет удалить лишний remote test . Как это сделать?

Подсказка: Удаление не трогает коммиты — только ссылку на сервер.

Ответ:

- **git remote remove test**
- git remote delete test
- git rm test
- git drop remote test

Шаг 04:

Миссия 4

Маша хочет подтянуть изменения не из origin , а из backup . Какой командой это сделать?

Подсказка: Та же команда, что и обычно, но имя remote — другое.

Ответ:

- **git fetch backup**
- git pull backup/master
- git sync backup
- git update backup

🔮 Шаг 05:

Миссия 5

Сопоставьте команду с её результатом. Представьте, что вы дежурите в DevOps-ночь и чините чужие remotes.

Подсказка: Вспомни, что делает каждая команда с remote.

Ответ:

- **git remote add backup URL** — Добавляет новый удалённый репозиторий
- **git remote rename old new** — Меняет имя удалённого репозитория
- **git remote remove temp** — Удаляет ссылку на удалённый репозиторий
- **git remote -v** — Показывает список всех удалённых репозитория

🔮 Шаг 06:

Миссия 6

Федя ведёт проект с зеркалами на GitHub и GitLab. Сопоставьте remote с его назначением.

Подсказка: GitHub чаще origin , остальное — резерв.

Ответ:

- **origin** — Основной репозиторий на GitHub
- **backup** — Зеркало на GitLab
- **deploy** — Сервер автодепоя
- **archive** — Старое хранилище для истории

🔮 Шаг 07:

Миссия 7

Саша анализирует вывод git remote -v . Сопоставьте URL с названием remotes.

Подсказка: В названии сервиса — подсказка.

Ответ:

- **origin** — [https://github.com/user/project.git
- **mirror** — https://gitlab.com/user/project.git
- **deploy** — ssh://server.com/var/repo.git
- **backup** — https://bitbucket.org/user/project.git

🔮 Шаг 08:

Миссия 8

Хакер Василий сканирует репозиторий. Сопоставьте угрозу и действие.

Подсказка: Правильная структура remotes снижает риски.

Ответ:

- **Публичный remote с секретами** — Удалить или закрыть доступ
- **Сломанный URL** — Обновить адрес через set-url
- **Случайное дублирование remotes** — Переименовать или удалить лишний
- **Неизвестное имя remote** — Переименовать для ясности

🔮 Шаг 09:

Миссия 9

Команда создала хаос: все называли remotes по-разному. Сопоставьте старое имя с новым стандартом.

Подсказка: Стандарт: origin — основной, upstream — оригинальный источник.

Ответ:

- **mainRepo** — origin
- **source** — upstream
- **mirrorServer** — backup
- **prodPush** — deploy

Шаг 10:

Миссия 10

Расставьте шаги при добавлении второго remote для коллаборации.

Подсказка: Сначала проверить текущее состояние, потом добавить.

Ответ:

- Проверить текущие remotes через `git remote -v`
- Добавить новый remote через `git remote add name URL`
- Убедиться, что он появился в списке
- Синхронизировать изменения через `git fetch name`

Шаг 11:

Миссия 11

Порядок действий при переезде на новый сервер.

Подсказка: Не удаляй раньше времени!

Ответ:

- Добавить новый remote через `git remote add new URL`
- Перенести ветки через `git push new --all`
- Сделать его основным: `git remote rename origin old && git remote rename new origin`
- Удалить старый remote через `git remote remove old`

Шаг 12:

Миссия 12

Команда делает форк. Расставьте шаги.

Подсказка: Форк — сначала clone, потом upstream.

Ответ:

- Клонировать свой форк через `git clone URL`
- Добавить оригинальный репозиторий как upstream
- Проверить, что upstream появился
- Получить свежие изменения `git fetch upstream`

Шаг 13:

Миссия 13

Порядок синхронизации между origin и backup.

Подсказка: Сперва забрать, потом отправить.

Ответ:

- Забрать изменения из origin через `git fetch origin`
- Слить их в локальную ветку
- Отправить результат в backup через `git push backup`
- Проверить состояние через `git remote -v`

 Шаг 14:

Миссия 14

Экстренный случай: сервер origin упал. Нужно переключиться на backup как основной.

Подсказка: Переименование спасает.

Ответ:

- Переименовать origin в old через `git remote rename origin old`
- Переименовать backup в origin через `git remote rename backup origin`
- Проверить список remotes
- При необходимости обновить URL

 Модуль 11 - Совместная разработка

 Урок 1 - Fork и работа с форками

 Шаг 01:

Миссия 1

Тимур впервые решил внести правку в популярный open-source проект. Что ему нужно сделать, прежде чем отправлять Pull Request?

Подсказка: Представь, что ты не можешь писать в чужую тетрадку напрямую.

Ответ:

- **Сделать fork репозитория**
- Удалить репозиторий автора
- Сразу пушнуть в main оригинала
- Создать issue и ждать вечность

 Шаг 02:

Миссия 2

Лена сделала fork проекта, но не видит свои изменения в оригинале. Почему?

Подсказка: Fork — это не магический портал, а отдельная копия.

Ответ:

- **Потому что форк** — это независимый репозиторий
- Потому что GitHub сломался
- Потому что она недостаточно известный разработчик
- Потому что нужно перезагрузить компьютер

 Шаг 03:

Миссия 3

Игорь сделал fork, но автор проекта уже внёс новые изменения. Что Игорю нужно сделать, чтобы их получить?

Подсказка: Нужно добавить источник правды.

Ответ:

- **Добавить remote upstream и сделать fetch**
- Удалить свой fork и начать заново
- Спросить автора по e-mail
- Залить свои изменения поверх

🔮 Шаг 04:

Миссия 4

Катя сделала Pull Request, но автор попросил «rebase onto upstream/main». Что от неё хотят?

Подсказка: Нужно выровнять историю с оригиналом.

Ответ:

- **Перенести свои коммиты поверх свежей версии оригинала**
- Удалить все коммиты
- Сменить ник на GitHub
- Пересобрать проект в Docker

🔮 Шаг 05:

Миссия 5

Петя создал Pull Request, но забыл указать, какую проблему он решает. Что считается хорошим тоном?

Подсказка: Люди любят ясность больше, чем магию.

Ответ:

- **Добавить описание изменений и ссылку на issue**
- Написать «догадайтесь сами»
- Оставить всё как есть
- Закрывать Pull Request обидевшись

🔮 Шаг 06:

Миссия 6

Аня отправила Pull Request в чужой проект. Когда ей нужно удалять свою ветку?

Подсказка: Ветки — как одноразовые тарелки в походе.

Ответ:

- **После merge в основной репозиторий**
- Сразу после создания Pull Request
- **Никогда** — пусть хранится вечно
- Перед тем как вообще пушить

🔮 Шаг 07:

Миссия 7

Сопоставьте термин с его смыслом.

Подсказка: Представьте, что вы объясняете другу, который никогда не слышал о Git.

Ответ:

- **Fork** — Создание копии чужого репозитория в своём аккаунте

- **Pull Request** — Запрос на внесение изменений в оригинальный репозиторий
- **Upstream** — Источник оригинального репозитория для синхронизации
- **Clone** — Скачивание репозитория на локальный компьютер

🔧 Шаг 08:

Миссия 8

Кто что делает?

Подсказка: Вспомните, кто инициирует действие, а кто принимает.

Ответ:

- **Автор Pull Request** — Предлагает изменения
- **Мейнтейнер проекта** — Проверяет и принимает изменения
- **Контрибьютор** — Вносит изменения через fork
- **Ревьюер** — Оставляет комментарии к коду

🔧 Шаг 09:

Миссия 9

Сопоставьте команду Git с её действием.

Подсказка: Представь, что Git — это ресторан, а команды — заказы.

Ответ:

- **git fetch upstream** — Получить изменения из оригинала
- **git merge upstream/main** — Объединить изменения в свою ветку
- **git push origin feature** — Отправить свою ветку в fork
- **git branch -d feature** — Удалить локальную ветку

🔧 Шаг 10:

Миссия 10

Соедини ситуацию с правильным действием.

Подсказка: Представь, что ты саппорт самого себя.

Ответ:

- **Оригинальный репозиторий обновился** — Сделать fetch upstream
- **Ветки конфликтуют** — Решить конфликты вручную
- **Pull Request отклонён** — Исправить и отправить заново
- **Хочется начать заново** — Создать новую ветку

🔧 Шаг 11:

Миссия 11

Сопоставьте тип ошибки с её причиной.

Подсказка: Ошибка — это следствие, найди причину.

Ответ:

- **Permission denied** — Нет прав на запись в оригинальный репозиторий
- **Merge conflict** — Одинаковые строки изменены в двух ветках
- **404 Not Found** — Репозиторий был удалён или скрыт
- **Rejected push** — Попытка пуша в защищённую ветку

🔗 Шаг 12:

Миссия 12

Сопоставьте термин с его назначением.

Подсказка: Всё в Git чем-то управляет.

Ответ:

- **Issue** — Описание проблемы или задачи
- **Label** — Способ классификации задач
- **Milestone** — Группа связанных задач
- **Draft Pull Request** — Неоконченная работа для обсуждения

🔗 Шаг 13:

Миссия 13

Расположите шаги работы над задачей через fork.

Подсказка: Вспомни путь от идеи до Pull Request.

Ответ:

- Сделать fork репозитория
- Создать новую ветку
- Внести изменения
- Создать Pull Request

🔗 Шаг 14:

Миссия 14

Расположите шаги обновления форка.

Подсказка: Идём от оригинала к тебе.

Ответ:

- Добавить remote upstream
- Сделать fetch upstream
- Сделать merge или rebase
- Запустить в свой fork

🔗 Шаг 15:

Миссия 15

Восстановление после конфликтов.

Подсказка: Конфликт — не катастрофа.

Ответ:

- Открыть конфликтующий файл
- Решить конфликт вручную
- Сделать commit
- Продолжить merge или rebase

🔗 Шаг 16:

Миссия 16

Расположите шаги после принятия Pull Request.

Подсказка: Не оставляй мусор после себя.

Ответ:

- Удалить локальную ветку
- Удалить удалённую ветку
- Синхронизировать свой fork
- Закрыть задачу, если нужно

🔗 **Шаг 17:**

Миссия 17

Что делать, если Pull Request завис без ответа?

Подсказка: Представь, ты не навязчивый, а вежливый робот.

Ответ:

- Написать вежливый комментарий-напоминание
- Подождать разумное время
- Спросить в issue, нужна ли помощь
- Закрыть PR, если он больше не актуален

📖 **Урок 2 - Pull Request и code review**

🔗 **Шаг 01:**

Миссия 1

Разработчик Саша впервые отправляет Pull Request в общий репозиторий. Какой главный смысл Pull Request?

Подсказка: Это не просто загрузка кода, а приглашение к обсуждению.

Ответ:

- **Запрос на внесение изменений с возможностью обсуждения и проверки**
- Прямое добавление кода в основную ветку
- Сохранение изменений только локально
- Автоматическое тестирование без участия людей

🔗 **Шаг 02:**

Миссия 2

Тимлид Анна оставила комментарий в Pull Request: "Нужно переименовать переменную на более понятную". Что должен сделать автор?

Подсказка: Игнорировать — худший вариант 😊

Ответ:

- **Внести исправление и обновить Pull Request**
- Закрыть Pull Request и создать новый
- Удалить комментарий тимлида
- Проигнорировать замечание и ждать одобрения

🔗 **Шаг 03:**

Миссия 3

Вова сделал Pull Request и увидел, что автоматические тесты упали. Что правильнее всего сделать?

Подсказка: Тесты — не враги, а помощники.

Ответ:

- **Исправить ошибку и снова запустить изменения**
- Удалить тесты
- Попросить коллег слить без тестов
- **Оставить как есть** — вдруг само пройдет

🔮 Шаг 04:

Миссия 4

Иногда в Pull Request просят добавить описание изменений. Что туда лучше включить?

Подсказка: Коллеги не экстрасенсы.

Ответ:

- **Кратко объяснить, что и зачем было изменено**
- Случайный набор эмодзи 🤪🔥
- **Ничего** — и так понятно
- Вставить мем без контекста

🔮 Шаг 05:

Миссия 5

Петя сделал Pull Request, но забыл обновить ветку, и теперь там конфликты. Его лучший шаг?

Подсказка: Конфликты сами не исчезают.

Ответ:

- **Сделать merge или rebase основной ветки и вручную решить конфликты**
- Удалить репозиторий и переехать в другую страну
- **Ничего не делать** — пусть тимлид решает
- Создать новый Pull Request без изменений

🔮 Шаг 06:

Миссия 6

На code review Маша увидела небезопасную обработку паролей. Как ей корректнее всего отреагировать?

Подсказка: Цель review — не критиковать, а улучшать.

Ответ:

- **Написать в комментарии предложение с безопасной альтернативой**
- Оставить злой смайлик 😡
- **Промолчать** — не её дело
- Переименовать весь файл без обсуждения

🔮 Шаг 07:

Миссия 7

Сопоставьте роли в процессе Pull Request с их действиями.

Подсказка: Вспомните, кто создаёт, кто проверяет, а кто утверждает.

Ответ:

- **Автор Pull Request** — Создаёт изменения и отправляет их на проверку.

- **Ревьюер** — Просматривает код и оставляет комментарии.
- **Тимлид** — Принимает финальное решение о слиянии.

🔮 Шаг 08:

Миссия 8

🔍 Сопоставьте типы комментариев на code review с их смыслом.

Подсказка: Есть замечание, есть предложение, есть похвала 😊

Ответ:

- **Suggestion** — Рекомендация по улучшению кода.
- **Issue** — Необходимо исправить перед слиянием.
- **Praise** — Положительный отзыв о хорошем решении.

🔮 Шаг 09:

Миссия 9

⚙️ Сопоставьте команды Git с их ролью при работе с Pull Request.

Подсказка: merge и rebase не одно и то же!

Ответ:

- **git fetch** — Получить обновления без изменения текущей ветки.
- **git merge** — Объединить ветки, создав новый коммит.
- **git rebase** — Перенести коммиты поверх другой ветки.

🔮 Шаг 10:

Миссия 10

💔 Сопоставьте типичные ошибки в Pull Request с последствиями.

Подсказка: Много изменений = сложно ревьюить.

Ответ:

- **Слишком большой Pull Request** — Трудно ревьюить, выше шанс пропустить баги.
- **Отсутствие описания изменений** — Ревьюер не понимает цели.
- **Игнорирование комментариев** — Возникают конфликты в команде.

🔮 Шаг 11:

Миссия 11

🗨️ Сопоставьте стили общения на review с их восприятием.

Подсказка: Формулировка решает всё.

Ответ:

- **"Это ужасно, перепиши!"** — Вызывает сопротивление и обиду.
- **"А если попробовать вот так?"** — Воспринимается как конструктивное предложение.
- **"Отличное решение!"** — Мотивирует продолжать в том же духе.

🔮 Шаг 12:

Миссия 12

Сопоставьте статусы Pull Request с их значением.

Подсказка: Draft — ещё не готово!

Ответ:

- **Open** — Pull Request ожидает review.
- **Draft** — Работа ещё ведётся, ревью пока рано.
- **Merged** — Изменения приняты и объединены.

 Шаг 13:

Миссия 13

✂ Расставьте шаги создания Pull Request в правильном порядке:

Подсказка: Сперва изменения, потом ветка, потом запрос.

Ответ:

- Создать новую ветку
- Внести изменения и закоммитить
- Запустить ветку в удалённый репозиторий
- Открыть Pull Request на платформе

 Шаг 14:

Миссия 14

Расположите шаги корректного code review:

Подсказка: Сначала понять, потом комментировать.

Ответ:

- Ознакомиться с описанием Pull Request
- Просмотреть изменения в коде
- Оставить конструктивные комментарии
- Одобрить или запросить правки

 Шаг 15:

Миссия 15

✂ Расположите шаги решения конфликта при merge:

Подсказка: Обновиться, исправить, проверить.

Ответ:

- Получить последние изменения из основной ветки
- Вручную решить конфликт в файлах
- Протестировать изменения
- Закоммитить и обновить Pull Request

 Шаг 16:

Миссия 16

✓ Расставьте этапы перед закрытием Pull Request:

Подсказка: Проверка → одобрение → слияние.

Ответ:

- Убедиться, что все тесты зелёные
- Получить хотя бы одно одобрение ревьюера
- Убедиться в отсутствии конфликтов
- Нажать кнопку Merge

Шаг 17:

Миссия 17

 Расположите шаги после успешного merge:

Подсказка: Удаление мусора — это уважение к репозиторию.

Ответ:

- Удалить рабочую ветку
- Сообщить команде о слиянии
- Обновить локальную основную ветку
- Запланировать релиз (если нужно)

Урок 3 - Практика создание PR

Шаг 01:

Миссия 1

Разработчик Маша сделала форк репозитория на GitHub. Что ей нужно сделать перед тем, как отправить Pull Request?

Подсказка: PR — это предложение изменений, но сначала нужно внести эти изменения.

Ответ:

- **Сделать изменения в своём форке и закоммитить их**
- Удалить оригинальный репозиторий
- Создать Pull Request без изменений
- Переименовать репозиторий в uppercase

Шаг 02:

Миссия 2

Борис создал ветку feature/login и хочет открыть Pull Request в main. Что важно указать в описании PR?

Подсказка: Хороший PR отвечает на вопрос «Зачем?».

Ответ:

- **Кратко описать, какие изменения внесены и зачем**
- Приложить мем с котиками
- Удалить весь предыдущий код
- Сделать описание в одной букве

Шаг 03:

Миссия 3

Настя получила замечания к своему PR от тимлида. Какой лучший способ отреагировать?

Подсказка: PR — это диалог, а не битва.

Ответ:

- **Внести правки и поблагодарить за отзыв**
- Закрыть PR и обидеться
- Проигнорировать замечания
- Сделать ещё один PR с тем же содержимым

🔗 Шаг 04:

Миссия 4

Алексей открыл PR, но CI показал ошибки. Что ему стоит сделать?

Подсказка: Никто не любит красные крестики.

Ответ:

- **Исправить ошибки и обновить PR**
- Удалить CI из проекта
- Добавить комментарий «оно само починится»
- Сделать merge несмотря на ошибки

🔗 Шаг 05:

Миссия 5

Саша хочет, чтобы его PR приняли быстрее. Что улучшит шансы?

Подсказка: Сделайте жизнь ревьюера приятнее.

Ответ:

- **Разбить изменения на небольшие логические части**
- Вылить 5000 строк в одном коммите
- Удалить все комментарии в коде
- Добавить 20 случайных эмодзи

🔗 Шаг 06:

Миссия 6

Катя случайно создала PR не в ту ветку. Что делать?

Подсказка: Всегда можно переоткрыть — мир не рухнет.

Ответ:

- **Закрыть PR и создать новый в правильную ветку**
- Удалить весь репозиторий
- Сделать вид, что так и задумано
- Попросить всех игнорировать ошибку

🔗 Шаг 07:

Миссия 7

Сопоставьте термины, связанные с Pull Request.

Подсказка: Представьте, что PR — это заявление в ЖЭК: его надо заполнить правильно.

Ответ:

- **Review** — Процесс проверки изменений командой.
- **Merge** — Объединение ветки с основной.
- **Fork** — Создание собственной копии репозитория.
- **Conflict** — Ситуация, когда изменения пересекаются.

🔗 Шаг 08:

Миссия 8

Команда безопасников проверяет PR. Сопоставьте, что они ищут.

Подсказка: Уязвимости прячутся там, где их не ждут.

Ответ:

- **Hardcoded password** — Пароль вписан прямо в код.
- **SQL injection** — Использование данных без экранирования.
- **Open redirect** — Переход на произвольный URL.
- **Unvalidated input** — Нет проверки данных пользователя.

Шаг 09:

Миссия 9

Сопоставьте элементы PR с их назначением.

Подсказка: Представьте, что вы оформляете заявку на отпуск.

Ответ:

- **Title** — Краткое описание изменений.
- **Description** — Подробности и обоснование.
- **Labels** — Категории для фильтрации.
- **Assignee** — Ответственный за проверку.

Шаг 10:

Миссия 10

Сопоставьте команды git с их действием при работе над PR.

Подсказка: Каждая команда — как кнопка в лифте.

Ответ:

- **git fetch** — Получить новые изменения без слияния.
- **git pull** — Получить и объединить изменения.
- **git rebase** — Перестроить историю поверх другой ветки.
- **git cherry-pick** — Перенести конкретный коммит.

Шаг 11:

Миссия 11

Сопоставьте тип комментария в PR с его стилем.

Подсказка: Комментарий можно написать как робот, а можно — как человек.

Ответ:

- **Nitpick** — Небольшое предложение по улучшению.
- **Blocking** — Без исправления нельзя принять PR.
- **Suggestion** — Предлагаемое изменение кода.
- **Praise** — Позитивная реакция на хорошее решение.

Шаг 12:

Миссия 12

Расположите шаги создания PR в правильном порядке.

Подсказка: Ветка → изменения → коммит → PR.

Ответ:

- Создать новую ветку
- Внести изменения

- Закоммитить изменения
- Открыть Pull Request

Шаг 13:

Миссия 13

Вы команда опенсорс-разработчиков. Расположите процесс ревью.

Подсказка: Сначала читаем, потом критикуем.

Ответ:

- Ознакомиться с изменениями
- Оставить комментарии
- Дождаться исправлений
- Одобрить PR

Шаг 14:

Миссия 14

Как исправить конфликт перед merge?

Подсказка: Извлеки → реши → подтверди.

Ответ:

- Сделать git pull или fetch
- Вручную исправить конфликт
- Закоммитить исправления
- Продолжить merge

Шаг 15:

Миссия 15

Отправка PR в крупный проект. Расставьте приоритеты.

Подсказка: Не начинай с «я гений».

Ответ:

- Почитать CONTRIBUTING.md
- Создать ветку
- Внести изменения
- Открыть PR с уважительным описанием

Шаг 16:

Миссия 16

Процесс закрытия PR после принятия.

Подсказка: Сначала слить, потом убрать следы.

Ответ:

- Сделать squash или merge
- Закрыть ветку
- Отметить issue как решённый
- Поблагодарить ревьюеров

Шаг 17:

Миссия 17

Финальная миссия! Вы — мейнтейнер. Расставьте действия при получении хаотичного PR.

Подсказка: Вежливость → структура → решение.

Ответ:

- Поблагодарить за вклад
- Попросить разбить PR на части
- Предложить конкретные улучшения
- Переоткрыть после исправлений

Урок 4 - Обработка комментариев ревью

Шаг 01:

Миссия 1

Тимлид Антон оставил комментарий в ревью: «Лучше переименовать переменную data в userData — сейчас непонятно, что внутри». Что нужно сделать в первую очередь?

Подсказка: Сначала важно понять, согласен ли ты с замечанием.

Ответ:

- **Обсудить с Антоном, если возникли сомнения по смыслу переименования**
- **Игнорировать комментарий** — ведь код работает
- Сразу сделать merge без изменений
- Удалить весь файл на всякий случай

Шаг 02:

Миссия 2

Аня оставила комментарий: «Этот метод слишком длинный — можно ли разбить его на части?». Как лучше отреагировать?

Подсказка: Даже если замечание кажется очевидным, покажи уважение.

Ответ:

- **Ответить: «Хорошая идея, сейчас попробую упростить»**
- Написать: «Сама разбивай!»
- Удалить её комментарий через админку
- Сделать вид, что не заметил

Шаг 03:

Миссия 3

Денис оставил в ревью: «Не понимаю, почему здесь используется именно такой алгоритм». Как лучше объяснить?

Подсказка: Чем конкретнее — тем лучше.

Ответ:

- **Написать комментарий с объяснением и, возможно, ссылкой на документацию**
- Ответить: «Потому что так надо»
- Стереть его комментарий
- Обвинить Дениса в некомпетентности

🔗 Шаг 04:

Миссия 4

В ревью возник спор: один разработчик предлагает переписать код на другой стиль, но функциональность не меняется. Как поступить?

Подсказка: Вопросы вкуса лучше решать общими стандартами.

Ответ:

- **Уточнить, есть ли в проекте стиль-гайд, и следовать ему**
- Сказать, что твой стиль объективно лучше
- Удалить весь проект из репозитория
- Притвориться, что ноутбук сел

🔗 Шаг 05:

Миссия 5

Рецензент оставил 10 однотипных замечаний по форматированию (пробелы и табы). Что сделать, чтобы не повторять ошибки?

Подсказка: Компьютеры отлично справляются с рутинной.

Ответ:

- **Подключить pre-commit hooks с автоформатированием**
- Ручками править каждый раз
- Обижаться и игнорировать
- Проставить пробелы в случайных местах

🔗 Шаг 06:

Миссия 6

Ты получил комментарий: «Этот код может упасть при пустом вводе». Что лучше сделать?

Подсказка: Надёжность — залог спокойствия продакшена.

Ответ:

- **Проверить кейсы и добавить обработку ошибок**
- Ответить: «У нас всегда есть ввод»
- Закомментировать весь код
- Случайно удалить репозиторий

🔗 Шаг 07:

Миссия 7

Сопоставь тип комментария ревью с правильной реакцией автора.

Подсказка: Иногда нужно исправить, иногда — обсудить.

Ответ:

- **Непонятное название функции** — Предложить более ясное имя или уточнить смысл
- **Неверная логика** — Исправить и добавить тест
- **Спорный стиль форматирования** — Сослаться на стиль-гайд команды
- **Неочевидное решение** — Добавить поясняющий комментарий

🔗 Шаг 08:

Миссия 8

Сопоставь фразу ревьюера с её настоящим смыслом.

Подсказка: Разработчики — мастера дипломатии.

Ответ:

- **Интересный подход** — Я бы сделал иначе, но ладно
- **Можно чуть подробнее?** — Я ничего не понял
- **Возможно, стоит подумать над оптимизацией** — Этот код тормозит как улитка
- **Как насчёт других вариантов?** — Это решение спорное

🔮 Шаг 09:

Миссия 9

Сопоставь действие с результатом, который оно даёт в командной работе.

Подсказка: Коммуникация решает всё.

Ответ:

- **Задать уточняющий вопрос** — Избежать неправильного исправления
- **Поблагодарить за замечание** — Повысить уровень доверия
- **Приложить ссылку на источник** — Убедить ревьюера в своей правоте
- **Саркастично ответить** — Создать токсичность

🔮 Шаг 10:

Миссия 10

Сопоставь тип ревьюера с его стилем комментариев.

Подсказка: Узнай тип — поймёшь поведение.

Ответ:

- **Перфекционист** — Исправляет даже пробелы
- **Новичок** — Спрашивает, как работает каждая строка
- **Архитектор** — Обсуждает влияние на всю систему
- **Шутник** — Добавляет мемы в ревью

🔮 Шаг 11:

Миссия 11

Сопоставь действие с тем, как его воспримет команда.

Подсказка: Предсказуемость — залог доверия.

Ответ:

- **Запустить правки без комментария** — Коллеги не поймут, что изменилось
- **Ответить на каждый комментарий** — Показать уважение к ревьюеру
- **Игнорировать замечание** — Снизить качество кода
- **Спросить совета у команды** — Улучшить общее решение

🔮 Шаг 12:

Миссия 12

Сопоставь результат ревью с вероятной причиной.

Подсказка: Проблема всегда оставляет след.

Ответ:

- **Ревью задерживается** — Комментарии остались без ответа

- **Быстрый approve** — Правки были понятными и малыми
- **Возврат на доработку** — Код содержит критическую ошибку
- **Уточняющие вопросы от всех** — Описание задачи было туманным

🔗 Шаг 13:

Миссия 13

Расположи действия разработчика после получения комментариев в правильном порядке.

Подсказка: Не всё сразу — сначала понять, потом править.

Ответ:

- Прочитать все комментарии внимательно
- Задать вопросы, если что-то непонятно
- Внести исправления
- Отправить обновление на повторное ревью

🔗 Шаг 14:

Миссия 14

Отсортируй комментарии ревью по степени серьёзности — от лёгких до критических.

Подсказка: В продакшн должен идти стабильный код.

Ответ:

- Опечатка в комментариях к коду
- Неверная проверка прав доступа
- Неоптимальный SQL-запрос
- Отсутствие обработки null

🔗 Шаг 15:

Миссия 15

Расставь шаги успешного обсуждения спорного замечания в правильном порядке.

Подсказка: Аргументы важнее эмоций.

Ответ:

- Сформулировать свою позицию спокойно
- Привести аргументы или примеры
- Предложить компромисс
- Уточнить, согласен ли оппонент

🔗 Шаг 16:

Миссия 16

Упорядочи этапы подготовки к отправке кода на ревью.

Подсказка: Сделай жизнь ревьюера проще.

Ответ:

- Перепроверить код локально
- Запустить линтер и тесты
- Добавить читаемое описание изменений
- Создать Pull Request

Шаг 17:

Миссия 17

Отсортируй реакции на ревью по уровню профессионализма (от лучшего к худшему).

Подсказка: Эмоции — враг продуктивности.

Ответ:

- Поблагодарить и внести правки
- Спросить уточнения
- Игнорировать вопросы
- Оскорбить ревьюера

Модуль 12 - Конфликты и их решение

Урок 1 - Merge conflicts при слиянии

Шаг 01:

Миссия 1

Разработчик Петя и тестировщик Маша одновременно правят один и тот же файл. Петя делает git merge и видит странные символы <<<<<<. Что произошло?

Подсказка: Эти символы появляются не просто так — Git растерялся, какую версию оставить.

Ответ:

- **Произошёл merge conflict**
- Git решил удалить файл
- Это скрытые комментарии Пети
- Это символы форматирования Markdown

Шаг 02:

Миссия 2

Frontend-разработчица Лена случайно изменила одну и ту же строку CSS в двух ветках. Git запросил её помощь. Что Лене нужно сделать?

Подсказка: Конфликты не исчезают сами — нужен ручной выбор.

Ответ:

- **Открыть файл и вручную выбрать нужную версию строки**
- **Нажать Enter 10 раз** — Git сам поймёт
- Удалить обе ветки и сделать всё заново
- **Сменить редактор** — конфликт исчезнет

Шаг 03:

Миссия 3

Бэкендер Антон замечает в конфликте строки вида =====. Что они разделяют?

Подсказка: Git показывает две версии — сверху одна, снизу другая.

Ответ:

- **Две конфликтующие версии кода**
- Комментарии от коллег
- Закрывающий тег HTML

- Маркер конца файла

🔗 Шаг 04:

Миссия 4

DevOps-инженер Игорь решил не разбираться с конфликтом и сделать `git merge --abort`. Что произойдёт?

Подсказка: Эта команда откручивает merge, как будто его не было.

Ответ:

- **Отмена слияния и возврат к состоянию до merge**
- Удаление всей истории коммитов
- Файл становится пустым
- Git перезагружается

🔗 Шаг 05:

Миссия 5

QA-инженер Вика разобралась с конфликтом вручную. Как завершить процесс?

Подсказка: Git ждёт от тебя двух действий: фикс и подтверждение.

Ответ:

- **Сделать add и commit**
- Сделать только pull
- Закрыть терминал
- Удалить репозиторий

🔗 Шаг 06:

Миссия 6

Какая команда помогает увидеть файлы в конфликте?

Подсказка: Это не merge, но рядом.

Ответ:

- **git status**
- git conflict-list
- git show-conflicts
- git reveal

🔗 Шаг 07:

Миссия 7

Сопоставьте маркеры Git с их назначением.

Подсказка: Представьте их как рамки начала и конца спора.

Ответ:

- <<<<<< **HEAD** — Начало текущей версии
- ===== — Разделение версий
- >>>>>> **branch** — Конец чужой версии

🔗 Шаг 08:

Миссия 8

DevOps Оля использует инструменты для разрешения конфликтов. Соотнесите инструмент и его тип.

Подсказка: Некоторые встроены в IDE, а другие — консольные.

Ответ:

- **VS Code Merge Tool** — Графический редактор
- **vimdiff** — Терминальный инструмент
- **P4Merge** — Внешний визуальный тул

🔗 Шаг 09:

Миссия 9

Сопоставьте команды с их назначением.

Подсказка: Одна отменяет, другая принимает.

Ответ:

- **git merge --abort** — Отменить слияние
- **git add** — Подтвердить решение конфликта
- **git commit** — Завершить процесс разрешения

🔗 Шаг 10:

Миссия 10

Программист Саша использует стратегии merge. Соотнесите стратегию с её описанием.

Подсказка: Fast-forward — как перемотка, ours — как «я прав!».

Ответ:

- **Fast-forward** — Перемотка без создания нового коммита
- **Recursive** — Стандартный merge с анализом изменений
- **Ours** — Принять свою версию и игнорировать чужую

🔗 Шаг 11:

Миссия 11

Сопоставьте тип конфликта и пример.

Подсказка: Изменили одну и ту же строку — это одно, удалили — другое.

Ответ:

- **Text conflict** — Обе ветки меняют одну строку кода
- **Delete/Modify** — Одна ветка удаляет файл, другая редактирует
- **Add/Add** — Обе ветки добавляют файл с одинаковым именем

🔗 Шаг 12:

Миссия 12

Расставьте шаги разрешения конфликта.

Подсказка: Сначала изучить, потом исправить, потом подтвердить.

Ответ:

- Открыть файл и изучить конфликт
- Исправить нужный вариант
- git add
- git commit

🔗 Шаг 13:

Миссия 13

Упорядочьте команды для ручного merge.

Подсказка: Слияние — фиксы — подтверждение.

Ответ:

- git merge feature
- Решить конфликт
- git add
- git commit

🔗 Шаг 14:

Миссия 14

Frontender Артём использует IDE для конфликтов. Упорядочьте его шаги.

Подсказка: Он выбирает версию, потом сохраняет.

Ответ:

- Открыть конфликт в IDE
- Выбрать нужную версию блока
- Удалить маркеры Git вручную, если остались
- Сохранить файл

🔗 Шаг 15:

Миссия 15

Backender Роман делает rebase и ловит конфликт. Что он делает?

Подсказка: Rebase — как merge, но с продолжением после commit.

Ответ:

- Решить конфликт вручную
- git add
- git rebase --continue
- Проверить результат

🔗 Шаг 16:

Миссия 16

Упорядочьте действия при конфликте в pull request.

Подсказка: Сперва локально, потом пуш.

Ответ:

- Склонировать ветку
- Сделать merge локально
- Решить конфликт и commit
- Запустить изменения

🔗 Шаг 17:

Миссия 17

Проект в продакшене! Срочно нужно решить конфликт быстрее всех. Расставь приоритетные действия.

Подсказка: Главное — не сломать прод.

Ответ:

- Сделать бэкап текущего состояния
- Создать временную ветку для фикса
- Решить конфликт в изолированной ветке
- Слить обратно после тестирования

Урок 2 - Конфликты при pull и push

Шаг 01:

Миссия 1

Разработчик Коля попытался выполнить git pull , но получил сообщение: Automatic merge failed; fix conflicts and then commit the result. Что ему нужно сделать в первую очередь?

Подсказка: Git не может объединить изменения сам — нужно вмешательство человека.

Ответ:

- **Открыть конфликтующие файлы и исправить конфликт вручную**
- Сразу выполнить git push --force
- Удалить локальный репозиторий
- Перезапустить компьютер

Шаг 02:

Миссия 2

Аня сделала изменения и пытается выполнить git push , но получила ошибку: rejected - non-fast-forward . Почему это произошло?

Подсказка: Если кто-то уже залил изменения до тебя — тебе нужно их учесть.

Ответ:

- **Потому что на сервере есть коммиты, которых нет локально**
- Потому что она не указала имя ветки
- GitHub временно недоступен
- Её коммит слишком большой

Шаг 03:

Миссия 3

Senior-разработчик Вова рекомендует перед каждым git push выполнять...

Подсказка: Нужно забрать изменения с сервера, чтобы не было сюрпризов.

Ответ:

- **git pull**
- git stash drop
- git status --verbose
- git branch -D main

Шаг 04:

Миссия 4

Маша увидела в файле конфликтные метки <<<<< HEAD и >>>>> origin/main . Что это за части?

Подсказка: Git показывает две версии одного фрагмента.

Ответ:

- **Это разделители между локальной и удалённой версией кода**
- Комментарии, оставленные предыдущим разработчиком
- Ошибки форматирования
- Остатки после компиляции

🔮 Шаг 05:

Миссия 5

DevOps-инженер Лена использовала `git merge --abort`. Зачем?

Подсказка: Иногда лучше начать с чистого листа.

Ответ:

- **Чтобы отменить начатое слияние с конфликтами**
- Чтобы удалить все локальные изменения навсегда
- Чтобы перезаписать историю коммитов
- Чтобы переключиться на другую ветку

🔮 Шаг 06:

Миссия 6

Frontender Серёжа хочет автоматически решать простые конфликты. Какую команду он может использовать?

Подсказка: Она пытается применить изменения без участия человека.

Ответ:

- **`git merge -X ours`**
- `git init --force`
- `git revert all`
- `git delete-conflicts`

🔮 Шаг 07:

Миссия 7

Саша изучает типы конфликтов в Git. Помогите ему сопоставить тип с описанием.

Подсказка: Конфликты бывают не только в коде.

Ответ:

- **Textual Conflict** — Оба разработчика изменили одну и ту же строку текста.
- **Add/Add Conflict** — Один и тот же файл добавлен в двух ветках независимо.
- **Delete/Modify Conflict** — Файл удалён в одной ветке, но изменён в другой.
- **Binary Conflict** — Файл слишком сложный для автоматического слияния.

🔮 Шаг 08:

Миссия 8

Backendер Петя сравнивает стратегии решения конфликтов. Найди соответствия.

Подсказка: Иногда выбираем свою версию, иногда — чужую.

Ответ:

- **Принять текущую версию (ours)** — Сохранить изменения из локальной ветки.

- **Принять входящую версию (theirs)** — Взять изменения с удалённой ветки.
- **Оставить обе версии** — Вставить обе части кода в файл:
- **Отложить решение** — Сохранить конфликт для ручного разбора позже.

🔗 Шаг 09:

Миссия 9

DevOps Катя настроила хуки. Сопоставь действия с соответствующими hook-скриптами.

Подсказка: Конфликты можно ловить заранее.

Ответ:

- **pre-commit** — Запускается перед созданием коммита.
- **pre-push** — Запускается перед отправкой на сервер.
- **post-merge** — Срабатывает после объединения веток.
- **prepare-commit-msg** — Добавляет шаблон сообщения коммита.

🔗 Шаг 10:

Миссия 10

Junior-разработчик Федя путает команды. Помоги ему разобраться.

Подсказка: Все команды связаны с конфликтами.

Ответ:

- **git pull --rebase** — Подтягивает историю и переписывает локальные коммиты.
- **git fetch** — Загружает изменения без слияния.
- **git merge** — Объединяет изменения двух веток.
- **git rebase --abort** — Отменяет начатую перебазировку.

🔗 Шаг 11:

Миссия 11

Team Lead Миша разбирает причины конфликтов. Что с чем связано?

Подсказка: Один конфликт — одна причина.

Ответ:

- **Отсутствие pull перед push** — Удалённая история ушла вперёд.
- **Работа в одной ветке всем отделом** — Частые пересечения изменений.
- **Долгоживущие feature-ветки** — Сильно расходится история.
- **Отсутствие код-ревью** — Незамеченные конфликтующие участки.

🔗 Шаг 12:

Миссия 12

Архитектор Женя классифицирует инструменты. Найди связь.

Подсказка: Всё связано с визуальной помощью при конфликтах.

Ответ:

- **VS Code Merge Editor** — Показывает три панели с вариантами.
- **GitKraken** — Графическое управление ветками и конфликтами.
- **Meld** — Визуальное сравнение и слияние файлов.
- **KDiff3** — Показывает различия в трёх версиях файлов.

🔮 Шаг 13:

Миссия 13

Расположи шаги ручного решения конфликта:

Подсказка: Сначала исправляем, потом фиксируем.

Ответ:

- Открыть конфликтующий файл
- Удалить конфликтные метки и выбрать нужный код
- Сохранить файл
- Выполнить git add и git commit

🔮 Шаг 14:

Миссия 14

Упорядочите шаги безопасного push при командной работе:

Подсказка: Помни: сначала получаем, потом отправляем.

Ответ:

- Сделать git pull
- Решить возможные конфликты
- Выполнить git push
- Проверить статус git status

🔮 Шаг 15:

Миссия 15

Как действовать при конфликте в git rebase? Расположи шаги:

Подсказка: Rebase строгий — его нужно завершить вручную.

Ответ:

- Открыть файл и исправить конфликт
- Выполнить git add
- Продолжить слияние через git rebase --continue
- Проверить историю через git log

🔮 Шаг 16:

Миссия 16

От Junior до Senior: порядок мыслей при виде конфликта:

Подсказка: Не паникуем, анализируем.

Ответ:

- Прочитать конфликтные участки
- Понять, что хотел сделать второй разработчик
- Принять взвешенное решение
- Проверить, не ломает ли код логику

🔮 Шаг 17:

Миссия 17

Как избежать конфликтов в будущем? Расставь практики по приоритету:

Подсказка: Меньше веток — меньше хаоса.

Ответ:

- Делать короткие feature-ветки
- Чаще делать pull
- Обсуждать изменения с командой заранее
- Писать понятные сообщения коммитов

Урок 3 - Практика симуляция конфликта

Шаг 01:

Миссия 1

Аня и Богдан одновременно изменили файл README.md в одной строке. Что произойдёт при попытке слияния?

Подсказка: Git не умеет читать мысли — ему нужно, чтобы строки были разными.

Ответ:

- **Возникнет конфликт слияния**
- Git автоматически выберет версию Ани
- Файл удалится
- Слияние пройдёт без проблем

Шаг 02:

Миссия 2

Разработчик Костя заметил конфликт после git merge . Что он должен сделать в первую очередь?

Подсказка: Паниковать — плохой вариант.

Ответ:

- **Открыть файл с конфликтом и изучить различия**
- Сразу выполнить git add .
- Удалить конфликтующий файл
- Запустить компьютер заново

Шаг 03:

Миссия 3

Катя решила отменить конфликтное слияние. Какой командой она может прервать процесс merge?

Подсказка: Это как "отмена", но для Git.

Ответ:

- **git merge --abort**
- git cancel merge
- git stop
- git revert .

Шаг 04:

Миссия 4

Миша вручную исправил конфликт и сохранил файл. Что он должен сделать дальше?

Подсказка: Git не читает мысли — ему нужно явно сказать, что конфликт решён.

Ответ:

- **Выполнить git add для исправленного файла**
- Сразу запустить в origin
- Перезагрузить IDE
- **Ничего не делать** — Git сам поймёт

🔮 Шаг 05:

Миссия 5

Саша использует git pull и получает конфликт. Какой внутренний процесс стоит за pull ?

Подсказка: Это два шага в одном.

Ответ:

- **git fetch + git merge**
- git clone + git merge
- git add + git push
- git rebase + git reset

🔮 Шаг 06:

Миссия 6

Юля случайно оставила конфликтные маркеры <<<<<< HEAD в коде и закоммитила. Как лучше всего это исправить?

Подсказка: Историю можно переписать.

Ответ:

- **Использовать git reset --soft или git revert**
- Сделать ещё один коммит с извинениями
- Удалить репозиторий
- **Оставить как есть** — вдруг никто не заметит

🔮 Шаг 07:

Миссия 7

Сопоставьте команды с их назначением при решении конфликтов.

Подсказка: Одни команды отменяют, другие продолжают.

Ответ:

- **git merge --abort** — Отменить текущее слияние
- **git add** — Пометить файл как решённый
- **git status** — Показать список конфликтующих файлов
- **git log** — Показать историю коммитов

🔮 Шаг 08:

Миссия 8

Женя оставил в файле конфликтные маркеры. Соотнесите маркер с его назначением.

Подсказка: Верх — твоя версия, низ — их версия.

Ответ:

- <<<<<< **HEAD** — Начало текущей версии

- ===== — Разделитель вариантов
- >>>>>> **other-branch** — Конец версии из другой ветки
- **Конфликтный блок** — Фрагмент, содержащий обе версии кода

🔗 Шаг 09:

Миссия 9

Сопоставьте тип конфликта с ситуацией.

Подсказка: Бывают текстовые и структурные.

Ответ:

- **Text conflict** — Изменены одни и те же строки
- **Delete/modify conflict** — Один удалил, другой изменил файл
- **Rename/rename conflict** — Файл переименован в двух ветках по-разному
- **Binary conflict** — Изменён файл изображения в обеих ветках

🔗 Шаг 10:

Миссия 10

Определи, что делать с конфликтом.

Подсказка: Иногда нужен компромисс, иногда — победитель.

Ответ:

- **Принять обе версии** — Сохранить обе строки как список
- **Принять текущую версию** — Удалить альтернативный блок
- **Принять входящую версию** — Удалить текущий блок
- **Написать новую версию** — Переписать участок вручную

🔗 Шаг 11:

Миссия 11

Сопоставь инструмент с его задачей.

Подсказка: IDE тоже умеют решать конфликты.

Ответ:

- **VS Code Merge Editor** — Визуальное решение конфликтов
- **git diff** — Показ различий между версиями
- **meld** — Графический дифф инструмент
- **git rebase** — Перепроигрывание коммитов поверх новой базы

🔗 Шаг 12:

Миссия 12

Соотнеси ошибку с последствиями.

Подсказка: Оставил маркеры — получишь хаос.

Ответ:

- **Не удалить конфликтные маркеры** — Сломанный код в продакшене
- **Закоммитить нерешённый конфликт** — Коллеги не смогут продолжать работу
- **Забыть сделать git add** — Merge не завершится
- **Случайно принять обе версии** — Дублирование логики

🔧 Шаг 13:

Миссия 13

Расположи действия при решении конфликта в правильном порядке.

Подсказка: Сначала анализ, потом фиксация.

Ответ:

- Открыть конфликтующий файл
- Выбрать нужные изменения
- Удалить маркеры
- `git add` и `git commit`

🔧 Шаг 14:

Миссия 14

Как происходит автоматическое слияние?

Подсказка: Git старается быть умным, но не всегда получается.

Ответ:

- Git сравнивает строки по блокам
- **Если строки разные** — создаёт конфликт
- **Если строки одинаковые** — объединяет
- Показывает результат пользователю

🔧 Шаг 15:

Миссия 15

Последовательность команд для отмены merge после ошибки.

Подсказка: Выход через аварийный люк.

Ответ:

- `git status`
- `git merge --abort`
- Проверить директорию
- Сделать новую попытку

🔧 Шаг 16:

Миссия 16

Как действует команда `git rebase` при конфликте?

Подсказка: Это merge наоборот.

Ответ:

- Останавливается на конфликте
- Требуется решения вручную
- `git add` после исправления
- `git rebase --continue`

🔧 Шаг 17:

Миссия 17

Шаги проверки после решения конфликта.

Подсказка: Лучше проверить дважды, чем чинить в продакшене.

Ответ:

- Запустить тесты
- Открыть приложение вручную
- Попросить коллегу сделать ревью
- Запустить в репозиторий

Урок 4 - Решение сложных конфликтов

Шаг 01:

Миссия 1

Лена случайно изменила один и тот же файл вместе с коллегой, и Git показал конфликт при merge. Что это значит?

Подсказка: Git не может решить за людей, какой вариант правильный.

Ответ:

- **Изменения в одном и том же месте не могут быть автоматически объединены**
- Git потерял один из файлов
- Репозиторий поврежден
- Merge нужно запускать ещё раз

Шаг 02:

Миссия 2

Саша увидел в файле маркеры <<<<<< и ||||| . Что он должен сделать?

Подсказка: Эти маркеры — не украшения, их нужно убрать.

Ответ:

- **Открыть файл и вручную выбрать нужную версию кода**
- Удалить репозиторий и клонировать заново
- Спросить у Git, какая версия лучше
- Игнорировать и продолжать работу

Шаг 03:

Миссия 3

Петя хочет отменить конфликтный merge и начать заново. Какую команду он может использовать?

Подсказка: Она откатывает текущее состояние merge.

Ответ:

- **git merge --abort**
- git rollback-now
- git clean-conflict
- git undo last

Шаг 04:

Миссия 4

Наташа использует VS Code и видит кнопки “Accept Current”, “Accept Incoming”. Что произойдёт при выборе “Accept Incoming”?

Подсказка: Incoming — это то, что пришло из другой ветки.

Ответ:

- **Останутся изменения из сливаемой (входящей) ветки**
- Оба варианта объединятся автоматически
- Git создаст копию файла
- Файл будет удалён

🔮 Шаг 05:

Миссия 5

Макс хочет заранее избежать конфликтов. Что из этого — хорошая практика?

Подсказка: Обновляйся чаще!

Ответ:

- **Чаще делать git pull перед началом работы**
- Писать весь код в одной большой ветке main
- Никогда не коммитить мелкие изменения
- Хранить весь код в одном файле

🔮 Шаг 06:

Миссия 6

Ира решила использовать rebase вместо merge, но столкнулась с конфликтом. Что нужно сделать после ручного разрешения конфликта?

Подсказка: Git ждёт подтверждения.

Ответ:

- **git add и затем git rebase --continue**
- git merge --abort
- git resolve-finish
- git push --force сразу

🔮 Шаг 07:

Миссия 7

Кирилл хочет сохранить оба варианта конфликтного кода, чтобы потом объединить вручную. Как он может это сделать?

Подсказка: Никто не запрещает копировать текст.

Ответ:

- **Скопировать оба варианта в разные части файла, удалить маркеры конфликта**
- Закоммитить конфликт как есть
- Спросить у Git, какая версия лучше
- Просто закрыть файл

🔮 Шаг 08:

Миссия 8

Сопоставьте команды с их назначением.

Подсказка: Каждая команда отвечает за свой этап работы с конфликтом.

Ответ:

- **git merge --abort** — Отменить текущее слияние с конфликтом
- **git add** — Подтвердить вручную решённый файл
- **git rebase --continue** — Продолжить перебазирование после конфликта
- **git status** — Посмотреть, какие файлы ещё содержат конфликты

🔮 Шаг 09:

Миссия 9

Свяжите тип конфликта с его описанием.

Подсказка: Обратите внимание, где именно возникло несогласие Git.

Ответ:

- **Conflict in file content** — Изменения сделаны в одной и той же строке файла
- **Conflict in file deletion** — Одна ветка удалила файл, другая — изменила его
- **Conflict in rename** — Файл был переименован по-разному в двух ветках
- **Conflict in permissions** — Права доступа к файлу различаются

🔮 Шаг 10:

Миссия 10

Соотнесите инструмент с его преимуществом при разрешении конфликтов.

Подсказка: Одни показывают визуально, другие — помогают автоматически.

Ответ:

- **VS Code Merge Editor** — Показывает различия в трёх панелях
- **git mergetool** — Открывает внешний инструмент для сравнения
- **Sourcetree** — Позволяет решать конфликты через интерфейс
- **IntelliJ IDEA** — Предлагает автоматические варианты слияния

🔮 Шаг 11:

Миссия 11

Соедините ошибочное действие с его последствием.

Подсказка: Конфликты наказывают за спешку.

Ответ:

- **Игнорировать маркеры <<<<<< и >>>>>>** — Оставить файл в невалидном состоянии
- **Сразу сделать git push после конфликта** — Отправить сломанный код в репозиторий
- **Удалить конфликтный файл полностью** — Потерять нужные данные
- **Не сделать git add после правок** — Git решит, что конфликт не исправлен

🔮 Шаг 12:

Миссия 12

Выберите правильную стратегию для каждой ситуации.

Подсказка: Иногда лучше rebase, иногда merge.

Ответ:

- **Хочется сохранить историю изменений в порядке** — Использовать rebase
- **Важна наглядность всех точек слияния** — Использовать merge

- **Нужно быстро обновить ветку feature** — Выполнить `git pull --rebase`
- **Требуется совместная работа с историей коммитов** — Сначала синхронизировать `main`

🔧 Шаг 13:

Миссия 13

Расставьте шаги разрешения конфликта при `merge`.

Подсказка: Начинается с проверки, заканчивается коммитом.

Ответ:

- Открыть `git status`
- Исправить конфликт в файле
- Сделать `git add`
- Выполнить `git commit`

🔧 Шаг 14:

Миссия 14

Упорядочьте шаги при `rebase`-конфликте.

Подсказка: Не забудьте про `rebase --continue`.

Ответ:

- Открыть файл с конфликтом
- Выбрать правильный код
- Сделать `git add`
- Выполнить `git rebase --continue`

🔧 Шаг 15:

Миссия 15

Действия Тимура при серьёзном конфликте в продакшене.

Подсказка: Главное — не навреди.

Ответ:

- Сделать резервную ветку
- Проанализировать конфликт
- Создать `hotfix`-ветку
- Аккуратно перенести нужные изменения

🔧 Шаг 16:

Миссия 16

Последовательность при использовании внешнего `mergetool`.

Подсказка: Инструмент лишь помогает, но решает человек.

Ответ:

- Запустить `git mergetool`
- Выбрать нужные изменения в интерфейсе
- Сохранить файл
- Сделать `git add`

Шаг 17:

Миссия 17

Финальная битва! Расставьте шаги предотвращения конфликтов в команде.

Подсказка: Сначала договорённости, потом автоматизация.

Ответ:

- Создать git-flow соглашение
- Внедрить code review
- Настроить CI с проверкой конфликтов
- Обучить команду разрешению конфликтов

Модуль 13 - Git hooks

Урок 1 - pre-commit, pre-push, post-merge

Шаг 01:

Миссия 1

Аня настраивает pre-commit , чтобы случайно не коммитить отладочные console.log . Какое действие должен выполнять такой хук?

Подсказка: Он должен проверить файлы ДО того, как они попадут в коммит.

Ответ:

- **Проверять содержимое файлов перед созданием коммита**
- Запускаться после выполнения git push
- Отправлять код-ревьюерам письмо
- Удалять все изменения автоматически

Шаг 02:

Миссия 2

Сергей случайно закоммитил огромный .mp4 -файл. Какой хук поможет предотвратить такие ошибки?

Подсказка: Он должен срабатывать ДО того, как изменения попадут в удалённый репозиторий.

Ответ:

- post-merge
- **pre-push**
- post-commit
- prepare-commit-msg

Шаг 03:

Миссия 3

Игорь устал от конфликтов после обновления ветки. Он хочет автоматически запускать npm install после git pull . Какой хук для этого подходит?

Подсказка: Этот хук срабатывает уже ПОСЛЕ того, как изменения объединены.

Ответ:

- pre-commit
- pre-push
- **post-merge**

- `commit-msg`

🔮 Шаг 04:

Миссия 4

Лена настроила `pre-commit`, но он игнорирует файлы. Что она могла забыть?

Подсказка: Скрипт должен быть исполняемым.

Ответ:

- **Выставить права на выполнение (`chmod +x`)**
- Переименовать хук в `hook-pre`
- Добавить комментарии внутрь
- Перезагрузить Git

🔮 Шаг 05:

Миссия 5

Марк хочет, чтобы `pre-push` запрещал пуш, если не пройдены тесты. Что должен вернуть скрипт при ошибке?

Подсказка: Git понимает только один тип ответа.

Ответ:

- **Ненулевой код завершения**
- Текст ошибки на английском
- Файл `log.txt`
- HTML-страницу

🔮 Шаг 06:

Миссия 6

Соотнеси хук и его момент срабатывания:

Подсказка: Один из них срабатывает ещё ДО отправки на сервер.

Ответ:

- **`pre-commit`** — Перед созданием коммита
- **`pre-push`** — Перед отправкой на удалённый репозиторий
- **`post-merge`** — После объединения веток
- **`commit-msg`** — При формировании сообщения коммита

🔮 Шаг 07:

Миссия 7

Программисты описали, какие проверки хотят. Укажи, каким хуком это можно реализовать.

Подсказка: Если надо запретить пуш — думай глобально!

Ответ:

- **Запретить пуш без обновления ветки** — `pre-push`
- **Запретить коммит без `format`** — `pre-commit`
- **Автообновление зависимостей после `pull`** — `post-merge`
- **Контроль описания коммита** — `commit-msg`

🔮 Шаг 08:

Миссия 8

Сопоставь языки скриптов с типичным использованием в Git hooks:

Подсказка: Не все любят Bash — иногда Python проще!

Ответ:

- **Bash** — Быстрые проверки файлов и прав доступа
- **Python** — Сложная логика с библиотеками
- **Node.js** — Проверка фронтенда и lint
- **Ruby** — Скрипты для внутренних тулов Rails

🔮 Шаг 09:

Миссия 9

Кто из команды за что отвечает? Сопоставь роли и хуки:

Подсказка: Безопасник следит, чтобы никто не пронёс «лишнего»!

Ответ:

- **DevOps** — post-merge: запуск обновлений
- **Backend-разработчик** — pre-commit: проверка форматирования
- **Security-инженер** — pre-push: запрет секретов
- **Team Lead** — commit-msg: стандарты описаний

🔮 Шаг 10:

Миссия 10

Сопоставь популярные инструменты с их назначением в pre-commit:

Подсказка: Не все знают про ShellCheck!

Ответ:

- **ESLint** — Проверка JavaScript-кода
- **ShellCheck** — Проверка bash-скриптов
- **Black** — Форматирование Python
- **Secretlint** — Поиск случайных паролей

🔮 Шаг 11:

Миссия 11

Расположи шаги подключения кастомного pre-commit скрипта:

Ответ:

- Сделать файл исполняемым
- Создать файл .git/hooks/pre-commit
- Написать в него нужный код
- Сохранить и протестировать коммит

Подсказка: Сначала файл, потом права!

- Создать файл .git/hooks/pre-commit
- Написать в него нужный код
- Сделать файл исполняемым
- Сохранить и протестировать коммит

🔮 Шаг 12:

Миссия 12

Упорядочь события при выполнении `git push` с ошибкой в `pre-push`:

Ответ:

- Git вызывает `pre-push`
- Скрипт возвращает ошибку
- Пуш отменяется
- Пользователь видит сообщение

Подсказка: Сначала вызов — потом последствия.

- Git вызывает `pre-push`
- Скрипт возвращает ошибку
- Пуш отменяется
- Пользователь видит сообщение

🔮 Шаг 13:

Миссия 13

Размести хуки по возрастанию «строгости» контроля:

Ответ:

- `commit-msg`
- `pre-commit`
- `pre-push`
- `post-merge`

Подсказка: Самый строгий — тот, кто ближе к удалённому серверу!

- `post-merge`
- `pre-commit`
- `commit-msg`
- `pre-push`

🔮 Шаг 14:

Миссия 14

Сергей описал, как работает `post-merge`. Упорядочь его рассказ:

Ответ:

- Скрипт запускается
- Изменения объединены
- Скрипт устанавливает зависимости
- Пользователь продолжает работу

Подсказка: Сначала Git, потом магия!

- Изменения объединены
- Скрипт запускается
- Скрипт устанавливает зависимости
- Пользователь продолжает работу

🔧 Шаг 15:

Миссия 15

Тебе нужно включить pre-commit в командной разработке. Расставь шаги:

Ответ:

- Добавить pre-commit в репозиторий
- Описать установку в README
- Синхронизировать с коллегами
- Настроить выполнение при клонировании

Подсказка: Документация — раньше хаоса!

- Добавить pre-commit в репозиторий
- Описать установку в README
- Настроить выполнение при клонировании
- Синхронизировать с коллегами

📖 Урок 2 - Настройка простых хуков

🔧 Шаг 01:

Миссия 1

Аня настроила свой первый репозиторий и внезапно поняла, что постоянно забывает писать осмысленные сообщения к коммитам. Какой Git hook может автоматически проверять формат commit message перед сохранением?

Подсказка: Представь строгого тимлида, который не пропускает коммиты с сообщением «123».

Ответ:

- **commit-msg**
- pre-push
- applypatch-msg
- post-merge

🔧 Шаг 02:

Миссия 2

Серёжа любит экспериментировать, но иногда нечаянно пушит временные файлы прямо в origin. Какой hook способен остановить отправку, если найдены запрещённые файлы?

Подсказка: Он срабатывает прямо перед тем, как ракета улетает в origin.

Ответ:

- post-commit
- **pre-push**
- prepare-commit-msg
- pre-rebase

🔧 Шаг 03:

Миссия 3

Катя хочет автоматически запускать линтер перед каждым коммитом, чтобы не позориться перед CI. Какой hook ей нужен?

Подсказка: Он такой: «Ты точно хочешь закоммитить ЭТО?»

Ответ:

- **pre-commit**
- update
- post-checkout
- post-receive

🔗 Шаг 04:

Миссия 4

Даня настроил pre-commit, но забыл сделать его исполняемым. В какой команде ему нужно добавить +x, чтобы всё заработало?

Подсказка: Это классика: «chmod ...»

Ответ:

- **chmod +x .git/hooks/pre-commit**
- chmod 777 pre-commit
- chmod -R +x .
- git hook enable pre-commit

🔗 Шаг 05:

Миссия 5

Федя хочет, чтобы после каждого merge в main автоматически выводилось сообщение «Не забудь задеплоить!». Какой hook для этого подходит?

Подсказка: Он срабатывает, когда ветки уже соединились в единое целое.

Ответ:

- pre-merge
- **post-merge**
- commit-msg
- pre-push

🔗 Шаг 06:

Миссия 6

Ира задумалась: если у неё есть локальный pre-commit, сработает ли он на сервере GitHub?

Подсказка: Представь, что сервер ничего не знает о твоём ноутбуке.

Ответ:

- **Нет, локальные хуки не передаются на сервер**
- Да, если включить настройку sync-hooks
- Да, только для ветки main
- Да, но только один раз

🔗 Шаг 07:

Миссия 7

🧠 Представьте команду, где каждый забыл, какой hook зачем. Помогите ребятам сопоставить названия с их описаниями.

Подсказка: Прочитай и представь, как будто каждый hook — это охранник с конкретной задачей.

Ответ:

- **pre-commit** — Запускается перед созданием коммита
- **commit-msg** — Проверяет текст сообщения коммита
- **post-merge** — Срабатывает после объединения веток
- **pre-push** — Выполняется перед отправкой в удалённый репозиторий

🔗 Шаг 08:

Миссия 8

👤 Тимлид Саша придумал собственные названия хуков. Сопоставь их с реальными.

Подсказка: «Смотритель push-а» явно занимается проверкой отправки.

Ответ:

- **Ворота перед коммитом** — pre-commit
- **Инспектор сообщения** — commit-msg
- **Смотритель push-а** — pre-push
- **Глашатай после merge** — post-merge

🔗 Шаг 09:

Миссия 9

🔗 Разработчики спорят: какой hook когда применяется? Разрули ситуацию!

Подсказка: Вспомни порядок действий: сначала пишем коммит, потом пушим.

Ответ:

- **Перед пушем** — pre-push
- **После merge** — post-merge
- **Перед коммитом** — pre-commit
- **Проверка сообщения коммита** — commit-msg

🔗 Шаг 10:

Миссия 10

✂ Представь хуки как фильтры. Сопоставь, что они могут делать.

Подсказка: pre — значит «не пушу, пока не проверю».

Ответ:

- **pre-commit** — Может отменить коммит при ошибке
- **commit-msg** — Может отклонить коммит с плохим текстом
- **pre-push** — Может остановить отправку
- **post-merge** — Может вывести напоминание после объединения

🔗 Шаг 11:

Миссия 11

👤 Какой hook поможет кому?

Подсказка: Коммит ≠ push.

Ответ:

- **Лера хочет проверять eslint** — pre-commit
- **Дима проверяет commit message** — commit-msg
- **Олег блокирует пуш без тестов** — pre-push

- **Юля напоминает про деплой** — post-merge

🔧 Шаг 12:

Миссия 12

🤖 Кто за что отвечает? Сопоставь роли хуков с их задачами.

Подсказка: commit-msg — самый придирчивый граммар-наци.

Ответ:

- **pre-commit** — Не пропускает неотформатированный код
- **commit-msg** — Проверяет шаблон сообщения
- **pre-push** — Останавливает при ошибках тестов
- **post-merge** — Печатает уведомление после merge

🔧 Шаг 13:

Миссия 13

Саша решил настроить pre-commit. Расставь шаги по порядку.

Подсказка: Сначала пишем файл, потом даём права.

Ответ:

- Создать файл .git/hooks/pre-commit
- Написать скрипт проверки
- Сделать его исполняемым через chmod +x
- Сделать коммит и проверить

🔧 Шаг 14:

Миссия 14

🔗 Как происходит срабатывание commit-msg hook? Упорядочь события.

Подсказка: Сообщение пишется ДО проверки.

Ответ:

- Пользователь вводит commit message
- Запускается commit-msg hook
- Hook проверяет сообщение
- Коммит завершается успешно или отклоняется

🔧 Шаг 15:

Миссия 15

⚙️ Хуки как охранники пропуска. Кто стоит первым, кто последним?

Подсказка: Сначала commit, потом push.

Ответ:

- pre-commit
- commit-msg
- pre-push
- post-merge

🔧 Шаг 16:

Миссия 16

📦 Выкатка через merge. Что происходит по шагам?

Подсказка: Merge → hook → напоминание → deploy.

Ответ:

- Слияние веток
- Запуск post-merge
- Вывод сообщения "Deploy now!"
- Деплой на сервер

💡 **Шаг 17:**

Миссия 17

Локальная проверка перед пушем. Расставь действия.

Подсказка: Push — это последняя ступень.

Ответ:

- Разработчик пишет код
- Запускается pre-commit
- Commit создаётся
- Запускается pre-push
- Push в удалённый репозиторий

📖 Урок 3 - Практика линтер через hook

💡 **Шаг 01:**

Миссия 1

Аня впервые настроила pre-commit hook с линтером. Но он не срабатывает при коммите. Что она забыла сделать?

Подсказка: Любой скрипт должен стать исполняемым, иначе Git его игнорирует.

Ответ:

- **Выдать файлу права на исполнение: `chmod +x .git/hooks/pre-commit`**
- Переустановить Git
- Создать новую ветку
- Добавить комментарий в начале файла

💡 **Шаг 02:**

Миссия 2

Борис устал от бесконечных пробелов в конце строк. Какой линтер чаще всего используют для автоудаления таких артефактов в JavaScript-проектах?

Подсказка: Этот линтер часто идёт в паре с Prettier.

Ответ:

- **ESLint**
- Flake8
- ShellCheck
- Checkstyle

💡 **Шаг 03:**

Миссия 3

Катя настроила hook, но он проверяет только файлы, уже находящиеся в репозитории. Как сделать, чтобы линтер анализировал только изменённые перед коммитом файлы?

Подсказка: Git предоставляет специальный аргумент для вывода только staged-файлов.

Ответ:

- **Использовать команду `git diff --cached --name-only`**
- Удалить hook и перезапустить Git
- Вызвать `git push` вместо `git commit`
- Сделать ребут компьютера

🔧 Шаг 04:

Миссия 4

Сергей настроил hook, но линтер слишком шумный и заваливает коммит псевдоошибками. Что правильное сделать?

Подсказка: Жёсткий линт — хорошо, но его можно настраивать.

Ответ:

- **Сконфигурировать правила линтера под проект**
- Полностью удалить линтер
- Запускать линтер только на продакшн-сервере
- Заменить Git на SVN

🔧 Шаг 05:

Миссия 5

DevOps-инженер Марат хочет добавить вывод "× Линтер не прошёл!" при ошибках. Как завершить hook с ошибкой, чтобы Git отменил коммит?

Подсказка: В Unix любой скрипт говорит об ошибке одним способом.

Ответ:

- **`exit 1`**
- `return "error"`
- `git abort`
- `stop now`

🔧 Шаг 06:

Миссия 6

Frontend-разработчик Вика использует Husky для управления hook'ами. Где обычно лежит конфигурация Husky в проекте?

Подсказка: Это почти всегда папка внутри репозитория.

Ответ:

- **`.husky/`**
- `.gitconfig/`
- `hooks/`
- `.vscode/`

🔧 Шаг 07:

Миссия 7

Сопоставь тип hook с его задачей.

Подсказка: Один из них срабатывает перед коммитом, другой — после пуша.

Ответ:

- **pre-commit** — Запускается перед фиксацией изменений.
- **pre-push** — Запускается перед отправкой кода на удалённый репозиторий.
- **commit-msg** — Проверяет содержимое сообщения коммита.
- **post-merge** — Срабатывает после слияния веток.

🔮 Шаг 08:

Миссия 8

Бэкендер Роман сортирует инструменты. Сопоставь линтер с языком.

Подсказка: Один из них строго следит за Python.

Ответ:

- **Flake8** — Python
- **ESLint** — JavaScript
- **Pylint** — Python (более строгий анализ)
- **ShellCheck** — Bash-скрипты

🔮 Шаг 09:

Миссия 9

DevOps-команда решает, какие ошибки ловить заранее.

Подсказка: Тип ошибки — тип проверки.

Ответ:

- **Стиль кода** — Несоблюдение форматирования.
- **SyntaxError** — Некорректный код, не выполняемый интерпретатором.
- **SecurityError** — Использование потенциально опасных конструкций.
- **NamingError** — Неподходящие названия переменных.

🔮 Шаг 10:

Миссия 10

Сопоставь фрагмент hook-скрипта с его действием.

Подсказка: Взгляни, какая команда что делает.

Ответ:

- **exit 0** — Завершить успешно.
- **exit 1** — Отклонить коммит.
- **git diff --cached** — Показать изменения в стейдже.
- **echo "Lint failed"** — Вывести сообщение в консоль.

🔮 Шаг 11:

Миссия 11

Света хочет разобраться, какие задачи должен выполнять линтер.

Подсказка: Не всё линтер делает одинаково.

Ответ:

- **Форматирование** — Приведение к единому стилю.
- **Анализ ошибок** — Выявление потенциальных проблем.

- **Фикс автоисправлений** — Использование опции вроде `--fix`.
- **Игнорирование файлов** — Пропуск по `.eslintignore` или аналогичному.

🔧 Шаг 12:

Миссия 12

Расположи этапы настройки линтера через `pre-commit hook` в правильном порядке:

Подсказка: Сначала ставим инструмент, только потом пишем `hook`.

Ответ:

- Установить линтер в проект.
- Создать файл `.git/hooks/pre-commit`.
- Добавить вызов линтера в `hook`.
- Выдать права на выполнение скрипта.

🔧 Шаг 13:

Миссия 13

Сергей чинит хаос. Расставь шаги обработки ошибки линтера:

Подсказка: Ошибка → исправление → повтор.

Ответ:

- Получить сообщение от линтера.
- Исправить код.
- Повторно закоммитить.
- Убедиться, что `hook` прошёл успешно.

🔧 Шаг 14:

Миссия 14

Frontend-команда пишет идеальный `workflow`. Расставь шаги проверки перед пушем:

Подсказка: Сначала всё локально, потом пуш.

Ответ:

- Запустить локальные тесты.
- Прогнать линтер.
- Сделать `commit`.
- Выполнить `push`.

🔧 Шаг 15:

Миссия 15

Расположи действия при внедрении Husky в проект:

Подсказка: `npm` — не просто для сборки.

Ответ:

- Установить Husky через `npm`.
- Создать папку `.husky`.
- Добавить файл `pre-commit`.
- Прописать команду линтера.

Шаг 16:

Миссия 16

DevOps внедряет CI-проверку. Расставь шаги:

Подсказка: Коммиты — не финал.

Ответ:

- Запустить linters на CI.
- Получить отчёт.
- Отклонить merge при ошибках.
- Разрешить merge при успехе.

Шаг 17:

Миссия 17

Финальный бой! Расставь последовательность полного цикла защиты от кривого кода:

Подсказка: От IDE до CI.

Ответ:

- Локальный линтер при сохранении файла.
- pre-commit hook с линтером.
- pre-push hook с тестами.
- CI-проверка на сервере.

Урок 4 - Автоматизация тестов через hook

Шаг 01:

Миссия 1

Маша настроила проект на GitHub и хочет, чтобы перед каждым коммитом автоматически запускались юнит-тесты. Какой hook ей стоит использовать?

Подсказка: Этот hook срабатывает до записи коммита.

Ответ:

- **pre-commit**
- post-merge
- pre-push
- applypatch-msg

Шаг 02:

Миссия 2

Петя случайно коммитит временные файлы .log. Какой hook поможет ему автоматически отменять такие коммиты?

Подсказка: Этот hook может прервать процесс до создания коммита.

Ответ:

- **pre-commit**
- post-commit
- post-checkout
- pre-rebase

Шаг 03:

Миссия 3

Саша работает в команде безопасников. Он хочет, чтобы каждый пуш блокировался, если в коде встречается слово "password". Какой hook ему поможет?

Подсказка: Этот hook отвечает за контроль перед отправкой в удалённый репозиторий.

Ответ:

- **pre-push**
- post-commit
- prepare-commit-msg
- post-merge

Шаг 04:

Миссия 4

Оля добавила hook, который автоматически форматирует Python-код с помощью Black. Но форматирование происходит уже после того, как коммит создан. Какой hook она, вероятно, использовала?

Подсказка: Этот hook срабатывает сразу после успешного коммита.

Ответ:

- pre-commit
- **post-commit**
- pre-merge
- commit-msg

Шаг 05:

Миссия 5

Андрей пишет commit message в стиле «Fix stuff», и тимлид уже плачет. Какой hook можно использовать, чтобы проверить формат сообщения коммита?

Подсказка: Этот hook работает с самим текстом сообщения.

Ответ:

- **commit-msg**
- post-commit
- post-rewrite
- applypatch-msg

Шаг 06:

Миссия 6

Команда настроила Git hook, который после слияния веток автоматически отправляет сообщение в Slack. Какой hook используется?

Подсказка: Он срабатывает после объединения веток.

Ответ:

- pre-merge
- pre-push
- **post-merge**
- pre-rebase

🔗 Шаг 07:

Миссия 7

Лена подключает разные hook'и и путается. Помогите ей сопоставить hook и действие.

Подсказка: Смотри, что происходит: commit, push или merge.

Ответ:

- **pre-commit** — Проверка кода перед созданием коммита
- **pre-push** — Проверка перед отправкой в удалённый репозиторий
- **post-merge** — Действие после объединения веток
- **commit-msg** — Валидация текста сообщения коммита

🔗 Шаг 08:

Миссия 8

Валера подключает pre-commit, но не знает, что в нём можно проверять. Сопоставь инструмент и его назначение внутри hook.

Подсказка: Подумай, какой инструмент форматирует, а какой ищет уязвимости.

Ответ:

- **flake8** — Проверка стиля и ошибок Python
- **black** — Автоформатирование кода
- **bandit** — Поиск потенциальных уязвимостей
- **pytest** — Запуск тестов

🔗 Шаг 09:

Миссия 9

Серёжа пишет bash-скрипт для hook'а. Помогите ему понять, какой статус что означает.

Подсказка: В Linux успешное выполнение — это не всегда 1.

Ответ:

- **exit 0** — Сигнал, что всё прошло успешно
- **exit 1** — Отменить текущую операцию
- **echo "Error"** — Вывести сообщение об ошибке
- **chmod +x hookfile** — Сделать файл исполняемым

🔗 Шаг 10:

Миссия 10

Алина хочет запускать тесты только если изменились файлы в директории /tests. Сопоставь команду и её назначение.

Подсказка: Вспомни git diff.

Ответ:

- **git diff --name-only** — Получить список изменённых файлов
- **grep "tests/"** — Отфильтровать тестовые файлы
- **pytest** — Запустить тестовый фреймворк
- **if [-n "\$FILES"]** — Проверить, есть ли изменения

🔗 Шаг 11:

Миссия 11

Паша устал писать хардкод для путей. Помоги ему сопоставить переменные Git hook'ов с их значением.

Подсказка: commit-msg получает аргумент с текстом.

Ответ:

- **\$1 в commit-msg** — Путь к файлу с сообщением коммита
- **\$PWD** — Текущая директория
- **\$0** — Имя текущего скрипта
- **\$?** — Код завершения предыдущей команды

🔗 Шаг 12:

Миссия 12

Ира настраивает разные hook'и под разные стадии разработки. Помоги понять, для чего каждый.

Подсказка: Один предотвращает коммит, другой — пуш, третий — проверяет сообщение.

Ответ:

- **pre-commit** — Блокирует неверный код до создания коммита
- **pre-push** — Останавливает отправку в удалённый репозиторий
- **commit-msg** — Валидация текста коммита
- **post-merge** — Автодействие после слияния веток

🔗 Шаг 13:

Миссия 13

Расположи шаги создания кастомного pre-commit hook.

Подсказка: Сначала файл, потом права, потом логика, потом проверка.

Ответ:

- Создать файл .git/hooks/pre-commit
- Выдать права на выполнение (chmod +x)
- Написать скрипт с проверками
- Сделать тестовый коммит

🔗 Шаг 14:

Миссия 14

Тимофей хочет, чтобы перед пушем запускались тесты. Расположи нужные шаги.

Подсказка: Надо написать скрипт, подключить его и проверить.

Ответ:

- Создать .git/hooks/pre-push файл
- Добавить запуск pytest в скрипт
- Выдать chmod +x
- Сделать git push для проверки

🔗 Шаг 15:

Миссия 15

Наташа пишет hook, который блокирует коммит без JIRA-номера. Расположи логику проверки.

Подсказка: Сначала читаем текст, потом ищем паттерн.

Ответ:

- Прочитать commit message из файла
- Проверить наличие JIRA-формата (например ABC-123)
- **Если нет** — вывести ошибку
- Завершить exit 1

 Шаг 16:

Миссия 16

Даня хочет, чтобы после merge автоматически запускалась сборка Docker-образа. Расположи шаги.

Подсказка: Hook только иницирует действие, сборка — задача скрипта.

Ответ:

- Создать .git/hooks/post-merge файл
- Добавить docker build в скрипт
- Выдать chmod +x
- Сделать merge для проверки

 Шаг 17:

Миссия 17

Финальный бой! Собери универсальный pipeline из действий при коммите.

Подсказка: Сначала анализ, потом форматирование, потом тесты.

Ответ:

- Запустить линтер (flake8)
- Запустить автоформатирование (black)
- Запустить юнит-тесты (pytest)
- **Если всё ок** — завершить exit 0

 Модуль 14 - Конфигурация и alias

 Урок 1 - Глобальная и локальная конфигурация

 Шаг 01:

Миссия 1

Разработчик Никита установил Git и хочет задать своё имя и email глобально. Какой файл будет изменён при этом?

Подсказка: Глобальные настройки хранятся в домашней директории пользователя.

Ответ:

- **~/.gitconfig**
- .git/config
- /etc/gitconfig
- config.yml

 Шаг 02:

Миссия 2

Марина настроила имя и email для одного конкретного проекта. Где Git сохранит эти данные?

Подсказка: Локальные настройки применяются только внутри папки репозитория.

Ответ:

- ~/.gitconfig
- **.git/config**
- /etc/gitconfig
- global.conf

Шаг 03:

Миссия 3

Инженер безопасности Антон хочет убедиться, что его глобальная конфигурация не содержит личных данных. Какую команду он должен выполнить для проверки?

Подсказка: Эта команда показывает все уровни конфигурации и их значения.

Ответ:

- **git config --list --show-origin**
- git show-config
- git info
- git global --print

Шаг 04:

Миссия 4

Юлия работает на общем сервере. Она хочет, чтобы настройки применялись для всех пользователей системы. Где Git будет хранить такую конфигурацию?

Подсказка: Этот уровень конфигурации расположен в системной папке.

Ответ:

- .git/config
- **/etc/gitconfig**
- ~/.gitconfig
- config/global

Шаг 05:

Миссия 5

Фрилансер Дима настраивает Git на новом ноутбуке. Он хочет проверить, какое имя установлено для коммитов в текущем проекте. Какую команду нужно выполнить?

Подсказка: Можно запросить конкретный параметр у Git с помощью --get .

Ответ:

- **git config user.name**
- git show user.name
- git info user.name
- git check name

Шаг 06:

Миссия 6

Сопоставьте уровни конфигурации Git с их областью действия.

Подсказка: Чем ближе к проекту, тем выше приоритет настроек.

Ответ:

- **Системная конфигурация** — Применяется ко всем пользователям на компьютере.
- **Глобальная конфигурация** — Относится к конкретному пользователю.
- **Локальная конфигурация** — Действует только в одном репозитории.
- **Приоритет настроек** — Локальная > Глобальная > Системная.

Шаг 07:

Миссия 7

Свяжите команды Git с их назначением при работе с конфигурацией.

Подсказка: Обратите внимание на флаги `--global` и `--list`.

Ответ:

- **git config --list** — Показать все настройки.
- **git config --global user.name** — Установить имя пользователя глобально.
- **git config --local user.email** — Установить email для текущего проекта.
- **git config --system core.editor** — Изменить редактор для всей системы.

Шаг 08:

Миссия 8

Алиса хочет быстро проверить, где именно Git хранит каждую настройку. Сопоставьте команду с её результатом.

Подсказка: Вспомните опцию `--show-origin`.

Ответ:

- **git config --list --show-origin** — Показывает путь к каждому файлу конфигурации.
- **git config --global --edit** — Открывает глобальный файл настроек.
- **git config --system --list** — Показывает только системные настройки.
- **git config --local --edit** — Редактирует настройки текущего проекта.

Шаг 09:

Миссия 9

Николай случайно переопределил глобальные настройки локальными. Сопоставьте понятия с тем, что происходит при конфликте конфигураций.

Подсказка: Уровни настроек накладываются друг на друга, и нижний имеет приоритет.

Ответ:

- **Локальные настройки** — Переопределяют глобальные и системные.
- **Глобальные настройки** — Могут быть изменены локальными.
- **Системные настройки** — Базовые, на уровне всей машины.
- **Приоритет** — Локальная > Глобальная > Системная.

Шаг 10:

Миссия 10

Расположите шаги настройки Git для нового пользователя в правильном порядке.

Подсказка: Сначала установка, затем глобальные данные и проверка.

Ответ:

- Установить Git на компьютер

- Настроить глобальное имя пользователя
- Настроить глобальный email
- Проверить конфигурацию командой `git config --list`

Шаг 11:

Миссия 11

Расположите действия при изменении локальных настроек проекта в верном порядке.

Подсказка: Сначала нужно перейти в нужный репозиторий.

Ответ:

- Перейти в папку проекта
- Изменить настройки командой `git config --local ...`
- Проверить результат через `git config --list`
- Сделать тестовый коммит, чтобы убедиться в корректности имени

Шаг 12:

Миссия 12

Расположите этапы устранения конфликта конфигурации между глобальными и локальными настройками.

Подсказка: Важно понять, где именно задано неверное значение.

Ответ:

- Проверить все настройки с `git config --list --show-origin`
- Найти конфликтующее значение
- Исправить локальную или глобальную конфигурацию
- Перепроверить результат

Шаг 13:

Миссия 13

Расположите шаги для полной переинициализации конфигурации Git.

Подсказка: Иногда проще начать с чистого листа.

Ответ:

- Удалить `~/.gitconfig`
- Создать новый глобальный файл настроек
- Добавить имя и email
- Проверить корректность через `git config --list`

Шаг 14:

Миссия 14

Расположите шаги настройки Git при работе на новом сервере для нескольких пользователей.

Подсказка: Администратор должен начать с системных настроек.

Ответ:

- Настроить системный `/etc/gitconfig`
- Добавить глобальные данные для каждого пользователя
- Создать локальные репозитории
- Проверить корректность всех уровней через `git config --list --show-origin`

Урок 2 - Настройка сокращений команд

Шаг 01:

Миссия 1

Маша-бэкендер устала писать `git status` по 100 раз в день. Какой alias поможет ей сократить команду?

Подсказка: В alias сначала пишут короткое имя, потом полную команду.

Ответ:

- **`git config --global alias.st status`**
- `git alias status=st`
- `git add --alias status st`
- `git status --as st`

Шаг 02:

Миссия 2

Фронтендер Паша решил сократить `git commit -m` до `git cm`. Как правильно?

Подсказка: Кавычки вокруг команды обязательны.

Ответ:

- **`git config --global alias.cm "commit -m"`**
- `git config alias.cm commit -m`
- `git alias cm commit -m`
- `cm = git commit -m`

Шаг 03:

Миссия 3

DevOps-инженер Лена хочет создать alias `git last` для показа последнего коммита. Какой вариант верный?

Подсказка: Используйте `log -1`.

Ответ:

- **`git config --global alias.last "log -1"`**
- `git alias log last -1`
- `git config alias.last log 1`
- `git log -1 = last`

Шаг 04:

Миссия 4

Безопасник Слава хочет чтобы `git lg` показывал красивые логи с графами. Как настроить?

Подсказка: Команда начинается с `log --oneline`.

Ответ:

- **`git config --global alias.lg "log --oneline --graph --decorate"`**
- `git alias lg graph`
- `git config lg "log graph"`
- `git log --lg --pretty`

💡 Шаг 05:

Миссия 5

Фрилансер Олег хочет, чтобы alias работал только в одном проекте. Что он должен сделать?

Подсказка: Alias можно задавать и без флага --global .

Ответ:

- **Выполнить `git config alias.st status` без `--global`**
- Добавить alias в README
- Создать .aliases файл в проекте
- Использовать `git alias-local`

💡 Шаг 06:

Миссия 6

Катя случайно создала alias с опечаткой. Как удалить `git alias oops` ?

Подсказка: Используйте `--unset` .

Ответ:

- **`git config --global --unset alias.oops`**
- `git alias --remove oops`
- `git config alias.oops remove`
- `git delete alias oops`

💡 Шаг 07:

Миссия 7

Стажёр Дима нашёл в проекте несколько alias и пытается понять, что они делают. Соотнеси alias и результат.

Подсказка: Вспомни, какие команды логически связаны.

Ответ:

- **co** — checkout
- **br** — branch
- **ci** — commit
- **rb** — rebase

💡 Шаг 08:

Миссия 8

TeamLead Жанна хочет, чтобы все использовали одни и те же alias. Что к чему относится?

Подсказка: Подумай, в каких файлах хранится конфигурация.

Ответ:

- **`~/.gitconfig`** — Глобальные alias
- **`.git/config`** — Локальные alias
- **`--global`** — Применяется ко всем проектам
- **`--system`** — Настройка для всех пользователей

💡 Шаг 09:

Миссия 9

DevOps Серёга сравнивает alias разных команд. Сопоставь команду и результат.

Подсказка: Смотри на ключевые слова.

Ответ:

- **lg** — `log --oneline --graph`
- **st** — `status`
- **hist** — `log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short`
- **df** — `diff`

🧭 Шаг 10:

Миссия 10

Андрей хочет alias для работы с ветками. Соотнеси alias и смысл.

Подсказка: Обрати внимание на похожие сокращения.

Ответ:

- **nb** — `checkout -b`
- **del** — `branch -d`
- **pub** — `push -u origin`
- **pl** — `pull`

🧭 Шаг 11:

Миссия 11

Саша внедряет alias для поиска ошибок. Что к чему относится?

Подсказка: Вспомни команды поиска в истории.

Ответ:

- **grep** — Поиск по содержимому коммитов
- **blame** — Кто последний изменил строку
- **pick** — `cherry-pick`
- **fix** — `commit --amend`

🧭 Шаг 12:

Миссия 12

Настя создаёт alias для логирования. Установи связь.

Подсказка: Формат вывода помогает понять стиль.

Ответ:

- **short** — `log --oneline`
- **full** — `log --stat`
- **name** — `log --name-only`
- **patch** — `log -p`

🧭 Шаг 13:

Миссия 13

Расположи шаги по созданию alias в правильном порядке.

Подсказка: Сначала реши, что сокращаем.

Ответ:

- Определить длинную команду
- Выбрать короткое имя alias

- Выполнить `git config ...`
- Проверить новый `alias`

Шаг 14:

Миссия 14

Сисадмин Гриша автоматизирует dotfiles. Расставь шаги.

Подсказка: Сначала копируем, потом применяем.

Ответ:

- Создать `.gitconfig` в репозитории dotfiles
- Скопировать его в домашнюю директорию
- Добавить `alias` в `.gitconfig`
- Проверить работу `alias`

Шаг 15:

Миссия 15

Расставь шаги проверки, что `alias` работает в другом проекте.

Подсказка: Важно сменить директорию.

Ответ:

- Перейти в другой проект
- Выполнить `alias`-команду
- Сравнить результат с оригинальной командой
- При необходимости исправить `alias`

Шаг 16:

Миссия 16

Junior Вася случайно создал `alias`, который ломает `push`. Как он должен поступить?

Подсказка: Помни про `unset`.

Ответ:

- Обнаружить, что `push` не работает
- Проверить список `alias` командой `git config --global --get-regexp alias`
- Удалить проблемный `alias`
- Повторно выполнить `push`

Шаг 17:

Миссия 17

Гуру Петя делится `alias`'ами с командой. Расставь по порядку.

Подсказка: Экспорт — потом импорт.

Ответ:

- Экспортировать `.gitconfig`
- Отправить файл коллегам
- Коллеги копируют в домашнюю директорию
- Все запускают `alias` и радуются

Урок 3 - Полезные alias для повседневной работы

Шаг 01:

Миссия 1

Бэкендер Серёжа устал каждый раз набирать `git status` и хочет создать alias `st`. Какой командой он это сделает?

Подсказка: Вспомни: alias создаётся через `git config` с параметром `--global`.

Ответ:

- **`git config --global alias.st status`**
- `git alias st = status`
- `git set-alias st status`
- `git create alias st status`

Шаг 02:

Миссия 2

Тимлид Дима хочет удалить надоевший alias `br`. Какой командой?

Подсказка: Удаление alias — это всё та же команда `config`, только без значения.

Ответ:

- **`git config --global --unset alias.br`**
- `git alias --remove br`
- `git config --delete br`
- `git unset alias.br`

Шаг 03:

Миссия 3

Сопоставьте alias с тем, что он делает — помоги джуниору Вите разобраться.

Подсказка: Внимательно смотри на флаги команд!

Ответ:

- **`co`** — checkout
- **`ci`** — commit
- **`rb`** — rebase
- **`st`** — status

Шаг 04:

Миссия 4

Архитектор Андрей нашёл старые alias, но не помнит, что они делают. Восстанови смысл.

Подсказка: Смотри на ключевые флаги команд.

Ответ:

- **`lg`** — `log --oneline --graph`
- **`br`** — branch
- **`pl`** — `pull --rebase`
- **`df`** — diff

🔗 Шаг 05:

Миссия 5

DevOps Игорь решил назвать alias по-другому. Сопоставь новое имя со старым действием.

Подсказка: Похоже на сокращённые версии команд.

Ответ:

- **hist** — log --decorate --graph
- **unst** — reset HEAD~1
- **amend** — commit --amend
- **lgall** — log --all --oneline

🔗 Шаг 06:

Миссия 6

Аня нашла alias друга, но не поняла смысл. Помоги разобраться.

Подсказка: Подумай, какая команда отменяет изменения.

Ответ:

- **undo** — reset --soft HEAD~1
- **wipe** — checkout -- .
- **clean** — reset --hard
- **save** — stash

🔗 Шаг 07:

Миссия 7

Фуллстек Вова хочет унифицировать alias на всех проектах. Найди пары с одинаковым смыслом.

Подсказка: Сравни по ключевым аргументам.

Ответ:

- **rbm** — rebase main
- **rbdev** — rebase develop
- **rbf** — rebase feature
- **rbh** — rebase hotfix

🔗 Шаг 08:

Миссия 8

Системный админ Коля любит чистоту. Сопоставь alias с его целью.

Подсказка: Все связаны с удалением мусора.

Ответ:

- **prune** — remote prune origin
- **wipeall** — reset --hard HEAD
- **clr** — clean -fd
- **drop** — stash drop

🔗 Шаг 09:

Миссия 9

Расположи alias в порядке от самого безопасного к самому опасному.

Подсказка: Сначала чтение, потом изменение, потом удаление.

Ответ:

- st (status)
- lg (log --oneline --graph)
- pl (pull --rebase)
- clean (reset --hard)

Шаг 10:

Миссия 10

Стажёр Дима хочет автоматизировать процесс. Размести alias по логике рабочего дня.

Подсказка: Сначала обновление, потом работа, потом фиксация.

Ответ:

- pl (pull --rebase)
- co (checkout branch)
- ci (commit)
- ps (push)

Шаг 11:

Миссия 11

UX-дизайнер Саша экспериментирует с alias. Размести их от короткого к длинному.

Подсказка: Считаем количество символов в имени alias.

Ответ:

- st
- rbm
- hist
- wipeall

Шаг 12:

Миссия 12

Фрилансер Женя пишет alias для деплоя. Размести по степени автоматизации.

Подсказка: Чем больше действий делает alias — тем он автоматизированнее.

Ответ:

- ps (push)
- build (build project)
- deploy (build + push)
- autodeploy (build + push + restart)

Шаг 13:

Миссия 13

Младший разработчик Лена устала каждый раз писать длинную команду git status . Она решила создать alias. Какой вариант сделает это глобально?

Подсказка: Вспомни, как используется ключ --global в git config .

Ответ:

- **git config --global alias.st status**

- `git alias st status`
- `git config alias.st status`
- `git alias --global st status`

🔗 Шаг 14:

Миссия 14

DevOps-инженер Марина хочет создать alias для `git commit -am`. Какой вариант будет корректным?

Подсказка: Не забудь об экранировании кавычек внутри команды.

Ответ:

- **`git config --global alias.ca '!git commit -am'`**
- `git config alias.ca commit -am`
- `git alias ca commit -am`
- `git config --global alias.commit -am`

🔗 Шаг 15:

Миссия 15

Бэкендер Артём заметил, что его alias не работает. Он добавил строку `git config alias.co checkout`. Почему alias не применяется в других репозиториях?

Подсказка: Вспомни разницу между локальной и глобальной конфигурацией.

Ответ:

- **Alias записан только в локальный `.git/config`**
- Alias требует перезапуска Git
- Alias сохранён в системе, но не активирован
- Alias перезаписан из-за другого профиля

🔗 Шаг 16:

Миссия 16

Разработчица Нина хочет удалить ненужный alias. Как это сделать?

Подсказка: Используется та же команда, что и для добавления alias, но с ключом удаления.

Ответ:

- **`git config --global --unset alias.st`**
- `git alias --remove st`
- `git remove alias.st`
- `git config alias.st off`

🔗 Шаг 17:

Миссия 17

QA-инженер Саша хочет увидеть все свои alias в Git. Какой командой это можно сделать?

Подсказка: Вспомни, как вывести всю конфигурацию Git или поискать только alias.

Ответ:

- **`git config --get-regexp alias`**
- `git alias --list`
- `git show-alias`
- `git config --show alias`

🔗 Шаг 18:

Миссия 18

Сопоставьте alias с их функцией. Каждый из них облегчает повседневную работу в Git.

Подсказка: Подумай, какие команды часто повторяются при работе с ветками и коммитами.

Ответ:

- **co** — Переключение между ветками (checkout)
- **br** — Создание и просмотр веток (branch)
- **st** — Проверка состояния репозитория (status)
- **ci** — Создание коммита (commit)

🔗 Шаг 19:

Миссия 19

Фронтендер Катя использует alias для логов и откатов. Сопоставьте их с действиями.

Подсказка: Вспомни, как выглядят команды git log и git reset .

Ответ:

- **lg** — Просмотр истории коммитов в компактной форме
- **undo** — Отмена последнего коммита
- **last** — Показ последнего коммита
- **hist** — История с графом и датами

📖 Урок 4 - Практика ускорение работы через alias

🔗 Шаг 01:

Миссия 1

Андрей создал alias, но он работает только в одном проекте. Что он забыл указать?

Подсказка: Alias доступен глобально только при использовании специального флага.

Ответ:

- **Флаг --global**
- Флаг --local-only
- Кавычки вокруг alias
- Символ @ перед alias

🔗 Шаг 02:

Миссия 2

Петя хочет сделать alias undo , который откатывает последний коммит. Какую команду он должен использовать внутри alias?

Подсказка: Команда должна переместить указатель HEAD назад на один шаг.

Ответ:

- **reset --soft HEAD~1**
- revert HEAD
- clean --last
- drop HEAD

🔗 Шаг 03:

Миссия 3

Катя добавила `alias st = status` в файл `~/.gitconfig` вручную, но он не работает. В каком разделе должна быть эта строка?

Подсказка: В Git все `alias` находятся в одном специальном блоке.

Ответ:

- **[alias]**
- [core]
- [user]
- [config]

🔗 Шаг 04:

Миссия 4

Ассистент безопасности Серёжа ввёл кучу `alias` и запутался. Помоги ему вспомнить, что делает каждый!

Подсказка: Смотри на ключевые слова в командах.

Ответ:

- **last** — `log -1 --stat`
- **unstage** — `reset HEAD`
- **graph** — `log --oneline --decorate --graph`
- **amend** — `commit --amend`

🔗 Шаг 05:

Миссия 5

Даня — фанат чистоты и хочет идеальные названия для `alias`. Сопоставь команды и логичные имена.

Подсказка: Подумай, как бы ты сам назвал это действие.

Ответ:

- **pull --rebase** — `sync`
- **branch -a** — `branches`
- **stash apply** — `unstash`
- **diff --cached** — `stagediff`

🔗 Шаг 06:

Миссия 6

Бэкендер Илья хочет, чтобы `alias` были короткие, но понятные. Найди подходящие пары.

Подсказка: Смотри на первые буквы слов.

Ответ:

- **checkout -b** — `newb`
- **tag -l** — `tags`
- **remote -v** — `remotes`
- **merge --no-ff** — `mrg`

🔗 Шаг 07:

Миссия 7

Фронтендер Юля создала alias, но забыла, какой что делает. Помоги ей вспомнить!

Подсказка: Обрати внимание на поведение команд.

Ответ:

- **clean -fd** — wipe
- **fetch --all** — grab
- **reflog** — history
- **push --force** — override

🔗 Шаг 08:

Миссия 8

DevOps-инженер Митя хочет автоматизировать всё. Сопоставь его alias и смысл.

Подсказка: Представь, что ты объясняешь это новичку.

Ответ:

- **status -sb** — shortstat
- **commit -v** — verbose
- **add -p** — patchadd
- **blame** — whodid

🔗 Шаг 09:

Миссия 9

Тимлид Жора оформляет alias для rebase, чтобы исправить историю. Упорядочь процесс.

Подсказка: Вначале — выбор commit, затем редактирование, затем сохранение.

Ответ:

- git rebase -i HEAD~3
- Изменение коммитов
- Сохранение и выход

🔗 Шаг 10:

Миссия 10

Юра пишет alias для полного клонирования чужого проекта и перехода в его папку. Расставь шаги.

Подсказка: После клонирования ты оказываешься в новой директории?

Ответ:

- git clone URL
- cd project-folder
- git status

🔗 Шаг 11:

Миссия 11

Финальная битва! Создаём alias rescue, который отменяет локальные изменения и подтягивает свежие с удалённого. Упорядочь действия.

Подсказка: Сначала очищаем, потом обновляем.

Ответ:

- `git reset --hard`
- `git pull`
- `git clean -fd`

Шаг 12:

Миссия 12

Алексей заметил, что он часто вводит длинные команды Git вроде `git status` и `git checkout main`. Что он может сделать, чтобы ускорить работу?

Подсказка: Подумай о способе создания коротких команд, которые выполняют длинные.

Ответ:

- **Создать alias в конфигурации Git**
- Использовать более короткое имя пользователя
- Отключить проверки коммитов
- Удалить историю репозитория

Шаг 13:

Миссия 13

Марина хочет, чтобы команда `git st` выполняла `git status`. Какую настройку ей нужно добавить?

Подсказка: Вспомни, как указываются псевдонимы в Git через конфиг.

Ответ:

- **`git config --global alias.st status`**
- `git alias st=status`
- `git add alias st`
- `alias.st=git status`

Шаг 14:

Миссия 14

Денис создал alias, но Git не распознает его. Что он сделал не так?

Подсказка: Подумай, где Git ищет alias — в файле конфигурации, а не в терминале напрямую.

Ответ:

- **Он добавил alias не через `git config`, а напрямую в `bashrc`**
- Он назвал alias слишком коротко
- Он забыл установить Git
- Он создал alias без прав администратора

Шаг 15:

Миссия 15

Николай хочет, чтобы его alias `git hist` работал у всех на проекте. Где ему нужно его записать?

Подсказка: Разные уровни конфигурации Git влияют на разные окружения.

Ответ:

- **В файле `.git/config` внутри проекта**
- В `~/.bashrc`
- В `/etc/hosts`

- В ~/.gitignore

🔧 Шаг 16:

Миссия 16

Сопоставь уровни конфигурации Git с их назначением.

Подсказка: Чем выше уровень — тем шире охват.

Ответ:

- **--system** — Настройки для всех пользователей на машине
- **--global** — Настройки для конкретного пользователя
- **--local** — Настройки для текущего репозитория
- **.git/config** — Файл, где хранятся локальные alias проекта

🔧 Шаг 17:

Миссия 17

Сопоставь alias с выполняемой командой.

Подсказка: Заметь закономерность в именах — многие просто сокращают команду.

Ответ:

- **alias.st** — status
- **alias.co** — checkout
- **alias.br** — branch
- **alias.cm** — commit

🔧 Шаг 18:

Миссия 18

Сопоставь команды Git alias с тем, что они позволяют делать быстрее.

Подсказка: Alias — это не просто сокращение, но и способ объединять действия.

Ответ:

- **alias.lg** — Просмотр истории в графическом виде
- **alias.last** — Показ последнего коммита
- **alias.undo** — Отмена последнего коммита без потери изменений
- **alias.s** — Быстрый просмотр состояния

🔧 Шаг 19:

Миссия 19

Сопоставь команду с её конфигурационным файлом.

Подсказка: Вспомни, где Git хранит разные уровни настроек.

Ответ:

- **git config --system** — /etc/gitconfig
- **git config --global** — ~/.gitconfig
- **git config --local** — .git/config
- **git config --worktree** — .git/config.worktree

Модуль 15 - Git в CICD

Урок 1 - Git в пайплайнах CI

Шаг 01:

Миссия 1

DevOps-инженер Артём настраивает CI/CD пайплайн для проекта на GitLab. Он хочет, чтобы сборка запускалась только при изменениях в ветке `main`. Какой параметр в `.gitlab-ci.yml` ему нужно использовать?

Подсказка: В пайплайнах GitLab можно ограничить запуск по веткам с помощью специальных директив.

Ответ:

- **only:** `[main]`
- `trigger:` `[main]`
- `branch:` `main`
- `if:` `main`

Шаг 02:

Миссия 2

Анастасия подключает GitHub Actions для тестирования приложения. Она хочет, чтобы пайплайн выполнялся при каждом пуше. Какое событие в блоке `on:` ей нужно указать?

Подсказка: GitHub Actions запускается по событиям: `push`, `pull_request` и т.д.

Ответ:

- **push**
- `commit`
- `deploy`
- `start`

Шаг 03:

Миссия 3

Инженер Сергей заметил, что пайплайн GitHub Actions не запускается для новых веток. Что ему нужно добавить в настройках события `push`?

Подсказка: Пуш можно ограничивать конкретными ветками или шаблонами — например, с помощью `branches:`.

Ответ:

- **branches:** `['']`
- `runs-on:` `all`
- `trigger:` `any`
- `if:` `true`

Шаг 04:

Миссия 4

Backend-разработчик Миша хочет, чтобы сборка Docker-образа выполнялась только при изменениях в директории `backend/`. Что ему нужно указать в настройках GitHub Actions?

Подсказка: GitHub Actions поддерживает фильтрацию по путям файлов.

Ответ:

- **paths:** ['backend/']
- include: backend
- scope: backend
- on_change: backend

Шаг 05:

Миссия 5

DevOps-команда настроила Jenkins. Какой Git-плагин чаще всего используется для интеграции Git с Jenkins pipeline?

Подсказка: Этот плагин автоматически клонирует репозиторий и отслеживает коммиты.

Ответ:

- **Git Plugin**
- RepoSync
- CIConnector
- AutoCommit

Шаг 06:

Миссия 6

Сопоставьте элементы пайплайна с их назначением.

Подсказка: Каждый этап отвечает за конкретную часть процесса CI/CD — от сборки до развёртывания.

Ответ:

- **Build** — Сборка исходного кода и зависимостей
- **Test** — Проверка кода автоматическими тестами
- **Deploy** — Развёртывание приложения на сервере
- **Lint** — Проверка качества и стиля кода

Шаг 07:

Миссия 7

Сопоставьте типы CI/CD систем с их основными особенностями.

Подсказка: Подумайте, где хранится код конфигурации — в UI или в виде файлов.

Ответ:

- **Jenkins** — Открытая система, скрипты на Groovy, гибкость
- **GitLab CI** — Конфигурация в YAML-файле, интеграция с GitLab
- **GitHub Actions** — CI/CD прямо в GitHub, события push/pull_request
- **CircleCI** — Интеграция с GitHub, быстрые контейнерные билды

Шаг 08:

Миссия 8

Сопоставьте команды Git с их ролью в CI/CD пайплайне.

Подсказка: Некоторые команды отвечают за сбор состояния, другие — за отправку изменений.

Ответ:

- **git clone** — Клонирование репозитория для выполнения сборки

- **git fetch** — Получение новых изменений без слияния
- **git merge** — Объединение веток для сборки
- **git push** — Отправка успешных изменений в удалённый репозиторий

🔧 Шаг 09:

Миссия 9

DevOps-команда внедряет контроль версий для конфигураций пайплайнов. Сопоставьте понятие с его объяснением.

Подсказка: Вспомните, что инфраструктура как код — это не только про Terraform.

Ответ:

- **Pipeline as Code** — Хранение описания пайплайна в виде кода (YAML) в Git
- **Rollback** — Возврат к предыдущему успешному состоянию
- **Artifact** — Результат сборки — бинарный файл, контейнер, архив
- **Stage** — Логический этап пайплайна, например, build или test

🔧 Шаг 10:

Миссия 10

Артём настраивает GitLab Runner. Сопоставьте тип Runner'a с его особенностью.

Подсказка: Есть общий Runner и приватный для конкретного проекта.

Ответ:

- **Shared Runner** — Используется всеми проектами в GitLab
- **Specific Runner** — Назначен для конкретного проекта
- **Docker Executor** — Запускает задачи в контейнерах
- **Shell Executor** — Выполняет команды напрямую на сервере

🔧 Шаг 11:

Миссия 11

Сопоставьте ошибки в CI/CD с причинами, по которым они могут возникать.

Подсказка: Иногда причина не в коде, а в окружении.

Ответ:

- **Pipeline failed** — Ошибка теста или отсутствующий шаг
- **Permission denied** — Неправильный токен доступа или SSH-ключ
- **Job timeout** — Пайплайн выполняется слишком долго
- **Missing artifact** — Не найден файл после сборки

🔧 Шаг 12:

Миссия 12

Расположите этапы запуска пайплайна GitHub Actions в правильном порядке.

Подсказка: Пайплайн сначала получает код, затем выполняет задачи и проверяет результаты.

Ответ:

- Получение кода из репозитория
- Установка зависимостей
- Выполнение тестов
- Деплой успешной сборки

Шаг 13:

Миссия 13

Упорядочьте действия для настройки GitLab Runner.

Подсказка: Runner сначала устанавливается, затем регистрируется и активируется.

Ответ:

- Установка GitLab Runner
- Регистрация с токеном проекта
- Выбор типа Executor
- Запуск сервиса Runner

Шаг 14:

Миссия 14

Расположите шаги от написания коммита до автоматического деплоя в CI/CD.

Подсказка: Вспомните, что всё начинается с разработчика.

Ответ:

- Разработчик делает коммит и пушит изменения
- CI запускает сборку и тесты
- CD деплоит на staging
- CD деплоит на production после успешных проверок

Шаг 15:

Миссия 15

Расположите шаги для добавления badge статуса сборки в README GitLab.

Подсказка: Сначала найдите ссылку в настройках пайплайна, потом вставьте её в Markdown.

Ответ:

- Открыть настройки проекта и найти badge
- Скопировать Markdown-ссылку
- Добавить ссылку в README.md
- Закоммитить и запустить изменения

Шаг 16:

Миссия 16

Расположите шаги настройки кэширования зависимостей в GitHub Actions.

Подсказка: Важно указать путь и ключ, зависящий от файла зависимостей.

Ответ:

- Добавить шаг actions/cache
- Указать путь к зависимостям
- Задать ключ кэша с hashFiles
- Протестировать сборку

Шаг 17:

Миссия 17

Расположите этапы от настройки Git-проекта до работающего пайплайна CI/CD.

Подсказка: Начинается всё с кода, а заканчивается успешным деплоем.

Ответ:

- Создание репозитория в Git
- Добавление .gitlab-ci.yml или workflow-файла
- Настройка Runner или Actions
- Запуск пайплайна и проверка результатов

Урок 2 - Автоматизация через GitHub Actions

Шаг 01:

Миссия 1

Ты настроил GitHub Actions, но workflow не запускается. Что стоит проверить в первую очередь?

Подсказка: Иногда проблема не в коде, а в названии файла или его месте в репозитории.

Ответ:

- **Путь и имя файла:** `.github/workflows/main.yml`
- Имя ветки репозитория
- Формат README.md
- Файл .gitignore

Шаг 02:

Миссия 2

Разработчик Лена хочет, чтобы тесты автоматически запускались при каждом push в ветку main. Какой триггер нужно использовать в GitHub Actions?

Подсказка: Вспомни, какой event реагирует на обновления ветки.

Ответ:

- **on: push**
- on: workflow_dispatch
- on: pull_request
- on: schedule

Шаг 03:

Миссия 3

DevOps-инженер Игорь хочет запустить workflow вручную. Какой event следует использовать?

Подсказка: Этот триггер позволяет запускать пайплайн прямо из интерфейса GitHub.

Ответ:

- **on: workflow_dispatch**
- on: push
- on: issue_comment
- on: fork

Шаг 04:

Миссия 4

Саша хочет, чтобы workflow запускался каждый день в 9 утра. Какой синтаксис cron ему подойдёт?

Подсказка: cron использует формат: минуты, часы, день месяца, месяц, день недели.

Ответ:

- **schedule:** `cron('0 9 * * *')`
- `schedule: cron('9 0 * * *')`
- `schedule: cron('0 * 9 * *')`
- `schedule: cron('* 9 * 0 *')`

🔗 Шаг 05:

Миссия 5

Сопоставь элементы GitHub Actions с их назначением.

Подсказка: Каждый раздел отвечает за свой уровень структуры workflow.

Ответ:

- **jobs** — Определяют, какие задачи выполняются в пайплайне.
- **steps** — Шаги, выполняемые в рамках одной задачи.
- **runs-on** — Указывает тип виртуальной машины (например, ubuntu-latest).
- **name** — Имя workflow, отображаемое в интерфейсе.

🔗 Шаг 06:

Миссия 6

Ты настраиваешь автоматизацию тестов. Соотнеси тип действия с его описанием.

Подсказка: Одни шаги готовят окружение, другие — выполняют код.

Ответ:

- **checkout** — Клонировать репозиторий на виртуальную машину.
- **setup-node** — Устанавливает версию Node.js для сборки проекта.
- **run** — Выполняет произвольные shell-команды.
- **uses** — Подключает готовый экшен из репозитория GitHub.

🔗 Шаг 07:

Миссия 7

Твоя команда хочет уведомления о статусе сборки. Сопоставь метод уведомления и его назначение.

Подсказка: Некоторые методы требуют токены, другие работают напрямую через GitHub.

Ответ:

- **GitHub Status** — Показывает статус выполнения прямо в PR.
- **Slack Notification** — Отправляет сообщение в канал Slack.
- **Email Notification** — Посылает письмо участникам проекта.
- **Webhook** — Передаёт информацию о событии на внешний сервер.

🔗 Шаг 08:

Миссия 8

Настраиваем секреты для безопасности. Соотнеси объект с его предназначением.

Подсказка: Никогда не храни токены в коде — только в Secrets!

Ответ:

- **Secrets** — Хранят чувствительные данные вроде API-ключей.
- **Environment variables** — Хранят временные данные среды выполнения.

- **GitHub Token** — Используется для авторизации внутри workflow.
- **Artifacts** — Сохраняют результаты сборок и тестов.

🔧 Шаг 09:

Миссия 9

Сопоставь ситуацию и подходящее решение в GitHub Actions.

Подсказка: Иногда нужно просто выбрать правильный уровень автоматизации.

Ответ:

- **Нужно запустить CI вручную** — workflow_dispatch
- **Нужно проверять код при pull request** — pull_request
- **Запускать тесты по расписанию** — schedule
- **Запуск при каждом коммите** — push

🔧 Шаг 10:

Миссия 10

Расставь шаги создания простого workflow для CI в правильном порядке.

Подсказка: Вспомни, с чего начинается настройка: структура, имя, шаги, проверка.

Ответ:

- Создать файл .github/workflows/ci.yml
- Задать имя и события запуска (on: push)
- Определить виртуальную машину (runs-on)
- Добавить шаги для тестов и сборки

🔧 Шаг 11:

Миссия 11

Расположи этапы работы GitHub Actions от триггера до завершения пайплайна.

Подсказка: Сначала происходит событие, затем — выполнение, потом — завершение.

Ответ:

- Событие (push/pull_request)
- Запуск workflow
- Выполнение шагов (steps)
- Отчёт о результате

🔧 Шаг 12:

Миссия 12

Определи последовательность действий для публикации пакета на npm через Actions.

Подсказка: Сначала тесты, потом сборка и только после — публикация.

Ответ:

- Установка зависимостей
- Запуск тестов
- Сборка проекта
- Публикация на npm

🔧 Шаг 13:

Миссия 13

Расположи шаги деплоя приложения на сервер через GitHub Actions в правильном порядке.

Подсказка: Перед деплоем убедись, что всё собрано и протестировано.

Ответ:

- Сборка проекта
- Запуск тестов
- Подключение к серверу через SSH
- Копирование артефактов и перезапуск сервиса

🔧 Шаг 14:

Миссия 14

Выстраиваем CI/CD для Docker-контейнера. Расставь этапы пайплайна.

Подсказка: Сначала сборка, потом тестирование, потом публикация образа.

Ответ:

- Сборка Docker-образа
- Запуск тестов контейнера
- Публикация в Docker Hub
- Развёртывание на сервере

🔧 Шаг 15:

Миссия 15

Расположи шаги настройки кэширования зависимостей для ускорения CI.

Подсказка: Сначала проверяется кэш, потом скачиваются зависимости, затем обновляется кэш.

Ответ:

- Проверка наличия кэша
- Загрузка зависимостей, если кэш отсутствует
- Запуск сборки
- Сохранение нового кэша

🔧 Шаг 16:

Миссия 16

Ты создаёшь многоступенчатый пайплайн. Расставь стадии выполнения в логической последовательности.

Подсказка: Продумай зависимость между этапами: сборка → тест → деплой → уведомление.

Ответ:

- Сборка приложения
- Запуск тестов
- Деплой на сервер
- Уведомление команды о результате

Урок 3 - Практика простой pipeline

Шаг 01:

Миссия 1

Ты — разработчик в стартапе, где впервые решили автоматизировать тесты через GitLab CI. Что нужно создать в репозитории, чтобы pipeline вообще запустился?

Подсказка: Без этого файла GitLab просто не знает, какие этапы выполнять.

Ответ:

- **.gitlab-ci.yml**
- pipeline.config
- build.json
- ci.sh

Шаг 02:

Миссия 2

Твой коллега Алексей удивлён, что pipeline не стартует после пуша. Репозиторий подключён к GitLab, но ничего не происходит. Что, скорее всего, забыли?

Подсказка: GitLab CI не будет работать, если не включен этот ключевой параметр.

Ответ:

- **Включить GitLab Runner**
- Добавить README.md
- Создать новый branch
- Переименовать проект

Шаг 03:

Миссия 3

Команда решила, что первый шаг пайплайна должен просто выводить 'Hello, CI/CD!'. Какой блок должен быть в .gitlab-ci.yml?

Подсказка: Команда должна быть указана под ключом, который определяет список действий для задачи.

Ответ:

- **script**
- before_script
- after_script
- runs_on

Шаг 04:

Миссия 4

Петя заметил, что pipeline падает, потому что нет прав на установку зависимостей. Что нужно добавить в job, чтобы команды выполнялись от root?

Подсказка: Иногда нужно явно указать пользователя, под которым работает Runner.

Ответ:

- **Добавить тег image с образом, где есть нужные права**
- Выполнить `sudo apt install` внутри script
- Создать файл `run_as_root.txt`

- Отключить все проверки безопасности

🔧 Шаг 05:

Миссия 5

Pipeline работает, но теперь нужно добавить второй шаг — деплой на staging. Что важно указать, чтобы второй шаг выполнялся только после успешного первого?

Подсказка: Подумай, как задать зависимость между job'ами в YAML.

Ответ:

- **stage: deploy**
- next: true
- depends_on: build
- pipeline: chain

🔧 Шаг 06:

Миссия 6

Тебе нужно сопоставить понятия GitLab CI с их назначением.

Подсказка: Вспомни, что описывает, какие шаги есть, а что — конкретную задачу.

Ответ:

- **Job** — Конкретная задача в пайплайне
- **Stage** — Этап, объединяющий несколько задач
- **Pipeline** — Последовательность выполнения стадий
- **.gitlab-ci.yml** — Файл, описывающий структуру пайплайна

🔧 Шаг 07:

Миссия 7

Сопоставь команду с её назначением в контексте CI/CD.

Подсказка: Подумай, что из этого выполняется перед, во время и после задачи.

Ответ:

- **before_script** — Команды, выполняемые перед основной задачей
- **script** — Основные команды задачи
- **after_script** — Действия после выполнения job
- **cache** — Механизм сохранения артефактов между job'ами

🔧 Шаг 08:

Миссия 8

Сопоставь тип runner'а с его особенностью.

Подсказка: Один работает локально, другой — через облако GitLab.

Ответ:

- **Shared Runner** — Предоставляется GitLab и используется многими проектами
- **Specific Runner** — Настраивается под конкретный проект
- **Group Runner** — Общий для нескольких проектов группы
- **Docker Runner** — Выполняет job внутри контейнера Docker

🔗 Шаг 09:

Миссия 9

Сопоставь стадию пайплайна с её типичным содержимым.

Подсказка: Разные этапы отвечают за разные цели: проверка, сборка, деплой.

Ответ:

- **build** — Сборка приложения
- **test** — Проверка кода и тестирование
- **deploy** — Развёртывание приложения
- **cleanup** — Очистка временных файлов

🔗 Шаг 10:

Миссия 10

Сопоставь элемент YAML с его примером.

Подсказка: Заметь, что ключевые слова отличаются по контексту.

Ответ:

- **stages** — ['build', 'test', 'deploy']
- **image** — python:3.9
- **script** — pytest tests/
- **only** — ['main']

🔗 Шаг 11:

Миссия 11

Сопоставь возможную ошибку с её причиной.

Подсказка: Иногда дело в отступах, иногда — в конфигурации Runner.

Ответ:

- **Pipeline не запускается** — Отсутствует .gitlab-ci.yml
- **Job висит в ожидании** — Не назначен Runner
- **Ошибка YAML** — Неправильные отступы
- **Pipeline падает** — Ошибка в скрипте задачи

🔗 Шаг 12:

Миссия 12

Расположи шаги создания простого pipeline в правильном порядке.

Подсказка: Подумай: сначала структура, потом тест, потом запуск.

Ответ:

- Создать .gitlab-ci.yml
- Определить stages
- Добавить job с простым script
- Пушить изменения в main

🔗 Шаг 13:

Миссия 13

Определи порядок этапов типичного CI/CD pipeline.

Подсказка: Обычно проверка идёт до деплоя.

Ответ:

- build
- test
- deploy

Шаг 14:

Миссия 14

Расположи действия разработчика при отладке упавшего pipeline.

Подсказка: Сначала проверь логи, потом правь код.

Ответ:

- Открыть логи pipeline
- Найти упавший job
- Исправить ошибку
- Запустить pipeline снова

Шаг 15:

Миссия 15

Расположи шаги настройки нового GitLab Runner.

Подсказка: Нужно сначала зарегистрировать, потом проверить.

Ответ:

- Установить GitLab Runner
- Зарегистрировать Runner
- Добавить теги
- Проверить запуск job

Шаг 16:

Миссия 16

Расположи этапы улучшения pipeline по мере усложнения проекта.

Подсказка: Начни с автоматизации тестов и закончи полноценным CI/CD.

Ответ:

- Добавление автоматических тестов
- Добавление сборки Docker-образов
- Внедрение деплоя на staging
- Настройка деплоя на production

Урок 4 - Стратегии версионирования в CI

Шаг 01:

Миссия 1

Команда стартапа обсуждает, как нумеровать версии приложения в CI. Какой подход отражает стратегию семантического версионирования?

Подсказка: Подумайте о трёх цифрах: major.minor.patch.

Ответ:

- **Версия формата X.Y.Z, где X** — критические изменения, Y — новые функции, Z — исправления
- Нумерация только по количеству релизов: v1, v2, v3...
- Использование случайных чисел для каждой версии
- Указание только даты релиза

Шаг 02:

Миссия 2

Разработчик Лена внедряет CI для микросервиса. Какой тип изменения должен привести к увеличению только последней цифры (patch)?

Подсказка: Вспомните: patch означает исправление багов без добавления новых возможностей.

Ответ:

- **Исправление бага в функции без изменения интерфейса**
- Добавление новой функции
- Переписывание API и поломка обратной совместимости
- Изменение архитектуры базы данных

Шаг 03:

Миссия 3

Команда обсуждает CI/CD-пайплайн. При добавлении новой совместимой функциональности, что следует увеличить?

Подсказка: Подумайте: minor — это новые возможности без ломки старых функций.

Ответ:

- **Среднюю цифру (minor)**
- Старшую цифру (major)
- Последнюю цифру (patch)
- Оставить версию без изменений

Шаг 04:

Миссия 4

Бэкэнд-команда в CI случайно изменила API так, что старые клиенты перестали работать. Какое число версии нужно увеличить?

Подсказка: Major означает несовместимые изменения.

Ответ:

- **Major (первая цифра)**
- Minor (вторая цифра)
- Patch (третья цифра)
- Не изменять номер версии

Шаг 05:

Миссия 5

В CI/CD нужно автоматически формировать версию. Как лучше всего использовать Git-теги?

Подсказка: Теги помогают связать конкретный коммит с версией.

Ответ:

- **Создавать тег vX.Y.Z на стабильных релизах**
- Тегировать каждый коммит случайными номерами
- Не использовать теги, полагаясь на хэши коммитов
- Использовать теги только для hotfix-веток

Шаг 06:

Миссия 6

DevOps-команда хочет автоматически увеличивать patch при каждом успешном билде. Какая практика ближе к этому?

Подсказка: Это похоже на continuous delivery с автоинкрементом номера версии.

Ответ:

- **Автоматическое инкрементирование версии с помощью скриптов в CI**
- Ручное редактирование версии перед каждым коммитом
- Использование случайного генератора номеров
- Игнорирование версий и работа только с main-веткой

Шаг 07:

Миссия 7

Разработчики обсуждают, какие практики привязаны к CI/CD и версиям. Сопоставьте понятие и его описание:

Подсказка: Подумайте, где версии встречаются чаще всего в пайплайне.

Ответ:

- **Git-ter** — Метка, указывающая на конкретный коммит
- **Release branch** — Ветка для подготовки релиза
- **Hotfix** — Экстренное исправление с новым patch
- **Build number** — Автоматический номер сборки в CI

Шаг 08:

Миссия 8

Сопоставьте команду Git и её назначение в CI:

Подсказка: Вспомните команды: tag, describe, log, rev-parse.

Ответ:

- **git tag v1.0.0** — Создание тега версии
- **git describe --tags** — Показать ближайший тег для коммита
- **git log --oneline** — История коммитов кратко
- **git rev-parse HEAD** — Получить хэш текущего коммита

Шаг 09:

Миссия 9

Сопоставьте CI-инструмент и его особенности работы с версиями:

Подсказка: Jenkins, GitHub Actions, GitLab CI и CircleCI умеют по-разному работать с тегами.

Ответ:

- **Jenkins** — Использует плагины для управления версиями

- **GitHub Actions** — Поддерживает триггеры по тегам и релизам
- **GitLab CI** — Интеграция с GitLab Releases и Auto DevOps
- **CircleCI** — Гибкая конфигурация для workflow и версий

🔮 Шаг 10:

Миссия 10

Сопоставьте стратегию версионирования с её описанием:

Подсказка: Не все используют только semver, есть и другие подходы.

Ответ:

- **SemVer** — Версия X.Y.Z с чёткими правилами
- **Calendar Versioning** — Версии основаны на датах (например, 2025.10)
- **Commit Hash Versioning** — Использование хэша коммита как версии
- **Manual Naming** — Произвольные имена версий от команды

🔮 Шаг 11:

Миссия 11

Сопоставьте задачи DevOps и их связь с версиями:

Подсказка: В CI/CD версии часто нужны для артефактов и откатов.

Ответ:

- **Откат** — Возврат на стабильную версию
- **Деплой** — Выкат новой версии на сервер
- **Тестирование** — Проверка новой версии
- **Мониторинг** — Отслеживание работы версии в продакшене

🔮 Шаг 12:

Миссия 12

Расположите шаги семантического версионирования в CI/CD при выпуске нового релиза:

Подсказка: Вспомните: сначала коммит, потом тесты, потом тег и деплой.

Ответ:

- Сделать коммит изменений
- Прогнать тесты
- Создать тег версии
- Выполнить деплой новой версии

🔮 Шаг 13:

Миссия 13

Расположите действия DevOps-инженера при исправлении бага через hotfix:

Подсказка: Подумайте: баг-фикс начинается с ветки, заканчивается деплоем.

Ответ:

- Создать hotfix-ветку
- Исправить баг и закоммитить
- Создать новый patch-тег
- Задеплоить на прод

Шаг 14:

Миссия 14

Упорядочьте шаги автоинкремента версии в CI:

Подсказка: Логика всегда: билд → тест → bump → тег.

Ответ:

- Запустить билд
- Прогнать тесты
- Автообновить patch-номер
- Создать новый тег

Шаг 15:

Миссия 15

Упорядочьте шаги релиза с использованием GitFlow:

Подсказка: GitFlow включает release-ветку и merge в master и develop.

Ответ:

- Создать release-ветку
- Тестировать и фиксировать баги
- Смержить в master
- Смержить в develop

Шаг 16:

Миссия 16

Упорядочьте шаги при использовании Calendar Versioning в CI/CD:

Подсказка: Дата версии формируется автоматически после тестов и перед релизом.

Ответ:

- Собрать билд
- Прогнать тесты
- Сгенерировать версию по дате
- Задеплоить релиз

Модуль 16 - Практикум сложные конфликты

Урок 1 - Смоделировать конфликт веток

Шаг 01:

Миссия 1

Разработчик Костя создал новую ветку feature и сделал несколько изменений. Когда он попробовал слить её в main, Git сообщил о конфликте. Что это значит?

Подсказка: Подумай, что происходит, когда один и тот же участок файла изменён по-разному в двух ветках.

Ответ:

- Изменения в одной и той же части файла различаются в ветках
- Git не поддерживает слияние веток
- Файл был случайно удалён

- Ветки находятся в разных репозиториях

Шаг 02:

Миссия 2

Аня работает над веткой bugfix и замечает, что при merge Git сообщает о конфликте. Какой первый шаг стоит сделать?

Подсказка: Не пытайся сразу коммитить, сначала убедись, что именно изменилось.

Ответ:

- **Открыть файл и посмотреть конфликтующие изменения**
- Удалить все изменения из ветки bugfix
- Отменить merge навсегда
- Перезапустить Git

Шаг 03:

Миссия 3

Игорь заметил, что после разрешения конфликта и коммита в истории появилась новая запись. Что это за запись?

Подсказка: Git фиксирует факт объединения веток отдельным коммитом.

Ответ:

- **Merge commit**
- Rebase commit
- Detached HEAD commit
- Stash commit

Шаг 04:

Миссия 4

Света использует pull request и сталкивается с конфликтом в GitHub. Какое действие нужно сделать?

Подсказка: Даже в веб-интерфейсе GitHub придётся вручную выбрать правильные изменения.

Ответ:

- **Разрешить конфликт вручную и закоммитить изменения**
- Удалить pull request
- Создать новую ветку без конфликта
- Отключить проверки GitHub

Шаг 05:

Миссия 5

Сопоставьте команды Git с их назначением при работе с конфликтами.

Подсказка: Вспомни: merge объединяет ветки, rebase переписывает историю, а git status подскажет текущее состояние.

Ответ:

- **git status** — Показать файлы с конфликтами
- **git merge** — Попытаться объединить изменения из другой ветки
- **git rebase** — Перенести коммиты поверх другой ветки
- **git log** — Показать историю коммитов

🔗 Шаг 06:

Миссия 6

Сопоставьте символы в файле конфликта с их назначением.

Подсказка: Git вставляет специальные разделители, чтобы показать разные версии кода.

Ответ:

- <<<<<<< — Начало блока конфликтующих изменений
- ===== — Разделитель двух версий кода
- >>>>>>> — Конец блока конфликтующих изменений
- **HEAD** — Текущая ветка

🔗 Шаг 07:

Миссия 7

Сопоставьте стратегии разрешения конфликтов с их кратким описанием.

Подсказка: Иногда нужно объединять, иногда — выбирать одну сторону.

Ответ:

- **Accept Current Change** — Принять изменения из текущей ветки
- **Accept Incoming Change** — Принять изменения из вливаемой ветки
- **Accept Both Changes** — Оставить оба варианта
- **Compare Changes** — Сравнить и вручную выбрать итог

🔗 Шаг 08:

Миссия 8

Сопоставьте команды Git с их результатами при работе после конфликта.

Подсказка: После ручного редактирования обязательно нужно зафиксировать изменения.

Ответ:

- **git add** — Пометить файл как решённый
- **git commit** — Создать merge commit
- **git diff** — Показать различия между версиями
- **git checkout --ours** — Принять версию из текущей ветки

🔗 Шаг 09:

Миссия 9

Сопоставьте реальные сценарии с подходящей стратегией работы с ветками.

Подсказка: Иногда merge сохраняет историю, а rebase делает её линейной.

Ответ:

- **Хотим видеть все точки слияния** — Использовать git merge
- **Нужно линейное дерево истории** — Использовать git rebase
- **Конфликт при слиянии** — Открыть файлы и редактировать вручную
- **Хотим проверить текущее состояние** — Использовать git status

🔗 Шаг 10:

Миссия 10

Сопоставьте команду с её применением в реальной ситуации разрешения конфликта.

Подсказка: Вспомни про `ours` и `theirs` — они указывают, какую версию оставить.

Ответ:

- **`git checkout --ours`** — Принять изменения из текущей ветки
- **`git checkout --theirs`** — Принять изменения из вливаемой ветки
- **`git mergetool`** — Открыть визуальный инструмент для решения конфликтов
- **`git add`** — Пометить конфликт как решённый

🔗 Шаг 11:

Миссия 11

Расположите шаги разрешения конфликта в правильном порядке.

Подсказка: Сначала смотри на файлы, потом редактируй, затем фиксируй.

Ответ:

- Открыть файл и изучить конфликт
- Отредактировать конфликтующие участки
- Пометить файл как решённый (`git add`)
- Сделать коммит

🔗 Шаг 12:

Миссия 12

Упорядочите шаги при использовании `git rebase` с конфликтами.

Подсказка: `Rebase` переносит коммиты, и на каждом шаге может возникнуть конфликт.

Ответ:

- Запустить `git rebase`
- Разрешить конфликт вручную
- `git add` решённых файлов
- Продолжить процесс командой `git rebase --continue`

🔗 Шаг 13:

Миссия 13

Расположите шаги для разрешения конфликта в GitHub при `pull request`.

Подсказка: Даже в браузере принцип тот же — сначала редактируешь, потом коммитишь.

Ответ:

- Открыть конфликтующий файл в веб-интерфейсе
- Внести правки вручную
- Сохранить и пометить как решённый
- Закоммитить изменения

🔗 Шаг 14:

Миссия 14

Расположите шаги разрешения сложного конфликта в проекте с несколькими файлами.

Подсказка: Работай последовательно с каждым файлом.

Ответ:

- Просмотреть список конфликтующих файлов (`git status`)
- Открыть и отредактировать первый файл

- Открыть и отредактировать остальные файлы
- Закоммитить все изменения

Шаг 15:

Миссия 15

Упорядочите шаги при использовании git mergetool для конфликта.

Подсказка: Визуальные инструменты работают по тем же правилам: выбрать, исправить, сохранить.

Ответ:

- Запустить git mergetool
- Выбрать нужные изменения в визуальном редакторе
- Сохранить файл
- Закоммитить результат

Шаг 16:

Миссия 16

Расположите шаги при моделировании конфликта для обучения команды.

Подсказка: Чтобы создать конфликт, нужно редактировать один и тот же файл в разных ветках.

Ответ:

- Создать новую ветку
- Внести изменения в файл в обеих ветках
- Попробовать объединить ветки
- Разрешить конфликт вручную

Урок 2 - Решить конфликт и завершить merge

Шаг 01:

Миссия 1

У программиста Алексея возник конфликт при merge: файл index.js содержит разные изменения в двух ветках. Какой первый шаг нужно сделать?

Подсказка: Сначала нужно открыть файл и изучить разметку с конфликтами.

Ответ:

- **Открыть файл и найти конфликтные участки <<<< ===== >>>>**
- Сразу выполнить git commit без изменений
- Удалить конфликтные файлы из репозитория
- Откатить проект к предыдущему коммиту

Шаг 02:

Миссия 2

Катя видит в конфликте строки с маркерами <<<<<< , ===== и >>>>>> . Что они означают?

Подсказка: Это разделители между версиями кода из разных веток.

Ответ:

- **Начало, разделитель и конец конфликтного блока**
- Комментарии, оставленные автором кода

- Ошибки синтаксиса в JavaScript
- Специальные команды bash

🔗 Шаг 03:

Миссия 3

Олег решает, чью версию кода оставить. Что нужно сделать в файле, чтобы завершить merge?

Подсказка: Важно убрать маркеры конфликтов и оставить согласованную версию кода.

Ответ:

- **Удалить маркеры конфликтов и выбрать финальный вариант кода**
- Закомментировать конфликтные строки
- Оставить все варианты подряд
- Сделать stash и продолжить работу

🔗 Шаг 04:

Миссия 4

После ручного исправления конфликтов что нужно сделать следующим шагом?

Подсказка: Git должен понять, что файл теперь исправлен.

Ответ:

- **Выполнить git add для исправленных файлов**
- Сразу выполнить git push
- Удалить репозиторий и заново клонировать
- Сделать git log

🔗 Шаг 05:

Миссия 5

После git add, как завершить merge?

Подсказка: Финальная команда фиксирует результат.

Ответ:

- **Сделать git commit**
- Сделать git status
- Сделать git revert
- Сделать git tag

🔗 Шаг 06:

Миссия 6

Сопоставьте команды с их назначением при решении конфликтов merge:

Подсказка: Вспомните, какие действия выполняют команды add, commit, status и log.

Ответ:

- **git add** — Пометить файл как исправленный
- **git commit** — Зафиксировать изменения и завершить merge
- **git status** — Показать текущий статус, включая конфликты
- **git log** — Показать историю коммитов

Шаг 07:

Миссия 7

Иван исправил конфликты, но не уверен, всё ли готово. Сопоставьте команду проверки и результат:

Подсказка: После правок важно убедиться, что конфликтов больше нет.

Ответ:

- **git status** — Показывает, какие файлы ещё конфликтуют
- **git diff** — Показывает различия в коде до и после правок
- **git log --merge** — Показывает коммиты, участвующие в слиянии
- **git branch** — Показывает список веток

Шаг 08:

Миссия 8

В команде решили использовать стратегию merge tool. Сопоставьте инструмент и его назначение:

Подсказка: Эти программы помогают визуально объединять код.

Ответ:

- **vimdiff** — Работа с конфликтами прямо в терминале
- **meld** — Графическое сравнение и объединение файлов
- **kdifff3** — Объединение трёх версий файла
- **git mergetool** — Запуск установленного инструмента для решения конфликтов

Шаг 09:

Миссия 9

Сопоставьте ошибку при merge и её возможное решение:

Подсказка: Обратите внимание: не все проблемы решаются одинаково.

Ответ:

- **Конфликт в коде** — Вручную выбрать правильный вариант и удалить маркеры
- **Файл удалён в одной ветке** — Решить — оставить или восстановить
- **Бинарный файл изменён в обеих ветках** — Заменить нужной версией вручную
- **Много конфликтов по проекту** — Использовать mergetool для упрощения

Шаг 10:

Миссия 10

Андрей хочет откатить merge, так как исправил конфликт неправильно. Сопоставьте команду с её действием:

Подсказка: Вспомните reset и merge --abort.

Ответ:

- **git merge --abort** — Отменить процесс merge и вернуться в исходное состояние
- **git reset --hard** — Полностью откатить изменения в рабочем каталоге
- **git checkout --ours** — Оставить версию из текущей ветки
- **git checkout --theirs** — Оставить версию из вливаемой ветки

🔗 Шаг 11:

Миссия 11

Сопоставьте маркеры конфликта и их назначение:

Подсказка: Каждая часть маркеров имеет роль — начало, разделитель, конец.

Ответ:

- <<<<<<< **HEAD** — Начало изменений из текущей ветки
- ===== — Разделитель между версиями
- >>>>>>> **feature-branch** — Конец изменений из вливаемой ветки
- **Удаление маркеров** — Финальная правка после выбора версии

🔗 Шаг 12:

Миссия 12

Расставьте шаги решения конфликта merge в правильном порядке:

Подсказка: Сначала работа с кодом, потом фиксация.

Ответ:

- Открыть файл с конфликтом
- Удалить маркеры и выбрать правильный код
- Сделать git add
- Сделать git commit

🔗 Шаг 13:

Миссия 13

Выберите правильный порядок команд для отмены merge:

Подсказка: Нужна команда, которая прерывает процесс.

Ответ:

- git merge --abort
- git status
- git reset --hard
- git log

🔗 Шаг 14:

Миссия 14

В команде решили сохранить обе версии кода в разные функции. Расставьте действия:

Подсказка: Сначала нужно внести изменения, потом закоммитить.

Ответ:

- Создать новые функции для каждой версии
- Удалить маркеры конфликта
- Сделать git add
- Сделать git commit

🔗 Шаг 15:

Миссия 15

Определите порядок команд для запуска merge tool:

Подсказка: Сначала настройка, потом запуск.

Ответ:

- git config merge.tool meld
- git mergetool
- git add
- git commit

 **Шаг 16:**

Миссия 16

В процессе CI/CD пайплайна merge застрял из-за конфликта. Расставьте действия:

Подсказка: CI ждёт решения вручную.

Ответ:

- Зайти в проект локально
- Исправить конфликт
- Закоммитить изменения
- Сделать push

 **Шаг 17:**

Миссия 17

Финальное испытание: расположите шаги полного цикла merge с конфликтом и его решением:

Подсказка: Сначала merge, потом исправление, фиксация и push.

Ответ:

- git merge feature
- Исправить конфликты
- git add
- git commit
- git push

Урок 3 - Практика rebase с конфликтами

 **Шаг 01:**

Миссия 1

Анна делает git rebase main и сталкивается с конфликтом. Что Git делает в этот момент?

Подсказка: Вспомните: rebase перебрасывает коммиты один за другим поверх указанной ветки.

Ответ:

- **Приостанавливает процесс и предлагает решить конфликт вручную**
- Автоматически сливает изменения без проверки
- Отменяет все изменения в конфликтных файлах
- Удаляет конфликтующие коммиты

 **Шаг 02:**

Миссия 2

Дмитрий начал rebase и получил сообщение о конфликте. Какие действия он должен выполнить перед продолжением?

Подсказка: Нужно привести файлы в состояние, которое можно сохранить как рабочий коммит.

Ответ:

- **Открыть конфликтные файлы и исправить конфликты**
- **Выполнить `git add` для изменённых файлов**
- Сразу выполнить `git push`
- Удалить все конфликтующие файлы

Шаг 03:

Миссия 3

Ольга использует `git rebase` и хочет отменить текущий конфликтный rebase. Какой командой это можно сделать?

Подсказка: Есть команда, которая откатывает rebase полностью.

Ответ:

- **`git rebase --abort`**
- `git reset --hard`
- `git merge --abort`
- `git checkout --conflict`

Шаг 04:

Миссия 4

Иван выполнил rebase и решил конфликты. Какой командой он продолжит процесс?

Подсказка: После исправления и `git add` нужно возобновить процесс.

Ответ:

- **`git rebase --continue`**
- `git rebase --skip`
- `git commit --amend`
- `git merge --continue`

Шаг 05:

Миссия 5

Наталья делает интерактивный rebase (`git rebase -i`) и хочет пропустить один конфликтный коммит. Что ей нужно использовать?

Подсказка: Есть специальная опция, позволяющая игнорировать коммит.

Ответ:

- **`git rebase --skip`**
- `git commit --skip`
- `git reset --skip`
- `git rebase --omit`

Шаг 06:

Миссия 6

Сопоставьте команды Git с их назначением при rebase с конфликтами.

Подсказка: Каждая команда связана с определённым сценарием: отмена, пропуск или продолжение.

Ответ:

- **git rebase --continue** — Продолжить процесс rebase после разрешения конфликта
- **git rebase --abort** — Прервать процесс rebase и вернуть всё в исходное состояние
- **git rebase --skip** — Пропустить конфликтный коммит и продолжить процесс
- **git add** — Пометить файл как исправленный для продолжения rebase

🔮 Шаг 07:

Миссия 7

Сопоставьте сообщения Git с их значением во время rebase.

Подсказка: Эти сообщения подсказывают, что делать дальше.

Ответ:

- **CONFLICT (content)** — Конфликт содержимого в файле
- **You must edit all merge conflicts** — Необходимо вручную исправить все конфликты
- **Resolve all conflicts manually** — Git ожидает вмешательства программиста
- **Changes to be committed** — Список файлов, готовых к коммиту

🔮 Шаг 08:

Миссия 8

Сопоставьте сценарии с правильным действием при rebase.

Подсказка: Подумайте, в каких случаях нужно продолжить, пропустить или отменить.

Ответ:

- **Файл исправлен вручную** — Выполнить git add и git rebase --continue
- **Коммит не нужен в истории** — Использовать git rebase --skip
- **Rebase оказался неуместным** — Прервать процесс через git rebase --abort
- **Файл добавлен в индекс** — Git готов продолжить rebase

🔮 Шаг 09:

Миссия 9

Сопоставьте опции интерактивного rebase с их функцией.

Подсказка: Эти ключевые слова позволяют управлять коммитами при rebase.

Ответ:

- **pick** — Применить коммит как есть
- **reword** — Изменить сообщение коммита
- **edit** — Остановиться для изменения коммита
- **drop** — Удалить коммит из истории

🔮 Шаг 10:

Миссия 10

Сопоставьте тип конфликта с его описанием.

Подсказка: Обратите внимание: конфликты могут быть как в тексте файлов, так и в структуре репозитория.

Ответ:

- **Content conflict** — Разные изменения в одном и том же участке файла
- **Delete/modify conflict** — Один разработчик удалил файл, другой изменил

- **Add/add conflict** — Оба разработчика добавили файл с одинаковым именем
- **Rename/rename conflict** — Оба разработчика переименовали файл по-разному

🔗 Шаг 11:

Миссия 11

Сопоставьте ситуации с правильными командами Git для завершения rebase.

Подсказка: Важно правильно выбрать действие после решения конфликта.

Ответ:

- **Конфликты исправлены** — `git rebase --continue`
- **Коммит больше не нужен** — `git rebase --skip`
- **Возникла критическая ошибка** — `git rebase --abort`
- **Файлы добавлены в индекс** — `git add`

🔗 Шаг 12:

Миссия 12

Расположите шаги при решении конфликта в ходе rebase в правильном порядке.

Подсказка: Сначала исправляем, потом индексируем, затем продолжаем.

Ответ:

- Открыть конфликтный файл и исправить проблему
- Выполнить `git add` для исправленного файла
- Продолжить процесс через `git rebase --continue`
- Проверить изменения с помощью тестов

🔗 Шаг 13:

Миссия 13

Упорядочите действия для безопасной отмены rebase.

Подсказка: Есть команда, которая возвращает репозиторий в прежнее состояние.

Ответ:

- Понять, что rebase не нужен
- Выполнить `git rebase --abort`
- Проверить, что HEAD вернулся к прежнему состоянию
- Сообщить команде об отмене

🔗 Шаг 14:

Миссия 14

Расположите шаги интерактивного rebase при изменении истории коммитов.

Подсказка: Вспомните процесс: выбор опций, редактирование, сохранение.

Ответ:

- Выполнить `git rebase -i HEAD~3`
- Указать нужные действия (pick, reword, edit, drop)
- Сохранить и закрыть редактор
- Выполнить указанные изменения

🔮 Шаг 15:

Миссия 15

Расставьте действия при разрешении rename/rename конфликта в правильном порядке.

Подсказка: Нужно решить, какое имя оставить, и завершить процесс.

Ответ:

- Выбрать итоговое имя файла
- Переименовать файл вручную
- Выполнить git add
- Продолжить через git rebase --continue

🔮 Шаг 16:

Миссия 16

Расположите шаги при разрешении delete/modify конфликта.

Подсказка: Нужно решить: оставить файл или удалить.

Ответ:

- Понять, оставить ли изменения или удалить файл
- Внести решение вручную
- Выполнить git add
- Завершить процесс командой git rebase --continue

🔮 Шаг 17:

Миссия 17

Расположите общие шаги работы с rebase и конфликтами в правильном порядке.

Подсказка: Это финальное обобщение процесса rebase с конфликтами.

Ответ:

- Начать rebase
- Исправить конфликты в файлах
- Добавить исправленные файлы в индекс
- Продолжить процесс rebase
- Завершить процесс и протестировать код

📖 Урок 4 - Итоги лучший подход к конфликтам

🔮 Шаг 01:

Миссия 1

Андрей сделал merge и увидел конфликт в файле app.js . Какое первое действие нужно предпринять?

Подсказка: Сначала важно понять, где именно Git нашёл несовместимые изменения.

Ответ:

- Открыть файл и изучить конфликтные блоки с маркерами <<<<<<, =====, >>>>>>
- Сразу запустить команду git commit
- Удалить файл и добавить заново
- Откатить проект на предыдущий коммит

Шаг 02:

Миссия 2

Марина выполняет rebase и сталкивается с конфликтом в index.html . Что нужно сделать после ручного разрешения конфликта?

Подсказка: После редактирования файла Git ждёт подтверждения, что конфликт решён.

Ответ:

- **Выполнить `git add index.html`**
- Сразу сделать `git push`
- Удалить временные файлы `.git`
- Перезапустить rebase с нуля

Шаг 03:

Миссия 3

Сергей случайно выбрал неправильное решение конфликта. Как корректно отменить текущее слияние и начать заново?

Подсказка: Git позволяет откатить процесс merge/rebase без потери истории.

Ответ:

- **`git merge --abort`**
- `git stop-conflict`
- `git delete-conflict`
- `git commit --force`

Шаг 04:

Миссия 4

Игорь заметил, что после merge остались лишние маркеры <<<<<<. Что это значит?

Подсказка: Git оставляет подсказки в коде, чтобы программист сам выбрал правильный вариант.

Ответ:

- **Файл содержит неразрешённый конфликт**
- Файл повреждён и требует восстановления
- Git сам выбрал этот вариант как финальный
- Это просто комментарии для истории коммита

Шаг 05:

Миссия 5

 Сопоставьте команды Git с их назначением при разрешении конфликтов.

Подсказка: Вспомните, какая команда используется для завершения rebase, а какая — для отката.

Ответ:

- **`git add`** — Пометить файл как решённый
- **`git rebase --continue`** — Продолжить процесс после разрешения конфликта
- **`git merge --abort`** — Отменить текущее слияние
- **`git rebase --abort`** — Отменить текущий rebase

🔗 Шаг 06:

Миссия 6

📖 Сопоставьте тип конфликта с его примером.

Подсказка: Внимательно посмотрите: иногда конфликт вызван изменением одной и той же строки, а иногда — переименованием файла.

Ответ:

- **Конфликт содержимого** — Оба разработчика изменили одну и ту же строку кода
- **Конфликт переименования** — Файл переименован в разных ветках по-разному
- **Конфликт удаления** — Файл удалён в одной ветке и изменён в другой
- **Конфликт прав доступа** — Разные изменения прав на файл в ветках

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте действия с результатами при завершении rebase.

Подсказка: Важно помнить: rebase меняет историю, а merge — нет.

Ответ:

- **git rebase --continue** — Переход к следующему коммиту
- **git rebase --skip** — Пропустить проблемный коммит
- **git rebase --abort** — Вернуться к исходному состоянию ветки
- **git log --oneline** — Просмотреть историю коммитов после rebase

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте роль участника команды с его действием при конфликте.

Подсказка: Каждый выбирает свой подход: один решает в коде, другой консультируется.

Ответ:

- **Разработчик** — Разрешает конфликт вручную в коде
- **Тимлид** — Помогает принять архитектурное решение
- **DevOps** — Следит, чтобы CI не падал из-за конфликтов
- **Тестер** — Проверяет, что после merge/rebase всё работает

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте инструмент с его назначением при разрешении конфликтов.

Подсказка: GUI-инструменты тоже могут быть полезны, но принцип всегда один — помочь выбрать правильный вариант.

Ответ:

- **VS Code merge editor** — Позволяет визуально выбрать нужные изменения
- **git diff** — Показывает различия между версиями
- **git status** — Показывает список файлов в конфликте
- **meld** — Графический инструмент сравнения файлов

🔗 Шаг 10:

Миссия 10

 Сопоставьте стратегию с её характеристикой.

Подсказка: Иногда проще принять одну сторону полностью, а иногда — объединить изменения.

Ответ:

- **Принять изменения текущей ветки** — Сохранить только свои изменения
- **Принять изменения входящей ветки** — Сохранить только чужие изменения
- **Объединение вручную** — Выбрать части кода из обеих веток
- **Откат операции** — Прекратить процесс merge/rebase

Шаг 11:

Миссия 11

Расположите шаги разрешения конфликта в правильном порядке.

Подсказка: Сначала редактируем, потом подтверждаем, а потом продолжаем процесс.

Ответ:

- Открыть файл с конфликтом
- Отредактировать и удалить маркеры
- `git add [файл]`
- `git rebase --continue`

Шаг 12:

Миссия 12

Расположите действия при использовании команды `git mergetool`.

Подсказка: Помните, что `mergetool` открывает GUI-интерфейс, где нужно выбрать правильные изменения.

Ответ:

- Вызвать `git mergetool`
- Выбрать нужные изменения в интерфейсе
- Сохранить изменения
- `git commit` для завершения

Шаг 13:

Миссия 13

Расположите шаги при отмене `rebase` с конфликтами.

Подсказка: Главное — вернуть состояние репозитория к стабильному виду.

Ответ:

- Выполнить `git rebase --abort`
- Проверить `git status`
- Убедиться, что ветка вернулась к исходному состоянию
- Продолжить работу с чистой историей

Шаг 14:

Миссия 14

Расположите шаги подготовки команды к работе с конфликтами в проекте.

Подсказка: Важно сначала договориться о правилах, а потом закрепить их в документации.

Ответ:

- Определить правила разрешения конфликтов
- Закрепить их в README или wiki
- Провести обсуждение с командой
- Согласовать инструменты (mergetool, IDE)

 Шаг 15:

Миссия 15

Расположите шаги анализа конфликта перед его решением.

Подсказка: Сначала поймите причину, потом посмотрите изменения, и только после этого принимайте решение.

Ответ:

- Оценить историю изменений (git log)
- Сравнить ветки (git diff)
- Выбрать стратегию разрешения
- Внести правки в файл

 Шаг 16:

Миссия 16

Расположите шаги проверки после разрешения конфликтов.

Подсказка: Всегда важно убедиться, что проект работает и история осталась целой.

Ответ:

- Запустить тесты
- Собрать проект
- Проверить git log
- Сделать push в удалённый репозиторий

 Модуль 17 - Продвинутый rebase

 Урок 1 - Изменение сообщений в истории

 Шаг 01:

Миссия 1

Алиса делает rebase своей ветки и хочет изменить сообщение последнего коммита. Какую команду она должна использовать?

Подсказка: Вспомните короткий способ редактировать именно последний коммит без долгого интерактива.

Ответ:

- **git commit --amend**
- git rebase --continue
- git reset --hard
- git cherry-pick HEAD~1

 Шаг 02:

Миссия 2

Боб понял, что в истории есть старый коммит с некорректным описанием. Он запускает `git rebase -i HEAD~3`. Что нужно сделать в списке коммитов, чтобы изменить сообщение?

Подсказка: Подумайте, какая команда в интерактивном rebase позволяет редактировать сообщение коммита.

Ответ:

- **Заменить pick на reword**
- Заменить pick на drop
- Заменить pick на squash
- Заменить pick на edit и изменить код

Шаг 03:

Миссия 3

Карина случайно написала неинформативное сообщение «фикс». Она запускает интерактивный rebase. Какой вариант верный?

Подсказка: При изменении только текста коммита не нужно менять содержимое файлов.

Ответ:

- **Использовать reword для этого коммита**
- Использовать squash, чтобы объединить с предыдущим
- Использовать edit и внести изменения в файлы
- Удалить коммит с помощью drop

Шаг 04:

Миссия 4

Дима хочет изменить сообщение старого коммита, но при этом не менять его содержимое. Он выбрал команду edit в интерактивном rebase. Что произойдёт?

Подсказка: Подумайте: reword и edit — разные режимы.

Ответ:

- Ему сразу предложат ввести новое сообщение
- **Git остановит процесс и даст возможность менять коммит вручную**
- Коммит автоматически удалится
- Git перепишет сообщение на пустое

Шаг 05:

Миссия 5

Игорь делает rebase и видит несколько коммитов подряд с сообщениями «bugfix». Какой способ лучше всего для переименования всех сообщений на понятные?

Подсказка: Есть быстрый способ изменить именно текст без изменения кода.

Ответ:

- **Использовать reword для каждого нужного коммита**
- Сделать squash всех «bugfix» в один
- Применить drop для всех «bugfix»
- Просто сделать `git commit --amend` на HEAD

Шаг 06:

Миссия 6

Лена делает интерактивный rebase. Она выбрала команду reword для одного коммита. Что произойдёт?

Подсказка: Подумайте, чем отличается reword от edit.

Ответ:

- **Откроется редактор для изменения только сообщения коммита**
- Git остановит выполнение и даст менять файлы
- Коммит будет удалён
- Ничего не произойдёт

Шаг 07:

Миссия 7

Сергей случайно оставил ругательное слово в сообщении коммита. Какой путь самый правильный, чтобы поправить это, если коммит был три коммита назад?

Подсказка: Подумайте, какая команда поможет открыть список коммитов для редактирования.

Ответ:

- **Запустить git rebase -i HEAD~3 и заменить pick на reword**
- Сделать git reset --hard HEAD~3 и переписать коммиты
- Использовать git revert
- Запустить git merge --abort

Шаг 08:

Миссия 8

Юля делает rebase, но забыла дописать описание, зачем нужен коммит. Она хочет быстро исправить последний коммит. Что ей сделать?

Подсказка: Есть короткая команда, которая меняет только последний коммит.

Ответ:

- **git commit --amend**
- git rebase -i HEAD~5
- git checkout -b new
- git push --force

Шаг 09:

Миссия 9

 Задача: сопоставьте команды интерактивного rebase с их действием.

Подсказка: Вспомните: одни команды меняют сообщение, другие удаляют или объединяют коммиты.

Ответ:

- **pick** — Оставить коммит без изменений
- **reword** — Изменить только сообщение коммита
- **edit** — Остановиться для редактирования содержимого коммита
- **drop** — Удалить коммит из истории

Шаг 10:

Миссия 10

Сопоставьте ситуации и команды для изменения сообщений коммитов.

Подсказка: Смотрите на то, где именно находится коммит — последний или в середине истории.

Ответ:

- **Нужно изменить сообщение последнего коммита** — `git commit --amend`
- **Нужно изменить сообщение трёх коммитов назад** — `git rebase -i HEAD~3` с `reword`
- **Нужно удалить лишний коммит** — `git rebase -i` и `drop`
- **Нужно объединить два коммита с общим смыслом** — `git rebase -i` и `squash`

Шаг 11:

Миссия 11

Артём работает в команде. Он хочет переименовать коммит, который уже отправлен на общий репозиторий. Сопоставьте действие и его последствия.

Подсказка: При переписывании истории на удалённом репозитории есть риски.

Ответ:

- **Сделать `git commit --amend` и `git push --force`** — Изменит историю, может создать проблемы для коллег
- **Сделать `git revert`** — Не меняет старое сообщение, создаёт новый коммит
- **Сделать `git rebase -i` и `reword`** — Перепишет историю, требует `force-push`
- **Ничего не делать** — Сообщение останется некорректным, но история не пострадает

Шаг 12:

Миссия 12

Сопоставьте команды с реальными примерами использования.

Подсказка: Каждая команда решает отдельную задачу: переименование, объединение, удаление.

Ответ:

- **`git rebase -i HEAD~4`, заменить `pick` на `reword`** — Изменить сообщение старого коммита
- **`git commit --amend`** — Быстро поправить последний коммит
- **`git rebase -i` и `squash`** — Объединить коммиты в один
- **`git rebase -i` и `drop`** — Удалить ненужный коммит

Шаг 13:

Миссия 13

Сопоставьте термины с их описанием при изменении сообщений.

Подсказка: Смотрите на смысл слов — некоторые говорят о тексте, другие об истории.

Ответ:

- **Amend** — Изменение последнего коммита
- **Reword** — Изменение сообщения коммита в интерактивном `rebase`
- **Force push** — Перезапись удалённой истории
- **Interactive rebase** — Режим для ручного изменения истории

Шаг 14:

Миссия 14

Расположите шаги изменения сообщения старого коммита в правильном порядке.

Подсказка: Сначала запускаем `rebase`, потом выбираем команду, затем вносим изменения и продолжаем.

Ответ:

- Запустить `git rebase -i HEAD~N`
- Заменить `pick` на `reword` для нужного коммита
- Сохранить файл и изменить сообщение
- Продолжить выполнение `rebase`

 Шаг 15:

Миссия 15

Виталий хочет изменить сообщение трёх последних коммитов. Расположите шаги действий.

Подсказка: Не забудьте, что сначала нужно открыть список коммитов, а потом редактировать каждый.

Ответ:

- Выполнить `git rebase -i HEAD~3`
- Заменить `pick` на `reword` для всех нужных коммитов
- Сохранить изменения
- Ввести новые сообщения для каждого коммита

 Урок 2 - Перестановка коммитов

 Шаг 01:

Миссия 1

Разработчик Маша хочет поменять местами последние два коммита в своей ветке с помощью интерактивного `rebase`. Какой параметр команды ей нужен?

Подсказка: Обычно для изменения истории используют ключ, который открывает список коммитов в редакторе.

Ответ:

- **`git rebase -i HEAD~2`**
- `git commit --amend`
- `git merge --squash`
- `git reset --hard HEAD^`

 Шаг 02:

Миссия 2

Разработчик Иван заметил, что порядок его коммитов ломает логику кода. Он запустил `git rebase -i`. Какой символ в редакторе указывает, что коммит можно переместить?

Подсказка: При `rebase` мы видим список коммитов, и каждый начинается с определённого слова.

Ответ:

- **`pick`**
- `move`
- `apply`
- `swap`

 Шаг 03:

Миссия 3

На code review Артёму сказали переставить третий коммит на первое место. Что ему нужно сделать в интерактивном списке `rebase`?

Подсказка: Достаточно поменять порядок строк в редакторе.

Ответ:

- **Переместить строку с нужным коммитом наверх списка**
- Заменить слово pick на swap
- Удалить все коммиты и закоммитить заново
- Использовать git push -f без изменений

Шаг 04:

Миссия 4

Алиса случайно перепутала местами два важных коммита. Коллега посоветовал ей использовать rebase. Какой шаг ключевой для исправления?

Подсказка: Подумай: чтобы поменять местами, достаточно передвинуть строки.

Ответ:

- **Изменить порядок строк с коммитами в редакторе**
- Выполнить git checkout --ours
- Сделать новый merge
- Удалить ветку и создать заново

Шаг 05:

Миссия 5

Безопасник Стас проверяет историю и видит, что один коммит с фиксом нужно поставить раньше другого. Какой инструмент ему НЕ поможет?

Подсказка: Мы говорим о перестановке коммитов, а не об их отмене или создании заново.

Ответ:

- git rebase -i
- **git revert**
- редактор списка коммитов
- перемещение строк в интерактивном режиме

Шаг 06:

Миссия 6

Девопс Никита хочет убедиться, что после перестановки коммитов история будет линейной и чистой. Какой флаг rebase отвечает за автоматическое продолжение без лишних остановок?

Подсказка: Этот параметр сокращает количество ручных действий при rebase.

Ответ:

- **--autosquash**
- --hard
- --no-ff
- --abort

Шаг 07:

Миссия 7

 Сопоставьте команды и их описание в контексте перестановки коммитов.

Подсказка: Вспомни, какие команды применяются для редактирования истории, а какие нет.

Ответ:

- **git rebase -i** — Открывает список коммитов для редактирования
- **git revert** — Создаёт новый коммит, отменяющий изменения
- **git reset** — Перемещает HEAD на указанный коммит
- **git commit --amend** — Изменяет последний коммит

🔮 Шаг 08:

Миссия 8

Сопоставьте ситуации с правильным действием.

Подсказка: Подумайте, что выбрать: переставить, отменить или исправить.

Ответ:

- **Нужно поменять местами коммиты** — Использовать git rebase -i
- **Нужно отменить ненужный коммит** — Использовать git revert
- **Нужно откатиться на несколько шагов назад** — Использовать git reset
- **Нужно поправить сообщение последнего коммита** — Использовать git commit --amend

🔮 Шаг 09:

Миссия 9

Сопоставьте ключевые слова в редакторе rebase с их значением.

Подсказка: Эти слова определяют, что произойдёт с коммитом.

Ответ:

- **pick** — Применить коммит как есть
- **reword** — Изменить сообщение коммита
- **edit** — Остановиться для изменения содержимого
- **drop** — Удалить коммит

🔮 Шаг 10:

Миссия 10

Сопоставьте разработчиков с их действиями.

Подсказка: Каждое действие отражает разные сценарии работы с историей.

Ответ:

- **Маша** — Переставила два коммита
- **Иван** — Удалил один лишний коммит
- **Артём** — Изменил сообщение коммита
- **Алиса** — Остановила процесс для правок

🔮 Шаг 11:

Миссия 11

Сопоставьте понятия с их результатами.

Подсказка: Помни: перестановка — это не отмена, а изменение порядка.

Ответ:

- **Перестановка коммитов** — Изменение порядка без изменения содержимого
- **Отмена коммита** — Создание нового коммита с обратными изменениями
- **Редактирование коммита** — Изменение его содержимого

- **Слияние коммитов** — Объединение нескольких в один

🔗 Шаг 12:

Миссия 12

Сопоставьте термины с их определением.

Подсказка: Эти термины встречаются при работе с историей в Git.

Ответ:

- **HEAD** — Указатель на текущий коммит
- **Branch** — Указатель на последнюю вершину ветки
- **Rebase** — Переписывание истории коммитов
- **Merge** — Объединение изменений из двух веток

🔗 Шаг 13:

Миссия 13

Расположите шаги интерактивного rebase для перестановки коммитов в правильном порядке.

Подсказка: Подумай: сначала запускаем rebase, потом редактируем список, сохраняем и проверяем.

Ответ:

- Выполнить `git rebase -i`
- Изменить порядок строк
- Сохранить файл
- Проверить историю с помощью `git log`

🔗 Шаг 14:

Миссия 14

Расставьте шаги действий Ивана, который хочет переместить старый коммит наверх истории.

Подсказка: Действия всегда начинаются с вызова команды rebase.

Ответ:

- Запустить `git rebase -i HEAD~5`
- Переместить строку коммита вверх
- Сохранить и выйти из редактора
- Проверить `git log`

🔗 Шаг 15:

Миссия 15

Расставьте шаги для Алисы, которая хочет переставить два соседних коммита.

Подсказка: Сначала открывается список, потом строки меняются местами.

Ответ:

- Выполнить `git rebase -i HEAD~2`
- Поменять строки местами
- Сохранить файл
- Закончить rebase

Шаг 16:

Миссия 16

Расположите шаги для безопасника Стаса, который переставляет коммиты для чистой истории.

Подсказка: Последним шагом всегда проверка истории.

Ответ:

- Открыть интерактивный rebase
- Переставить строки
- Сохранить изменения
- Выполнить git log

Урок 3 - Практика squash и fixup

Шаг 01:

Миссия 1

Алиса случайно сделала два мелких коммита подряд, исправляющих одну и ту же опечатку.

Какой способ с помощью git rebase -i лучше использовать, чтобы объединить их в один?

Подсказка: Подумай о команде, которая объединяет содержимое, но оставляет возможность отредактировать сообщение.

Ответ:

- **Использовать squash для объединения и редактирования сообщения**
- Использовать fixup для объединения с сохранением сообщения
- Сделать revert и заново закоммитить
- Сделать merge этих коммитов

Шаг 02:

Миссия 2

Боб хочет быстро убрать ненужный коммит «debug print» и присоединить его изменения к предыдущему коммиту, не меняя сообщение. Что лучше выбрать?

Подсказка: Подумай о команде, которая не спрашивает про сообщение коммита.

Ответ:

- squash
- **fixup**
- drop
- edit

Шаг 03:

Миссия 3

Кира видит список коммитов в интерактивном rebase. Ей нужно объединить два коммита в один, но она хочет оставить финальное сообщение от второго коммита. Какой порядок действий?

Подсказка: Вспомни, чем отличаются squash и fixup по сообщению.

Ответ:

- **Использовать squash, так как оно позволяет редактировать сообщение**
- Использовать fixup, так как оно убирает сообщение
- Использовать drop, чтобы удалить коммит
- Использовать reword

Шаг 04:

Миссия 4

Дима случайно сделал коммит «WIP», а затем полноценный коммит с нужным кодом. Как объединить их, чтобы сообщение осталось только из второго?

Подсказка: Нужно избавиться от «WIP» сообщения.

Ответ:

- squash
- **fixup**
- merge
- revert

Шаг 05:

Миссия 5

Лена экспериментирует: она хочет объединить три подряд идущих коммита в один. Какую стратегию лучше выбрать?

Подсказка: Иногда проще объединить шаг за шагом.

Ответ:

- **Использовать несколько squash подряд**
- Использовать только один fixup
- Удалить два коммита и переписать код
- Сделать revert всех коммитов

Шаг 06:

Миссия 6

Сопоставьте команды rebase с их назначением:

Подсказка: Внимательно смотри, какие из них влияют на сообщение, а какие нет.

Ответ:

- **squash** — Объединяет изменения и позволяет редактировать сообщение
- **fixup** — Объединяет изменения, но отбрасывает сообщение
- **drop** — Удаляет коммит из истории
- **reword** — Изменяет только сообщение коммита

Шаг 07:

Миссия 7

Сопоставьте ситуацию с подходящей стратегией:

Подсказка: Думай, когда нужно сохранить сообщение, а когда — нет.

Ответ:

- **Исправление мелкой опечатки** — fixup
- **Два похожих коммита, оба важны** — squash
- **Лишний коммит «debug print»** — drop
- **Сообщение коммита слишком непонятное** — reword

Шаг 08:

Миссия 8

Соедини команду и результат её использования в интерактивном rebase:

Подсказка: Представь, что ты объясняешь новичку в команде.

Ответ:

- **pick** — Оставить коммит как есть
- **squash** — Объединить и редактировать сообщение
- **fixup** — Объединить и сохранить сообщение предыдущего
- **edit** — Остановиться и внести изменения вручную

Шаг 09:

Миссия 9

Сопоставьте команду и то, когда её лучше использовать:

Подсказка: Каждая команда решает конкретную проблему.

Ответ:

- **squash** — Нужно объединить несколько логически связанных коммитов
- **fixup** — Убрать ненужный промежуточный коммит
- **drop** — Полностью удалить ошибочный коммит
- **reword** — Сделать сообщение более понятным

Шаг 10:

Миссия 10

Свяжи команду с её результатом в истории проекта:

Подсказка: Обрати внимание на то, остаётся ли сообщение или исчезает.

Ответ:

- **fixup** — Убирает сообщение фикса и присоединяет изменения
- **squash** — Объединяет изменения и оставляет возможность редактировать сообщение
- **drop** — Полностью удаляет коммит
- **reword** — Меняет только сообщение коммита

Шаг 11:

Миссия 11

Расставьте шаги объединения коммитов с помощью squash:

Подсказка: Вспомни, с чего начинается rebase и чем он заканчивается.

Ответ:

- Запустить `git rebase -i HEAD~3`
- Пометить нужные коммиты как `squash`
- Отредактировать сообщение коммита
- Завершить rebase и проверить историю

Шаг 12:

Миссия 12

Определи порядок действий для использования `fixup`:

Подсказка: Разница в том, что сообщение не сохраняется.

Ответ:

- Запустить `git rebase -i HEAD~2`

- Пометить ненужный коммит как fixup
- Применить изменения к предыдущему коммиту
- Закончить rebase без редактирования сообщения

🔗 Шаг 13:

Миссия 13

Определи правильный порядок объединения трёх коммитов в один:

Подсказка: Помни, что squash можно применять последовательно.

Ответ:

- Выбрать три последних коммита
- Запустить git rebase -i
- Применить squash к двум нижним
- Оставить верхний как pick и редактировать сообщение

🔗 Шаг 14:

Миссия 14

Расположи шаги, чтобы объединить коммиты «draft» и «final» через fixup:

Подсказка: Важно, какое сообщение останется.

Ответ:

- Запустить git rebase -i HEAD~2
- Пометить «draft» как fixup
- Оставить «final» как pick
- Закончить rebase

🔗 Шаг 15:

Миссия 15

Определи последовательность действий при squash с изменением сообщения:

Подсказка: Нужно будет открыть редактор.

Ответ:

- Отметить второй коммит как squash
- Отметить первый как pick
- Открыть редактор и написать итоговое сообщение
- Завершить rebase

🔗 Шаг 16:

Миссия 16

Расположи шаги при удалении ненужного коммита с fixup:

Подсказка: Подумай, к какому коммиту он будет присоединён.

Ответ:

- Выбрать коммиты для rebase
- Пометить ненужный как fixup
- Применить изменения к предыдущему
- Закончить rebase без редактирования

Шаг 17:

Миссия 17

Определи порядок действий для полного объединения серии «hotfix» коммитов в один:

Подсказка: Нужно выбрать один главный коммит и присоединить остальные.

Ответ:

- Запустить `git rebase -i HEAD~4`
- Оставить первый hotfix как pick
- Пометить остальные как squash
- Редактировать сообщение и завершить

Урок 4 - Поддержание читаемой истории

Шаг 01:

Миссия 1

Разработчик Алекс решил навести порядок в истории проекта. Он хочет объединить три мелких коммита, исправляющих опечатки, в один. Какой приём rebase ему подойдёт?

Подсказка: Подумайте, какой способ позволяет слить несколько коммитов в один.

Ответ:

- **squash**
- fixup
- merge
- cherry-pick

Шаг 02:

Миссия 2

Безопасник Лена заметила в истории коммит с секретным паролем. Что лучше сделать с помощью интерактивного rebase, чтобы удалить этот коммит из истории?

Подсказка: Можно не просто изменить сообщение, а убрать коммит полностью.

Ответ:

- **drop**
- reword
- edit
- fixup

Шаг 03:

Миссия 3

Фронтендер Дима случайно сделал два одинаковых коммита. Какой вариант интерактивного rebase поможет избавиться от дубликата?

Подсказка: Думайте о том, что можно просто убрать один из них.

Ответ:

- **drop**
- squash
- fixup
- merge

Шаг 04:

Миссия 4

Бэкендер Игорь хочет оставить историю красивой: он исправил баг в отдельном коммите, но сообщение написал невнятно. Что можно сделать через rebase?

Подсказка: Иногда не нужно переписывать код, достаточно изменить заголовок коммита.

Ответ:

- **reword**
- drop
- squash
- merge

Шаг 05:

Миссия 5

QA-инженер Маша заметила, что коммит с тестами логически должен идти сразу после коммита с функцией, которую они проверяют. Какое действие при интерактивном rebase поможет упорядочить историю?

Подсказка: Иногда нужно менять порядок коммитов.

Ответ:

- **reorder**
- fixup
- squash
- drop

Шаг 06:

Миссия 6

 Задача: Сопоставьте команду rebase с её результатом.

Подсказка: Вспомните, какие ключевые слова используются при интерактивном rebase.

Ответ:

- **reword** — Изменить сообщение коммита
- **drop** — Удалить коммит
- **squash** — Объединить коммиты с сохранением сообщений
- **fixup** — Объединить коммиты без сохранения сообщений

Шаг 07:

Миссия 7

Аналитик Саша хочет объяснить стажёрам действия при поддержке читаемой истории. Сопоставьте шаг с его смыслом.

Подсказка: Сначала исправляем ошибки, потом приводим историю к аккуратному виду.

Ответ:

- **Использовать squash** — Объединить похожие коммиты
- **Использовать drop** — Убрать ненужный коммит
- **Использовать reword** — Сделать сообщение более понятным
- **Использовать reorder** — Поменять порядок коммитов

🔗 Шаг 08:

Миссия 8

Разработчик Коля хочет поделиться принципами читаемой истории. Сопоставьте принцип с его описанием.

Подсказка: Подумайте о том, что делает историю понятной для коллег.

Ответ:

- **Меньше коммитов с правками мелочей** — Использовать squash
- **Ясные сообщения** — Использовать reword
- **Удаление лишнего** — Использовать drop
- **Правильный порядок** — Использовать reorder

🔗 Шаг 09:

Миссия 9

Тимлид Оля хочет обучить команду: сопоставьте действие rebase с примером из жизни.

Подсказка: Представьте, что история — это рассказ, и нужно убрать повторы и ошибки.

Ответ:

- **squash** — Собрать три коммита с опечатками в один
- **fixup** — Убрать старое сообщение и оставить только новое
- **drop** — Удалить лишний коммит с тестом
- **reword** — Сделать сообщение понятным для команды

🔗 Шаг 10:

Миссия 10

У вас есть четыре коммита: 1) исправление опечатки, 2) добавление функции, 3) тест для функции, 4) удаление временного файла. Расположите их так, чтобы история была читаемой.

Подсказка: Сначала код, потом тест, потом мелочи.

Ответ:

- Добавление функции
- Тест для функции
- Исправление опечатки
- Удаление временного файла

🔗 Шаг 11:

Миссия 11

История проекта выглядит так: 1) тест без функции, 2) исправление бага, 3) добавление функции. Расположите коммиты так, чтобы история читалась логично.

Подсказка: Сначала должна появиться функция, потом тест, потом исправления.

Ответ:

- Добавление функции
- Тест без функции
- Исправление бага

🔗 Шаг 12:

Миссия 12

В истории есть: 1) черновик функции, 2) исправление опечатки в сообщении, 3) финальная версия функции. Упорядочите их для чистоты истории.

Подсказка: Сначала финальная версия, потом исправления.

Ответ:

- Финальная версия функции
- Исправление опечатки в сообщении
- Черновик функции

Шаг 13:

Миссия 13

Вы видите историю: 1) удаление файла .env, 2) добавление фичи, 3) тесты к фиче, 4) возврат файла .env. Расположите коммиты так, чтобы история выглядела осмысленно.

Подсказка: В истории не должно быть лишних движений с секретами.

Ответ:

- Добавление фичи
- Тесты к фиче
- Удаление файла .env
- Возврат файла .env

Шаг 14:

Миссия 14

История: 1) багфикс, 2) добавление логирования, 3) добавление фичи, 4) тест для фичи.

Упорядочите коммиты так, чтобы их легко было читать.

Подсказка: Фича → тест → улучшения → багфиксы.

Ответ:

- Добавление фичи
- Тест для фичи
- Добавление логирования
- Багфикс

Шаг 15:

Миссия 15

Вы видите коммиты: 1) исправление опечатки в readme, 2) добавление библиотеки, 3) настройка CI, 4) тесты. Упорядочите их для читаемой истории.

Подсказка: Сначала код и инфраструктура, потом документация.

Ответ:

- Добавление библиотеки
- Тесты
- Настройка CI
- Исправление опечатки в readme

Шаг 16:

Миссия 16

Финальный квест: 1) мелкие исправления кода, 2) добавление API, 3) тесты к API, 4) оптимизация алгоритма. Сделайте историю максимально логичной.

Подсказка: Главная идея: сначала функционал, потом тесты, затем улучшения и мелочи.

Ответ:

- Добавление API
- Тесты к API
- Оптимизация алгоритма
- Мелкие исправления кода

Модуль 18 - Восстановление удалённых веток

Урок 1 - Reflog для поиска потерянных веток

Шаг 01:

Миссия 1

Вы случайно удалили ветку feature-X, но помните, что вчера работали с ней. Что стоит первым делом проверить?

Подсказка: Git хранит историю ссылок даже после удаления.

Ответ:

- **git reflog**
- git fsck
- git prune

Шаг 02:

Миссия 2

Вы запустили git reflog и увидели множество записей HEAD@{n}. Как понять, какая запись — это ветка feature-X?

Подсказка: В названии записи часто фигурирует имя ветки перед удалением.

Ответ:

- **Найти запись, где в сообщении упоминается feature-X**
- Выбрать самую старую запись
- Выбрать запись без комментария

Шаг 03:

Миссия 3

Вы нашли HEAD@{5} с коммитом из ветки feature-X, как восстановить ветку с этой точки?

Подсказка: Создать новую ветку от указанного reflog-снимка.

Ответ:

- **git branch feature-X HEAD@{5}**
- git checkout HEAD@{5}
- git reset --hard HEAD@{5}

Шаг 04:

Миссия 4

После восстановления локальной ветки feature-X вы хотите запустить её в удалённый репозиторий. Что нужно сделать?

Подсказка: Удалённый репозиторий не знает о восстановленной ветке.

Ответ:

- **git push -u origin feature-X**
- git push origin :feature-X
- git push origin HEAD

Шаг 05:

Миссия 5

Вы не уверены, что reflog всё ещё содержит нужную запись (раньше прошло много времени). Какова политика reflog по срокам хранения?

Подсказка: По умолчанию записи хранятся ограниченное время.

Ответ:

- **90 дней по умолчанию**
- 365 дней
- 7 дней

Шаг 06:

Миссия 6

Сопоставьте приём reflog с его описанием:

Приём:

Ответ:

- Найти удалённую ветку
- Просмотреть недавние коммиты
- Вернуть HEAD на старую позицию
- Создать новую ветку из reflog

Описание:

1. git reflog показывает список изменений HEAD
2. git branch + HEAD@{n}
3. git checkout HEAD@{n}
4. использовать ключи reflog (например, --expire)

Подсказка: Обратите внимание на ключевые действия: просмотр → восстановление → перемещение → фильтрация.

- **Найти удалённую ветку** — использовать ключи reflog (например, --expire)
- **Просмотреть недавние коммиты** — git reflog показывает список изменений HEAD
- **Вернуть HEAD на старую позицию** — git checkout HEAD@{n}
- **Создать новую ветку из reflog** — git branch + HEAD@{n}

Шаг 07:

Миссия 7

Соотнесите режим reflog и его результат:

Режим:

Ответ:

- HEAD@{n}
- git reflog show branch-name
- git reflog expire

- `git reflog delete`

Результат:

1. удаление записи
2. принудительное истечение старых записей
3. просмотр истории отдельной ветки
4. доступ к записи из списка HEAD

Подсказка: Первый режим — числовой доступ, второй — выбор ветки, третий/четвёртый — очистка.

- **HEAD@{n}** — доступ к записи из списка HEAD
- **git reflog show branch-name** — просмотр истории отдельной ветки
- **git reflog expire** — принудительное истечение старых записей
- **git reflog delete** — удаление записи

Шаг 08:

Миссия 8

Сопоставьте ситуацию с действием в reflog:

Ситуация:

Ответ:

- Вы хотите очистить старые записи
- Вы хотите ограничить число записей
- Вы хотите показать reflog ветки dev
- Вы хотите полностью сбросить reflog

Действие:

1. `git reflog expire --expire=now --all`
2. `git reflog delete --rewrite HEAD@{n}`
3. `git reflog show dev`
4. `git reflog --max-count=N`

Подсказка: «expire» — о сроках, «max-count» — по количеству.

- **Вы хотите очистить старые записи** — `git reflog expire --expire=now --all`
- **Вы хотите ограничить число записей** — `git reflog --max-count=N`
- **Вы хотите показать reflog ветки dev** — `git reflog show dev`
- **Вы хотите полностью сбросить reflog** — `git reflog delete --rewrite HEAD@{n}`

Шаг 09:

Миссия 9

Соотнесите ключ reflog с его назначением:

Ключ:

Ответ:

- `--expire`
- `--verbose`
- `--all`
- `--relative-date`

Назначение:

1. показывать время в относительном формате
2. применять к всем ссылкам
3. устанавливать время истечения записей
4. показывать подробную информацию

Подсказка: «verbose» типично даёт много вывода, «relative» — отношение времени.

- **--expire** — устанавливать время истечения записей
- **--verbose** — показывать подробную информацию
- **--all** — применять к всем ссылкам
- **--relative-date** — показывать время в относительном формате

Шаг 10:

Миссия 10

Сопоставьте проблему с возможным объяснением в reflog:

Проблема:

Ответ:

- Запись отсутствует
- Запись слишком старая
- Выбран не тот HEAD@{n}
- Reflog был очищен

Объяснение:

1. expire уже применялся
2. слишком долгий срок с момента удаления
3. ошибка при подборе индекса
4. reflog был «prune»/удалён вручную

Подсказка: Сравните сценарий с тем, что могло произойти с историей.

- **Запись отсутствует** — reflog был «prune»/удалён вручную
- **Запись слишком старая** — слишком долгий срок с момента удаления
- **Выбран не тот HEAD@{n}** — ошибка при подборе индекса
- **Reflog был очищен** — expire уже применялся

Шаг 11:

Миссия 11

Сопоставьте сценарий и действие, чтобы минимизировать риск потери ветки:

Сценарий:

Ответ:

- Перед удалением ветки
- После резервного сохранения
- При частой работе над ветками
- В CI/CD среде

Действие:

1. push ветка на удалённый

2. создать локальный reflog-лог
3. использовать git reflog expire осторожно
4. настроить автоматическое бэкапирование

Подсказка: Подумайте: что помогает перед удалением, что — в автоматизации.

- **Перед удалением ветки** — push ветка на удалённый
- **После резервного сохранения** — создать локальный reflog-лог
- **При частой работе над ветками** — настроить автоматическое бэкапирование
- **В CI/CD среде** — использовать git reflog expire осторожно

🔮 Шаг 12:

Миссия 12

Расположите шаги восстановления потерянной ветки через reflog:

1. Создать ветку от HEAD@{n}
2. Запустить git reflog
3. Найти нужную запись с веткой
4. Пуш ветки в origin

Подсказка: Сначала находят запись, затем восстанавливают и публикуют.

Ответ:

- Запустить git reflog
- Найти нужную запись с веткой
- Создать ветку от HEAD@{n}
- Пуш ветки в origin

🔮 Шаг 13:

Миссия 13

Расположите шаги безопасной работы с reflog и ветками:

1. Проверить reflog на наличие удалённой ветки
2. Сделать резервную копию репозитория
3. Восстановить ветку через reflog
4. Пуш восстановленной ветки в origin

Подсказка: Резерв — прежде, чем что-либо менять.

Ответ:

- Сделать резервную копию репозитория
- Проверить reflog на наличие удалённой ветки
- Восстановить ветку через reflog
- Пуш восстановленной ветки в origin

🔮 Шаг 14:

Миссия 14

Упорядочите шаги чистки reflog после восстановления веток:

1. git reflog expire --expire=now --all
2. git reflog delete --rewrite HEAD@{n}
3. git gc --prune=now

4. git reflog show

Подсказка: Сначала задают срок, потом удаляют конкретно, потом проводят GC, и, наконец, проверяют.

Ответ:

- git reflog expire --expire=now --all
- git reflog delete --rewrite HEAD@{n}
- git gc --prune=now
- git reflog show

Шаг 15:

Миссия 15

Упорядочите действия, если вы обнаружили, что reflog не содержит нужной записи:

1. Проверить удалённый репозиторий на ветку
2. Посмотреть reflog других локальных клонов
3. Спросить коллег, возможно, у них есть копия
4. Начать работу заново или восстановить вручную

Подсказка: Используйте все внешние источники перед радикальным шагом.

Ответ:

- Проверить удалённый репозиторий на ветку
- Посмотреть reflog других локальных клонов
- Спросить коллег, возможно, у них есть копия
- Начать работу заново или восстановить вручную

Шаг 16:

Миссия 16

Упорядочите этапы безопасного удаления веток при настроенном reflog:

1. Убедиться, что ветка запущена или сохранена
2. Удалить ветку локально
3. Удалить ветку удалённо
4. Проверить reflog, что запись осталась

Подсказка: Не удаляйте, пока не убедились в наличии резервной копии.

Ответ:

- Убедиться, что ветка запущена или сохранена
- Удалить ветку локально
- Удалить ветку удалённо
- Проверить reflog, что запись осталась

Шаг 17:

Миссия 17

Упорядочите полную процедуру выявления и восстановления потерянной ветки:

1. Запустить git reflog
2. Найти подходящий HEAD@{n}
3. Создать ветку от найденного коммита

4. Пуш ветки в удалённый репозиторий
5. Очистить или оптимизировать reflog

Подсказка: Восстановление — это цикл: поиск, восстановление, публикация, уборка.

Ответ:

- Запустить `git reflog`
- Найти подходящий `HEAD@{n}`
- Создать ветку от найденного коммита
- Пуш ветки в удалённый репозиторий
- Очистить или оптимизировать reflog

Урок 2 - Reset и checkout для возврата

Шаг 01:

Миссия 1

Вы случайно переключились на старый коммит: хочется вернуться к ветке `main`. Что сделать?

Подсказка: Обычный `checkout` позволяет переключаться.

Ответ:

- **`git checkout main`**
- `git reset --hard main`
- `git checkout HEAD~3`

Шаг 02:

Миссия 2

Вы находитесь на ветке `feature`, сделали пару коммитов, но потом поняли — нужно было отталкиваться от текущего состояния `develop`. Как вы вернётесь?

Подсказка: `Reset` может передвинуть ветку на указанный коммит.

Ответ:

- **`git reset --hard develop`**
- `git checkout develop`
- `git reset develop --soft`

Шаг 03:

Миссия 3

Вы хотите временно посмотреть историю другой ветки `bugfix`, но не хотите переключаться окончательно. Что подойдёт?

Подсказка: `Checkout` может работать с указанием коммита.

Ответ:

- **`git checkout bugfix`**
- `git reset bugfix`
- `git checkout bugfix --detach`

Шаг 04:

Миссия 4

Вы уже сделали `git reset --hard HEAD~2`, но поняли, что потеряли полезные коммиты. Как быстро вернуться назад?

Подсказка: Git хранит reflog, который можно использовать для возврата.

Ответ:

- **git checkout HEAD@{1}**
- git reset --hard HEAD
- git reset HEAD@{1}

💡 Шаг 05:

Миссия 5

Вы на ветке feature , сделали git reset --soft HEAD~1 , но обнаружили, что индекс тоже сбросился. Что делать?

Подсказка: Soft-режим не трогает рабочее дерево, но индекс может быть перезаписан.

Ответ:

- **git reset --mixed HEAD@{1}**
- git reset --hard HEAD
- git checkout .

💡 Шаг 06:

Миссия 6

Сопоставьте команду с её действием:

Команды:

Ответ:

- git reset --hard
- git reset --soft
- git reset --mixed
- git checkout --

Действие:

1. сбрасывает рабочее дерево и индекс к указанному коммиту
2. сохраняет изменения в индексе и рабочем дереве, меняет только HEAD
3. сбрасывает индекс, но оставляет рабочее дерево
4. откатывает файл к версии в индексе

Подсказка: Помните: soft — мягко, hard — жёстко, mixed — средний уровень.

- **git reset --hard** — сбрасывает рабочее дерево и индекс к указанному коммиту
- **git reset --soft** — сохраняет изменения в индексе и рабочем дереве, меняет только HEAD
- **git reset --mixed** — сбрасывает индекс, но оставляет рабочее дерево
- **git checkout --** — откатывает файл к версии в индексе

💡 Шаг 07:

Миссия 7

Сопоставьте тип reset/checkout и его эффект:

Тип:

Ответ:

- checkout с коммитом (detached)
- checkout ветки

- reset HEAD~2 hard
- reset HEAD~1 soft

Эффект:

1. вы становитесь «оторванным» от ветки
2. переключаетесь на ветку
3. сбрасывает всё на два коммита назад
4. двигает HEAD, оставляя изменения в индексе и дереве

Подсказка: Checkout может быть detached, reset имеет разные уровни воздействия.

- **checkout с коммитом (detached)** — вы становитесь «оторванным» от ветки
- **checkout ветки** — переключаетесь на ветку
- **reset HEAD~2 hard** — сбрасывает всё на два коммита назад
- **reset HEAD~1 soft** — двигает HEAD, оставляя изменения в индексе и дереве

Шаг 08:

Миссия 8

Сопоставьте ситуацию и действие:

Ситуация:

Ответ:

- Вы хотите откатить отдельный файл
- Вы хотите отбросить все рабочие изменения
- Вы хотите вернуть ветку к определённому коммиту
- Вы хотите сместить HEAD без потери файлов

Действие:

1. git checkout --
2. git reset --hard
3. git reset --soft
4. git reset --mixed

Подсказка: Каждый режим reset работает по-разному: hard — всё, mixed — индекс, soft — только HEAD.

- **Вы хотите откатить отдельный файл** — git checkout --
- **Вы хотите отбросить все рабочие изменения** — git reset --hard
- **Вы хотите вернуть ветку к определённому коммиту** — git reset --mixed
- **Вы хотите сместить HEAD без потери файлов** — git reset --soft

Шаг 09:

Миссия 9

Сопоставьте флаг reset/checkout с пояснением:

Флаг:

Ответ:

- --hard
- --soft
- --mixed
- --detach

Пояснение:

1. отключает HEAD от ветки
2. удаляет всё, включая рабочее дерево
3. не влияет на рабочее дерево и индекс
4. сбрасывает индекс, но остаётся рабочее дерево

Подсказка: detach — не режим reset, а режим checkout без ветки.

- **--hard** — удаляет всё, включая рабочее дерево
- **--soft** — не влияет на рабочее дерево и индекс
- **--mixed** — сбрасывает индекс, но остаётся рабочее дерево
- **--detach** — отключает HEAD от ветки

Шаг 10:

Миссия 10

Упорядочите действия при возврате к старому состоянию и сохранении работы:

1. Сохранить изменения в stash
2. `git reset --hard`
3. `git checkout`
4. `git stash pop`

Подсказка: Если есть несохранённые изменения — сначала сохранить их.

Ответ:

- Сохранить изменения в stash
- `git reset --hard`
- `git checkout`
- `git stash pop`

Шаг 11:

Миссия 11

Упорядочите шаги safe-работы с reset:

1. Проверить состояние и изменения (`git status`)
2. Сохранить незакоммиченные изменения (stash или commit)
3. Выполнить reset нужного типа
4. Проверить результат (`git log`, `git diff`)

Подсказка: Безопасность = сначала сохранить, потом менять, потом проверять.

Ответ:

- Проверить состояние и изменения (`git status`)
- Сохранить незакоммиченные изменения (stash или commit)
- Выполнить reset нужного типа
- Проверить результат (`git log`, `git diff`)

Шаг 12:

Миссия 12

Упорядочите последовательность, чтобы откатить ветку до коммита A , сохранив изменения:

1. `git reset --soft A`

2. `git commit --amend` или новый коммит
3. `git push --force`
4. удалить старые коммиты

Подсказка: Soft сначала, потом фиксация, затем публикация.

Ответ:

- `git reset --soft A`
- `git commit --amend` или новый коммит
- удалить старые коммиты
- `git push --force`

Шаг 13:

Миссия 13

Упорядочите действия при откате и одновременном создании новой ветки:

1. `git reset --hard`
2. `git branch new-feature`
3. `git checkout new-feature`
4. документировать изменения

Подсказка: Сначала передвинуть HEAD, потом создать и переключиться.

Ответ:

- `git reset --hard`
- `git branch new-feature`
- `git checkout new-feature`
- документировать изменения

Шаг 14:

Миссия 14

Упорядочите шаги для безопасного возврата с удалённого сервера:

1. `git fetch origin`
2. **`git checkout origin/feature`** — detach
3. `git reset --hard origin/feature`
4. `git checkout feature`

Подсказка: Сначала обновить, потом переключиться, потом сброс.

Ответ:

- `git fetch origin`
- **`git checkout origin/feature`** — detach
- `git reset --hard origin/feature`
- `git checkout feature`

Шаг 15:

Миссия 15

Упорядочите шаги, если вы хотите откатиться, но сохранить текущую работу:

1. `git stash`
2. `git reset --hard`

3. git stash apply
4. урегулировать конфликты

Подсказка: Stash прежде, чем hard-сброс.

Ответ:

- git stash
- git reset --hard
- git stash apply
- урегулировать конфликты

Шаг 16:

Миссия 16

Упорядочите шаги для отладки после неудачного reset:

1. git reflog
2. найти нужный HEAD@{n}
3. git reset --hard HEAD@{n}
4. git log проверить историю

Подсказка: Reflog поможет найти утраченный коммит.

Ответ:

- git reflog
- найти нужный HEAD@{n}
- git reset --hard HEAD@{n}
- git log проверить историю

Шаг 17:

Миссия 17

Упорядочите полную стратегию возврата ветки:

1. Оценить изменения, которые хотите вернуть
2. Сохранить незакоммиченные изменения (stash или commit)
3. Выбрать подходящий commit
4. Сделать reset нужного типа
5. Вернуть сохранённые изменения

Подсказка: Планируйте шаги: подготовка → выбор → действие → восстановление.

Ответ:

- Оценить изменения, которые хотите вернуть
- Сохранить незакоммиченные изменения (stash или commit)
- Выбрать подходящий commit
- Сделать reset нужного типа
- Вернуть сохранённые изменения

Урок 3 - Практика восстановление ветки

Шаг 01:

Миссия 1

Во время пятничной уборки Серёжа случайно удалил ветку feature/login . Как он может проверить, существует ли шанс её вернуть?

Подсказка: Git хранит историю перемещений HEAD.

Ответ:

- **git reflog**
- git log --deleted
- git restore --lost-branches
- git branch --deleted

🔮 Шаг 02:

Миссия 2

Юля случайно удалила ветку, в которой были важные коммиты. Какой командой ей стоит восстановить ветку, если она нашла нужный хеш в reflog?

Подсказка: Ветка — это просто имя, указывающее на коммит.

Ответ:

- **git branch**
- git checkout -b
- git revert
- git create-branch

🔮 Шаг 03:

Миссия 3

После rebase Паша потерял ветку fix/api-bug . Он помнит, что работал с ней вчера. Что поможет найти её последний коммит?

Подсказка: Reflog содержит все переходы HEAD за последние дни.

Ответ:

- **git reflog show**
- git fsck --lost-found
- git rebase --undo
- git commit-log --since=yesterday

🔮 Шаг 04:

Миссия 4

Таня удалила ветку, но потом вспомнила, что там был незапущенный важный коммит. Что произойдёт, если просто ввести git checkout HEAD@{1} ?

Подсказка: HEAD@{1} — предыдущее положение HEAD.

Ответ:

- **Вы перейдёте на временное состояние (detached HEAD)**
- Будет создана новая ветка
- Ветка восстановится автоматически
- Коммит удалится окончательно

🔮 Шаг 05:

Миссия 5

Игорь случайно удалил ветку в репозитории и не помнит, был ли в ней хоть один уникальный коммит. Как быстро это проверить?

Подсказка: Команда сравнения поможет увидеть отличия между ветками.

Ответ:

- **git reflog** и **git log ^main**
- **git merge-base HEAD main**
- **git branch --compare**
- **git graph-diff**

Шаг 06:

Миссия 6

Вадим восстановил ветку с помощью `git branch lost dev123`. Но теперь ему нужно продолжить работу — что сделать дальше?

Подсказка: После создания ветки важно на неё переключиться.

Ответ:

- **git checkout lost**
- **git restore lost**
- **git push lost**
- **git reset --hard lost**

Шаг 07:

Миссия 7

Сопоставьте между собой фразу из описания ситуации и наиболее подходящее действие при восстановлении ветки:

Подсказка: Подумайте, какое действие логически соответствует ситуации.

Ситуации:

Ответ:

- Хеш найден в **reflog**, но не уверены, это тот коммит
- Нужно посмотреть содержимое ветки без переключения
- Создать ветку прямо из текущего состояния **HEAD**
- Нужно вернуться к состоянию до удаления

Действия:

1. `git checkout --detach HEAD@{n}`
2. `git diff HEAD@{n}..HEAD`
3. `git branch recovered HEAD@{n}`
4. `git reset --hard HEAD@{n}`

- **Хеш найден в reflog, но не уверены, это тот коммит** — `git diff HEAD@{n}..HEAD`
- **Нужно посмотреть содержимое ветки без переключения** — `git checkout --detach HEAD@{n}`
- **Создать ветку прямо из текущего состояния HEAD** — `git branch recovered HEAD@{n}`
- **Нужно вернуться к состоянию до удаления** — `git reset --hard HEAD@{n}`

Шаг 08:

Миссия 8

Сопоставьте термин с тем, на чём он акцентирует внимание в процессе восстановления:

Подсказка: Reflog, detached, reset, branch — разные аспекты восстановления.

Термины:

Ответ:

- detached HEAD
- reflog expire
- branch recovery
- reset hard

Акцент:

1. временный просмотр точки без ветки
 2. удаление старых записей reflog
 3. создание новой ветки из коммита
 4. полный возврат дерева и индекса
- **detached HEAD** — временный просмотр точки без ветки
 - **reflog expire** — удаление старых записей reflog
 - **branch recovery** — создание новой ветки из коммита
 - **reset hard** — полный возврат дерева и индекса

Шаг 09:

Миссия 9

Сопоставьте возможную ошибку и её причину при попытке восстановления ветки:

Подсказка: Ошибки часто связаны с неправильным индексом reflog или очищением.

Ошибка:

Ответ:

- Неверный HEAD@{n}
- Запись не найдена
- Ветка создана, но пуста
- Невозможно переключиться

Причина:

1. индекс не соответствует ожидаемому
 2. reflog уже обрезан
 3. коммит пустой или не содержит изменений
 4. состояние detached без ветки
- **Неверный HEAD@{n}** — индекс не соответствует ожидаемому
 - **Запись не найдена** — reflog уже обрезан
 - **Ветка создана, но пуста** — коммит пустой или не содержит изменений
 - **Невозможно переключиться** — состояние detached без ветки

Шаг 10:

Миссия 10

Расположите шаги логического процесса восстановления удалённой ветки:

1. Посмотреть reflog
2. Найти подходящую запись
3. Создать ветку от неё

4. Переключиться на восстановленную ветку
5. Пуш её в origin

Подсказка: Сначала найти, потом создать, затем переключиться и опубликовать.

Ответ:

- Посмотреть reflog
- Найти подходящую запись
- Создать ветку от неё
- Переключиться на восстановленную ветку
- Пуш её в origin

Шаг 11:

Миссия 11

Расположите шаги, если вы хотите восстановить ветку и затем аккуратно очистить reflog:

1. `git branch recovered HEAD@{n}`
2. `git push -u origin recovered`
3. `git reflog expire --expire=now --all`
4. `git gc --prune=now`
5. `git reflog show`

Подсказка: Сначала восстановить и опубликовать, затем очистить и проверить.

Ответ:

- `git branch recovered HEAD@{n}`
- `git push -u origin recovered`
- `git reflog expire --expire=now --all`
- `git gc --prune=now`
- `git reflog show`

Шаг 12:

Миссия 12

Расположите шаги безопасного восстановления в командной среде, чтобы не мешать коллегам:

1. `git fetch origin`
2. `git reflog`
3. `create branch locally`
4. `git push origin recovered`
5. удалить локальный временный reflog записи при необходимости

Подсказка: Сначала обновить удалёнку, затем локально восстановить, потом опубликовать, потом чистить.

Ответ:

- `git fetch origin`
- `git reflog`
- `create branch locally`
- `git push origin recovered`
- удалить локальный временный reflog записи при необходимости

Шаг 13:

Миссия 13

Расположите шаги, если нужно восстановить ветку и затем слить изменения в текущую ветку:

1. git branch recovered HEAD@{n}
2. git checkout master
3. git merge recovered
4. git push origin master
5. git branch -d recovered

Подсказка: Сначала восстановить ветку, потом переключиться, слить, опубликовать и убрать временную ветку.

Ответ:

- git branch recovered HEAD@{n}
- git checkout master
- git merge recovered
- git push origin master
- git branch -d recovered

Шаг 14:

Миссия 14

Расположите этапы восстановления потерянной ветки с проверкой её содержимого:

1. git reflog
2. git checkout --detach HEAD@{n}
3. git log или git diff для проверки
4. git branch restored HEAD@{n}
5. git checkout restored

Подсказка: Используйте detached режим, чтобы убедиться, что коммит тот, который нужен, прежде чем закреплять ветку.

Ответ:

- git reflog
- git checkout --detach HEAD@{n}
- git log или git diff для проверки
- git branch restored HEAD@{n}
- git checkout restored

Шаг 15:

Миссия 15

Расположите шаги возврата после случайного git reset --hard , если reflog под рукой:

1. git reflog
2. найти подходящий HEAD@{n}
3. git checkout --detach HEAD@{n}
4. git branch recovered
5. git checkout recovered

Подсказка: Используйте detached состояние для проверки перед возвратом на ветку.

Ответ:

- git reflog
- найти подходящий HEAD@{n}
- git checkout --detach HEAD@{n}
- git branch recovered
- git checkout recovered

 Шаг 16:

Миссия 16

Расположите полную стратегию восстановления и публикации удалённой ветки:

1. git reflog
2. git checkout --detach HEAD@{n}
3. git log / diff для оценки
4. git branch recovered HEAD@{n}
5. git checkout recovered
6. git push -u origin recovered

Подсказка: Сначала исследовать, затем создать ветку, переключиться и опубликовать.

Ответ:

- git reflog
- git checkout --detach HEAD@{n}
- git log / diff для оценки
- git branch recovered HEAD@{n}
- git checkout recovered
- git push -u origin recovered

 Урок 4 - Ошибки при восстановлении и их решение

 Шаг 01:

Миссия 1

Вы случайно удалили локальную ветку — `git branch -D featureX`. Что вы сделаете в первую очередь?

Подсказка: Сначала нужно оценить, есть ли ссылка на ветку в reflog или удалённом репозитории.

Ответ:

- **Посмотреть `git reflog` для поиска хэша удалённой ветки**
- Немедленно пересоздать ветку с нуля
- **Игнорировать удаление** — ветка уже удалена навсегда

 Шаг 02:

Миссия 2

Вы нашли в `git reflog` запись вроде `abc1234 HEAD@{3}: checkout featureX`. Что это означает?

Подсказка: Обратите внимание, что reflog показывает перемещения HEAD.

Ответ:

- **Была сделана попытка перейти на ветку featureX** — хэш может быть началом её истории

- Это означает, что branch featureX был удалён удалённо
- **Это** — автоматический коммит слияния

🔗 Шаг 03:

Миссия 3

Вы хотите восстановить удалённую ветку `featureX` по хэшу `abc1234`. Какой из следующих вариантов наиболее корректен?

Подсказка: Помните, что восстановление ветки — это просто создание новой ссылки на коммит.

Ответ:

- **`git branch featureX abc1234`**
- `git checkout featureX`
- `git reset --hard abc1234`

🔗 Шаг 04:

Миссия 4

Восстановили ветку локально, но при попытке `git push origin featureX` получаете ошибку: `error: src refspec featureX does not match any`. Что сейчас неверно?

Подсказка: Подумайте, существует ли локальная ветка под таким именем.

Ответ:

- **Локальная ветка featureX ещё не создана (не указывает на коммит)**
- Удалённый репозиторий не разрешает push
- Неправиль хэш был использован при восстановлении

🔗 Шаг 05:

Миссия 5

Вы восстановили ветку локально, пушите и получаете: `error: failed to push some refs to ... (non-fast-forward)`. Что вероятнее всего произошло?

Подсказка: Часто удалённая ветка была перезаписана или обнулялась.

Ответ:

- **Удалённая ветка изменилась (есть более свежие коммиты), push требует force**
- Нет у вас прав пушить
- Хэш был неверным

🔗 Шаг 06:

Миссия 6

Сопоставьте ошибки с их причинами при попытке восстановить ветку:

Ошибки:

Ответ:

- error: src refspec does not match any
- error: failed to push some refs to (non-fast-forward)
- error: remote ref not found
- fatal: ambiguous argument

Причины:

1. Нет локальной ветки с указанным именем

2. Удалённая ветка уже была удалена в удалённом репозитории
3. **Локальная ветка отстаёт от удалённой** — требуется force-пуш
4. Хэш или имя ветки указаны неверно или неоднозначно

Подсказка: Обратите внимание на ключевые слова: "does not match any" — локальной ссылки не существует и т.д.

- **error: src refspec does not match any** — Нет локальной ветки с указанным именем
- **error: failed to push some refs to (non-fast-forward)** — Локальная ветка отстаёт от удалённой — требуется force-пуш
- **error: remote ref not found** — Удалённая ветка уже была удалена в удалённом репозитории
- **fatal: ambiguous argument** — Хэш или имя ветки указаны неверно или неоднозначно

Шаг 07:

Миссия 7

Сопоставьте действия с описанием поведения при восстановлении ветки:

Действия:

Ответ:

- ``git branch featureX abc1234``
- ``git checkout -b featureX abc1234``
- ``git push origin featureX``
- ``git push --force origin featureX``

Поведение:

1. Создаёт локальную ветку указывающую на указанный коммит
2. То же самое + сразу переключается на неё
3. Отправляет ветку на удалённый репозиторий, если нет конфликтов
4. Перезаписывает удалённую ветку принудительно

Подсказка: Команда ``checkout -b`` автоматически переключается; ``--force`` говорит о перезаписи.

- **git branch featureX abc1234** — Создаёт локальную ветку указывающую на указанный коммит
- **git checkout -b featureX abc1234** — То же самое + сразу переключается на неё
- **git push origin featureX** — Отправляет ветку на удалённый репозиторий, если нет конфликтов
- **git push --force origin featureX** — Перезаписывает удалённую ветку принудительно

Шаг 08:

Миссия 8

Сопоставьте стадии восстановления ветки с их задачами:

Стадии:

Ответ:

- Поиск в reflog
- Создание локальной ветки
- Проверка изменений
- Пуш на удалённый

Задачи:

1. Найти потенциальный коммит, где ветка существовала
2. Установить новую ссылку на коммит как ветку
3. Убедиться, что всё корректно (новые коммиты, нет потерь)
4. Передать ветку в удалённый репозиторий

Подсказка: Восстановление — это последовательность: поиск → локальная регистрация → проверка → пуш.

- **Поиск в reflog** — Найти потенциальный коммит, где ветка существовала
- **Создание локальной ветки** — Установить новую ссылку на коммит как ветку
- **Проверка изменений** — Убедиться, что всё корректно (новые коммиты, нет потерь)
- **Пуш на удалённый** — Передать ветку в удалённый репозиторий

Шаг 09:

Миссия 9

Сопоставьте варианты force-пуша с их эффектами:

Варианты:

Ответ:

- ``git push --force``
- ``git push --force-with-lease``
- ``git push --force origin featureX``
- ``git push --force-with-lease origin featureX``

Эффекты:

1. Принудительно перезапишет удалённую ветку, проигнорировав изменения
2. То же, но проверит, что удалённая ветка не сменилась с момента последнего fetch
3. Принудительный push конкретной ветки
4. Безопасный force на конкретной ветке с проверкой lease

Подсказка: Разница между ``--force`` и ``--force-with-lease`` в защите от затирания чужих изменений.

- **git push --force** — Принудительно перезапишет удалённую ветку, проигнорировав изменения
- **git push --force-with-lease** — То же, но проверит, что удалённая ветка не сменилась с момента последнего fetch
- **git push --force origin featureX** — Принудительный push конкретной ветки
- **git push --force-with-lease origin featureX** — Безопасный force на конкретной ветке с проверкой lease

Шаг 10:

Миссия 10

Расположите шаги восстановления удалённой ветки в правильном порядке:

1. Проверка запретов на force-push в удалённом
2. Поиск коммита через reflog
3. Создание локальной ветки на найденном коммите
4. Пуш восстановленной ветки в удалённый репозиторий
5. Проверка истории и коммитов

Подсказка: Не пытайтесь пушить, если удалённый не разрешает force; проверьте это заранее.

Ответ:

- Поиск коммита через reflog
- Проверка запретов на force-push в удалённом
- Создание локальной ветки на найденном коммите
- Проверка истории и коммитов
- Пуш восстановленной ветки в удалённый репозиторий

 Шаг 11:

Миссия 11

Расположите шаги диагностики ошибки push:

1. Получить свежие ссылки с удалённого (``git fetch``)
2. Проверить локальный HEAD vs удалённый
3. Убедиться, что локальная ветка существует
4. Попытаться выполнить ``git push`` снова
5. Восстановить ветку, если она отсутствует

Подсказка: Ошибки push часто проявляются из-за рассинхронизации refs.

Ответ:

- Получить свежие ссылки с удалённого (``git fetch``)
- Проверить локальный HEAD vs удалённый
- Убедиться, что локальная ветка существует
- Восстановить ветку, если она отсутствует
- Попытаться выполнить ``git push`` снова

 Шаг 12:

Миссия 12

Расположите шаги восстановления ветки с защитой lease:

1. Запросить ``git fetch``
2. Сравнить локальный и удалённый refs
3. Выполнить ``git branch featureX``
4. Запустить ``git push --force-with-lease origin featureX``
5. Проверить наличие изменений в удалённой ветке перед пушем

Подсказка: ``--force-with-lease`` должен гарантировать, что вы не перезапишете чужие изменения.

Ответ:

- Запросить ``git.fetch``
- Сравнить локальный и удалённый refs
- Выполнить ``git branch featureX``
- Проверить наличие изменений в удалённой ветке перед пушем
- Запустить ``git push --force-with-lease origin featureX``

 Шаг 13:

Миссия 13

Расположите шаги, когда восстановление ветки влияет на CI/CD:

1. Восстановить локально ветку
2. Протестировать ветку локально или в staging
3. Пушить с `--force-with-lease`
4. Мониторить пуш и проверку CI
5. **Восстановление, если CI падает** — отменить или исправить

Подсказка: CI может отозвать ветку, если история кардинально изменится.

Ответ:

- Восстановить локально ветку
- Протестировать ветку локально или в staging
- Пушить с `--force-with-lease`
- Мониторить пуш и проверку CI
- **Восстановление, если CI падает** — отменить или исправить

Шаг 14:

Миссия 14

Расположите шаги восстановления ветки после ребейза:

1. Найти старый хэш ветки в reflog
2. Создать временную ветку на старом коммите
Слить изменения из временной ветки в текущую ветку
3. Уничтожить временную ветку
4. Пуш принудительно с `--force-with-lease`

Подсказка: Ребейз мог увести ветку в новую историю — нужно вернуть старую как базу.

Ответ:

- Найти старый хэш ветки в reflog
- Создать временную ветку на старом коммите
- Слить изменения из временной ветки в текущую ветку
- Уничтожить временную ветку
- Пуш принудительно с `--force-with-lease`

Шаг 15:

Миссия 15

Расположите шаги восстановления нескольких удалённых веток в пакете:

1. Сделать `git fetch --all`
2. **Для каждой ветки** — найти хэш в reflog
3. Создать локальные ветки по хэсам
4. Проверить локальные коммиты и конфликты
5. Пушить все ветки (с `--force-with-lease`) к удалённому

Подсказка: Массовое восстановление — это просто последовательное восстановление многих веток.

Ответ:

- Сделать `git fetch --all`
- **Для каждой ветки** — найти хэш в reflog
- Создать локальные ветки по хэсам

- Проверить локальные коммиты и конфликты
- Пушить все ветки (с `--force-with-lease`) к удалённому

Шаг 16:

Миссия 16

Расположите действия при восстановлении ветки, если удалённый репозиторий запретил force-пуш:

1. Связаться с администратором репозитория
2. Предложить создать новую ветку под другим именем
3. Проверить историю удалённой ветки
4. Объяснить причину восстановления
5. **После разрешения** — выполнить пуш с force-with-lease

Подсказка: Иногда технические запреты нужно разрешать через политику, а не через force.

Ответ:

- Связаться с администратором репозитория
- Предложить создать новую ветку под другим именем
- Проверить историю удалённой ветки
- Объяснить причину восстановления
- **После разрешения** — выполнить пуш с force-with-lease

Модуль 19 - Автоматизация через Git hooks

Урок 1 - pre-commit для линтинга и тестов

Шаг 01:

Миссия 1

Вы — разработчик в компании «CodeGuard». Вам нужно настроить pre-commit, чтобы запускать линтер перед коммитом. Что первым делом вы должны сделать?

Подсказка: Git hooks лежат в каталоге .git/hooks.

Ответ:

- **Создать файл .git/hooks/pre-commit и сделать его исполняемым**
- Добавить скрипт в package.json (npm)
- Запустить линтер вручную после коммита

Шаг 02:

Миссия 2

Ваша команда использует Python и flake8. Какой способ лучше всего интегрировать запуск flake8 в pre-commit?

Подсказка: Нужно, чтобы линтер проверял изменённые файлы, а не весь репозиторий.

Ответ:

- **Запустить flake8 только на изменённых файлах через git diff**
- Запускать flake8 на весь проект при каждом коммите
- Запустить flake8 на файлы .py во всей рабочей директории

🔮 Шаг 03:

Миссия 3

Вы написали скрипт pre-commit, но он не запускается — вы получаете «permission denied». Что вероятнее всего забыли?

Подсказка: Файлы хук должны быть исполняемыми.

Ответ:

- **Не поставили бит «execute» на файл pre-commit**
- Не добавили shebang в начало файла
- Не переименовали файл в pre-commit.sample

🔮 Шаг 04:

Миссия 4

Вы хотите игнорировать lintr'ом определённые директории (например, tests/fixtures). Где лучше это настроить?

Подсказка: Почитайте, как flake8 или ESLint игнорируют пути.

Ответ:

- **Добавить исключения в конфигурацию линтера**
- Жёстко прописать if в скрипте pre-commit
- Игнорировать эти директории через .gitignore

🔮 Шаг 05:

Миссия 5

Ваша команда добавила вызов запуска модульных тестов в pre-commit. Какое поведение будет наиболее дружелюбным?

Подсказка: Пользователи не любят медленные коммиты.

Ответ:

- **Запускать только тесты, затрагивающие изменённые файлы**
- Запускать весь тест-набор при каждом коммите
- Запасной вариант: делать коммит и потом запускать тесты вручную

🔮 Шаг 06:

Миссия 6

Сопоставьте команды Git hook и их назначение.

Команды Git hook:

Ответ:

- pre-commit
- pre-push
- prepare-commit-msg
- commit-msg

Назначения:

1. Проверка кода перед коммитом
2. Изменение сообщения коммита перед отображением пользователю
3. Проверка сообщения коммита (lint сообщения)
4. Запуск перед пушем, например, интеграционные тесты

Подсказка: pre-commit — перед фиксацией, commit-msg — после сообщения, pre-push — перед push.

- **pre-commit** — Проверка кода перед коммитом
- **pre-push** — Запуск перед пушем, например, интеграционные тесты
- **prepare-commit-msg** — Изменение сообщения коммита перед отображением пользователю
- **commit-msg** — Проверка сообщения коммита (lint сообщения)

🔮 Шаг 07:

Миссия 7

Сопоставьте фразу с её смыслом в контексте pre-commit:

Ответ:

- Hook fails
 - Hook passes
 - Skip hook
 - Staged files mismatch
1. Хук возвращает ненулевой код => коммит блокируется
 2. Хук завершился с кодом 0 => коммит продолжается
 3. Файлы в staging изменились после проверки линтером
 4. Hook пропущен (например, при --no-verify)

Подсказка: У разных терминов — разные состояния выполнения hooks.

- **Hook fails** — Хук возвращает ненулевой код => коммит блокируется
- **Hook passes** — Хук завершился с кодом 0 => коммит продолжается
- **Skip hook** — Hook пропущен (например, при --no-verify)
- **Staged files mismatch** — Файлы в staging изменились после проверки линтером

🔮 Шаг 08:

Миссия 8

Сопоставьте термины с их назначением:

Ответ:

- shebang
 - chmod +x
 - git diff --name-only
 - exit status
1. Указывает интерпретатор командного файла
 2. Сделать файл исполняемым
 3. Список изменённых в индексе файлов
 4. Код возврата процесса (0 или ≠0)

Подсказка: Комбинация shebang + execute bit + проверка exit status — основа hook-скриптов.

- **shebang** — Указывает интерпретатор командного файла
- **chmod +x** — Сделать файл исполняемым
- **git diff --name-only** — Список изменённых в индексе файлов
- **exit status** — Код возврата процесса (0 или ≠0)

Шаг 09:

Миссия 10

Расположите действия при запуске pre-commit в правильной последовательности:

1. Проверить изменённые файлы (staged)
2. Запустить линтер / тесты
3. Возвратить код выхода
4. **Если код $\neq 0$** — остановить коммит
5. **Если код $= 0$** — продолжить коммит

Подсказка: Hook-интерпретатор выполняет шаги строго по порядку.

Ответ:

- Проверить изменённые файлы (staged)
- Запустить линтер / тесты
- Возвратить код выхода
- **Если код $\neq 0$** — остановить коммит
- **Если код $= 0$** — продолжить коммит

Шаг 10:

Миссия 11

Расположите шаги отладки, когда pre-commit не запускается:

1. Проверить права chmod
2. Убедиться, что файл называется pre-commit
3. Добавить shebang (например, `#!/usr/bin/env bash`)
4. Посмотреть логи или echo в начале скрипта
5. Убедиться, что `.git/hooks/pre-commit` присутствует

Подсказка: Проблемы часто — в имени, правах или shebang.

Ответ:

- Проверить права chmod
- Убедиться, что файл называется pre-commit
- Добавить shebang (например, `#!/usr/bin/env bash`)
- Посмотреть логи или echo в начале скрипта
- Убедиться, что `.git/hooks/pre-commit` присутствует

Шаг 11:

Миссия 12

Расположите этапы добавления тестов в hook:

1. Собрать список изменённых файлов
2. Запустить соответствующие тесты по этому списку
3. Проанализировать результат (exit status)
4. В случае ошибки вернуть $\neq 0$
5. **Если всё ОК** — вернуть 0 и продолжить коммит

Подсказка: Логика похожа на линтинг.

Ответ:

- Собрать список изменённых файлов

- Запустить соответствующие тесты по этому списку
- Проанализировать результат (exit status)
- В случае ошибки вернуть $\neq 0$
- **Если всё ОК** — вернуть 0 и продолжить коммит

🔧 Шаг 12:

Миссия 13

Расположите шаги настройки локального репозитория для автоматического включения hook в clone:

1. Создать директорию scripts/hooks внутри репозитория
2. Добавить копирование при install (например, через setup.py или Makefile)
3. Поместить шаблон pre-commit в scripts/hooks/pre-commit
4. После клонирования запустить скрипт установки hook
5. Убедиться, что скопированный hook исполняемый (chmod +x)

Подсказка: Hook не вернётся сам — нужен механизм установки.

Ответ:

- Создать директорию scripts/hooks внутри репозитория
- Добавить копирование при install (например, через setup.py или Makefile)
- Поместить шаблон pre-commit в scripts/hooks/pre-commit
- После клонирования запустить скрипт установки hook
- Убедиться, что скопированный hook исполняемый (chmod +x)

🔧 Шаг 13:

Миссия 14

Расположите стадии, если hook должен проверять и линт и тесты, но запуск тестов только в случае, если линт пройден:

1. Запустить линт
2. Проверить exit status линта
3. **Если линт не прошёл** — вернуть $\neq 0$ и остановить
4. **Если линт прошёл** — запустить тесты на изменённых файлах
5. Вернуть exit status тестов или 0

Подсказка: Тестировать нужно только если линт успешно.

Ответ:

- Запустить линт
- Проверить exit status линта
- **Если линт не прошёл** — вернуть $\neq 0$ и остановить
- **Если линт прошёл** — запустить тесты на изменённых файлах
- Вернуть exit status тестов или 0

🔧 Шаг 14:

Миссия 15

Расположите шаги, когда вы добавляете автозапуск hook в CI pipeline:

1. Сделать скрипт hook утилитой в проекте
2. Добавить шаг в CI перед коммитами или merge

3. Запустить линт и тесты через тот же скрипт
4. Обработать exit code и остановить CI при ошибке
5. Убедиться, что локальные hook и CI запускают одинаковые наборы правил

Подсказка: CI и локальный pre-commit должны быть синхронизированы.

Ответ:

- Сделать скрипт hook утилитой в проекте
- Добавить шаг в CI перед коммитами или merge
- Запустить линт и тесты через тот же скрипт
- Обработать exit code и остановить CI при ошибке
- Убедиться, что локальные hook и CI запускают одинаковые наборы правил

Шаг 15:

Миссия 16

Расположите последовательность действий для миграции с ручного линтинга на автоматический pre-commit:

1. Добавить конфиг линтера
2. Настроить локальный pre-commit
3. Сообщить команде о новой политике
4. Промигрировать существующие коммиты (проверить весь код)
5. Поддерживать CI и локальные проверки синхронизированными

Подсказка: Нельзя внедрить хуки без коммуникации и первого полного прогона линта.

Ответ:

- Добавить конфиг линтера
- Настроить локальный pre-commit
- Сообщить команде о новой политике
- Промигрировать существующие коммиты (проверить весь код)
- Поддерживать CI и локальные проверки синхронизированными

Шаг 16:

Миссия 17

Расположите этапы, если при merge приходит коммит с кодом, который ломает линт, и вы хотите откатить merge:

1. Hook обнаруживает ошибку линта
2. Hook возвращает код $\neq 0$ и блокирует merge
3. Пользователь видит сообщения об ошибках
4. Он исправляет код в ветке и делает новый merge/pull request
5. Hook проверяет снова и разрешает merge при успехе

Подсказка: Merge не должен проходить, если линт сломался.

Ответ:

- Hook обнаруживает ошибку линта
- Hook возвращает код $\neq 0$ и блокирует merge
- Пользователь видит сообщения об ошибках
- Он исправляет код в ветке и делает новый merge/pull request
- Hook проверяет снова и разрешает merge при успехе

Урок 2 - pre-push для проверки формата

Шаг 01:

Миссия 1

Разработчик Катя хочет добавить pre-push hook, чтобы автоматически проверять формат кода перед отправкой изменений в репозиторий. Что первым делом нужно сделать?

Подсказка: Подумайте, какой файл отвечает за pre-push hook в Git.

Ответ:

- **Создать или отредактировать файл `.git/hooks/pre-push`**
- Написать скрипт для post-commit hook
- Настроить remote репозиторий
- Добавить файл `.pre-push` в корень проекта

Шаг 02:

Миссия 2

Дмитрий хочет, чтобы pre-push hook проверял только изменённые файлы с расширением `.js`. Какой командой можно получить список таких файлов в hook?

Подсказка: Используйте `git diff` с флагами для выбора изменённых файлов.

Ответ:

- **`git diff --cached --name-only -- '*.js'`**
- `git status --short '*.js'`
- `git ls-files '*.js'`
- `git show HEAD -- '*.js'`

Шаг 03:

Миссия 3

Анна написала скрипт проверки форматирования кода с помощью ESLint. Что нужно сделать, чтобы pre-push hook не позволял пушить при ошибках форматирования?

Подсказка: Важно, какой код возврата возвращает скрипт при ошибках.

Ответ:

- **Сделать так, чтобы скрипт возвращал ненулевой код выхода при ошибках**
- Выводить ошибки в консоль, но возвращать 0
- Запускать скрипт в фоне и всегда возвращать 0
- Игнорировать ошибки форматирования

Шаг 04:

Миссия 4

Свяжите термин с правильным объяснением в контексте pre-push hook.

Подсказка: Подумайте о ролях и целях этих элементов.

Термины:

Ответ:

- Hook
- Code style check
- Exit code
- Staged files

Определения:

1. Файлы, подготовленные для коммита
 2. Механизм Git для запуска скриптов на события
 3. Проверка соответствия кода стандартам оформления
 4. Код, который сигнализирует об успехе или ошибке выполнения
- **Hook** — Механизм Git для запуска скриптов на события
 - **Code style check** — Проверка соответствия кода стандартам оформления
 - **Exit code** — Код, который сигнализирует об успехе или ошибке выполнения
 - **Staged files** — Файлы, подготовленные для коммита

Шаг 05:

Миссия 5

Сопоставьте части скрипта pre-push с их назначением.

Подсказка: Вспомните, как выглядит базовая структура hook.

Части скрипта:

Ответ:

- Получение списка файлов
- Запуск проверки форматирования
- Обработка результата проверки
- Вывод сообщения об ошибке

Назначения:

1. Проверка возвращаемого кода и прерывание пуша при ошибке
 2. Отбор файлов для проверки
 3. Выводит предупреждение пользователю
 4. Запускает ESLint или другой формater
- **Получение списка файлов** — Отбор файлов для проверки
 - **Запуск проверки форматирования** — Запускает ESLint или другой формater
 - **Обработка результата проверки** — Проверка возвращаемого кода и прерывание пуша при ошибке
 - **Вывод сообщения об ошибке** — Выводит предупреждение пользователю

Шаг 06:

Миссия 6

Сопоставьте распространённые ошибки pre-push hook с их последствиями.

Подсказка: Подумайте, что происходит, если hook не настроен корректно.

Ошибки:

Ответ:

- Hook не имеет права на исполнение
- Hook всегда возвращает 0
- Скрипт вызывает ошибку синтаксиса
- Неправильный путь к ESLint

Последствия:

1. Hook не запускается

2. Пуш разрешён даже при ошибках
3. Hook прерывается с ошибкой
4. Проверка не выполняется

- **Hook не имеет права на исполнение** — Hook не запускается
- **Hook всегда возвращает 0** — Пуш разрешён даже при ошибках
- **Скрипт вызывает ошибку синтаксиса** — Hook прерывается с ошибкой
- **Неправильный путь к ESLint** — Проверка не выполняется

🔧 Шаг 07:

Миссия 7

Иван хочет, чтобы pre-push hook проверял не только формат кода, но и запускал тесты. Как лучше объединить эти проверки в hook?

Подсказка: Подумайте о последовательности и логике выполнения команд.

Ответ:

- **Сначала запускать проверку форматирования, затем тесты, прерывая пуш при любой ошибке**
- Запускать тесты и форматирование параллельно, не прерывая пуш
- Проверять только форматирование, тесты запускать вручную
- Игнорировать ошибки тестов, если форматирование прошло

🔧 Шаг 08:

Миссия 8

Мария хочет, чтобы pre-push hook проверял файлы на большом проекте, но не замедлял процесс слишком сильно. Что она может сделать?

Подсказка: Подумайте, как можно ограничить проверяемый объём.

Ответ:

- **Проверять только изменённые или добавленные файлы**
- Проверять весь проект каждый раз
- Запускать проверки только по ночам
- Игнорировать проверки при большом количестве файлов

🔧 Шаг 09:

Миссия 9

Сопоставьте фрагменты кода shell-скрипта для pre-push с их функциями.

Подсказка: Обратите внимание на команды git и их назначения.

Фрагменты:

Ответ:

- `files=$(git diff --cached --name-only -- '*.py')`
- `eslint $files`
- `if [ $? -ne 0 ]; then exit 1; fi`
- `echo "Code style errors found! Push aborted."`

Назначения:

1. Получить список изменённых файлов с расширением .py
2. Запустить ESLint на этих файлах

3. Прервать пуш при ошибках
4. Вывести сообщение об ошибках

- **files=\$(git diff --cached --name-only -- '*.py')** — Получить список изменённых файлов с расширением .py
- **eslint \$files** — Запустить ESLint на этих файлах
- **if [\$? -ne 0]; then exit 1; fi** — Прервать пуш при ошибках
- **echo "Code style errors found! Push aborted."** — Вывести сообщение об ошибках

Шаг 10:

Миссия 10

Свяжите инструменты с их назначением при автоматизации pre-push hook.

Подсказка: Некоторые из них помогают проверять стиль, другие — тесты.

Инструменты:

Ответ:

- ESLint
- Prettier
- Jest
- Husky

Назначения:

1. Автоматизация запуска Git хуков
 2. Проверка и исправление стиля JavaScript кода
 3. Проверка правильности форматирования кода
 4. Запуск юнит-тестов
- **ESLint** — Проверка и исправление стиля JavaScript кода
 - **Prettier** — Проверка правильности форматирования кода
 - **Jest** — Запуск юнит-тестов
 - **Husky** — Автоматизация запуска Git хуков

Шаг 11:

Миссия 11

Соотнесите возможные причины с ситуациями, при которых pre-push hook не срабатывает.

Подсказка: Иногда проблема кроется в правах или конфигурации.

Причины:

Ответ:

- Файл pre-push не имеет права на исполнение
- Путь к скрипту указан неверно
- Hook отключён в настройках Git
- Пуш происходит через GUI-клиент без поддержки хуков

Ситуации:

1. Скрипт не запускается
2. Проверка не проводится
3. Hook не вызывается
4. Пуш проходит без проверок

- **Файл pre-push не имеет права на исполнение** — Скрипт не запускается
- **Путь к скрипту указан неверно** — Проверка не проводится
- **Hook отключён в настройках Git** — Hook не вызывается
- **Пуш происходит через GUI-клиент без поддержки хуков** — Пуш проходит без проверок

🔧 Шаг 12:

Миссия 12

Отсортируйте шаги настройки pre-push hook для проверки форматирования кода:

1. Записать скрипт проверки форматирования
2. Создать или отредактировать .git/hooks/pre-push
3. Дать права на исполнение файлу hook
4. Проверить работу hook на тестовом пуше
5. Добавить проверки для только изменённых файлов

🔧 Подсказка:

Начинайте с написания логики, потом интегрируйте в Git, и обязательно тестируйте.

Ответ:

- Записать скрипт проверки форматирования
- Создать или отредактировать .git/hooks/pre-push
- Дать права на исполнение файлу hook
- Добавить проверки для только изменённых файлов
- Проверить работу hook на тестовом пуше

🔧 Шаг 13:

Миссия 13

Разработчик Вася добавил pre-push hook, который проверяет формат кода перед пушем. Что произойдет, если в проекте есть ошибки форматирования?

Подсказка: Подумайте, что делает pre-push hook с ошибками.

Ответ:

- **Пуш не пройдет, пока ошибки не исправлены**
- Пуш пройдет, но с предупреждением
- Пуш пройдет без проверок

🔧 Шаг 14:

Миссия 14

📄 Сопоставьте команды и их назначение при использовании pre-push hook для проверки формата кода.

Команды:

Ответ:

- git diff --cached
- eslint --fix
- exit 1
- git push

Назначения:

1. Автоматическое исправление ошибок форматирования
2. Проверка изменений перед пушем
3. Прерывание операции пуша при ошибках
4. Отправка коммитов на сервер

Подсказка:

Подумайте, какие команды помогают проверить, исправить и отправить код.

- **git diff --cached** — Проверка изменений перед пушем
- **eslint --fix** — Автоматическое исправление ошибок форматирования
- **exit 1** — Прерывание операции пуша при ошибках
- **git push** — Отправка коммитов на сервер

Шаг 15:

Миссия 15

 Сопоставьте ошибки форматирования с действиями pre-push hook.

Ошибки:

Ответ:

- Неправильные отступы
- Отсутствие точки с запятой
- Длинные строки кода
- Использование табуляции вместо пробелов

Действия:

1. Автоматическая замена табуляции на пробелы
2. Сообщение об ошибке и отмена пуша
3. Автоматическое добавление точек с запятой
4. Подсказка исправить вручную

Подсказка:

Обратите внимание, какие ошибки можно исправить автоматически, а какие требуют вмешательства.

- **Неправильные отступы** — Подсказка исправить вручную
- **Отсутствие точки с запятой** — Автоматическое добавление точек с запятой
- **Длинные строки кода** — Сообщение об ошибке и отмена пуша
- **Использование табуляции вместо пробелов** — Автоматическая замена табуляции на пробелы

Шаг 16:

Миссия 16

Расположите этапы работы pre-push hook для проверки формата в правильном порядке:

1. Проверка изменений на формат
2. Автоматическое исправление ошибок
3. Вывод сообщения об ошибках
4. Прерывание пуша при критических ошибках
5. Разрешение пуша, если ошибок нет

Подсказка:

Подумайте, что делает хук перед отправкой кода и как реагирует на ошибки.

Ответ:

- Проверка изменений на формат
- Автоматическое исправление ошибок
- Вывод сообщения об ошибках
- Прерывание пуша при критических ошибках
- Разрешение пуша, если ошибок нет

Шаг 17:

Миссия 17

Упорядочьте шаги настройки pre-push hook для проверки формата кода:

1. Создание скрипта pre-push с проверками
2. Добавление прав на исполнение скрипта
3. Тестирование скрипта на локальном репозитории
4. Коммит и пуш скрипта в общий репозиторий
5. Информирование команды о новых проверках

Подсказка:

Подумайте о правильной последовательности от создания до внедрения хука в рабочий процесс.

Ответ:

- Создание скрипта pre-push с проверками
- Добавление прав на исполнение скрипта
- Тестирование скрипта на локальном репозитории
- Коммит и пуш скрипта в общий репозиторий
- Информирование команды о новых проверках

Урок 3 - Практика свой hook на Go или Python

Шаг 01:

Миссия 1

Вы программист Алексей, который впервые решил написать Git hook на Python для автоматической проверки кода перед коммитом. Какой из вариантов лучше всего описывает назначение pre-commit hook?

Подсказка: Подумайте, когда именно должен сработать этот hook, чтобы предотвратить ошибки.

Ответ:

- **Запуск проверки кода перед фиксацией изменений в локальном репозитории**
- Автоматическое слияние веток при конфликте
- Отправка уведомлений о новых коммитах на почту
- Автоматическое обновление удалённого репозитория

Шаг 02:

Миссия 2

Программистка Марина хочет написать hook на Go, который будет блокировать коммит, если в сообщении отсутствует Jira-таск. Что из ниже перечисленного будет наиболее подходящим способом реализовать такую проверку?

Подсказка: Сообщение коммита можно получить с помощью специального файла.

Ответ:

- **Считать содержимое файла `.git/COMMIT_EDITMSG` и проверить наличие шаблона Jira-таска**
- Считать все файлы в рабочей директории и искать Jira-таск в них
- Вызвать `git log` и проанализировать последние коммиты
- Отправить запрос к Jira API для проверки задачи

Шаг 03:

Миссия 3

Дмитрий создал hook на Python, который должен запускать тесты при каждом пуше. Что из ниже перечисленного НЕ относится к возможным событиям, к которым можно привязать hook?

Подсказка: Подумайте, какие действия в Git вызывают хуки, а какие — нет.

Ответ:

- **Открытие нового терминала в системе**
- Команда `git push`
- Команда `git commit`
- Обновление рефов на удалённом сервере

Шаг 04:

Миссия 4

Наталья хочет, чтобы её hook на Go автоматически форматировал изменённые файлы перед коммитом. Что из перечисленного нужно учитывать при написании такого hook?

Подсказка: Автоматизация форматирования влияет на файлы, которые попадут в коммит.

Ответ:

- **Hook должен изменить файлы и добавить их в индекс `git` (`git add`) перед завершением коммита**
- Hook должен просто вывести предупреждение, но не изменять файлы
- Hook должен создавать новый коммит с отформатированными файлами
- Hook должен удалять все незакоммиченные изменения

Шаг 05:

Миссия 5

Иван написал hook, который должен проверять стиль кода на Python с помощью `flake8`. При запуске hook обнаруживается, что `flake8` не установлен в системе. Как лучше обработать эту ситуацию?

Подсказка: Hook должен быть информативным и не ломать рабочий процесс без причины.

Ответ:

- **Вывести понятное сообщение об ошибке и прервать коммит**
- Игнорировать отсутствие `flake8` и пропустить проверку
- Автоматически устанавливать `flake8` без уведомления пользователя
- Отправить ошибку в лог и продолжить коммит

Шаг 06:

Миссия 6

Сопоставьте тип Git hook с его основным назначением.

Подсказка: Внимательно подумайте, когда срабатывает каждый из hook и что обычно выполняется в этот момент.

Ответ:

- pre-commit
 - commit-msg
 - post-commit
 - pre-push
1. Проверка или изменение сообщения коммита
 2. Запуск проверок и изменений перед сохранением изменений
 3. Действия после успешного коммита (например, уведомления)
 4. Проверка перед отправкой изменений на удалённый сервер
- **pre-commit** — Запуск проверок и изменений перед сохранением изменений
 - **commit-msg** — Проверка или изменение сообщения коммита
 - **post-commit** — Действия после успешного коммита (например, уведомления)
 - **pre-push** — Проверка перед отправкой изменений на удалённый сервер

Шаг 07:

Миссия 7

Василий хочет оптимизировать работу своего hook, написанного на Python. Соотнесите команды Git с их назначением в рамках hook.

Подсказка: Подумайте, какие команды помогают узнать состояние и содержимое индекса или рабочего каталога.

Ответ:

- git diff --cached
 - git rev-parse --verify HEAD
 - git ls-files --modified
 - git status --porcelain
1. Показать изменения, подготовленные к коммиту
 2. Проверить, существует ли коммит HEAD
 3. Показать изменённые, но ещё не подготовленные файлы
 4. Вывести краткий статус рабочих файлов
- **git diff --cached** — Показать изменения, подготовленные к коммиту
 - **git rev-parse --verify HEAD** — Проверить, существует ли коммит HEAD
 - **git ls-files --modified** — Показать изменённые, но ещё не подготовленные файлы
 - **git status --porcelain** — Вывести краткий статус рабочих файлов

Шаг 08:

Миссия 8

Сопоставьте сценарии использования hook с подходящими языками программирования для написания скриптов.

Подсказка: Обратите внимание на особенности систем и интеграции с инструментами.

Ответ:

- Маленький скрипт для проверки формата сообщения коммита
- Сложный hook для интеграции с внешними API
- Hook для быстрой проверки синтаксиса локальных файлов
- Hook для автоматической сборки и тестирования

1. Python
2. Go
3. Bash
4. Makefile / скрипты shell

- **Маленький скрипт для проверки формата сообщения коммита** — Bash
- **Сложный hook для интеграции с внешними API** — Go
- **Hook для быстрой проверки синтаксиса локальных файлов** — Python
- **Hook для автоматической сборки и тестирования** — Makefile / скрипты shell

 Шаг 09:

Миссия 9

Сопоставьте возможные ошибки hook с подходящими решениями.

Подсказка: Рассмотрите распространённые проблемы с hook и способы их устранения.

Ответ:

- Hook не срабатывает при коммите
- Hook срабатывает, но изменения не применяются
- Hook замедляет коммит
- Hook вызывает сбой при отсутствии определённых инструментов

1. Проверить права на выполнение и расположение файла
2. Убедиться, что hook добавляет изменённые файлы в индекс (git add)
3. Оптимизировать код, запускать только необходимые проверки
4. Добавить проверки наличия инструментов и информативные ошибки

- **Hook не срабатывает при коммите** — Проверить права на выполнение и расположение файла
- **Hook срабатывает, но изменения не применяются** — Убедиться, что hook добавляет изменённые файлы в индекс (git add)
- **Hook замедляет коммит** — Оптимизировать код, запускать только необходимые проверки
- **Hook вызывает сбой при отсутствии определённых инструментов** — Добавить проверки наличия инструментов и информативные ошибки

 Шаг 10:

Миссия 10

Определите соответствие между этапами разработки hook и их описанием.

Подсказка: Последовательность разработки важна для успешного результата.

Ответ:

- Планирование задачи
- Написание скрипта
- Тестирование hook в локальном репозитории

- Интеграция и автоматизация

1. Определение целей и критериев срабатывания hook
2. Кодирование логики на Python или Go
3. Запуск hook в разных сценариях и фиксация ошибок
4. Добавление hook в общий репозиторий и CI/CD

- **Планирование задачи** — Определение целей и критериев срабатывания hook
- **Написание скрипта** — Кодирование логики на Python или Go
- **Тестирование hook в локальном репозитории** — Запуск hook в разных сценариях и фиксация ошибок
- **Интеграция и автоматизация** — Добавление hook в общий репозиторий и CI/CD

Шаг 11:

Миссия 11

Расположите этапы создания собственного pre-commit hook на Python в правильном порядке:

1. Создать исполняемый файл с логикой проверки
2. Написать проверку на нужное условие (например, формат кода)
3. Разместить файл в папке .git/hooks с именем pre-commit
4. Проверить работу hook на тестовом коммите

Подсказка:

Ответ:

- Сначала пишется код проверки
- Потом создаётся файл с правильным именем и правами
- И наконец, проверяется работа на практике
- Написать проверку на нужное условие (например, формат кода)
- Создать исполняемый файл с логикой проверки
- Разместить файл в папке .git/hooks с именем pre-commit
- Проверить работу hook на тестовом коммите

Урок 4 - Интеграция hooks с CI

Шаг 01:

Миссия 1

Разработчик Анна хочет запускать линтер перед каждым пушем, чтобы не допускать ошибок в коде в CI. Какой hook Git подойдёт для этой задачи?

Подсказка: Этот hook срабатывает перед тем, как Git отправляет коммиты на удалённый репозиторий.

Ответ:

- pre-commit
- **pre-push**
- post-commit

Шаг 02:

Миссия 2

Борис хочет, чтобы после каждого пуша в ветку `main` запускались автоматические тесты через CI. Где лучше разместить hook?

Подсказка: Такой hook на стороне сервера часто делается через Git-сервер или CI, а не локально.

Ответ:

- **server-side hook (например, post-receive на сервере)**
- local pre-commit hook
- client-side post-checkout hook

🔗 Шаг 03:

Миссия 3

Ирина хочет, чтобы при попытке мержа веток проверялось, что все тесты зелёные, чтобы не сломать сборку. Какой подход логичнее?

Подсказка: Локальные хуки могут быть обойдены, если разработчик их удалит.

Ответ:

- Запустить тесты только в локальном pre-merge hook
- **Запретить merge через CI-платформу, если тесты падают**
- Запуск тестов в post-checkout hook

🔗 Шаг 04:

Миссия 4

Сергей хочет, чтобы при создании тега `vX.Y.Z` автоматически добавлялся changelog через CI. Какое событие Git можно использовать на сервере?

Подсказка: Этот hook срабатывает после того, как сервер получил новый реф (коммит, тег и др.).

Ответ:

- **post-receive**
- pre-receive
- update

🔗 Шаг 05:

Миссия 5

Команда хочет запретить push, если в коде есть TODO-комментарии. Где лучше реализовать проверку?

Подсказка: Локальные хуки могут быть изменены пользователями, а серверные — контролируются централизованно.

Ответ:

- pre-commit hook на каждом клиенте
- post-commit hook
- **pre-receive hook на Git-сервере**

🔗 Шаг 06:

Миссия 6

Екатерина хочет, чтобы после принятия Pull Request автоматически генерировался Docker-образ и был отправлен в регистр. Какой подход наиболее надежен?

Подсказка: CI обычно триггерится внешним сервисом, а не внутри локального репозитория.

Ответ:

- Hook pre-push запускает Docker в локальной машине
- **CI-система реагирует на merge event и строит образ**
- post-merge hook на клиенте

 Шаг 07:

Миссия 7

Команда захотела, чтобы перед отправкой кода в CI выполнялась проверка безопасности зависимостей (например, `npm audit`). Какой hook лучше использовать локально?

Подсказка: Локальный hook должен срабатывать перед push и может запускать произвольные скрипты.

Ответ:

- **pre-push hook**
- post-commit hook
- pre-receive hook

 Шаг 08:

Миссия 8

Задача: Сопоставьте hook с моментом его срабатывания

Hooks:

Ответ:

- pre-push
- pre-receive
- post-receive
- update

Моменты:

1. Перед отправкой данных с клиента на сервер
2. На сервере перед обновлением ветки
3. На сервере после обновления веток
4. На сервере при каждом обновлении конкретной ссылки (ветка или тег)

Подсказка: Разница между pre-receive и update — granular-уровень ссылки.

- **pre-push** — Перед отправкой данных с клиента на сервер
- **pre-receive** — На сервере перед обновлением ветки
- **post-receive** — На сервере после обновления веток
- **update** — На сервере при каждом обновлении конкретной ссылки (ветка или тег)

 Шаг 09:

Миссия 9

Задача: Сопоставьте термин с его ролью при интеграции hook и CI

Термины:

Ответ:

- Webhook
- CI job
- Hook script

- Payload

Роли:

1. HTTP-запрос от Git-сервера к CI при событии
2. Скрипт, запускаемый локально или на сервере при событии Git
3. Данные о событии, которые передаются между системами
4. Этап выполнения в CI (сборка, тесты, деплой)

Подсказка: Подумайте, что передаётся как JSON, и что запускает процесс CI.

- **Webhook** — HTTP-запрос от Git-сервера к CI при событии
- **Hook script** — Скрипт, запускаемый локально или на сервере при событии Git
- **Payload** — Данные о событии, которые передаются между системами
- **CI job** — Этап выполнения в CI (сборка, тесты, деплой)

Шаг 10:

Миссия 10

Задача: Сопоставьте событие CI с убедительной причиной интеграции hook

События CI:

Ответ:

- Запуск тестов
- Сборка артефактов
- Публикация пакета
- Принятие Pull Request

Причины:

1. Убедиться, что код корректен перед мёрджем
2. Подготовить бинарные файлы для деплоя
3. Немедленный релиз пакета после проверки
4. Принудительный запуск CI после внешнего пуша

Подсказка: Иногда hook выступает триггером к CI, а иногда CI выполняет задачи.

- **Запуск тестов** — Убедиться, что код корректен перед мёрджем
- **Сборка артефактов** — Подготовить бинарные файлы для деплоя
- **Публикация пакета** — Немедленный релиз пакета после проверки
- **Принятие Pull Request** — Принудительный запуск CI после внешнего пуша

Шаг 11:

Миссия 11

Задача: Сопоставьте инструмент с его ролью в цепочке hook → CI → deployment

Инструменты:

Ответ:

- GitLab CI/CD
- GitHub Actions
- Jenkins
- Bitbucket Pipelines

Роли:

1. Организация workflow в репозитории GitHub
2. Сборка и деплой через Docker-агенты внутри GitLab
3. Плагинный CI-сервер, интегрируемый в любой репозиторий
4. CI/CD прямо внутри Bitbucket проекта

Подсказка: Инструменты часто продаются вместе с платформами, но Jenkins — универсален.

- **GitHub Actions** — Организация workflow в репозитории GitHub
- **GitLab CI/CD** — Сборка и деплой через Docker-агенты внутри GitLab
- **Jenkins** — Плагинный CI-сервер, интегрируемый в любой репозиторий
- **Bitbucket Pipelines** — CI/CD прямо внутри Bitbucket проекта

Шаг 12:

Миссия 12

Задача: Сопоставьте шаг с его гарантией при интеграции hook + CI

Шаги:

Ответ:

- Hook проверяет стиль кода
- CI запускает unit-тесты
- CI запускает e2e-тесты
- CI делает деплой на staging

Гарантии:

1. Убедиться, что базовые модули не ломаются
2. Убедиться, что пользовательский поток работает
3. Код соответствует правилам командного стиля
4. Версия доступна для проверки QA

Подсказка: Hook лучше лёгкие проверки, тяжёлые — CI.

- **Hook проверяет стиль кода** — Код соответствует правилам командного стиля
- **CI запускает unit-тесты** — Убедиться, что базовые модули не ломаются
- **CI запускает e2e-тесты** — Убедиться, что пользовательский поток работает
- **CI делает деплой на staging** — Версия доступна для проверки QA

Шаг 13:

Миссия 13

Расположите этапы настройки интеграции hook → CI в правильном порядке:

1. Настроить скрипт hook, который делает HTTP-запрос к CI
2. Настроить CI-проекты и jobs, ожидающие вебхуки
3. Настроить права доступа и секреты между Git-сервером и CI
4. Проверить, что при пуше происходит запуск CI-job
5. Документировать как пользователи должны использовать workflow

Подсказка: Сначала инфраструктура, потом тест, потом документация.

Ответ:

- Настроить скрипт hook, который делает HTTP-запрос к CI
- Настроить CI-проекты и jobs, ожидающие вебхуки

- Настроить права доступа и секреты между Git-сервером и CI
- Проверить, что при пуше происходит запуск CI-job
- Документировать как пользователи должны использовать workflow

🔧 Шаг 14:

Миссия 14

Расположите шаги от простого тестового hook до полноценной CI-интеграции:

1. Создать простой скрипт, который возвращает статус 0/1
2. Добавить отправку HTTP-запроса к CI из hook
3. Настроить CI job, принимающий вебхук и выполняющий тесты
4. Обработать ошибки и логирование на стороне hook и CI
5. Развернуть механизм секретов (токены, подписи) между hook и CI

Подсказка: Постепенно усложняйте интеграцию и безопасность.

Ответ:

- Создать простой скрипт, который возвращает статус 0/1
- Добавить отправку HTTP-запроса к CI из hook
- Настроить CI job, принимающий вебхук и выполняющий тесты
- Обработать ошибки и логирование на стороне hook и CI
- Развернуть механизм секретов (токены, подписи) между hook и CI

🔧 Шаг 15:

Миссия 15

Расположите шаги реактивной отладки интеграции hook + CI:

1. Посмотреть логи CI и webhook request
2. Симулировать пуш или событие, чтобы повторить проблему
3. Изолировать, срабатывает ли hook локально
4. Проверить, корректно ли сформирован payload и заголовки
5. Исправить скрипт hook или настройки CI и повторно протестировать

Подсказка: Отладка идёт от внешнего к внутреннему и обратно.

Ответ:

- Посмотреть логи CI и webhook request
- Симулировать пуш или событие, чтобы повторить проблему
- Изолировать, срабатывает ли hook локально
- Проверить, корректно ли сформирован payload и заголовки
- Исправить скрипт hook или настройки CI и повторно протестировать

🔧 Шаг 16:

Миссия 16

Расположите шаги внедрения интеграции в командный процесс:

1. Внедрить проверку hook + CI в тестовый репозиторий
2. Попросить разработчиков протестировать на feature-ветках
3. Сбор обратной связи и доработка механизмов
4. Перенести конфигурацию на основные репозитории команды
5. Обучить команду правилам нового workflow

Подсказка: Не сразу «включайте» всем — сначала тестируйте на меньшем масштабе.

Ответ:

- Внедрить проверку hook + CI в тестовый репозиторий
- Попросить разработчиков протестировать на feature-ветках
- Сбор обратной связи и доработка механизмов
- Перенести конфигурацию на основные репозитории команды
- Обучить команду правилам нового workflow

🔔 **Шаг 17:**

Миссия 17

Расположите этапы эволюции интеграции hook и CI от простого к надежному:

1. Локальный pre-push с minimal checks
2. Серверный pre-receive для базовых проверок
3. Hook → webhook к CI с запуском тестов
4. CI jobs включая сборку, тесты, деплой
5. Обратная связь пользователю (статус ошибок) через hook или CI

Подсказка: Эволюция — от простого к интегрированному, от локального к распределенному.

Ответ:

- Локальный pre-push с minimal checks
- Серверный pre-receive для базовых проверок
- Hook → webhook к CI с запуском тестов
- CI jobs включая сборку, тесты, деплой
- Обратная связь пользователю (статус ошибок) через hook или CI

📁 **Модуль 20 - Submodules и Subtree**

📖 **Урок 1 - Что такое submodule и зачем нужен**

🔔 **Шаг 01:**

Миссия 1

Вы решили использовать git submodule в проекте. Какова основная причина добавить подмодуль?

Подсказка: Подумайте, когда один репозиторий зависит от другого.

Ответ:

- **Чтобы встроить независимый репозиторий как зависимость**
- Чтобы сжать историю проекта
- Чтобы объединить все ветки в один репозиторий
- Чтобы автоматически генерировать документацию

🔔 **Шаг 02:**

Миссия 2

Когда вы клонируете главный репозиторий с подмодулями, что нужно сделать, чтобы получить содержимое подмодулей?

Подсказка: Прочитайте про флаг при клонировании и команду init.

Ответ:

- **git clone --recurse-submodules** или потом **git submodule update --init**
- git clone и затем git fetch origin
- git clone и git pull submodule
- git clone --all-branches

 **Шаг 03:**

Миссия 3

Вы хотите обновить подмодуль до последнего коммита в его репозитории. Что вы сделаете?
Подсказка: Подумайте о командах внутри каталога подмодуля.

Ответ:

- **Перейти в каталог подмодуля и выполнить git pull, затем вернуться и git add/commit**
- В корне проекта выполнить git pull
- Удалить подмодуль и добавить заново
- git submodule push

 **Шаг 04:**

Миссия 4

Вы хотите удалить подмодуль из проекта. Какой шаг НЕ обязателен?
Подсказка: Удаление подмодуля — это несколько шагов.

Ответ:

- **Оставить .gitmodules без изменений**
- Удалить запись из .gitmodules
- Удалить каталог подмодуля
- git rm --cached путь_к_подмодулю

 **Шаг 05:**

Миссия 5

Когда удобно использовать git subtree вместо submodule?

Подсказка: Подумайте, когда вам нужно включить код зависимостей прямо в историю вашего проекта.

Ответ:

- **Когда хотите содержать внешний проект как внутреннюю часть слияния истории**
- Когда внешний проект очень большой
- Когда хотите, чтобы подмодуль был приватным
- Когда не хотите иметь отдельный репозиторий

 **Шаг 06:**

Миссия 6

Как вы синхронизируете subtree с удалённым репозиторием?

Подсказка: Команда subtree pull может помочь.

Ответ:

- **git subtree pull --prefix=путь remote branch**

- `git subtree push --prefix=путь remote branch`
- `git subtree clone remote branch`
- `git subtree update`

🔗 Шаг 07:

Миссия 7

Что произойдет при выполнении `git submodule update` без `init`?

Подсказка: `Init` нужен для первоначальной установки.

Ответ:

- **Подмодули не инициализируются и остаются пустыми**
- Подмодули обновятся без проблем
- Основной репозиторий откатится
- Подмодули удалятся

🔗 Шаг 08:

Миссия 8

Сопоставьте термин с его определением в контексте `submodule` и `subtree`:

Термины:

Ответ:

- Submodule
- Subtree
- `git submodule init`
- `git subtree push`

Определения:

1. Встраивание отдельного репозитория как поддиректории, сохраняя его независимым
2. Объединение истории внешнего проекта в ваш репозиторий
3. Команда инициализации записей из `.gitmodules`
4. Проталкивание изменений `subtree` обратно в внешний репозиторий

Подсказка: Подумайте, какая команда записывает изменения во внешний `remote`.

- **Submodule** — Встраивание отдельного репозитория как поддиректории, сохраняя его независимым
- **Subtree** — Объединение истории внешнего проекта в ваш репозиторий
- **`git submodule init`** — Команда инициализации записей из `.gitmodules`
- **`git subtree push`** — Проталкивание изменений `subtree` обратно в внешний репозиторий

🔗 Шаг 09:

Миссия 9

Сопоставьте команды с их действием:

Команды:

Ответ:

- `git submodule update --remote`
- `git submodule add URL путь`
- `git subtree split`
- `git subtree pull`

Действия:

1. Добавляет новый подмодуль
2. Сплит истории subtree в ветку для отправки
3. Обновляет подмодуль из его удалённого источника
4. Сликает изменения из внешнего репозитория subtree в ваш проект

Подсказка: Split используется перед push или pull.

- **git submodule update --remote** — Обновляет подмодуль из его удалённого источника
- **git submodule add URL путь** — Добавляет новый подмодуль
- **git subtree split** — Сплит истории subtree в ветку для отправки
- **git subtree pull** — Сликает изменения из внешнего репозитория subtree в ваш проект

Шаг 10:

Миссия 10

Сопоставьте сценарий с преимуществом submodule или subtree:

Сценарии:

Ответ:

- Нужна чёткая граница между проектами
- Хотите, чтобы внешние изменения шли как ваш код
- Изменения внешнего проекта редко
- Нужна независимость версий

Преимущества:

1. Хорошо подходит подмодуль
2. Хорошо подходит subtree
3. submodule минималистичен для редко меняющихся зависимостей
4. submodule позволяет независимые версии

Подсказка: Когда проекты должны быть строгими модулями — submodule; когда интеграция — subtree.

- **Нужна чёткая граница между проектами** — Хорошо подходит подмодуль
- **Хотите, чтобы внешние изменения шли как ваш код** — Хорошо подходит subtree
- **Изменения внешнего проекта редко** — submodule минималистичен для редко меняющихся зависимостей
- **Нужна независимость версий** — submodule позволяет независимые версии

Шаг 11:

Миссия 11

Сопоставьте файлы/записи с их роли при работе с submodule:

Элементы:

Ответ:

- .gitmodules
- .git/config запись
- .git/modules/...
- файл .gitmodules в корне проекта

Роли:

1. Указывает URL и путь подмодуля
2. Локальная настройка submodule
3. Содержит объекты репозитория подмодуля
4. Записи, используемые git для инициализации

Подсказка: .gitmodules и .git/config — это конфигурации на разных уровнях.

- **.gitmodules** — Указывает URL и путь подмодуля
- **.git/config запись** — Локальная настройка submodule
- **.git/modules/...** — Содержит объекты репозитория подмодуля
- **файл .gitmodules в корне проекта** — Записи, используемые git для инициализации

Шаг 12:

Миссия 12

Сопоставьте команды subtree с их назначением:

Команды:

Ответ:

- git subtree add
- git subtree pull
- git subtree push
- git subtree merge

Назначения:

1. Добавляет внешний проект как subtree впервые
2. Обновляет из внешнего проекта в интегрированную часть
3. Отправляет ваши изменения во внешний проект
4. Сливают изменение истории subtree в основной проект

Подсказка: Add — первый раз, pull — получать обновления, push — отдавать.

- **git subtree add** — Добавляет внешний проект как subtree впервые
- **git subtree pull** — Обновляет из внешнего проекта в интегрированную часть
- **git subtree push** — Отправляет ваши изменения во внешний проект
- **git subtree merge** — Сливают изменение истории subtree в основной проект

Шаг 13:

Миссия 13

Расположите шаги добавления submodule в правильном порядке:

1. git submodule add URL путь
2. git commit изменений .gitmodules и записей
3. git submodule update --init
4. git clone проекта, возможно с --recurse-submodules

Подсказка: Сначала добавляем, затем инициализируем, затем коммитим.

Ответ:

- git submodule add URL путь
- git submodule update --init
- git commit изменений .gitmodules и записей

- `git clone` проекта, возможно с `--recurse-submodules`

Шаг 14:

Миссия 14

Расположите шаги удаления submodule:

1. Удалить каталог подмодуля
2. Изменить `.gitmodules` и удалить запись
3. `git rm --cached` путь_к_подмодулю
4. `git commit` изменений

Подсказка: Удаление — сначала убрать индекс, потом очистить конфигурации, потом коммит.

Ответ:

- `git rm --cached` путь_к_подмодулю
- Изменить `.gitmodules` и удалить запись
- Удалить каталог подмодуля
- `git commit` изменений

Шаг 15:

Миссия 15

Расположите шаги синхронизации subtree:

1. `git subtree pull --prefix=путь remote branch`
2. `git commit` изменений после pull
3. `git subtree split --prefix=путь branch_out`
4. `git subtree push --prefix=путь remote branch_out`

Подсказка: Pull — получать, split — готовить к push, затем push.

Ответ:

- `git subtree pull --prefix=путь remote branch`
- `git commit` изменений после pull
- `git subtree split --prefix=путь branch_out`
- `git subtree push --prefix=путь remote branch_out`

Шаг 16:

Миссия 16

Расположите этапы, когда вы переключаетесь между ветками в проекте с подмодулями:

1. `git checkout` другая_ветка
2. `git submodule update --remote` или `--init`
3. `git fetch` в корневом репозитории
4. `git submodule status`

Подсказка: После checkout нужно синхронизировать подмодули.

Ответ:

- `git checkout` другая_ветка
- `git fetch` в корневом репозитории
- `git submodule update --remote` или `--init`
- `git submodule status`

Шаг 17:

Миссия 17

Расположите стадии, если вы хотите полностью перевести проект с submodule на subtree:

1. `git subtree add --prefix=путь URL branch`
2. Убедиться, что весь код подмодуля интегрирован
3. Удалить записи submodule и `.gitmodules`
4. Удалить конфигурации submodule и закоммитить изменения

Подсказка: Сначала добавить subtree, потом удалить submodule-инфраструктуру.

Ответ:

- `git subtree add --prefix=путь URL branch`
- Убедиться, что весь код подмодуля интегрирован
- Удалить записи submodule и `.gitmodules`
- Удалить конфигурации submodule и закоммитить изменения

Урок 2 - Добавление и обновление submodules

Шаг 01:

Миссия 1

Вы — разработчик, и ваш коллега предлагает использовать submodule для общих библиотек. Что нужно сделать в корне проекта при добавлении подмодуля?

Подсказка: Вспомните, что создаётся файл `.gitmodules`.

Ответ:

- **`git submodule add <путь>`**
- `git subtree add --prefix <путь> <ветка>`
- `git clone --recurse-submodules`
- `git fetch origin --all`

Шаг 02:

Миссия 2

После того как вы добавили подмодуль, что нужно сделать, чтобы начать работать с ним?

Подсказка: Подумайте, что нужно, чтобы инициализировать записи `.gitmodules`.

Ответ:

- **`git submodule update --init`**
- `git submodule push`
- `git submodule branch`
- `git submodule status`

Шаг 03:

Миссия 3

Вы добавили подмодуль и закоммитили. Что нужно сделать при клонировании проекта, чтобы сразу получить подмодули?

Подсказка: Подумайте о параметрах `git clone`.

Ответ:

- **`git clone --recurse-submodules`**

- `git clone && git pull submodules`
- `git clone --all`
- `git clone && git submodule push`

🔧 Шаг 04:

Миссия 4

Коллега говорит, что подмодули автоматически обновляются при `git pull`. Это верно?

Подсказка: Подумайте, нужен ли флаг `--remote`.

Ответ:

- **Нет** — нужно явно вызвать `git submodule update --remote`
- **Да** — по умолчанию подмодули подтягиваются
- Да, если включён автоматический хук
- **Нет** — подмодули запрещены по умолчанию

🔧 Шаг 05:

Миссия 5

Сопоставьте команды с их назначениями при работе с подмодулями:

Команды:

Ответ:

- `git submodule init`
- `git submodule update`
- `git submodule add`
- `git submodule status`

Назначения:

1. Инициализация записей из `.gitmodules`
2. Обновление рабочего дерева подмодуля
3. Добавление нового подмодуля
4. Показывает статус каждого подмодуля

Подсказка: `init` не загружает содержимого, `update` — загружает.

- **`git submodule init`** — Инициализация записей из `.gitmodules`
- **`git submodule update`** — Обновление рабочего дерева подмодуля
- **`git submodule add`** — Добавление нового подмодуля
- **`git submodule status`** — Показывает статус каждого подмодуля

🔧 Шаг 06:

Миссия 6

Сопоставьте параметры с их эффектом при обновлении подмодулей:

Параметры:

Ответ:

- `--remote`
- `--merge`
- `--checkout`
- `--force`

Эффекты:

1. Подтянуть изменения с удалённого репо подмодуля
2. Слить изменения с локальными
3. Принудительно чек-аутить ветки из подмодуля
4. Принудительно перезаписать локальные изменения

Подсказка: `remote` — извне, `merge` — слияние, `force` — принудительно.

- **--remote** — Подтянуть изменения с удалённого репо подмодуля
- **--merge** — Слить изменения с локальными
- **--checkout** — Принудительно чек-аутить ветки из подмодуля
- **--force** — Принудительно перезаписать локальные изменения

Шаг 07:

Миссия 7

Сопоставьте файлы и записи с их значением, когда вы добавили подмодуль:

Элементы:

Ответ:

- `.gitmodules` запись
- файл `.gitmodules`
- `.git/config` запись `submodule`
- индекс в главном репозитории

Значения:

1. содержит URL и путь подмодуля
2. локальная конфигурация `submodule` для этого клона
3. отражение записи в `.gitmodules`, локализованное
4. помечает подмодуль как отслеживаемый

Подсказка: Помните, `.gitmodules` — общая конфигурация, `.git/config` — локальная.

- **.gitmodules запись** — содержит URL и путь подмодуля
- **файл .gitmodules** — помечает подмодуль как отслеживаемый
- **.git/config запись submodule** — локальная конфигурация `submodule` для этого клона
- **индекс в главном репозитории** — отражение записи в `.gitmodules`, локализованное

Шаг 08:

Миссия 8

Расположите шаги добавления подмодуля и начала работы с ним:

1. `git submodule add <путь>`
2. `git commit` изменений
3. `git submodule update --init`
4. `git clone` проект или перейти в проект

Подсказка: Сначала вы добавляете, затем инициализируете, потом коммитите.

Ответ:

- `git submodule add <путь>`
- `git submodule update --init`
- `git commit` изменений

- git clone проект или перейти в проект

Шаг 09:

Миссия 9

Расположите шаги, когда вы хотите обновить подмодули до последних коммитов:

1. git submodule update --remote
2. git commit изменений
3. git add путь_к_подмодулю
4. git pull в корневом репозитории (если нужно)

Подсказка: Сначала подтягиваете remote, затем фиксируете ссылку в основном репо.

Ответ:

- git submodule update --remote
- git add путь_к_подмодулю
- git commit изменений
- git pull в корневом репозитории (если нужно)

Шаг 10:

Миссия 10

Расположите шаги, если при клонировании забыли рекурсивно загрузить подмодули:

1. git submodule init
2. git submodule update --remote
3. git submodule update --init
4. git submodule status

Подсказка: init перед update, затем — статус.

Ответ:

- git submodule init
- git submodule update --init
- git submodule update --remote
- git submodule status

Шаг 11:

Миссия 11

Расположите шаги, если вы хотите поменять URL подмодуля:

1. Изменить .gitmodules запись
2. git submodule sync
3. git add .gitmodules
4. git commit изменений

Подсказка: sync синхронизирует локальные записи после изменения .gitmodules.

Ответ:

- Изменить .gitmodules запись
- git submodule sync
- git add .gitmodules
- git commit изменений

💡 Шаг 12:

Миссия 12

Расположите шаги, при которых вы переключаетесь на новую ветку в проекте с подмодулями:

1. `git checkout новая_ветка`
2. `git submodule update --remote` или `--init`
3. `git fetch`
4. `git submodule status`

Подсказка: После checkout нужно убедиться, что подмодули синхронизированы.

Ответ:

- `git checkout новая_ветка`
- `git fetch`
- `git submodule update --remote` или `--init`
- `git submodule status`

💡 Шаг 13:

Миссия 13

Расположите шаги, если подмодуль перестал работать и вы хотите его пересоздать:

1. Удалить каталог подмодуля
2. Удалить запись из `.gitmodules`
3. `git rm --cached <путь>`
4. `git submodule add <путь>`

Подсказка: Сначала убрать ссылки, потом заново добавить.

Ответ:

- `git rm --cached <путь>`
- Удалить запись из `.gitmodules`
- Удалить каталог подмодуля
- `git submodule add <путь>`

💡 Шаг 14:

Миссия 14

Расположите все стадии, если вы хотите плавно мигрировать подмодуль в subtree:

1. `git subtree add --prefix=<путь> <ветка>`
2. Убедиться, что весь код подмодуля теперь внутри папки
3. Удалить записи `.gitmodules` и конфигурации `submodule`
4. Удалить каталог `.git/modules` для подмодуля и закоммитить изменения

Подсказка: Сначала интеграция, потом удаление старой инфраструктуры.

Ответ:

- `git subtree add --prefix=<путь> <ветка>`
- Убедиться, что весь код подмодуля теперь внутри папки
- Удалить записи `.gitmodules` и конфигурации `submodule`
- Удалить каталог `.git/modules` для подмодуля и закоммитить изменения

Шаг 15:

Миссия 15

Расположите шаги, если хотите инициировать сразу все подмодули после клонирования:

1. git clone
2. git submodule init
3. git submodule update
4. git submodule update --remote (если нужно)

Подсказка: init перед update, затем remote при необходимости.

Ответ:

- git clone
- git submodule init
- git submodule update
- git submodule update --remote (если нужно)

Шаг 16:

Миссия 16

Расположите шаги, если вы хотите откатить изменения submodule-ссылки в основном репозитории:

1. git checkout HEAD <путь_подмодуля>
2. git add <путь_подмодуля>
3. git commit или git reset (как нужно)
4. git submodule status, чтобы проверить ссылку

Подсказка: Сначала вернуть ссылку, потом зафиксировать, потом проверить.

Ответ:

- git checkout HEAD <путь_подмодуля>
- git add <путь_подмодуля>
- git commit или git reset (как нужно)
- git submodule status, чтобы проверить ссылку

Урок 3 - Subtree альтернатива submodules

Шаг 01:

Миссия 1

Аня хочет встроить библиотеку из другого репозитория и не хочет мучиться с submodule. Что из этого может быть альтернативой?

Подсказка: Подумай о механизме, где содержимое встраивается в основной репозиторий.

Ответ:

- Submodule
- **Subtree**
- Гитпатч

Шаг 02:

Миссия 2

Боб слышал, что subtrees проще в использовании по сравнению с submodules. В чём ключевое отличие?

Подсказка: Подумай, что хранится в вашем основном репозитории.

Ответ:

- **Содержимое subtree копируется внутрь основного репозитория**
- Subtree требует отдельной инициализации
- Subtree хранит только ссылку на внешний репозиторий

🔮 Шаг 03:

Миссия 3

Катя пытается обновить subtree до последней версии внешнего проекта. Какой командой она может попробовать?

Подсказка: Посмотри в документации к git subtree.

Ответ:

- **git subtree pull --prefix=путь remote branch**
- git pull subtree
- git subtree fetch

🔮 Шаг 04:

Миссия 4

Иван хочет отправить изменения из subtree обратно в исходный репозиторий. Какая команда ему понадобится?

Подсказка: Подумай как «push» работает с subtree.

Ответ:

- **git subtree push --prefix=путь remote branch**
- git push subtree
- git subtree send

🔮 Шаг 05:

Миссия 5

Маша думает: «А нельзя ли объединить несколько subtree в одну структуру?» Какой подход её интересует?

Подсказка: Подумай о том, чтобы вложить один subtree в другой.

Ответ:

- **Nested subtree**
- **Subtree в subdirectory**
- Subtree в подмодуле

🔮 Шаг 06:

Миссия 6

Сопоставьте команды с их описаниями в контексте subtree:

Команды:

Ответ:

- git subtree add

- `git subtree pull`
- `git subtree push`
- `git subtree split`

Описание:

1. Добавляет проект как subtree в ваш репозиторий
2. Обновляет ваш subtree из удалённого источника
3. Отправляет ваши изменения обратно в удалённый проект
4. Выделяет изменения subtree в отдельную ветку

Подсказка: Смотрите слово «add», «pull», «push», «split».

- **git subtree add** — Добавляет проект как subtree в ваш репозиторий
- **git subtree pull** — Обновляет ваш subtree из удалённого источника
- **git subtree push** — Отправляет ваши изменения обратно в удалённый проект
- **git subtree split** — Выделяет изменения subtree в отдельную ветку

Шаг 07:

Миссия 7

Сопоставьте аргументы к командам subtree:

Аргументы:

Ответ:

- `--prefix`
- `--squash`
- `--rejoin`
- `--annotate`

Назначение:

1. Указывает путь, куда вставлять subtree
2. Сжимает всё добавление в один коммит
3. Позволяет позже повторно объединять историю
4. Добавляет текст заметки о subtree

Подсказка: Подумай, что означает каждое слово.

- **--prefix** — Указывает путь, куда вставлять subtree
- **--squash** — Сжимает всё добавление в один коммит
- **--rejoin** — Позволяет позже повторно объединять историю
- **--annotate** — Добавляет текст заметки о subtree

Шаг 08:

Миссия 8

Сопоставьте проблемы с subtree с методами их решения:

Проблемы:

Ответ:

- История сильно разрослась
- Конфликты при pull
- Изменения основного репозитория мешают subtree
- Нечёткая граница мержей

Решения:

1. Использовать --squash
2. Разделять ветки и делать pull часто
Синхронизировать с --rejoin
3. Использовать git subtree split и явно назначать ветки

Подсказка: Подумайте, какая опция сглаживает историю или помогает управлять конфликтами.

- **История сильно разрослась** — Использовать --squash
- **Конфликты при pull** — Разделять ветки и делать pull часто
- **Изменения основного репозитория мешают subtree** — Синхронизировать с --rejoin
- **Нечёткая граница мержей** — Использовать git subtree split и явно назначать ветки

Шаг 09:

Миссия 9

Упорядочите этапы добавления subtree:

1. git subtree add --prefix=путь remote branch
2. commit слияния subtree
3. remote add (если ещё не добавлен)
4. fetch remote

Подсказка: Сначала подключаем удалённый репозиторий, потом получаем данные, потом добавляем как subtree, затем фиксируем.

Ответ:

- remote add (если ещё не добавлен)
- fetch remote
- git subtree add --prefix=путь remote branch
- commit слияния subtree

Шаг 10:

Миссия 10

Упорядочите шаги обновления subtree:

1. git subtree pull --prefix=путь remote branch
2. fetch remote
3. commit слияния

Подсказка: Сначала нужно получить изменения, затем применить их, потом зафиксировать.

Ответ:

- fetch remote
- git subtree pull --prefix=путь remote branch
- commit слияния

Шаг 11:

Миссия 11

Упорядочите шаги отправки изменений subtree:

1. commit изменений в subtree части локально
2. git subtree push --prefix=путь remote branch

3. fetch remote (необязательно, но полезно)

Подсказка: Коммит —> выталкивание —> чтение удалённого.

Ответ:

- commit изменений в subtree части локально
- git subtree push --prefix=путь remote branch
- fetch remote (необязательно, но полезно)

Шаг 12:

Миссия 12

Упорядочите шаги split subtree в ветку:

1. git subtree split --prefix=путь -b имя_ветки
2. commit созданной ветки
3. выполнение операции split

Подсказка: Команда split сама создаёт ветку, затем коммит уже внутри неё.

Ответ:

- выполнение операции split
- git subtree split --prefix=путь -b имя_ветки
- commit созданной ветки

Шаг 13:

Миссия 13

Упорядочите этапы удаления subtree:

1. удалить директорию subtree из файловой системы
2. commit удаления
3. удалить ветку subtree (если есть)
4. удалить remote (если больше не нужен)

Подсказка: Сначала убрать файлы, потом зафиксировать, затем чистить остатки.

Ответ:

- удалить директорию subtree из файловой системы
- commit удаления
- удалить ветку subtree (если есть)
- удалить remote (если больше не нужен)

Шаг 14:

Миссия 14

Упорядочите этапы комплексной работы с subtree в крупном проекте:

1. выбор внешнего репозитория и ветки
2. remote add и fetch
3. git subtree add или pull
4. внесение локальных изменений и коммит
5. git subtree push обратно, если нужно
6. регулярные обновления и разрешение конфликтов

Подсказка: Начинайте с подключения, потом добавляйте, правьте, синхронизируйте и поддерживайте.

Ответ:

- выбор внешнего репозитория и ветки
- remote add и fetch
- git subtree add или pull
- внесение локальных изменений и коммит
- git subtree push обратно, если нужно
- регулярные обновления и разрешение конфликтов

Урок 4 - Практика работа с зависимыми репозиториями

Шаг 01:

Миссия 1

Аня хочет встроить библиотеку из другого репозитория и не хочет мучиться с submodule. Что из этого может быть альтернативой?

Подсказка: Подумай о механизме, где содержимое встраивается в основной репозиторий.

Ответ:

- Submodule
- **Subtree**
- Git patch

Шаг 02:

Миссия 2

Боб слышал, что subtrees проще в использовании по сравнению с submodules. В чём ключевое отличие?

Подсказка: Подумай, что хранится в вашем основном репозитории.

Ответ:

- **Содержимое subtree копируется внутрь основного репозитория**
- Subtree требует отдельной инициализации
- Subtree хранит только ссылку на внешний репозиторий

Шаг 03:

Миссия 3

Катя пытается обновить subtree до последней версии внешнего проекта. Какой командой она может попробовать?

Подсказка: Посмотри в документации к git subtree.

Ответ:

- **git subtree pull --prefix=путь remote branch**
- git pull subtree
- git subtree fetch

Шаг 04:

Миссия 4

Иван хочет отправить изменения из subtree обратно в исходный репозиторий. Какая команда ему понадобится?

Подсказка: Подумай как «push» работает с subtree.

Ответ:

- **git subtree push --prefix=путь remote branch**
- git push subtree
- git subtree send

🔗 Шаг 05:

Миссия 5

Маша думает: «А нельзя ли объединить несколько subtree в одну структуру?» Какой подход её интересует?

Подсказка: Подумай о том, чтобы вложить один subtree в другой.

Ответ:

- **Nested subtree**
- **Subtree в subdirectory**
- Subtree в подмодуле

🔗 Шаг 06:

Миссия 6

Денис настроил submodule, но обнаружил, что при клонировании проекта коллега не получает содержимое зависимого репозитория. Что он забыл?

Подсказка: Submodule требует дополнительного шага после clone.

Ответ:

- **Запустить git submodule update --init**
- Добавить .submodules в .gitignore
- Создать папку вручную
- Запустить git fetch

🔗 Шаг 07:

Миссия 7

Сопоставьте команды с их действиями при работе с subtree:

Подсказка: В названиях команд уже зашифровано их предназначение.

Ответ:

- **git subtree add** — Добавляет внешний проект внутрь текущего репозитория
- **git subtree pull** — Обновляет содержимое вложенного репозитория
- **git subtree push** — Отправляет изменения обратно во внешний репозиторий
- **git subtree split** — Создаёт отдельную ветку из содержимого subtree

🔗 Шаг 08:

Миссия 8

Лена клонирует крупный проект с submodule, но не видит содержимого вложенного репозитория. Что ей стоит сделать?

Подсказка: После клона submodule не инициализируется автоматически.

Ответ:

- **git submodule update --init --recursive**
- git pull origin master
- git fetch && git merge

Шаг 09:

Миссия 9

Олег думает, что субмодули — это удобно, но коллеги часто жалуются на сложности. В каком сценарии subtree может быть предпочтительнее submodule?

Подсказка: Подумай о том, чтобы минимизировать шаги при клонировании проекта.

Ответ:

- **Когда хочется, чтобы всё сразу было в одном репозитории без extra init**
- Когда нужно строго контролировать версии зависимостей
- **Когда внешний репозиторий — приватный**

Шаг 10:

Миссия 10

Саша уже использует subtree для библиотеки А, и хочет интегрировать библиотеку В таким же способом. Что важнее всего при добавлении второго subtree?

Подсказка: Обрати внимание, куда ты вставляешь содержимое.

Ответ:

- **Выбрать уникальный путь (--prefix) для вставки**
- Использовать ту же директорию, что для библиотеки А
- Игнорировать конфликты имя-файлов

Шаг 11:

Миссия 11

Николай сделал локальные правки в subtree. Какой командой он скорее всего отправит свои изменения обратно?

Подсказка: Используй аналог git push, но для subtree.

Ответ:

- **git subtree push --prefix=путь remote branch**
- git push subtree
- git subtree send

Шаг 12:

Миссия 12

Тимур устал от конфликтов при pull subtree и слышал про ключ --squash. Что делает эта опция?

Подсказка: Эта опция касается истории коммитов.

Ответ:

- **Объединяет все изменения subtree в один коммит**
- Сбрасывает все изменения перед pull
- Разделяет коммиты на мелкие куски

🔗 Шаг 13:

Миссия 13

Игорь хочет, чтобы при обновлениях subtree изменения из основного проекта не мешали. Какую опцию стоит использовать?

Подсказка: Обрати внимание на опцию, которая «соединяет» истории.

Ответ:

- **--rejoin**
- --squash
- --depth

🔗 Шаг 14:

Миссия 14

Сопоставьте команды subtree с их ролями:

Подсказка: Названия команд дают подсказку.

Ответ:

- **git subtree add** — Вставляет внешний проект как subtree
- **git subtree pull** — Обновляет содержимое subtree из внешнего репозитория
- **git subtree push** — Отправляет ваши локальные изменения обратно
- **git subtree split** — Извлекает subtree в отдельную ветку

🔗 Шаг 15:

Миссия 15

Сопоставьте опции subtree с их действием:

Подсказка: Каждая опция отражена в её названии.

Ответ:

- **--prefix** — Указывает поддиректорию, куда вставлять subtree
- **--squash** — Объединяет коммиты subtree в один
- **--rejoin** — Разрешает позже повторно мержить историю
- **--annotate** — Добавляет метку или комментарий к коммитам subtree

🔗 Шаг 16:

Миссия 16

Сопоставьте типичные проблемы работы с зависимостями и способы решения:

Подсказка: Вспомни ключевые боли: конфликты, раздутую историю, непредвиденное поведение.

Ответ:

- **Обширная история subtree** — Использовать --squash
- **Частые конфликты при обновлении** — Обновлять часто и в маленьких партиях
- **Сложность мёржа с основной веткой** — Использовать --rejoin
- **Нечёткие границы кода** — Использовать git subtree split

🔗 Шаг 17:

Миссия 17

Упорядочите шаги при добавлении subtree в проект:

1. remote add (если ещё не добавлен)
2. git fetch remote
3. git subtree add --prefix=путь remote branch
4. commit слияния

Подсказка: Сначала подключаем удалёнку, получаем данные, затем вставляем и фиксируем.

Ответ:

- remote add (если ещё не добавлен)
- git fetch remote
- git subtree add --prefix=путь remote branch
- commit слияния

Шаг 18:

Миссия 18

Упорядочите шаги обновления subtree:

1. git fetch remote
2. git subtree pull --prefix=путь remote branch
3. commit слияния

Подсказка: Сначала забираем изменения, потом применяем и фиксируем.

Ответ:

- git fetch remote
- git subtree pull --prefix=путь remote branch
- commit слияния

Шаг 19:

Миссия 19

Упорядочите шаги отправки изменений subtree обратно:

1. commit изменений в subtree части
2. git subtree push --prefix=путь remote branch
3. git fetch remote (необязательно, но полезно)

Подсказка: Сначала фиксируем изменения, затем отправляем, потом можно проверить удалёнку.

Ответ:

- commit изменений в subtree части
- git subtree push --prefix=путь remote branch
- git fetch remote (необязательно, но полезно)

Модуль 21 - Git LFS

Урок 1 - Проблема больших файлов в Git

Шаг 01:

Миссия 1

Аня — backend-разработчик. Она случайно закоммитила огромный видеофайл в репозиторий и теперь push не проходит. Что стало причиной проблемы?

Подсказка: Git плохо работает с двоичными файлами — он хранит каждую версию целиком.

Ответ:

- **Git не оптимизирован для хранения больших бинарных файлов**
- Git не поддерживает видеофайлы
- Git автоматически удаляет большие файлы
- Git требует платную подписку для больших файлов

Шаг 02:

Миссия 2

DevOps-инженер Сергей замечает, что размер репозитория вырос с 200 МБ до 2 ГБ после добавления нескольких моделей машинного обучения. Что в этом случае стоит использовать?

Подсказка: Есть специальное расширение Git, которое хранит только ссылки на тяжёлые файлы.

Ответ:

- **Git LFS (Large File Storage)**
- Git Flow
- Git Hooks
- Git Submodules

Шаг 03:

Миссия 3

Фронтендер Кира установила Git LFS, но не понимает, зачем команда прописывает `git lfs track`. Что делает эта команда?

Подсказка: Вспомни, как Git определяет, какие файлы обрабатывать особым образом.

Ответ:

- **Добавляет шаблон файлов, которые Git LFS должен отслеживать**
- Удаляет большие файлы из репозитория
- Переносит файлы в облако
- Изменяет историю коммитов

Шаг 04:

Миссия 4

Тестировщик Вика случайно закоммитила большие бинарники без LFS. Что нужно сделать, чтобы убрать их из истории и добавить правильно?

Подсказка: Придётся переписать историю — иначе файлы так и останутся в коммитах.

Ответ:

- **Использовать `git filter-repo` или `BFG Repo-Cleaner`**
- Создать новую ветку
- Сделать rebase на origin
- Переустановить Git LFS

Шаг 05:

Миссия 5

Сопоставьте команды Git LFS с их назначением.

Подсказка: Команды track, untrack, и ls-files выполняют разные роли при управлении файлами LFS.

Ответ:

- **git lfs track** — Добавляет шаблон для отслеживания файлов LFS.
- **git lfs untrack** — Удаляет шаблон отслеживания из .gitattributes.
- **git lfs ls-files** — Показывает файлы, отслеживаемые через LFS.
- **git lfs install** — Инициализирует поддержку LFS в локальном репозитории.

Шаг 06:

Миссия 6

Сопоставьте проблему и решение при работе с большими файлами.

Подсказка: Вспомни, что делает LFS с двоичными файлами и как это влияет на размер репозитория.

Ответ:

- **Большие бинарники замедляют клон** — Использовать Git LFS для хранения ссылок вместо самих файлов.
- **GitHub отказывает в push** — Проверить лимит размера и использовать LFS для тяжёлых файлов.
- **История переполнена версиями видео** — Очистить историю с помощью BFG или filter-геро.
- **Файл не загружается через HTTPS** — Проверить настройки LFS-сервера и токен доступа.

Шаг 07:

Миссия 7

Сопоставьте понятие с его объяснением в контексте Git LFS.

Подсказка: LFS хранит только указатели, а сами файлы — отдельно.

Ответ:

- **Pointer file** — Маленький файл, содержащий ссылку на реальный объект.
- **LFS storage** — Отдельное хранилище, где находятся большие файлы.
- **Bandwidth limit** — Ограничение объёма данных при передаче LFS-файлов.
- **.gitattributes** — Файл, определяющий шаблоны файлов для LFS.

Шаг 08:

Миссия 8

Сопоставьте действия с правильными последствиями при использовании Git LFS.

Подсказка: Не все команды изменяют реальные файлы — некоторые работают только с метаданными.

Ответ:

- **git lfs pull** — Загружает большие файлы из удалённого хранилища.
- **git lfs push** — Отправляет файлы LFS на сервер.
- **git lfs fetch** — Получает данные без замены pointer-файлов.
- **git lfs status** — Показывает различия между LFS и рабочей копией.

Шаг 09:

Миссия 9

Сопоставьте элементы инфраструктуры с их ролью при работе Git LFS.

Подсказка: LFS добавляет отдельный сервер и клиентскую часть.

Ответ:

- **LFS client** — Отслеживает и заменяет файлы при коммитах.
- **LFS server** — Хранит бинарные объекты и управляет загрузкой.
- **Git repository** — Хранит метаданные и pointer-файлы.
- **Git remote** — Обеспечивает доступ к LFS-серверу и обычным данным.

Шаг 10:

Миссия 10

Сопоставьте распространённые ошибки при Git LFS с их возможными причинами.

Подсказка: Иногда проблема не в Git, а в конфигурации или правах доступа.

Ответ:

- **Ошибка 403 при push** — Недостаточно прав для LFS-загрузки.
- **Файл не заменяется pointer'ом** — Отсутствует запись в .gitattributes.
- **LFS не установлен** — Не выполнена команда git lfs install.
- **Файлы не загружаются на сервер** — Проблемы с LFS-эндпоинтом или сетью.

Шаг 11:

Миссия 11

Расположите шаги по настройке Git LFS в правильном порядке.

Подсказка: Сначала нужно установить LFS, затем подключить и отслеживать нужные файлы.

Ответ:

- Установить Git LFS в системе
- Выполнить git lfs install
- Добавить файлы через git lfs track
- Закоммитить изменения в .gitattributes

Шаг 12:

Миссия 12

Расположите шаги работы Git LFS при push больших файлов.

Подсказка: Git сначала отправляет обычные данные, а потом LFS загружает бинарники.

Ответ:

- Git создаёт pointer-файл
- Pointer сохраняется в коммит
- Git push отправляет метаданные
- LFS загружает реальные файлы на сервер

Шаг 13:

Миссия 13

Расположите шаги, чтобы удалить большие файлы из истории Git и перевести их под LFS.

Подсказка: Очистка истории должна предшествовать настройке LFS.

Ответ:

- Использовать git filter-repo для удаления больших файлов

- Установить Git LFS
- Добавить шаблон отслеживания через git lfs track
- Закоммитить обновлённую историю

🔗 Шаг 14:

Миссия 14

Расположите действия, если GitHub отказывается принимать push из-за превышения лимита.

Подсказка: Нужно удалить тяжёлые файлы, подключить LFS и повторить загрузку.

Ответ:

- Отменить push
- Очистить историю от больших файлов
- Настроить Git LFS
- Выполнить push снова

🔗 Шаг 15:

Миссия 15

Расположите шаги при клонировании репозитория с LFS.

Подсказка: После клона нужно скачать настоящие файлы через LFS.

Ответ:

- Клонировать репозиторий обычной командой git clone
- Инициализировать LFS в проекте
- Выполнить git lfs pull для загрузки бинарников
- Проверить наличие файлов в рабочей директории

📖 Урок 2 - Подключение Git LFS

🔗 Шаг 01:

Миссия 1

Вы хотите добавить большие двоичные файлы (например, изображения) в репозиторий. Что нужно сделать сначала, чтобы использовать Git LFS?

Подсказка: LFS — это расширение, его нужно инициализировать.

Ответ:

- Выполнить ``git lfs install`` в локальном репозитории
- Просто закоммитить файлы как обычно
- Добавить ``.gitattributes`` сразу

🔗 Шаг 02:

Миссия 2

Вы уже выполнили ``git lfs install``. Что теперь нужно сделать, чтобы Git начал отслеживать определённые типы файлов через LFS?

Подсказка: Подключение типов файлов — через атрибуты.

Ответ:

- Добавить правила в ``.gitattributes``, например ``*.png filter=lfs diff=lfs merge=lfs -text``
- Добавить файлы сразу в ветку remote

- Удалить файлы из истории вручную

🔗 Шаг 03:

Миссия 3

Вы добавили правило `*.mp4 filter=lfs diff=lfs merge=lfs -text` в .gitattributes`. Что делать далее?`

Подсказка: Правило действует для файлов, добавленных после него.

Ответ:

- **Добавить (git add) нужные файлы видео и закоммитить**
- Сразу пушить в удалённый репозиторий
- Игнорировать файлы .mp4 через .gitignore

🔗 Шаг 04:

Миссия 4

Вы закоммитили файлы с правилами LFS, но пуш отклоняется из-за большого файла. Что могло быть забыто?

Подсказка: Проверка LFS должна быть активирована.

Ответ:

- **Не выполнена команда `git lfs track` перед добавлением файла`**
- Не настроен SSH-ключ
- Не настроены права доступа к репозиторию

🔗 Шаг 05:

Миссия 5

Вы используете GitHub, и LFS уже настроен локально. Что необходимо для того, чтобы файлы действительно хранились через LFS на удалённом сервере?

Подсказка: Убедитесь, что удалённый репозиторий поддерживает LFS.

Ответ:

- **Убедиться, что репозиторий на GitHub имеет включённую поддержку LFS**
- Добавить файл `.gitattributes` только в ветку master`
- Слить ветки вручную

🔗 Шаг 06:

Миссия 6

Сопоставьте команды Git LFS с их назначениями:

Подсказка: Подумайте, что делает каждая команда.

Команды:

Ответ:

- `git lfs install``
- `git lfs track ``
- `git lfs untrack ``
- `git lfs ls-files``

Назначения:

1. Показывает файлы, отслеживаемые LFS
2. Останавливает отслеживание шаблона файлов

3. Инициализирует поддержку LFS в репозитории

4. Добавляет шаблон файлов в отслеживание LFS

- **git lfs install** — Инициализирует поддержку LFS в репозитории
- **git lfs track** — Добавляет шаблон файлов в отслеживание LFS
- **git lfs untrack** — Останавливает отслеживание шаблона файлов
- **git lfs ls-files** — Показывает файлы, отслеживаемые LFS

Шаг 07:

Миссия 7

Сопоставьте части `.gitattributes`` к тому, что они задают:

Подсказка: В записи ``pattern filter=... diff=... merge=... -text`` каждая часть что-то управляет.

Части:

Ответ:

- ``filter=lfs``
- ``diff=lfs``
- ``merge=lfs``
- ``-text``

Значения:

1. Отключение текстового режима (не как обычный текст)
2. Использование LFS при сортировке изменений (diff)
3. Использование LFS для фильтрации при коммите/checkout
4. Использование LFS при слиянии (merge)

- **filter=lfs** — Использование LFS для фильтрации при коммите/checkout
- **diff=lfs** — Использование LFS при сортировке изменений (diff)
- **merge=lfs** — Использование LFS при слиянии (merge)
- **-text** — Отключение текстового режима (не как обычный текст)

Шаг 08:

Миссия 8

Сопоставьте ситуацию и подходящее действие при работе с LFS:

Подсказка: Иногда нужно переносить файлы в LFS задним числом.

Ситуации:

Ответ:

- У вас уже есть история с большими файлами, не в LFS
- Файлы поменяли расширения и теперь не отслеживаются
- Хочется удалить файл из LFS отслеживания
- Хочется увидеть, какие файлы LFS хранит

Действия:

1. Использовать ``git lfs migrate import`` или ``git filter-repo`` с LFS-флагами
2. Обновить правило `.gitattributes`` и заново ``git add`` и коммитить
3. Вызвать ``git lfs untrack`` и удалить из индекса
4. Вызвать ``git lfs ls-files``

- **У вас уже есть история с большими файлами, не в LFS** — Использовать ``git lfs migrate import`` или ``git filter-repo`` с LFS-флагами
- **Файлы поменяли расширения и теперь не отслеживаются** — Обновить правило ``.gitattributes`` и заново ``git add`` и коммитить
- **Хочется удалить файл из LFS отслеживания** — Вызвать ``git lfs untrack`` и удалить из индекса
- **Хочется увидеть, какие файлы LFS хранит** — Вызвать ``git lfs ls-files``

🔧 Шаг 09:

Миссия 9

Сопоставьте шаг и причину, почему его важно делать при подключении LFS:

Подсказка: Процесс безопасного внедрения требует этапов.

Шаги:

Ответ:

- Запустить ``git lfs install``
- Добавить ``.gitattributes``
- Переключить или пересоздать коммиты (миграция)
- Проверить историю и отсутствие больших файлов вне LFS

Причины:

1. Чтобы LFS функционировал в этом репозитории
2. Чтобы Git знал, какие файлы отслеживать через LFS
3. Чтобы преобразовать уже закоммиченные файлы в LFS
4. Чтобы убедиться, что нет «утечек» больших файлов

- **Запустить ``git lfs install``** — Чтобы LFS функционировал в этом репозитории
- **Добавить ``.gitattributes``** — Чтобы Git знал, какие файлы отслеживать через LFS
- **Переключить или пересоздать коммиты (миграция)** — Чтобы преобразовать уже закоммиченные файлы в LFS
- **Проверить историю и отсутствие больших файлов вне LFS** — Чтобы убедиться, что нет «утечек» больших файлов

🔧 Шаг 10:

Миссия 10

Сопоставьте странный симптом и вероятную причину в контексте LFS:

Подсказка: Иногда ошибки — из-за несоответствия атрибутов или фильтров.

Симптомы:

Ответ:

- Файл остаётся большим и хранится как обычный blob
- При ``git pull`` скачивается пустой файл-ссылка
- Push отклоняется как «объект слишком большой»
- Файл не отображается в ``git lfs ls-files``

Причины:

1. Правило ``.gitattributes`` отсутствует или неправильное
2. LFS не активирован (не вызван ``git lfs install``)
3. Удалённый сервер не поддерживает LFS или превышен квота

4. Файл добавлен до настройки LFS, без миграции

- **Файл остаётся большим и хранится как обычный blob** — Файл добавлен до настройки LFS, без миграции
- **При `git pull` скачивается пустой файл-ссылка** — LFS не активирован (не вызван `git lfs install`)
- **Push отклоняется как «объект слишком большой»** — Удалённый сервер не поддерживает LFS или превышен квота
- **Файл не отображается в `git lfs ls-files`** — Правило `.gitattributes` отсутствует или неправильное

🔮 Шаг 11:

Миссия 11

Сопоставьте персонажа и его задачу при внедрении LFS:

Подсказка: У каждого своя роль — разработчик, DevOps, QA.

Персонажи:

Ответ:

- Разработчик
- DevOps инженер
- QA инженер
- Руководитель проекта

Задачи:

1. Проверить, что в сборке нет больших файлов вне LFS
 2. Подготовить конфигурацию в CI/CD для LFS (см. quota, pull, fetch)
 3. Внести `.gitattributes` и убедиться, что локально LFS работает
 4. Контролировать бюджет хранения LFS и квоты
- **Разработчик** — Внести `.gitattributes` и убедиться, что локально LFS работает
 - **DevOps инженер** — Подготовить конфигурацию в CI/CD для LFS (см. quota, pull, fetch)
 - **QA инженер** — Проверить, что в сборке нет больших файлов вне LFS
 - **Руководитель проекта** — Контролировать бюджет хранения LFS и квоты

🔮 Шаг 12:

Миссия 12

Расположите этапы подключения Git LFS к новому репозиторию:

Подсказка: Думайте от инициализации до проверки.

1. Добавление правил `.gitattributes` для нужных типов
2. Запуск `git lfs install`
3. Проверка, что файлы после push хранятся через LFS
4. Добавление файлов и коммит

Ответ:

- Запуск `git lfs install`
- Добавление правил `.gitattributes` для нужных типов
- Добавление файлов и коммит
- Проверка, что файлы после push хранятся через LFS

🔧 Шаг 13:

Миссия 13

Расположите шаги миграции существующих файлов в LFS:

Подсказка: Первый шаг — резервная копия.

1. Запуск ``git lfs migrate import --include="*.bin,*.zip"``
2. Сделать бэкап репозитория или клонировать в другое место
3. Пуш изменений в удалённый репозиторий
4. Проверить, что большие файлы теперь LFS отслеживаются

Ответ:

- Сделать бэкап репозитория или клонировать в другое место
- Запуск ``git lfs migrate import --include="*.bin,*.zip"``
- Проверить, что большие файлы теперь LFS отслеживаются
- Пуш изменений в удалённый репозиторий

🔧 Шаг 14:

Миссия 14

Расположите действия при обнаружении файла, который по ошибке был закоммичен как обычный вместо LFS:

Подсказка: Надо удалить из индекса и пересоздать коммит.

1. Удалить файл из индекса: ``git rm --cached path/to/file``
2. Добавить шаблон в ``.gitattributes``
3. Добавить файл снова (``git add``) и закоммитить
4. Пустить миграцию для старых коммитов (если нужно)

Ответ:

- Добавить шаблон в ``.gitattributes``
- Удалить файл из индекса: ``git rm --cached path/to/file``
- Добавить файл снова (``git add``) и закоммитить
- Пустить миграцию для старых коммитов (если нужно)

🔧 Шаг 15:

Миссия 15

Расположите этапы устранения конфликта при слиянии файла, который отслеживается LFS:

Подсказка: Обработайте конфликт, затем обновите LFS-ссылки.

1. Разрешить конфликт в файле вручную, если это возможно
2. Выполнить ``git lfs pull`` или ``git lfs fetch`` для получения нужных объектов
3. Добавить файл в индекс (``git add``) после разрешения конфликта
4. Закончить слияние через ``git commit``

Ответ:

- Разрешить конфликт в файле вручную, если это возможно
- Выполнить ``git lfs pull`` или ``git lfs fetch`` для получения нужных объектов
- Добавить файл в индекс (``git add``) после разрешения конфликта
- Закончить слияние через ``git commit``

🔧 Шаг 16:

Миссия 16

Расположите шаги удаления файла из репозитория и освобождения места, если он был ранее сохранён через LFS:

Подсказка: Удаление из истории, затем уборка.

1. Выполнить ``git lfs untrack`` и удалить файл из индекса
2. Применить ``git filter-repo`` или ``git lfs migrate export`` для удаления истории LFS объекта
3. Закоммитить и запустить изменения
4. Очистить локальные объекты: ``git lfs prune`` или ``git gc``

Ответ:

- Выполнить ``git lfs untrack`` и удалить файл из индекса
- Применить ``git filter-repo`` или ``git lfs migrate export`` для удаления истории LFS объекта
- Закоммитить и запустить изменения
- Очистить локальные объекты: ``git lfs prune`` или ``git gc``

📖 Урок 3 - Отслеживание и хранение бинарных файлов

🔧 Шаг 01:

Миссия 1

Вы — разработчик, и вам нужно добавить в репозиторий большие изображения (по 50 МБ каждый). Какой подход выбрать?

Подсказка: Git плохо хранит большие двоичные файлы в обычных ветках.

Ответ:

- **Использовать Git LFS для отслеживания таких файлов**
- Закоммитить их обычным способом прямо в репозиторий
- Хранить их в отдельной папке вне Git и игнорировать

🔧 Шаг 02:

Миссия 2

Вы решили подключить Git LFS к проекту. Что вы должны сделать в первую очередь?

Подсказка: Нужно инициализировать расширение в вашем локальном репозитории.

Ответ:

- **Запустить ``git lfs install``**
- Добавить LFS-файлы в `.gitignore`
- Создать ветку `lfs-setup`
- Удалить остальные файлы из репозитория

🔧 Шаг 03:

Миссия 3

После ``git lfs install`` вы хотите начать отслеживать файл формата `.psd`. Что вы делаете?

Подсказка: Используйте команду, которая инструктирует Git LFS отслеживать определённые расширения.

Ответ:

- **Выполнить ``git lfs track "*.psd"``**

- Добавить *.psd в .gitignore
- Коммитить .psd вручную как обычный файл

🔗 Шаг 04:

Миссия 4

Задача: Сопоставьте команды Git LFS с их назначением.

Команды:

Ответ:

- git lfs install
- git lfs track
- git lfs untrack
- git lfs ls-files

Назначения:

1. Отменить отслеживание шаблона
2. Установить Git LFS в репозиторий
3. Показать, какие файлы уже под управлением LFS
4. Добавить правило отслеживания для расширения или шаблона

Подсказка: «install» активирует LFS, «track» и «untrack» — противоположные функции.

- **git lfs install** — Установить Git LFS в репозиторий
- **git lfs track** — Добавить правило отслеживания для расширения или шаблона
- **git lfs untrack** — Отменить отслеживание шаблона
- **git lfs ls-files** — Показать, какие файлы уже под управлением LFS

🔗 Шаг 05:

Миссия 5

Задача: Сопоставьте состояния файла с тем, как он отображается после включения Git LFS.

Состояния:

Ответ:

- Обычный файл .psd
- Файл, уже отслеживаемый LFS
- Файл удалён локально, но в LFS
- Файл изменён в рабочей директории

Отображение LFS:

1. Содержит текстовый указатель (pointer), а не реальные данные
2. Показывается как модифицированный (modified)
3. Показывается как удалённый (deleted)
4. Обычный двоичный файл, пока не отслеживается LFS

Подсказка: Дума: если файл уже под LFS, коммит хранится как pointer, а не содержимое.

- **Обычный файл .psd** — Обычный двоичный файл, пока не отслеживается LFS
- **Файл, уже отслеживаемый LFS** — Содержит текстовый указатель (pointer), а не реальные данные
- **Файл удалён локально, но в LFS** — Показывается как удалённый (deleted)

- **Файл изменён в рабочей директории** — Показывается как модифицированный (modified)

🔗 Шаг 06:

Миссия 6

Вы случайно закоммитили большой файл .psd как обычный файл (не через LFS). Что разумно сделать?

Подсказка: Нужно переписать историю, чтобы файл не хранился в обычном Git.

Ответ:

- **Использовать ``git filter-repo`` или ``git lfs migrate import``**
- Коммитить ещё один коммит, который удалит файл
- Ничего не делать, пусть остаётся
- Создать новую ветку и начать заново

🔗 Шаг 07:

Миссия 7

Вы хотите мигрировать уже закоммиченные большие файлы в LFS. Какой инструмент вы можете использовать?

Подсказка: Есть встроенная поддержка миграции от Git LFS.

Ответ:

- **Использовать ``git lfs migrate import --include="*.psd"``**
- Скопировать репозиторий вручную
- Запустить ``git merge lfs``

🔗 Шаг 08:

Миссия 8

Задача: Сопоставьте стадии миграции в LFS с их действиями.

Стадии:

Ответ:

- Выбор шаблонов для миграции
- Пробный запуск миграции
- Перезапись истории
- Проверка истории и пуш

Действия:

1. Использовать ключ ``--include`` или ``--everything``
2. Запустить миграцию без пуша для проверки
3. Заменить коммиты с файлами на LFS-поинтеры
4. Убедиться, что новая история пушится без ошибок

Подсказка: Сначала указываем, что мигрировать, затем пробуем, потом переписываем, и в конце пушим.

- **Выбор шаблонов для миграции** — Использовать ключ ``--include`` или ``--everything``
- **Пробный запуск миграции** — Запустить миграцию без пуша для проверки
- **Перезапись истории** — Заменить коммиты с файлами на LFS-поинтеры
- **Проверка истории и пуш** — Убедиться, что новая история пушится без ошибок

🔗 Шаг 09:

Миссия 9

Задача: Сопоставьте ошибки с возможными причинами при работе с Git LFS.

Ошибки:

Ответ:

- Ошибка «batch response: Not Found»
- Ошибка «This repository is not allowed to use LFS»
- LFS файл не загружается при `git push`
- Слишком большое время ожидания при загрузке LFS

Причины:

1. Сервер не поддерживает LFS для данного репозитория
2. Отсутствие прав на push LFS объектов
3. Сеть медленная или таймаут
4. Файл не был отслежен через LFS

Подсказка: Часто проблемы связаны с настройками сервера или правами доступа.

- **Ошибка «batch response: Not Found»** — Сервер не поддерживает LFS для данного репозитория
- **Ошибка «This repository is not allowed to use LFS»** — Отсутствие прав на push LFS объектов
- **LFS файл не загружается при `git push`** — Файл не был отслежен через LFS
- **Слишком большое время ожидания при загрузке LFS** — Сеть медленная или таймаут

🔗 Шаг 10:

Миссия 10

Вы работаете в команде, и коллега забывает запускать `git lfs install` после клонирования. Что сделать?

Подсказка: Можно автоматизировать это через конфигурацию.

Ответ:

- **Добавить инструкцию в README и pre-check hook, проверяющий LFS**
- Игнорировать, пусть делает вручную
- Запретить коммиты без LFS локально

🔗 Шаг 11:

Миссия 11

Задача: Сопоставьте компоненты конфигурации Git с их влиянием на LFS.

Компоненты:

Ответ:

- Файл .gitattributes
- Настройки Git-клиента
- Серверная поддержка LFS
- Права на репозиторий

Влияние:

1. Определяет шаблоны файлов, которые LFS будет отслеживать

2. Позволяет включать LFS после clone (git lfs install)
3. Без сервера, поддерживающего LFS, пуш невозможен
4. Без прав push LFS объекты не будут приняты

Подсказка: Конфигурации работают вместе: клиент, атрибуты и сервер.

- **.gitattributes** — Определяет шаблоны файлов, которые LFS будет отслеживать
- **Настройки Git-клиента** — Позволяет включать LFS после clone (git lfs install)
- **Серверная поддержка LFS** — Без сервера, поддерживающего LFS, пуш невозможен
- **Права на репозиторий** — Без прав push LFS объекты не будут приняты

Шаг 12:

Миссия 12

Расположите шаги при работе с LFS после клонирования чужого проекта:

1. Запуск ``git lfs install``
2. Проверка файла `.gitattributes` на наличие шаблонов
3. Выполнение обычного ``git checkout`` ветки
4. Проверка, что большие файлы скачались корректно

Подсказка: После clone нужно активировать LFS, затем checkout, затем проверить результат.

Ответ:

- Запуск ``git lfs install``
- Проверка файла `.gitattributes` на наличие шаблонов
- Выполнение обычного ``git checkout`` ветки
- Проверка, что большие файлы скачались корректно

Шаг 13:

Миссия 13

Расположите этапы решения проблемы «файл не загружается через LFS»:

1. Проверить, что файл отслежен (`git lfs ls-files`)
2. Убедиться, что сервер поддерживает LFS
3. Проверить права доступа на репозиторий
4. Выполнить ``git push`` ещё раз или повторно отправить LFS объекты

Подсказка: Сначала проверяем локально, потом сервер и права, затем повторно пытаемся.

Ответ:

- Проверить, что файл отслежен (`git lfs ls-files`)
- Убедиться, что сервер поддерживает LFS
- Проверить права доступа на репозиторий
- Выполнить ``git push`` ещё раз или повторно отправить LFS объекты

Шаг 14:

Миссия 14

Расположите этапы миграции существующих файлов в LFS:

1. Запустить ``git lfs migrate import`` с нужными шаблонами
2. Проверить репозиторий, что все файлы заменены на pointer
3. Закоммитить изменения

4. Пушить новую историю на сервер

Подсказка: Миграция сначала преобразует, затем проверяем, затем фиксируем и пушим.

Ответ:

- Запустить ``git lfs migrate import`` с нужными шаблонами
- Проверить репозиторий, что все файлы заменены на pointer
- Закоммитить изменения
- Пушить новую историю на сервер

Шаг 15:

Миссия 15

Расположите шаги отката миграции, если что-то пошло не так:

1. Переключиться на резервную ветку перед миграцией
2. Сравнить историю до и после миграции
3. Вернуть указатели на резервную ветку
4. Пушнуть откат на сервер

Подсказка: Всегда имейте резервную копию перед рискованной операцией.

Ответ:

- Переключиться на резервную ветку перед миграцией
- Сравнить историю до и после миграции
- Вернуть указатели на резервную ветку
- Пушнуть откат на сервер

Шаг 16:

Миссия 16

Вам доверили репозиторий с множеством больших файлов разных форматов. Расположите глобальную стратегию внедрения Git LFS:

1. Обсудить с командой, какие типы файлов будут в LFS
2. Инициализировать LFS (``git lfs install``)
3. Добавить шаблоны и закоммитить `.gitattributes`
4. Мигрировать старые файлы в LFS
5. **Обучить команду** — добавить pre-commit hook и документацию
6. Следить и проверять корректность работы LFS

Подсказка: Стратегия — сначала план, потом техническая часть, потом контроль и обучение.

Ответ:

- Обсудить с командой, какие типы файлов будут в LFS
- Инициализировать LFS (``git lfs install``)
- Добавить шаблоны и закоммитить `.gitattributes`
- Мигрировать старые файлы в LFS
- **Обучить команду** — добавить pre-commit hook и документацию
- Следить и проверять корректность работы LFS

Урок 4 - Практика работа с медиаконтентом

Шаг 01:

Миссия 1

Вы решили добавить в репозиторий Git LFS большие медиафайлы — какие типы файлов стоит отслеживать с помощью LFS?

Подсказка: Обычно это файлы, занимающие много места и редко меняющиеся побитово.

Ответ:

- **.png файлов**
- **.mp4 файлов**
- .txt файлов
- .md файлов

Шаг 02:

Миссия 2

Вы добавили файл изображения, но он не попал в Git LFS и теперь лежит в обычном Git. Что вы сделаете?

Подсказка: Нужно мигрировать содержимое, но сохранить историю.

Ответ:

- **Использовать `git lfs migrate import` для этого файла**
- Просто удалить файл и добавить заново через LFS
- **Остаётся как есть** — оставить его в обычном Git

Шаг 03:

Миссия 3

Вы клонировали репозиторий с LFS-контентом — что потребуется, чтобы получить реальные медиафайлы?

Подсказка: Git LFS хранит ссылки, не содержимое.

Ответ:

- **Запустить `git lfs pull`**
- Запустить `git fetch`
- Запустить `git checkout`

Шаг 04:

Миссия 4

При попытке запустить большие медиафайлы вы получили ошибку “file exceeds maximum size”. Что делать?

Подсказка: Лимиты задаются Git-хостингом и LFS.

Ответ:

- **Убедиться, что файл отслеживается через LFS**
- **Разбить файл на части меньше лимита**
- Удалить файл из репозитория навсегда
- Игнорировать ошибку

🔗 Шаг 05:

Миссия 5

Коллега говорит, что удалил трек LFS для формата .mp3 — что теперь произойдёт при коммите новых .mp3 файлов?

Подсказка: LFS трекает только те, что определены сейчас.

Ответ:

- **Новые .mp3 файлы попадут в обычный Git**
- Они всё ещё пойдут в LFS
- Коммит вообще не пройдет

🔗 Шаг 06:

Миссия 6

Сопоставьте команды Git LFS и их назначение.

Подсказка: Помните, что migrate, track и pull имеют разные роли.

Команды:

Ответ:

- git lfs track
- git lfs migrate import
- git lfs pull
- git lfs ls-files

Назначения:

1. Скачать реальные медиафайлы после клона
 2. Показать, какие файлы находятся под LFS
 3. Добавить шаблоны файлов в LFS-трекеры
 4. Перенести существующие данные из Git в LFS, сохраняя историю
- **git lfs track** — Добавить шаблоны файлов в LFS-трекеры
 - **git lfs migrate import** — Перенести существующие данные из Git в LFS, сохраняя историю
 - **git lfs pull** — Скачать реальные медиафайлы после клона
 - **git lfs ls-files** — Показать, какие файлы находятся под LFS

🔗 Шаг 07:

Миссия 7

Сопоставьте сценарий с действием в LFS.

Подсказка: Вспомните, что делать после клона, после изменения треков и т.д.

Сценарии:

Ответ:

- После клона пустого репозитория с LFS-контентом
- После добавления правила track и коммита .png
- При желании увидеть версии LFS-файлов
- Для миграции старых файлов из Git

Действия:

1. git lfs pull
2. git lfs migrate import

3. `git lfs ls-files`

4. `git lfs track`

- После клона пустого репозитория с LFS-контентом — `git lfs pull`
- После добавления правила `track` и коммита `.png` — `git lfs track`
- При желании увидеть версии LFS-файлов — `git lfs ls-files`
- Для миграции старых файлов из Git — `git lfs migrate import`

🔮 Шаг 08:

Миссия 8

Сопоставьте команды для отката изменений и их эффект.

Подсказка: `git reset` и `git checkout` работают по-разному.

Команды:

Ответ:

- `git reset HEAD file.png`
- `git checkout -- file.png`
- `git lfs migrate import` (откат)
- `git lfs untrack file.png`

Эффекты:

1. Вернуть файл в незакоммиченный статус
 2. Отменить изменения рабочей копии
 3. Удалить файл из треков LFS
 4. Попытаться вернуть историю медиа к состоянию до миграции
- **`git reset HEAD file.png`** — Вернуть файл в незакоммиченный статус
 - **`git checkout -- file.png`** — Отменить изменения рабочей копии
 - **`git lfs migrate import (откат)`** — Попытаться вернуть историю медиа к состоянию до миграции
 - **`git lfs untrack file.png`** — Удалить файл из треков LFS

🔮 Шаг 09:

Миссия 9

Сопоставьте части конфигурации `.gitattributes` и их смысл для LFS.

Подсказка: Формат: путь фильт текст lock.

Части:

Ответ:

- `*.png filter=lfs`
- `*.png diff=lfs`
- `*.png merge=lfs`
- `*.png lockable`

Смыслы:

1. Позволяет блокировать файл от мерджа
2. Указывает, что файл идёт под фильтр LFS
3. Отключает дифф как обычный текст
4. Использует стратегию слияния LFS

- ***.png filter=lfs** — Указывает, что файл идёт под фильтр LFS
- ***.png diff=lfs** — Отключает дифф как обычный текст
- ***.png merge=lfs** — Использует стратегию слияния LFS
- ***.png lockable** — Позволяет блокировать файл от мерджа

🔗 Шаг 10:

Миссия 10

Сопоставьте подразделы workflow с действиями при работе с LFS в команде.

Подсказка: Порядок: трекинг → коммит → миграция → пуш → клонирование.

Этапы:

Ответ:

- Настройка треков LFS
- Миграция старых файлов в LFS
- Клонирование репозитория
- Пуш с LFS-контентом

Действия:

1. git clone + git lfs pull
 2. git commit + push с LFS после track
 3. git lfs migrate import
 4. git lfs track *.format
- **Настройка треков LFS** — git lfs track *.format
 - **Миграция старых файлов в LFS** — git lfs migrate import
 - **Клонирование репозитория** — git clone + git lfs pull
 - **Пуш с LFS-контентом** — git commit + push с LFS после track

🔗 Шаг 11:

Миссия 11

Расположите шаги миграции больших файлов в LFS в корректном порядке:

1. Запустить git lfs migrate import
2. Закоммитить файлы под LFS
3. Настроить .gitattributes с нужными шаблонами
4. Пуш в удалённый репозиторий
5. Провести тесты и убедиться, что всё работает

Подсказка: Сначала настройка, затем миграция, затем коммит и пуш, и только после — проверка.

Ответ:

- Настроить .gitattributes с нужными шаблонами
- Запустить git lfs migrate import
- Закоммитить файлы под LFS
- Пуш в удалённый репозиторий
- Провести тесты и убедиться, что всё работает

🔗 Шаг 12:

Миссия 12

Расположите этапы совместной работы над LFS-контентом в команде:

1. Другой участник запускает `git lfs pull` после клона
2. Вы создаёте правило `track` и коммитите
3. Вы пушите изменения и LFS-объекты
4. Другие получают изменения и запускают `git lfs pull`
5. В коммиты попадают реальные файлы через LFS

Подсказка: Подумайте, как команда синхронизирует LFS-файлы.

Ответ:

- Вы создаёте правило `track` и коммитите
- Вы пушите изменения и LFS-объекты
- В коммиты попадают реальные файлы через LFS
- Другие получают изменения и запускают `git lfs pull`
- Другой участник запускает `git lfs pull` после клона

Шаг 13:

Миссия 13

Упорядочите действия для отката LFS-миграции в крайней ситуации:

1. Восстановить резервную копию `.git`
2. Удалить записи LFS из `.gitattributes`
3. Переконфигурировать `.gitattributes` на обычные файлы
4. Откатить коммиты после миграции
5. Провести проверку целостности репозитория

Подсказка: Откат — это сложный набор шагов: сначала возврат истории, потом удаление треков, потом проверка.

Ответ:

- Восстановить резервную копию `.git`
- Откатить коммиты после миграции
- Удалить записи LFS из `.gitattributes`
- Переконфигурировать `.gitattributes` на обычные файлы
- Провести проверку целостности репозитория

Шаг 14:

Миссия 14

Расположите последовательность при клонировании репозитория с LFS и получении файлов:

1. `git clone` репозитория
2. `git lfs install` (если ещё не установлен)
3. `git lfs pull`
4. `git checkout` ветки
5. `git fetch`

Подсказка: Сначала установка, потом `fetch/clone`, `checkout`, затем `pull`.

Ответ:

- `git lfs install` (если ещё не установлен)
- `git clone` репозитория
- `git fetch`

- git checkout ветки
- git lfs pull

Шаг 15:

Миссия 15

Упорядочите этапы полного workflow с медиа через Git LFS:

1. Настройка треков
2. Миграция старых файлов
3. Добавление новых медиафайлов
4. Коммит и пуш содержимого
5. Клонирование и получение файлов командой pull

Подсказка: Это объединение всех предыдущих сценариев в единый цикл.

Ответ:

- Настройка треков
- Миграция старых файлов
- Добавление новых медиафайлов
- Коммит и пуш содержимого
- Клонирование и получение файлов командой pull

Модуль 22 - Подписи и безопасность

Урок 1 - GPG-подписи коммитов и тегов

Шаг 01:

Миссия 1

Разработчик Саша заметил, что в репозитории появилась опция Signed commit . Что это значит?

Подсказка: Подписи помогают подтвердить, что коммит действительно сделал тот, кто утверждает, что его сделал.

Ответ:

- **Коммит подписан криптографической GPG-подписью автора**
- Коммит помечен как одобренный менеджером
- Коммит автоматически отправлен на сервер
- Коммит содержит хеш с проверкой стиля кода

Шаг 02:

Миссия 2

Инженер Марина решила создать свою первую GPG-подпись. Какую команду она должна использовать?

Подсказка: Для генерации ключа используется команда с флагом --generate-key .

Ответ:

- **gpg --full-generate-key**
- git gpg init
- gpg --create-signature
- git config --create-gpg

🔧 Шаг 03:

Миссия 3

Дмитрий хочет проверить, подписан ли коммит в репозитории. Какую команду он использует?

Подсказка: Проверка подписи выполняется не просто просмотром лога, а с дополнительным флагом проверки.

Ответ:

- **git log --show-signature**
- git verify commit
- gpg --verify commit
- git check-signatures

🔧 Шаг 04:

Миссия 4

Андрей импортировал чужой публичный ключ, чтобы проверить подписи. Какой командой это делается?

Подсказка: Импорт выполняется через gpg, а не git.

Ответ:

- **gpg --import keyfile.asc**
- git import-key keyfile.asc
- gpg --add keyfile.asc
- git config --add gpg.key

🔧 Шаг 05:

Миссия 5

Программист Илья хочет, чтобы все его коммиты автоматически подписывались. Что нужно добавить в конфигурацию Git?

Подсказка: В параметрах конфигурации есть специальная настройка для подписи коммитов по умолчанию.

Ответ:

- **git config --global commit.gpgSign true**
- git config --global gpg.auto true
- git commit --sign-all
- git enable-signing

🔧 Шаг 06:

Миссия 6

Фронтендер Лена подписала коммит, но GitHub показывает «Unverified». Что может быть причиной?

Подсказка: GitHub должен знать публичный ключ, чтобы подтвердить подпись.

Ответ:

- **Публичный ключ не добавлен в профиль GitHub**
- Подпись выполнена устаревшей версией GPG
- Репозиторий приватный
- Не выполнена команда git verify-signatures

🔗 Шаг 07:

Миссия 7

DevOps-инженер Тимофей хочет подписать тег версии релиза. Какую команду он должен использовать?

Подсказка: Для подписанных тегов есть специальный флаг.

Ответ:

- **git tag -s v1.0 -m 'Release 1.0'**
- `git tag v1.0 --sign-now`
- `git commit -s v1.0`
- `git release-sign v1.0`

🔗 Шаг 08:

Миссия 8

Системный администратор Олег настраивает GPG. Сопоставьте команды с их назначением.

Подсказка: Подумайте, какие команды управляют ключами, а какие — подписывают данные.

Ответ:

- **gpg --list-keys** — Показать список доступных публичных ключей
- **gpg --delete-secret-keys** — Удалить секретный ключ
- **gpg --sign** — Подписать файл или сообщение
- **gpg --recv-keys** — Получить ключ с сервера

🔗 Шаг 09:

Миссия 9

Тимофей хочет разобраться, как Git взаимодействует с GPG. Сопоставьте конфигурационные параметры с их описанием.

Подсказка: Обратите внимание на параметры `commit.gpgSign` и `user.signingkey` — они тесно связаны.

Ответ:

- **commit.gpgSign** — Включает подпись коммитов по умолчанию
- **user.signingkey** — Указывает идентификатор ключа GPG
- **tag.gpgSign** — Включает подпись тегов по умолчанию
- **gpg.program** — Определяет, какая программа GPG используется

🔗 Шаг 10:

Миссия 10

Команда разработчиков создала корпоративную систему подписи релизов. Сопоставьте роли с их действиями.

Подсказка: Подумайте, кто должен подписывать, а кто проверять.

Ответ:

- **Разработчик** — Создает коммит с GPG-подписью
- **Тимлид** — Проверяет подписи и валидность ключей
- **CI/CD система** — Проверяет подписи перед деплоем
- **Безопасник** — Выдает доверенные ключи команде

Шаг 11:

Миссия 11

Разработчик Алексей проверяет, какие типы ключей существуют в GPG. Сопоставьте типы ключей с их описанием.

Подсказка: Обратите внимание на разницу между публичным и приватным ключом.

Ответ:

- **Публичный ключ** — Доступен всем, используется для проверки подписи
- **Приватный ключ** — Хранится в секрете, используется для создания подписи
- **Сертификат доверия** — Подтверждает подлинность ключа
- **Идентификатор ключа** — Короткий уникальный идентификатор для работы с ключом

Шаг 12:

Миссия 12

Инженер Кира подписывает теги релизов. Сопоставьте команды с их результатом.

Подсказка: Обратите внимание на разницу между -s и -u в тегах.

Ответ:

- **git tag -s v1.0** — Создает подписанный тег с текущим ключом
- **git tag -u 0A123BCD v1.1** — Создает тег с указанием конкретного ключа
- **git show v1.0** — Показывает информацию и подпись тега
- **git tag -v v1.0** — Проверяет подпись тега

Шаг 13:

Миссия 13

У Саши сбились настройки, и он решил полностью заново настроить GPG-подпись для Git. Расставьте шаги в правильном порядке.

Подсказка: Начните с генерации ключа и закончите проверкой подписи.

Ответ:

- Сгенерировать GPG-ключ
- Добавить ключ в git config как user.signingkey
- Включить commit.gpgSign
- Подписать коммит и проверить его подпись

Шаг 14:

Миссия 14

Компания внедряет политику обязательных подписей коммитов. Расположите этапы внедрения в правильном порядке.

Подсказка: Сначала формируется ключевая инфраструктура, потом настраиваются правила и проверка.

Ответ:

- Создание и распространение доверенных ключей
- Настройка commit.gpgSign=true у разработчиков
- Настройка проверки подписей на сервере
- Автоматическая проверка в CI/CD

Шаг 15:

Миссия 15

QA-инженер хочет проверить корректность цепочки доверия. Расставьте шаги процесса.

Подсказка: Сначала импорт, потом проверка и подтверждение доверия.

Ответ:

- Импорт публичного ключа разработчика
- Проверка отпечатка ключа
- Установка уровня доверия ключу
- Проверка подписанных коммитов

Шаг 16:

Миссия 16

DevOps-команда автоматизирует верификацию подписей тегов перед релизом. Упорядочьте действия.

Подсказка: Начните с получения тегов, завершите отчётом об их проверке.

Ответ:

- Получить список тегов
- Проверить подпись каждого тега
- Собрать результаты проверки
- Сформировать отчёт CI/CD

Урок 2 - Проверка подписанных коммитов

Шаг 01:

Миссия 1

☞ Сопоставьте термин и его значение в контексте подписи коммитов:

Ответ:

- Primary key
 - Subkey
 - Web of Trust
 - Revocation certificate
1. Ключ для подписания и шифрования публикации
 2. Дополнительный ключ, часто используемый для подписи коммитов
 3. Способ проверки доверия между пользователями
 4. Документ для аннулирования ключа

Подсказка: Подумайте, как строится доверие и различие между ключами.

- **Primary key** — Ключ для подписания и шифрования публикации
- **Subkey** — Дополнительный ключ, часто используемый для подписи коммитов
- **Web of Trust** — Способ проверки доверия между пользователями
- **Revocation certificate** — Документ для аннулирования ключа

Шаг 02:

Миссия 2

Сопоставьте шаг и команду для настройки подписи GPG в Git:

Ответ:

- Указать ключ для подписей
 - Включить подпись для всех коммитов
 - Установить email, связанный с ключом
 - Проверить текущую конфигурацию
1. ``git config --global user.signingKey <ключ>``
 2. ``git config --global commit.gpgSign true``
 3. ``git config --global user.email ``
 4. ``git config --list | grep gpg``

Подсказка: Запомните порядок: сначала email и ключ, потом включение, потом проверка.

- **Указать ключ для подписей** — `git config --global user.signingKey <ключ>`
- **Включить подпись для всех коммитов** — `git config --global commit.gpgSign true`
- **Установить email, связанный с ключом** — `git config --global user.email`
- **Проверить текущую конфигурацию** — `git config --list | grep gpg`

Шаг 03:

Миссия 3

Расположите шаги проверки подписанных коммитов по порядку:

1. Запрос публичного ключа
2. Вычисление хеша коммита
3. Проверка подписи с открытым ключом
4. Сравнение хеша с тем, что подписан

Подсказка: Сначала нужно получить ключ, потом проверить подпись, затем хеши.

Ответ:

- Запрос публичного ключа
- Вычисление хеша коммита
- Проверка подписи с открытым ключом
- Сравнение хеша с тем, что подписан

Шаг 04:

Миссия 4

У Петра несколько GPG-ключей. Он хочет подписать коммиты конкретным подключом. Расположите шаги:

1. Выбрать нужный подключ
2. Скопировать его идентификатор
Указать его как `user.signingKey`
3. Сделать коммит с подписью

Подсказка: Выбор — перед указанием в конфиге, потом уже коммит.

Ответ:

- Выбрать нужный подключ
- Скопировать его идентификатор
- Указать его как `user.signingKey`
- Сделать коммит с подписью

Шаг 05:

Миссия 5

При отзыве ключа нужно выполнить действия в порядке:

1. Сгенерировать revocation certificate
2. Опубликовать сертификат в публичном ключе
3. Удалить ключ из удалённых серверов
4. Сообщить коллегам о ревокации

Подсказка: Сначала готовим документ, потом публикуем и уведомляем.

Ответ:

- Сгенерировать revocation certificate
- Опубликовать сертификат в публичном ключе
- Удалить ключ из удалённых серверов
- Сообщить коллегам о ревокации

Шаг 06:

Миссия 6

Полный порядок действий, если пользователь хочет полностью перейти на обязательную подпись:

1. Сгенерировать GPG-ключ
2. Добавить публичный ключ в сервер и аккаунт
3. Настроить Git: email, signingKey, commit.gpgSign
4. Обновить CI/CD-скрипты для проверки

Подсказка: Вы не начнёте проверку подписи, пока не создадите ключ и не зарегистрируете его.

Ответ:

- Сгенерировать GPG-ключ
- Добавить публичный ключ в сервер и аккаунт
- Настроить Git: email, signingKey, commit.gpgSign
- Обновить CI/CD-скрипты для проверки

Шаг 07:

Миссия 7

Полная последовательность ответа на подозрительный коммит с неподтверждённой подписью:

1. Сообщить автору коммита
2. Запросить публичный ключ
3. Проверить подпись и лог истории
4. Откат или повторный коммит с подписанием

Подсказка: Безопасность требует проверок и обратной связи.

Ответ:

- Сообщить автору коммита
- Запросить публичный ключ
- Проверить подпись и лог истории
- Откат или повторный коммит с подписанием

🔧 Шаг 08:

Миссия 8

Расположите все этапы защиты процесса подписанных коммитов:

1. Генерация ключей с сильной защитой
2. Регистрация публичного ключа на сервере и в аккаунте
3. Настройка Git для подписания коммитов
4. Внедрение проверки подписей в CI/CD
5. Мониторинг и отзыв ключей при необходимости

Подсказка: Это полный жизненный цикл безопасности подписи.

Ответ:

- Генерация ключей с сильной защитой
- Регистрация публичного ключа на сервере и в аккаунте
- Настройка Git для подписания коммитов
- Внедрение проверки подписей в CI/CD
- Мониторинг и отзыв ключей при необходимости

🔧 Шаг 09:

Миссия 9

Денис настроил Git в новой компании и заметил, что его коммиты не помечаются как «подписанные». Что ему нужно сделать, чтобы коммиты отображались как GPG-подписанные?

Подсказка: Подумай о связке между ключом GPG и почтой, указанной в Git-конфигурации.

Ответ:

- **Создать GPG-ключ и привязать его к email в Git config**
- Изменить глобальный user.name на случайное значение
- Включить параметр core.autocrlf
- Установить alias для команды commit

🔧 Шаг 10:

Миссия 10

Инна заметила, что при попытке подписать коммит появляется ошибка: «secret key not available». Что может быть причиной?

Подсказка: Иногда Git не знает, где искать ключ, даже если он существует.

Ответ:

- **Ключ не добавлен в gpg-agent или не импортирован в систему**
- Файл .gitignore содержит *.gpg
- Отсутствует remote-репозиторий
- Репозиторий не был клонирован через SSH

🔧 Шаг 11:

Миссия 11

Андрей работает в команде, где требуется проверка подписей. Какой флаг нужно добавить при коммите, чтобы Git создал подписанный коммит?

Подсказка: Этот флаг напрямую указывает Git на необходимость использовать GPG-подпись при сохранении изменений.

Ответ:

- **git commit -S**
- git commit --verify
- git commit -p
- git commit --trusted

Шаг 12:

Миссия 12

Света проверяет коммиты в проекте и хочет убедиться, что они подписаны. Какой командой это можно сделать?

Подсказка: Есть специальный флаг, который показывает статус проверки подписи при просмотре лога.

Ответ:

- **git log --show-signature**
- git log --check
- git verify-tag
- git blame --sign

Шаг 13:

Миссия 13

Паша хочет, чтобы все его коммиты автоматически подписывались. Что нужно добавить в конфигурацию Git?

Подсказка: Это параметр, который можно задать через git config .

Ответ:

- **git config --global commit.gpgsign true**
- git config --global user.sign all
- git config --global push.signed true
- git config --global gpg.auto true

Шаг 14:

Миссия 14

Катя получила предупреждение: «BAD signature». Что это значит при проверке коммитов?

Подсказка: Это сообщение говорит о несоответствии подписи и содержимого коммита.

Ответ:

- **Коммит был изменён после подписания**
- Ключ подписания не найден
- Подпись не содержит времени создания
- Проблема с SSH-подключением

Шаг 15:

Миссия 15

Максим клонировал публичный репозиторий и хочет убедиться, что теги, отмечающие релизы, действительно подписаны автором. Что ему следует использовать?

Подсказка: Эта команда используется специально для проверки подлинности тегов.

Ответ:

- **git verify-tag**
- git verify-commit
- git show --sign
- git tag --check

🔧 Шаг 16:

Миссия 16

🔗 Сопоставь ситуации и возможные причины ошибок при подписании коммитов.

Подсказка: Внимательно посмотри на формулировки — они могут указывать на проблемы с настройкой ключей.

Ответ:

- **Ошибка «No secret key»** — Ключ не импортирован в gpg-agent
- **Ошибка «Inappropriate ioctl for device»** — Не настроен вывод pinentry
- **Ошибка «Unusable public key»** — Ключ не имеет параметра sign
- **Ошибка «BAD signature»** — Коммит был изменён после подписи

🔧 Шаг 17:

Миссия 17

🔗 Сопоставь команды Git с их назначением при работе с подписями.

Подсказка: Подумай, какие из них отвечают за создание, проверку и настройку.

Ответ:

- **git commit -S** — Создание подписанного коммита
- **git log --show-signature** — Проверка подписей в истории
- **git tag -s v1.0** — Создание подписанного тега
- **git verify-commit** — Проверка подписи конкретного коммита

🔧 Шаг 18:

Миссия 18

🔗 Сопоставь файлы и их роль при настройке подписей.

Подсказка: Эти файлы часто встречаются при работе с GPG и Git-конфигурациями.

Ответ:

- **~/.gnupg/pubring.kbx** — Хранилище публичных ключей
- **~/.gnupg/secring.gpg** — Хранилище приватных ключей
- **~/.gitconfig** — Файл с настройками Git, включая GPG
- **~/.bashrc** — Может содержать алиасы и переменные среды для gpg

🔧 Шаг 19:

Миссия 19

Сопоставь типы ключей и их особенности использования в Git.

Подсказка: Не все ключи подходят для подписи — некоторые только для шифрования или аутентификации.

Ответ:

- **RSA** — Поддерживается всеми системами, универсален

- **ED25519** — Современный, компактный и безопасный
- **DSA** — Устаревший, ограничен по длине ключа
- **ECC** — Быстрый и эффективный на новых системах

Урок 3 - Настройка trust chain

Шаг 01:

Миссия 1

Вася настраивает свой новый ноутбук и хочет подписывать коммиты. Что ему нужно сделать в первую очередь, чтобы Git знал, какой ключ использовать?

Подсказка: Без указания правильного ключа Git не сможет добавить подпись к коммиту.

Ответ:

- **Выполнить команду `git config --global user.signingkey <ключ>`**
- Сгенерировать SSH-ключ
- Настроить alias для git commit
- Изменить глобальный email пользователя

Шаг 02:

Миссия 2

Марина получила предупреждение: «gpg: Can't check signature: No public key». Что это значит?

Подсказка: Подпись есть, но Git не может проверить, кому она принадлежит.

Ответ:

- **На машине Марины нет публичного ключа автора коммита**
- Коммит не подписан вообще
- Файл .gitignore повреждён
- GPG отключён в настройках Git

Шаг 03:

Миссия 3

Разработчик Андрей хочет, чтобы все его коммиты по умолчанию подписывались. Какую настройку он должен включить?

Подсказка: Это можно сделать глобально, чтобы не писать флаг вручную при каждом коммите.

Ответ:

- **`git config --global commit.gpgsign true`**
- `git config --global core.editor vim`
- `git config --global user.sign true`
- `git config --global sign.default yes`

Шаг 04:

Миссия 4

В компании “SecureSoft” коммиты с неподписанными изменениями запрещены. Как можно отклонить непроверенные подписи при слиянии?

Подсказка: Есть специальный флаг, который делает проверку обязательной при merge или pull.

Ответ:

- **`git merge --verify-signatures`**

- `git commit --force`
- `git rebase --sign`
- `git pull --auth-only`

🔧 Шаг 05:

Миссия 5

Олег случайно импортировал неверный публичный ключ. Как безопасно удалить его из GPG-хранилища?

Подсказка: Помните, что удаляются именно публичные ключи, а не приватные.

Ответ:

- **`gpg --delete-key <идентификатор>`**
- `gpg --remove-secret-key`
- `git rm .gpg`
- `gpg --clean-all`

🔧 Шаг 06:

Миссия 6

Саша проверяет подписи в репозитории и сталкивается с разными типами статусов. Сопоставьте статус подписи с его смыслом.

Подсказка: Вспомните цвета и ключевые слова, которые выводит `git log --show-signature`.

Ответ:

- **Good signature** — Подпись действительная и доверена.
- **Bad signature** — Подпись неверная или изменён коммит.
- **Unknown key** — Подпись найдена, но ключ не известен системе.
- **No signature** — Коммит не содержит подписи.

🔧 Шаг 07:

Миссия 7

Компания внедряет систему доверенных ключей. Сопоставьте действия с их назначением.

Подсказка: Разберитесь, где добавляется доверие, а где создаются ключи.

Ответ:

- **`gpg --full-generate-key`** — Создание новой пары ключей GPG.
- **`gpg --edit-key`** — Изменение уровня доверия к ключу.
- **`gpg --export --armor`** — Экспорт публичного ключа в текстовом виде.
- **`git config --global user.signingkey`** — Указание ключа, который Git будет использовать для подписи.

🔧 Шаг 08:

Миссия 8

В проекте появилось требование хранить публичные ключи всех участников. Сопоставьте место хранения с его назначением.

Подсказка: Некоторые ключи доступны публично, другие — только внутри команды.

Ответ:

- **keyserver** — Публичный сервер для обмена ключами.
- **gnupg/pubring.kbx** — Локальное хранилище публичных ключей пользователя.

- **.git/config** — Настройки подписи и привязка ключа.
- **trustedkeys.gpg** — Файл доверенных ключей, определяющий chain доверия.

🔗 Шаг 09:

Миссия 9

Дима разбирается в уровнях доверия ключей. Сопоставьте уровень доверия с его описанием.

Подсказка: Чем выше уровень, тем больше уверенность, что владелец проверяет других корректно.

Ответ:

- **Unknown** — Доверие не определено.
- **None** — Владелец не проверяет других.
- **Marginal** — Можно частично доверять подтверждениям.
- **Full** — Полное доверие к тому, кого подтвердил владелец.

🔗 Шаг 10:

Миссия 10

Аня настраивает trust chain и хочет понять, как работает верификация. Сопоставьте понятие с его описанием.

Подсказка: Обратите внимание, кто кому доверяет в цепочке.

Ответ:

- **Web of Trust** — Сеть взаимных подтверждений ключей пользователей.
- **Root of trust** — Начальная точка, которой доверяют без проверки.
- **Intermediate signer** — Подтверждает ключи других участников.
- **Signature chain** — Цепочка подтверждений между ключами.

🔗 Шаг 11:

Миссия 11

Петя экспортирует свой публичный ключ на сервер. Сопоставьте команды с их результатом.

Подсказка: Не все команды публикуют, некоторые только отображают.

Ответ:

- **gpg --send-keys** — Отправка ключа на публичный сервер.
- **gpg --list-keys** — Просмотр списка известных публичных ключей.
- **gpg --recv-keys** — Загрузка ключа с сервера.
- **gpg --delete-key** — Удаление публичного ключа.

🔗 Шаг 12:

Миссия 12

Евгений заметил, что GitHub показывает зелёную метку «Verified». Сопоставьте причину метки с её смыслом.

Подсказка: Не все подписи проходят проверку одинаково — важен тип ключа.

Ответ:

- **Подпись совпадает с ключом пользователя** — Коммит «Verified».
- **Подпись выполнена неизвестным ключом** — Коммит «Unverified».
- **Коммит не подписан** — Метка отсутствует.
- **Подпись от другой учётной записи** — GitHub помечает как подозрительную.

Шаг 13:

Миссия 13

Расставьте шаги создания trust chain в правильном порядке.

Подсказка: Сначала ключи создаются, затем обмениваются, а потом подписываются.

Ответ:

- Создание собственной пары ключей
- Импорт ключей коллег
- Подписание чужих ключей
- Экспорт доверенных ключей для распространения

Шаг 14:

Миссия 14

Установите порядок проверки подписей в Git при pull из защищённого репозитория.

Подсказка: Git сперва скачивает данные, потом проверяет подписи, затем применяет изменения.

Ответ:

- Получение данных из удалённого репозитория
- Проверка подписей коммитов
- Проверка доверенности ключей
- Применение изменений в локальной ветке

Шаг 15:

Миссия 15

Составьте последовательность шагов для публикации подписанного релиза.

Подсказка: Подпись должна быть проверена до момента публикации.

Ответ:

- Создание тега релиза
- Подписание тега GPG-подписью
- Проверка подписи другими участниками
- Публикация релиза на сервере

Шаг 16:

Миссия 16

Упорядочьте шаги по настройке GPG для работы с GitHub Actions (подписанные CI-пайплайны).

Подсказка: Вспомните, что ключ должен быть создан, экспортирован и добавлен в секреты GitHub.

Ответ:

- Создание GPG-ключа
- Экспорт приватного ключа в переменную окружения
- Добавление ключа в GitHub Secrets
- Настройка действия для подписи коммитов

Урок 4 - Практика работа с подписанными релизами

Шаг 01:

Миссия 1

Вы — разработчик в компании, и вам надо создать подписанный релиз. Что первым делом вы должны проверить?

Подсказка: Без подписи релиз не будет считаться безопасным.

Ответ:

- **Наличие приватного ключа GPG/PGP, который будет использоваться для подписи**
- Текущий размер бинарных артефактов
- Наличие тестов в проекте

Шаг 02:

Миссия 2

Вы уже сгенерировали ключ и хотите подписать релиз автоматически на CI. Какой формат подписи наиболее часто используют?

Подсказка: Популярный формат подписей в экосистеме пакетов — ASCII-армор.

Ответ:

- **ASCII-armored detached signature (.asc)**
- Inline подпись в бинарнике
- Base64-встроенная подпись в комментариях

Шаг 03:

Миссия 3

Сопоставьте термин и его роль при подписанных релизах.

Термины:

Ответ:

- Detached signature
- Web of Trust
- Revocation certificate
- Trust level (ultimate / full / marginal)

Роли / описания:

1. Подпись, хранимая отдельно от содержимого
2. Сеть доверительных отношений между пользователями
3. Документ, который позволяет аннулировать ключ
4. Уровень доверия к публичному ключу

Подсказка: Подумайте, зачем нужна отзываемость и доверие в экосистеме ключей.

- **Detached signature** — Подпись, хранимая отдельно от содержимого
- **Web of Trust** — Сеть доверительных отношений между пользователями
- **Revocation certificate** — Документ, который позволяет аннулировать ключ
- **Trust level (ultimate / full / marginal)** — Уровень доверия к публичному ключу

Шаг 04:

Миссия 4

Свяжите этапы процесса CI-подписания релиза с их назначением.

Этапы:

Ответ:

- Сборка артефактов
- Подпись артефактов
- Публикация публичного ключа
- Верификация на стороне потребителя

Назначения:

1. Генерация релизов из исходников
2. Создание подписи к артефактам
3. Предоставление ключа для проверки
4. Проверка подписи конечным пользователем или системой

Подсказка: Подумайте, в каком порядке эти шаги происходят и зачем публиковать публичный ключ.

- **Сборка артефактов** — Генерация релизов из исходников
- **Подпись артефактов** — Создание подписи к артефактам
- **Публикация публичного ключа** — Предоставление ключа для проверки
- **Верификация на стороне потребителя** — Проверка подписи конечным пользователем или системой

Шаг 05:

Миссия 5

Сопоставьте проблему с возможным последствием в системе подписанных релизов.

Проблемы:

Ответ:

- Компрометирован приватный ключ
- Неиспользование сертификата отзыва
- Публичный ключ не проверен
- Ключ истёк

Последствия:

1. Злоумышленник может подделать подписи
2. Пользователи продолжают доверять небезопасному ключу
3. Подписанные релизы не признаются доверенными
4. Старая подпись не считается действительной после истечения

Подсказка: Представьте, что произойдёт, если ключ потеряет срок или его не отзовут.

- **Компрометирован приватный ключ** — Злоумышленник может подделать подписи
- **Неиспользование сертификата отзыва** — Пользователи продолжают доверять небезопасному ключу
- **Публичный ключ не проверен** — Подписанные релизы не признаются доверенными
- **Ключ истёк** — Старая подпись не считается действительной после истечения

🔗 Шаг 06:

Миссия 6

Расположите шаги, которые выполняются при загрузке подписанного релиза через CI.

1. Сборка проекта
2. Генерация подписи
3. Публикация артефактов и подписи
4. Проверка подписи потребителем

Подсказка: Убедитесь, что подпись создаётся до публикации, и только после публикации проверяется.

Ответ:

- Сборка проекта
- Генерация подписи
- Публикация артефактов и подписи
- Проверка подписи потребителем

🔗 Шаг 07:

Миссия 7

Рассортируйте уровни доверия по убыванию (от наивысшего к низшему): ultimate, marginal, full.

Подсказка: Уровень ultimate ставится владельцу ключа.

Ответ:

- ultimate
- full
- marginal

🔗 Шаг 08:

Миссия 8

Упорядочите шаги для отзыва скомпрометированного ключа:

1. Опубликовать revocation certificate
2. Оп уведомить пользователей
3. Удалить приватный ключ локально
4. Обновить публичный ключ в ключевых серверах

Подсказка: Сперва публикуем, затем убираем ключ и уведомляем.

Ответ:

- Опубликовать revocation certificate
- Обновить публичный ключ в ключевых серверах
- Удалить приватный ключ локально
- Оп уведомить пользователей

🔗 Шаг 09:

Миссия 9

Сопоставьте команды GPG и их назначения при подписании релизов:

Команды:

Ответ:

- `gpg --armor --detach-sign`
- `gpg --verify`
- `gpg --list-keys`
- `gpg --gen-revoke`

Назначения:

1. Создание ASCII-подписи к файлу
2. Проверка подписи файла
3. Показ публичных ключей в локальном кольце
4. Генерация сертификата отзыва ключа

Подсказка: Обратите внимание на ключевые слова в командах: `verify`, `list`, `revoke`.

- **`gpg --armor --detach-sign`** — Создание ASCII-подписи к файлу
- **`gpg --verify`** — Проверка подписи файла
- **`gpg --list-keys`** — Показ публичных ключей в локальном кольце
- **`gpg --gen-revoke`** — Генерация сертификата отзыва ключа

 Шаг 10:

Миссия 10

Сопоставьте атрибуты подписи с их функцией:

Атрибуты:

Ответ:

- Дата подписи
- Имя подписавшего
- Алгоритм
- Хэш-функция

Функции:

1. Определяет, когда была сделана подпись
2. Определяет, каким ключом подписано
3. Указывает, каким алгоритмом шифрования подписано
4. Указывает, какая хеш-функция использована для подписи

Подсказка: Подумайте, какие детали важны для безопасности подписи.

- **Дата подписи** — Определяет, когда была сделана подпись
- **Имя подписавшего** — Определяет, каким ключом подписано
- **Алгоритм** — Указывает, каким алгоритмом шифрования подписано
- **Хэш-функция** — Указывает, какая хеш-функция использована для подписи

 Шаг 11:

Миссия 11

Упорядочите уровни подписи от наименее безопасного к наиболее безопасному:

1. Без подписи
2. Подпись с неизвестным ключом
3. Подпись с доверенным ключом
4. Подпись с ключом, которому вы поставили `ultimate` доверие

Подсказка: Без подписи — самое слабое звено, доверие ключа усиливает защиту.

Ответ:

- Без подписи
- Подпись с неизвестным ключом
- Подпись с доверенным ключом
- Подпись с ключом, которому вы поставили ultimate доверие

Шаг 12:

Миссия 12

Расположите шаги реакции на инцидент компрометации ключа:

1. Оповестить пользователей и команду
2. Сгенерировать и опубликовать revocation certificate
3. Убрать ключ из CI и репозитория
4. Выдать новый ключ и обновить подписи релизов

Подсказка: Последовательность: уведомление, отзыв, удаление, реэмиссия.

Ответ:

- Оповестить пользователей и команду
- Сгенерировать и опубликовать revocation certificate
- Убрать ключ из CI и репозитория
- Выдать новый ключ и обновить подписи релизов

Модуль 23 - Теги и релизы

Урок 1 - Лёгкие и аннотированные теги

Шаг 01:

Миссия 1

Разработчик Лена создала стабильную версию своего проекта и хочет отметить её в истории Git. Что она должна использовать?

Подсказка: Это не ветка и не коммит, но указывает на конкретный момент времени.

Ответ:

- **Тег (tag)**
- Ветку (branch)
- Слияние (merge)
- Удалённый репозиторий

Шаг 02:

Миссия 2

Андрей хочет создать тег для версии 1.0, но без дополнительных комментариев. Какой тип тега ему нужен?

Подсказка: Он просто указывает на коммит, без метаданных или описания.

Ответ:

- **Лёгкий тег (lightweight tag)**
- Аннотированный тег (annotated tag)
- Подписанный тег (signed tag)

- Слияние тегов

Шаг 03:

Миссия 3

Инженер по безопасности Вика хочет, чтобы в теге хранились автор, дата и комментарий. Какой тип тега она должна использовать?

Подсказка: Этот тип больше похож на полноценный объект в Git.

Ответ:

- **Аннотированный тег**
- Лёгкий тег
- Подпись GPG
- Хеш коммита

Шаг 04:

Миссия 4

Петя создал тег, но его коллега на сервере не видит. Что Петя забыл сделать?

Подсказка: Не все теги автоматически отправляются в удалённый репозиторий.

Ответ:

- **Выполнить команду `git push origin --tags`**
- Сделать merge
- Переключиться на тег
- Удалить старые ветки

Шаг 05:

Миссия 5

Фрилансер Костя создал тег v2.0, но понял, что ошибся коммитом. Как правильно его обновить?

Подсказка: Git не любит, когда теги просто перезаписывают без флага.

Ответ:

- **Использовать `git tag -f v2.0`**
- Удалить тег с помощью `git tag -d` и пересоздать
- Использовать `git amend`
- Переименовать тег через `git rename-tag`

Шаг 06:

Миссия 6

QA-инженер Мария хочет проверить список всех тегов в проекте. Какую команду нужно использовать?

Подсказка: Эта команда выводит только список тегов без коммитов.

Ответ:

- **`git tag`**
- `git log --tags`
- `git show-tags`
- `git branch -t`

🔗 Шаг 07:

Миссия 7

Разработчик Егор хочет удалить тег v1.2 из локального репозитория. Какой командой это сделать?

Подсказка: Это почти как удалить ветку, только короче.

Ответ:

- **git tag -d v1.2**
- git delete v1.2
- git rm-tag v1.2
- git remove v1.2

🔗 Шаг 08:

Миссия 8

Сопоставьте тип тега с его характеристикой:

Подсказка: Один из типов хранит больше метаданных.

Ответ:

- **Лёгкий тег** — Просто ссылка на коммит без дополнительных данных.
- **Аннотированный тег** — Отдельный объект с именем, датой и автором.
- **Подписанный тег** — Аннотированный тег, дополнительно защищённый GPG-подписью.
- **Удалённый тег** — Тег, отправленный в репозиторий через push.

🔗 Шаг 09:

Миссия 9

Команда DevOps использует теги для релизов. Сопоставьте команду Git с её назначением:

Подсказка: Одна команда показывает информацию, другая — создаёт, а третья удаляет.

Ответ:

- **git tag** — Показать список всех тегов.
- **git show v2.0** — Показать информацию о конкретном теге.
- **git tag -a v3.0 -m 'Release 3.0'** — Создать аннотированный тег.
- **git tag -d v1.0** — Удалить тег из локального репозитория.

🔗 Шаг 10:

Миссия 10

Сопоставьте, где обычно применяются разные типы тегов:

Подсказка: Некоторые теги нужны для внутренних отметок, другие — для официальных релизов.

Ответ:

- **Лёгкий тег** — Временные внутренние отметки.
- **Аннотированный тег** — Официальные версии проекта.
- **Подписанный тег** — Безопасные релизы с подтверждением подлинности.
- **Удалённый тег** — Тег, доступный всей команде после push.

🔗 Шаг 11:

Миссия 11

Сопоставьте ошибку с возможным решением:

Подсказка: Иногда нужно добавить флаг, иногда пересоздать тег.

Ответ:

- **Тег не отображается на GitHub** — Отправить его через `git push origin --tags`.
- **Тег создан не на том коммите** — Использовать `git tag -f` для обновления.
- **Тег нельзя подписать** — Проверить наличие GPG-ключа.
- **Тег удалён локально, но есть на сервере** — Удалить удалённый тег через `git push origin :refs/tags/`.

🔗 Шаг 12:

Миссия 12

Сопоставьте ситуации с правильной командой:

Подсказка: Всё зависит от того, хотите вы создать, посмотреть или удалить тег.

Ответ:

- **Создать тег v1.0 без комментария** — `git tag v1.0`
- **Создать тег с сообщением** — `git tag -a v1.0 -m 'Первый релиз'`
- **Показать детали тега** — `git show v1.0`
- **Удалить тег v1.0** — `git tag -d v1.0`

🔗 Шаг 13:

Миссия 13

Расположите шаги создания аннотированного тега в правильном порядке:

Подсказка: Сначала убедитесь, что вы на нужном коммите, потом создавайте тег и публикуйте.

Ответ:

- Проверить текущий коммит (`git log -1`)
- Создать тег с сообщением (`git tag -a v1.0 -m 'Релиз 1.0'`)
- Отправить теги в удалённый репозиторий (`git push origin --tags`)
- Проверить, что тег появился (`git tag`)

🔗 Шаг 14:

Миссия 14

Расположите действия при обновлении ошибочного тега:

Подсказка: Сначала нужно удалить старый, потом создать новый и опубликовать.

Ответ:

- Удалить старый тег (`git tag -d v1.0`)
- Создать новый тег с правильным коммитом
- Отправить обновление на сервер (`git push origin --tags`)
- Проверить список тегов

🔗 Шаг 15:

Миссия 15

Расположите шаги для удаления тега с сервера:

Подсказка: Локальные и удалённые теги удаляются по-разному.

Ответ:

- Удалить тег локально (`git tag -d v1.0`)
- Удалить тег на сервере (`git push origin :refs/tags/v1.0`)
- Проверить, что тег исчез (`git tag -l`)

 Шаг 16:

Миссия 16

Расположите шаги для подписания тега с помощью GPG:

Подсказка: Перед этим убедитесь, что у вас настроен GPG-ключ.

Ответ:

- Проверить GPG-ключ (`gpg --list-keys`)
- Создать подписанный тег (`git tag -s v2.0 -m 'Подписанный релиз'`)
- Проверить подпись (`git tag -v v2.0`)
- Отправить теги на сервер (`git push origin --tags`)

 Урок 2 - Семантическое версионирование

 Шаг 01:

Миссия 1

Вы работаете над библиотекой и собираетесь выпускать первую стабильную версию. Какой номер версии вы укажете по семантическому версионированию?

Подсказка: Обычно первая стабильная версия начинается с 1.0.0.

Ответ:

- **1.0.0**
- 0.1.0
- 0.0.1

 Шаг 02:

Миссия 2

Вы выпускаете патч, в котором исправили одну мелкую ошибку и не меняли API. Какой версии вы присвоите новой версии?

Подсказка: Патч-версия меняется при обратимых исправлениях.

Ответ:

- **1.0.1**
- 1.1.0
- 2.0.0

 Шаг 03:

Миссия 3

Вы добавляете новую функцию, совместимую с предыдущим API — её можно использовать без изменений клиентского кода. Какую версию вы присвоите?

Подсказка: Минорная версия увеличивается при добавлении совместимой функциональности.

Ответ:

- **1.1.0**
- 2.0.0

- 1.0.2

Шаг 04:

Миссия 4

Вы внедряете изменение, которое ломает обратную совместимость API (например, меняете сигнатуру функции). Как поднимете номер версии?

Подсказка: Мажорная версия меняется при несовместимых изменениях.

Ответ:

- **2.0.0**
- 1.2.0
- 1.1.1

Шаг 05:

Миссия 5

Сопоставьте каждый тип изменения с нужным изменением версии:

Тип изменения:

Ответ:

- Патч
- Минорное изменение
- Мажорное изменение
- Предрелизная отметка

Версия должна быть обновлена как:

1. Добавление обратимо-совместной функции
2. Исправление багов без изменения API
3. Нарушение совместимости API
4. Версия типа 2.0.0-alpha

Подсказка: Подумайте, к какому случаю должна применяться каждая категория изменений.

- **Патч** — Исправление багов без изменения API
- **Минорное изменение** — Добавление обратимо-совместной функции
- **Мажорное изменение** — Нарушение совместимости API
- **Предрелизная отметка** — Версия типа 2.0.0-alpha

Шаг 06:

Миссия 6

Сопоставьте компонент семантической версии с его значением:

Части версии:

Ответ:

- Major
- Minor
- Patch
- Prerelease

Значения:

1. Номер несовместимых изменений

2. Дополнительные функции, обратимо-совместные
3. Исправления багов
4. Тег как alpha, beta и т.д.

Подсказка: Minor ≠ мажор, patch — мелкие исправления, prerelease часто с суффиксом.

- **Major** — Номер несовместимых изменений
- **Minor** — Дополнительные функции, обратимо-совместные
- **Patch** — Исправления багов
- **Prerelease** — Тег как alpha, beta и т.д.

Шаг 07:

Миссия 7

Сопоставьте теги Git с подходящим описанием релиза:

Теги:

Ответ:

- v1.0.0
- v2.1.5-beta
- v3.0.0-rc.1
- v1.2.3-alpha.2

Описание:

1. Первый стабильный релиз
2. Бета-версия перед мажором
3. Релиз-кандидат перед финальным
4. Ранний альфа-стадийный релиз

Подсказка: beta — бета, rc — release candidate, alpha — ранняя версия.

- **v1.0.0** — Первый стабильный релиз
- **v2.1.5-beta** — Бета-версия перед мажором
- **v3.0.0-rc.1** — Релиз-кандидат перед финальным
- **v1.2.3-alpha.2** — Ранний альфа-стадийный релиз

Шаг 08:

Миссия 8

Расположите версии по возрастанию:

Ответ:

- 1.0.1
- 1.0.0
- 1.0.0-alpha
- 1.0.0-beta.2

Подсказка: Сначала преленки, затем стабильные версии по major/minor/patch.

- 1.0.0-alpha
- 1.0.0-beta.2
- 1.0.0
- 1.0.1

Шаг 09:

Миссия 9

Убедитесь, что порядок тегов правильный (от старого к новому):

Ответ:

- v1.2.0
- v1.1.0
- v1.2.0-alpha.1
- v1.1.5

Подсказка: Прелизы — до стабильных, затем сортировка по номерам.

- v1.1.0
- v1.1.5
- v1.2.0-alpha.1
- v1.2.0

Шаг 10:

Миссия 10

У вас есть версии:

Ответ:

- 2.0.0-rc.1
- 2.0.0
- 1.9.9
- 2.0.0-alpha.2

Отсортируйте их от наименьшей к наибольшей.

Подсказка: Прелизы меньше стабильных, буквально сравнивайте части версии.

- 1.9.9
- 2.0.0-alpha.2
- 2.0.0-rc.1
- 2.0.0

Шаг 11:

Миссия 11

Дан список:

Ответ:

- v1.0.0
- v1.0.0-alpha.1
- v1.0.0-alpha
- v1.0.0-beta

Отсортируйте по возрастанию.

Подсказка: Альфа до бета, и первой альфы может быть без числа.

- v1.0.0-alpha
- v1.0.0-alpha.1
- v1.0.0-beta
- v1.0.0

🔮 Шаг 12:

Миссия 12

Список версий:

Ответ:

- 3.0.0
- 2.5.4
- 3.0.0-beta
- 2.5.5

Упорядочьте по возрастанию.

Подсказка: Сначала меньший major/minor, затем релизы перед стабильной.

- 2.5.4
- 2.5.5
- 3.0.0-beta
- 3.0.0

🔮 Шаг 13:

Миссия 13

У вас версии:

Ответ:

- 1.10.0
- 1.2.0
- 1.2.10
- 1.10.0-alpha

Отсортируйте от старой к новой.

Подсказка: Внимательно сравнивайте minor цифры, не строково.

- 1.2.0
- 1.2.10
- 1.10.0-alpha
- 1.10.0

🔮 Шаг 14:

Миссия 14

Сравните набор тегов:

Ответ:

- v2.0.0
- v2.0.0-alpha.1
- v2.0.1
- v2.1.0

Упорядочьте по возрастанию.

Подсказка: Alpha до стабильной, патчи между минорными.

- v2.0.0-alpha.1
- v2.0.0

- v2.0.1
- v2.1.0

Шаг 15:

Миссия 15

Расположите эти версии:

Ответ:

- 1.0.0
- 1.0.1
- 1.1.0-alpha
- 1.1.0

По порядку от старой к новой.

Подсказка: Альфа перед стабильной минорной.

- 1.0.0
- 1.0.1
- 1.1.0-alpha
- 1.1.0

Шаг 16:

Миссия 16

Вот версии, выпущенные с течением времени:

Ответ:

- v1.1.0
- v1.0.5
- v1.1.0-beta.1
- v1.0.5-alpha

Упорядочьте по возрастанию.

Подсказка: Альфа, затем бета, затем стабильные по номерам.

- v1.0.5-alpha
- v1.0.5
- v1.1.0-beta.1
- v1.1.0

Урок 3 - Практика создание релизов

Шаг 01:

Миссия 1

Артём готовит первый релиз своего pet-проекта. Он хочет пометить стабильную версию, чтобы команда знала, какую сборку считать «официальной». Что ему нужно сделать?

Подсказка: Подумай о метках, которые сохраняют состояние репозитория в определённый момент.

Ответ:

- Создать аннотированный тег
- Удалить старую ветку

- Создать новый коммит
- Сделать ребейз master

🔗 Шаг 02:

Миссия 2

Вика из команды DevOps хочет добавить сообщение с описанием изменений прямо к тегу релиза. Какой тип тега ей нужно использовать?

Подсказка: Обычный тег — это просто ссылка, но аннотированный — как мини-коммит.

Ответ:

- **Аннотированный тег**
- Лёгкий тег
- Merge-тег
- Branch-тег

🔗 Шаг 03:

Миссия 3

Разработчик Илья создал тег v2.0 локально, но на сервере его нет. Как опубликовать этот тег в общий репозиторий?

Подсказка: Помни, что теги не пушатся автоматически.

Ответ:

- **git push origin v2.0**
- git push --all
- git push tags
- git sync tags

🔗 Шаг 04:

Миссия 4

После релиза v1.1 команда заметила ошибку. Никита хочет удалить этот тег на локальной машине. Что ему стоит выполнить?

Подсказка: Удаление тега — почти как удаление ветки, но команда короче.

Ответ:

- **git tag -d v1.1**
- git branch -d v1.1
- git rm v1.1
- git delete v1.1

🔗 Шаг 05:

Миссия 5

Инженер QA Марина видит, что в репозитории уже есть тег v3.0 . Как ей узнать, к какому коммиту он относится?

Подсказка: Иногда полезно «раскрыть» тег, чтобы увидеть его данные.

Ответ:

- **git show v3.0**
- git log v3.0
- git diff v3.0

- `git describe v3.0`

Шаг 06:

Миссия 6

DevOps-команда компании хочет автоматически формировать changelog на основе тегов. Какие теги лучше использовать, чтобы GitHub распознал их как релизы?

Подсказка: Вспомни, какие теги обычно обозначают версии.

Ответ:

- **Семантические версии (v1.0.0, v2.1.3)**
- Теги с именем `feature/`
- Теги с именем `hotfix/`
- Теги, совпадающие с ветками

Шаг 07:

Миссия 7

Сопоставьте типы тегов с их характеристиками.

Подсказка: Один тип хранит метаданные, другой — просто ссылку на коммит.

Ответ:

- **Аннотированный тег** — Содержит автора, дату и сообщение.
- **Лёгкий тег** — Является прямой ссылкой на коммит.
- **Тег релиза** — Используется для фиксации стабильной версии проекта.
- **Тег snapshot** — Создаётся для промежуточной сборки перед релизом.

Шаг 08:

Миссия 8

Соотнеси команды Git с их назначением при работе с тегами.

Подсказка: Думай о том, что каждая команда делает: создаёт, показывает или удаляет.

Ответ:

- **`git tag -a v1.0 -m 'release'`** — Создаёт аннотированный тег.
- **`git tag -l`** — Показывает список тегов.
- **`git show v1.0`** — Выводит подробности тега.
- **`git tag -d v1.0`** — Удаляет тег локально.

Шаг 09:

Миссия 9

 Команда релиза использует GitHub. Сопоставь действия с тем, где они происходят.

Подсказка: Некоторые вещи делаются локально, а другие — прямо через веб-интерфейс.

Ответ:

- **Создание тега через интерфейс GitHub** — Выполняется на сервере, создаёт релиз автоматически.
- **Создание тега через терминал** — Происходит локально в Git.
- **Публикация тега** — Отправка локального тега на удалённый репозиторий.
- **Удаление удалённого тега** — Требуется команда `git push origin --delete`.

🔗 Шаг 10:

Миссия 10

Сопоставьте действия DevOps-команды с их целями при релизе.

Подсказка: Релиз — это не просто тег, это способ зафиксировать и описать стабильную сборку.

Ответ:

- **Создать аннотированный тег** — Добавить описание к версии.
- **Пушнуть тег в origin** — Поделиться релизом с командой.
- **Создать GitHub release** — Добавить бинарники и changelog.
- **Удалить тег** — Отозвать ошибочную версию.

🔗 Шаг 11:

Миссия 11

В проекте несколько релизов. Сопоставь версию и её тип обновления.

Подсказка: Вспомни правила семантического версионирования (MAJOR.MINOR.PATCH).

Ответ:

- **v2.0.0** — Мажорное обновление, есть несовместимые изменения.
- **v2.1.0** — Минорное обновление, добавлены новые функции.
- **v2.1.1** — Патч, исправлены баги.
- **v3.0.0** — Следующее крупное изменение API.

🔗 Шаг 12:

Миссия 12

Сопоставь команды с ситуациями из практики релизов.

Подсказка: Представь, что ты релиз-менеджер, выбираешь нужную команду под конкретную задачу.

Ответ:

- **git push origin --tags** — Отправить все локальные теги.
- **git describe** — Показать ближайший тег и количество коммитов после него.
- **git checkout v1.0.0** — Перейти к состоянию конкретного релиза.
- **git tag -f v1.0.0** — Перезаписать существующий тег.

🔗 Шаг 13:

Миссия 13

Расположи шаги создания аннотированного тега в правильном порядке.

Подсказка: Сначала — коммит, потом — тег, потом — публикация.

Ответ:

- Сделать коммит с изменениями
- Создать аннотированный тег с описанием
- Проверить тег командой git show
- Отправить тег на сервер с помощью git push origin

🔗 Шаг 14:

Миссия 14

🔗 Расставь шаги обновления версии по правилам семантического версионирования.

Подсказка: Когда меняется логика API — увеличивается major, при новых функциях — minor, при фиксе — patch.

Ответ:

- Определить тип изменений (bugfix, feature, breaking)
- Изменить номер версии в формате MAJOR.MINOR.PATCH
- Создать тег с новым номером версии
- Задеплоить релиз

Урок 4 - Работа с тегами в CI/CD

Шаг 01:

Миссия 1

 Разработчик Саша выпустил стабильную версию проекта и хочет пометить её в Git. Какой тип объекта в Git используется для создания отметки конкретного состояния кода?

Подсказка: Это не коммит и не ветка, но напрямую указывает на коммит.

Ответ:

- **Тег (tag)**
- Ветка (branch)
- Метка версии в README
- Снапшот (snapshot)

Шаг 02:

Миссия 2

 Тимлид Ира хочет отличать релизы от обычных тегов. Какой тег считается аннотированным (annotated)?

Подсказка: Он хранит больше, чем просто ссылку на коммит — ещё и метаданные.

Ответ:

- **Созданный командой `git tag -a v1.0 -m 'Release'`**
- Созданный командой `git tag v1.0`
- Добавленный через GitHub UI без описания
- Созданный автоматически при коммите

Шаг 03:

Миссия 3

 Девопс Катя настроила CI/CD и хочет, чтобы пайплайн запускался только при появлении нового тега. Что чаще всего проверяется в YAML-конфигурации?

Подсказка: В GitLab и GitHub Actions есть специальные условия для этого события.

Ответ:

- **Событие push на тег**
- Событие merge в ветку main
- Событие pull request
- Событие push на любую ветку

Шаг 04:

Миссия 4

👤 Программист Миша случайно создал тег не на том коммите. Как безопасно удалить локальный тег?

Подсказка: Удаление — это не удаление ветки, команда начинается с той же утилиты.

Ответ:

- **git tag -d v1.0**
- git branch -d v1.0
- git rm-tag v1.0
- git reset --tag v1.0

💡 Шаг 05:

Миссия 5

👤 Разработчик Лена хочет отправить тег на удалённый репозиторий. Какой командой это делается?

Подсказка: Та же команда, что и для веток, но с другим аргументом.

Ответ:

- **git push origin v1.0**
- git send-tag v1.0
- git push-tag origin v1.0
- git publish v1.0

💡 Шаг 06:

Миссия 6

👤 Команда проекта хочет быстро просмотреть все доступные теги. Какой командой это можно сделать?

Подсказка: Похожая команда выводит список веток, только слово другое.

Ответ:

- **git tag**
- git show-tags
- git list tags
- git tags list

💡 Шаг 07:

Миссия 7

В репозитории появилось несколько релизов. Какой тег принято использовать для мажорной версии по семантическому версионированию?

Подсказка: Обычно формат версии состоит из трёх чисел через точки.

Ответ:

- **v2.0.0**
- 2
- release_2
- ver-2

💡 Шаг 08:

Миссия 8

🔗 Свяжите действия с правильными командами, чтобы команда DevOps могла автоматизировать релизы.

Подсказка: Обратите внимание на то, что аннотированные теги содержат сообщение.

Ответ:

- **Создать аннотированный тег** — `git tag -a v1.0 -m 'Release'`
- **Удалить тег локально** — `git tag -d v1.0`
- **Отправить тег на сервер** — `git push origin v1.0`
- **Посмотреть все теги** — `git tag`

🔗 Шаг 09:

Миссия 9

🧠 Сопоставьте ситуации в команде с нужными действиями по работе с тегами.

Подсказка: Подумайте, где нужен новый тег, а где наоборот — удалить старый.

Ответ:

- **Ошибка в названии тега** — Удалить тег и создать новый
- **Выход новой версии** — Создать аннотированный тег
- **CI не запускается на релиз** — Проверить событие push на тег
- **Нужно увидеть все доступные релизы** — Посмотреть список тегов

🔗 Шаг 10:

Миссия 10

💬 Сопоставьте типы тегов с их характеристиками.

Подсказка: Один тип хранит только ссылку, другой — описание и подпись.

Ответ:

- **Лёгкий тег** — Хранит только ссылку на коммит
- **Аннотированный тег** — Содержит метаданные, автора и сообщение
- **Подписанный тег** — Защищён GPG-подписью
- **CI-тег** — Создаётся автоматически при сборке

🔗 Шаг 11:

Миссия 11

📦 Сопоставьте команды Git с их результатом, чтобы помочь новичку Диме настроить релизы.

Подсказка: У каждой команды своя роль в цикле создания тега.

Ответ:

- **`git show v1.0`** — Показывает информацию о теге и связанном коммите
- **`git tag -s v1.1`** — Создаёт GPG-подписанный тег
- **`git push --tags`** — Отправляет все теги на сервер
- **`git tag -l 'v1.*'`** — Фильтрует список тегов по шаблону

🔗 Шаг 12:

Миссия 12

⚙️ Сопоставьте типы событий CI/CD с их запуском, чтобы разобраться, какие сценарии активируются при теге.

Подсказка: Теги обычно обрабатываются отдельно от веток.

Ответ:

- **push на ветку main** — Запускает тесты и линтер
- **push тега v1.0** — Запускает релизный пайплайн
- **merge request** — Запускает проверку кода
- **push в feature-ветку** — Запускает сборку в песочнице

Шаг 13:

Миссия 13

 Расставьте шаги подготовки релиза с тегом в правильном порядке.

Подсказка: Сначала подготовка кода, потом создание тега и его публикация.

Ответ:

- Проверить, что все изменения закоммичены
- Создать аннотированный тег с сообщением
- Отправить тег на сервер
- Проверить запуск CI/CD пайплайна

Шаг 14:

Миссия 14

Расположите действия для исправления ошибочного тега в правильном порядке.

Подсказка: Нельзя просто поверх старого создать новый — сначала нужно удалить.

Ответ:

- Удалить локальный тег командой `git tag -d`
- Удалить тег с сервера командой `git push origin :tagname`
- Создать новый тег с правильным именем
- Отправить новый тег на сервер

Шаг 15:

Миссия 15

 Расположите шаги автоматической публикации релиза при пуше тега.

Подсказка: Сначала CI должен среагировать, затем собрать и выложить.

Ответ:

- Push нового тега в репозиторий
- CI/CD определяет событие push на тег
- Сборка артефактов проекта
- Публикация релиза на GitHub или сервере

Модуль 24 - Оптимизация и чистка репозитория

Урок 1 - Git gc, prune, repack

Шаг 01:

Миссия 1

Вы — герой команды DevOps, ваш репозиторий разросся до гигабайтов. Какой из команд Git позволяет автоматически запускать сборку мусора (garbage collection)?

Подсказка: Это команда, которая очищает неиспользуемые объекты.

Ответ:

- **git gc**
- git clean
- git prune

Шаг 02:

Миссия 2

После клонирования большого проекта коллега жалуется: «У меня слишком много мусора». Какой флаг можно передать git gc для агрессивной очистки?

Подсказка: Один из флагов заставляет делать full pack.

Ответ:

- **--aggressive**
- --dry-run
- --quiet

Шаг 03:

Миссия 3

Ваша команда работает над experimental-веткой, но теперь нужно избавиться от всех неиспользуемых объектов. Какой из этих вызовов безопасно применить?

Подсказка: prune удаляет объекты, недостижимые из reflog или веток.

Ответ:

- **git prune**
- git reset --hard HEAD
- git gc --prune=now

Шаг 04:

Миссия 4

Вы заметили файл pack-файла растущего размера. Что делает команда git repack?

Подсказка: Она пересобирает объекты в компактные пакеты.

Ответ:

- **Пересобирает объекты в один или несколько pack-файлов**
- Удаляет рабочий каталог
- Синхронизирует с удалённым репозиторием

Шаг 05:

Миссия 5

Какой параметр к git gc управляет максимальным временем хранения объектов до удаления?

Подсказка: Этот параметр определяет порог, после которого объекты удаляются.

Ответ:

- **--auto**
- --expire
- --keep

Шаг 06:

Миссия 6

При запуске `git gc` вы видите строку “prune obsolete packs”. Что это означает?

Подсказка: Obsolete — означает устаревшие.

Ответ:

- **Удаление устаревших пак-файлов**
- Создание новых пак-файлов
- Синхронизация удалённых веток

Шаг 07:

Миссия 7

Сопоставьте команды `git` и их описания в контексте оптимизации репозитория:

Команды:

Ответ:

- `git gc`
- `git prune`
- `git repack`
- `git gc --aggressive`

Описания:

1. Пересобирает объекты для более плотной упаковки
2. Удаляет недостижимые объекты
3. Запускает стандартную очистку и оптимизацию репозитория
4. Запускает более тщательную и медленную оптимизацию

Подсказка: Обратите внимание: `aggressive` вариант делает дополнительную работу.

- **`git gc`** — Запускает стандартную очистку и оптимизацию репозитория
- **`git prune`** — Удаляет недостижимые объекты
- **`git repack`** — Пересобирает объекты для более плотной упаковки
- **`git gc --aggressive`** — Запускает более тщательную и медленную оптимизацию

Шаг 08:

Миссия 8

Сопоставьте параметры и их функции при работе с `git gc / prune`:

Параметры:

Ответ:

- `--prune=now`
- `--expire=`
- `--aggressive`
- `--auto`

Функции:

1. Удаляет сразу все недостижимые объекты
2. Указывает срок давности для `prune`
3. Запускает только при пороге мусора

4. Усиливает и замедляет упаковку объектов

Подсказка: Параметры обычно встречаются вместе с `git gc`.

- **--prune=now** — Удаляет сразу все недостижимые объекты
- **--expire=** — Указывает срок давности для `prune`
- **--auto** — Запускает только при пороге мусора
- **--aggressive** — Усиливает и замедляет упаковку объектов

Шаг 09:

Миссия 9

Сопоставьте флаги `git repack` и их эффекты:

Флаги:

Ответ:

- -a
- -d
- -l
- -f

Эффекты:

1. Удаляет старые пак-файлы после пересборки
2. Принудительно пересобирает даже если уже оптимизировано
3. Создаёт локальный `pack` без обращения к `remotes`
4. Добавляет все объекты в пак (включая свободные)

Подсказка: Флаги часто комбинируются для получения нужного результата.

- **-a** — Добавляет все объекты в пак (включая свободные)
- **-d** — Удаляет старые пак-файлы после пересборки
- **-l** — Создаёт локальный `pack` без обращения к `remotes`
- **-f** — Принудительно пересобирает даже если уже оптимизировано

Шаг 10:

Миссия 10

Сопоставьте этапы внутреннего процесса `git gc` с их действием:

Этапы:

Ответ:

- `pre-gc`
- `repack`
- `prune`
- `post-gc`

Действия:

1. Удаляет недостижимые объекты
2. Выполняет пользовательские хуки после `gc`
3. Пересобирает пак-файлы
4. Проверяет, стоит ли запускать `gc` (`threshold`)

Подсказка: Сначала проверяется порог, затем репак, затем `prune`, затем завершающие шаги.

- **pre-gc** — Проверяет, стоит ли запускать gc (threshold)
- **repack** — Пересобирает пак-файлы
- **prune** — Удаляет недостижимые объекты
- **post-gc** — Выполняет пользовательские хуки после gc

Шаг 11:

Миссия 11

Расположите этапы базовой оптимизации git gc в порядке выполнения:

1. repack объектов
2. prune недостижимых объектов
3. проверка порогов (auto)
4. post-gc действия

Подсказка: Сначала проверка условий, потом тяжелые операции, в конце — финализация.

Ответ:

- проверка порогов (auto)
- repack объектов
- prune недостижимых объектов
- post-gc действия

Шаг 12:

Миссия 12

Интересная задача: вы откатали ветку, но остались мусорные патчи. Расположите шаги для безопасной очистки репозитория:

1. git reflog expire --expire=now --all
2. git gc --prune=now
3. git repack -adf
4. git fsck

Подсказка: Проверять целостность стоит после очистки.

Ответ:

- git reflog expire --expire=now --all
- git repack -adf
- git gc --prune=now
- git fsck

Шаг 13:

Миссия 13

Вам нужно минимизировать размер локального репозитория перед отправкой на CI. Расставьте операции по порядку:

1. git prune
2. git repack -ad
3. git gc --aggressive --prune=now
4. git reflog expire --expire=now --all

Подсказка: Сначала избавляемся от ссылок, потом репак и GC с агрессией.

Ответ:

- `git reflog expire --expire=now --all`
- `git prune`
- `git repack -ad`
- `git gc --aggressive --prune=now`

 Шаг 14:**Миссия 14**

Ниже даны шаги по очистке репозитория после удаления большой ветки. Упорядочите их корректно:

1. `git gc --prune=now`
2. `git reflog expire --expire=now --all`
3. `git repack -Add`
4. `git fsck --full`

Подсказка: Проверка ошибок должна быть последним шагом.

Ответ:

- `git reflog expire --expire=now --all`
- `git repack -Add`
- `git gc --prune=now`
- `git fsck --full`

 Шаг 15:**Миссия 15**

Упорядочите действия для сценария: вы клонируете репозиторий и хотите очистить его сильно:

1. `git gc --aggressive`
2. `git repack -a -d -f`
3. `git prune --expire now`
4. `git reflog expire --expire=now --all`

Подсказка: `reflog expire` должен идти до `prune`, а `repack` перед `gc` агрессивным.

Ответ:

- `git reflog expire --expire=0`
- `git prune --expire now`
- `git repack -a -d -f`
- `git gc --aggressive`

 Шаг 16:**Миссия 16**

Вам предстоит очистить весь репозиторий вручную. Упорядочите шаги:

1. `git reflog expire --all --expire=now`
2. `git gc --prune=now --aggressive`
3. `git repack -Adf`
4. `git fsck --lost-found`

Подсказка: Заключительная проверка — восстановление потерянных объектов.

Ответ:

- `git reflog expire --all --expire=now`
- `git repack -Adf`
- `git gc --prune=now --aggressive`
- `git fsck --lost-found`

 Шаг 17:

Миссия 17

 **Финальная миссия:** сложный сценарий очистки. Упорядочите шаги:

1. `git reflog expire --expire=now --all`
2. `git repack -a -d -f --window=250 --depth=250`
3. `git gc --aggressive --prune=now`
4. `git prune --expire now`
5. `git fsck --full`

Подсказка: `reflog expire` снимает ссылки, потом репак, далее `gc`, далее принудительный `prune`, а в конце — проверка целостности.

Ответ:

- `git reflog expire --expire=now --all`
- `git repack -a -d -f --window=250 --depth=250`
- `git gc --aggressive --prune=now`
- `git prune --expire now`
- `git fsck --full`

 Урок 2 - Очистка больших файлов из истории

 Шаг 01:

Миссия 1

Разработчик Саша случайно добавил в репозиторий огромный файл логов размером 500 МБ. Что нужно сделать в первую очередь, чтобы избавиться от него и не повредить историю?

Подсказка: Просто удалить файл — недостаточно. История Git хранит всё!

Ответ:

- **Использовать инструмент `git filter-repo` для удаления файла из истории**
- Просто удалить файл и сделать новый коммит
- Создать новую ветку без этого файла
- Добавить файл в `.gitignore`

 Шаг 02:

Миссия 2

Аня обнаружила, что старые бинарники случайно попали в историю коммитов. Какой инструмент из стандартных возможностей Git поможет очистить историю?

Подсказка: Этот инструмент позволяет переписать всю историю, удаляя нежелательные файлы из каждого коммита.

Ответ:

- **`git filter-branch`**
- `git rm`

- git stash
- git bisect

🔧 Шаг 03:

Миссия 3

Фронтендер Лена запустила очистку через git filter-repo . Какое действие необходимо сделать после очистки, чтобы изменения применились на удалённом репозитории?

Подсказка: После переписывания истории старые коммиты больше не существуют.

Ответ:

- **Выполнить git push --force**
- Создать pull request
- Сделать обычный push
- Удалить локальный репозиторий

🔧 Шаг 04:

Миссия 4

Дима хочет проверить, какие крупные файлы спрятаны в истории репозитория. Какой инструмент поможет быстро найти все файлы, превышающие 10 МБ?

Подсказка: Этот инструмент сканирует весь Git-объектный граф и выводит подозрительные файлы.

Ответ:

- **git-sizer или git rev-list с анализом blob**
- git status
- git diff
- git branch

🔧 Шаг 05:

Миссия 5

DevOps-инженер Олег очистил репозиторий, но коллеги жалуются, что не могут делать pull. Что он должен сообщить команде?

Подсказка: После переписывания истории все коммиты поменяли хэши.

Ответ:

- **Каждому нужно заново клонировать репозиторий или выполнить git fetch --all и git reset --hard origin/main**
- Нужно снова добавить файл в .gitignore
- Нужно пересоздать все ветки вручную
- Нужно сделать git merge

🔧 Шаг 06:

Миссия 6

Сопоставьте команды с их описаниями — Петя готовит гайд по чистке истории Git.

Подсказка: Каждая команда имеет конкретное назначение — вспомните, какая из них используется для фильтрации истории, а какая для анализа.

Ответ:

- **git filter-repo** — Современный инструмент для переписывания истории и удаления файлов
- **git filter-branch** — Старый инструмент для фильтрации истории коммитов
- **git rev-list** — Позволяет просматривать список всех коммитов
- **git gc** — Очищает и сжимает внутреннее хранилище репозитория

🔗 Шаг 07:

Миссия 7

Бэкендер Миша нашёл старый архив data.zip в истории. Сопоставьте действия с их назначением.

Подсказка: Некоторые команды только удаляют файлы из текущей версии, а другие — из всей истории.

Ответ:

- **git rm data.zip** — Удаляет файл из текущего состояния репозитория
- **git filter-repo --path data.zip --invert-paths** — Удаляет файл из всей истории
- **git commit -m 'remove data.zip'** — Фиксирует удаление файла
- **git push --force** — Перезаписывает историю на удалённом сервере

🔗 Шаг 08:

Миссия 8

Настя готовит чек-лист по безопасной чистке истории. Сопоставьте шаги с объяснениями.

Подсказка: Безопасность прежде всего — сделайте резервную копию, прежде чем начинать.

Ответ:

- **Создать бэкап репозитория** — Позволяет вернуть данные, если что-то пойдёт не так
- **Запустить git filter-repo** — Удаляет крупные или нежелательные файлы из истории
- **Проверить размер репозитория** — Убедиться, что очистка действительно помогла
- **Уведомить команду** — Чтобы избежать конфликтов при форс-пуше

🔗 Шаг 09:

Миссия 9

Сисадмин Катя хочет автоматизировать чистку. Сопоставьте сценарии с командами, которые их решают.

Подсказка: Подумайте, какая команда нужна для каждого конкретного сценария.

Ответ:

- **Удалить все большие файлы больше 100МБ** — `git filter-repo --strip-blobs-bigger-than 100M`
- **Удалить конкретный файл** — `git filter-repo --path filename --invert-paths`
- **Проверить использование диска Git** — `git count-objects -vH`
- **Сжать хранилище** — `git gc --prune=now`

🔗 Шаг 10:

Миссия 10

Инженер Игорь хочет разобраться в типах фильтрации при очистке истории. Сопоставьте фильтры с их назначениями.

Подсказка: Каждый фильтр отвечает за разный аспект — дерево, сообщения, авторы и т.д.

Ответ:

- **--path** — Фильтрует изменения по пути файлов
- **--blob-callback** — Позволяет выполнять операции с содержимым файлов
- **--message-callback** — Изменяет сообщения коммитов
- **--replace-text** — Позволяет заменить строки во всех коммитах

🔗 Шаг 11:

Миссия 11

DevOps Лиля сравнивает инструменты Git. Сопоставьте инструмент с его особенностями.

Подсказка: Некоторые из них встроенные, другие — внешние, но более удобные.

Ответ:

- **git filter-repo** — Быстрый, современный, написан на Python
- **git filter-branch** — Медленный, встроенный в Git, но устаревший
- **BFG Repo-Cleaner** — Быстрая Java-альтернатива для удаления больших файлов
- **git rebase -i** — Используется для ручного редактирования истории

🔗 Шаг 12:

Миссия 12

Расставьте шаги очистки истории Git в правильном порядке, как делает опытный инженер Андрей.

Подсказка: Сначала нужно сохранить данные, потом очистить, проверить и отправить.

Ответ:

- Создать бэкап репозитория
- Запустить git filter-repo
- Проверить результат очистки
- Сделать git push --force

🔗 Шаг 13:

Миссия 13

Определи порядок действий для удаления секретного ключа из истории.

Подсказка: Нельзя просто удалить файл — нужно полностью убрать его следы из истории.

Ответ:

- Создать резервную копию
- Удалить ключ с помощью git filter-repo
- Проверить, что файл больше не существует в истории
- Сделать принудительный push

🔗 Шаг 14:

Миссия 14

Тестировщик Влад восстанавливает порядок действий после чистки. Расставь этапы логично.

Подсказка: Проверь, что сначала идёт анализ, а потом действия с историей.

Ответ:

- Проверить размер репозитория
- Найти крупные файлы с помощью git rev-list

- Удалить файлы git filter-repo
- Сделать push изменений

Шаг 15:

Миссия 15

У Марка возникли проблемы после очистки. Как правильно восстановить репозиторий коллегам?
Подсказка: После принудительного пуша старая история становится недействительной.

Ответ:

- Сделать git fetch --all
- Сделать git reset --hard origin/main
- Проверить актуальность ветки
- Продолжить работу

Шаг 16:

Миссия 16

DevOps-команда автоматизирует процесс очистки больших файлов. Расставь этапы пайплайна в нужном порядке.

Подсказка: Начни с анализа, затем применяй фильтрацию и обновляй историю.

Ответ:

- Анализ истории Git на крупные файлы
- Применение git filter-repo для очистки
- Сжатие хранилища git gc
- Принудительный пуш обновлений

Шаг 17:

Миссия 17

Финальная миссия! Составь полный сценарий очистки репозитория от больших файлов.

Подсказка: Вспомни все шаги — от поиска проблем до публикации изменений.

Ответ:

- Найти крупные файлы с помощью git rev-list
- Создать бэкап
- Удалить ненужные файлы git filter-repo
- Проверить результат
- Сделать git push --force

Урок 3 - Практика ускорение старого репозитория

Шаг 01:

Миссия 1

Разработчик Алекс заметил, что его старый репозиторий весит больше 1 ГБ. Он хочет понять, что именно занимает так много места. Какой инструмент Git поможет ему это определить?

Подсказка: Этот инструмент показывает, какие файлы занимают больше всего места в истории коммитов.

Ответ:

- git-sizer

- git-grep
- git-diff
- git-status

Шаг 02:

Миссия 2

Бэкендер Лена случайно закоммитила огромный архив с логами. Какой инструмент поможет ей удалить этот файл из всей истории проекта?

Подсказка: Этот инструмент был создан специально для переписывания истории репозитория, включая удаление файлов из всех коммитов.

Ответ:

- git clean
- **git filter-repo**
- git rm
- git reset

Шаг 03:

Миссия 3

Разработчик Иван хочет избавиться от случайно добавленных временных файлов перед коммитом. Какой файл ему поможет исключить их в будущем?

Подсказка: Этот файл лежит в корне репозитория и управляет тем, что попадает в индекс Git.

Ответ:

- **.gitignore**
- .gitattributes
- .gitconfig
- README.md

Шаг 04:

Миссия 4

Команда DevOps решила очистить репозиторий от больших файлов, сохранив их в отдельном хранилище. Какое расширение Git для этого используется?

Подсказка: Это решение часто применяют для хранения видео, бинарников и моделей машинного обучения.

Ответ:

- Git Hooks
- **Git LFS**
- Git Flow
- Git Worktree

Шаг 05:

Миссия 5

Разработчица Маша решила почистить историю от ненужных коммитов. Какой инструмент поможет ей объединить несколько старых коммитов в один?

Подсказка: Этот инструмент позволяет редактировать историю интерактивно — изменять, удалять или объединять коммиты.

Ответ:

- **git rebase -i**
- git cherry-pick
- git stash
- git merge

Шаг 06:

Миссия 6

Помогите DevOps-инженеру Кате подобрать правильные команды Git для оптимизации старого репозитория.

Подсказка: Каждая команда делает уникальное действие — от удаления мусора до упаковки объектов.

Ответ:

- **git gc** — Удаляет ненужные файлы и оптимизирует локальное хранилище объектов.
- **git prune** — Удаляет недостижимые объекты, не связанные с ветками.
- **git fsck** — Проверяет целостность объектов в репозитории.
- **git reflog expire** — Удаляет старые записи из reflog для экономии места.

Шаг 07:

Миссия 7

 Сопоставьте команды с их назначением при удалении больших файлов из истории.

Подсказка: Вспомните, что один инструмент заменил устаревший BFG и теперь официально рекомендован GitHub.

Ответ:

- **git filter-repo** — Современный инструмент для очистки истории репозитория.
- **bfg-repo-cleaner** — Устаревший инструмент для удаления больших файлов.
- **git rm** — Удаляет файл из текущего коммита, но не из истории.
- **git reset --hard** — Откатывает рабочее дерево к состоянию определённого коммита.

Шаг 08:

Миссия 8

Сопоставьте команды Git с их функциями при проверке размера и производительности репозитория.

Подсказка: Подумайте, какая команда показывает размер объектов, а какая проверяет структуру пак-файлов.

Ответ:

- **git count-objects** — Показывает количество неупакованных объектов и их общий размер.
- **git verify-pack** — Проверяет целостность пак-файлов и их структуру.
- **git rev-list --objects --all** — Список всех объектов с привязкой к коммитам.
- **git ls-tree** — Отображает содержимое дерева Git для заданного коммита.

Шаг 09:

Миссия 9

 Разработчик Дима готовит репозиторий к публикации на GitHub. Сопоставьте действия с целями оптимизации.

Подсказка: Вспомните, что компрессия и чистка логов — разные процессы.

Ответ:

- **Удалить ненужные ветки** — Уменьшить количество ссылок на устаревшие коммиты.
- **Сжать объекты командой `git gc`** — Ускорить операции за счёт упаковки данных.
- **Очистить `reflog`** — Сократить место, занятое старыми записями.
- **Удалить временные файлы** — Снизить вес репозитория перед публикацией.

🔧 Шаг 10:

Миссия 10

Сопоставьте ситуации с правильными Git-командами для чистки и оптимизации.

Подсказка: Если речь о старых ветках — подумайте о `prune`, а если о сжатии — о `gc`.

Ответ:

- **Неиспользуемые ветки занимают место** — `git prune`
- **Репозиторий стал медленно откликаться** — `git gc`
- **Нужно проверить целостность после очистки** — `git fsck`
- **История коммитов засорена логами** — `git filter-repo`

🔧 Шаг 11:

Миссия 11

🔧 Помогите инженеру Саше выбрать, какие команды использовать на разных этапах оптимизации.

Подсказка: Очистка — не то же самое, что упаковка или проверка!

Ответ:

- **Подготовка** — поиск больших файлов — `git rev-list --objects --all`
- **Очистка** — удаление старых объектов — `git prune`
- **Проверка** — тестирование целостности — `git fsck`
- **Финал** — упаковка и сжатие — `git gc`

🔧 Шаг 12:

Миссия 12

Расположите шаги очистки старого репозитория в правильном порядке.

Подсказка: Сначала анализ, потом чистка, потом упаковка.

Ответ:

- Проверка репозитория с помощью `git-sizer`
- Удаление больших файлов через `git filter-repo`
- Очистка `reflog` и старых ссылок
- Оптимизация с помощью `git gc`

🔧 Шаг 13:

Миссия 13

📦 Упорядочьте действия для настройки Git LFS в проекте.

Подсказка: Сначала установка, потом подключение, затем добавление файлов.

Ответ:

- Установить Git LFS

- Активировать Git LFS в репозитории (git lfs install)
- Отметить нужные типы файлов (git lfs track *.bin)
- Закоммитить изменения и запустить

Шаг 14:

Миссия 14

Расставьте шаги оптимизации веток в правильном порядке.

Подсказка: Сначала проверьте, что ветка не нужна, потом удалите локально и удалённо.

Ответ:

- Проверить список веток git branch -a
- Удалить локальные ненужные ветки git branch -d
- Удалить удалённые ветки git push origin --delete
- Выполнить git prune для очистки ссылок

Шаг 15:

Миссия 15

Упорядочьте шаги переписывания истории для удаления секрета из репозитория.

Подсказка: Главное — не забудьте про пересоздание ссылок и форс-пуш.

Ответ:

- Сделать резервную копию репозитория
- Использовать git filter-repo для удаления секрета
- Проверить репозиторий на остаточные упоминания
- Перезаписать удалённый репозиторий с помощью git push --force

Шаг 16:

Миссия 16

Упорядочьте шаги анализа производительности Git после оптимизации.

Подсказка: Измерение всегда идёт до и после изменений.

Ответ:

- Замерить текущий размер репозитория git count-objects
- Провести оптимизацию git gc
- Перепроверить состояние git fsck
- Повторно измерить размер для сравнения

Урок 4 - Поддержание производительности

Шаг 01:

Миссия 1

Андрей заметил, что его репозиторий Git стал работать медленно. Какое первое действие стоит предпринять, чтобы выяснить причину замедления?

Подсказка: Иногда достаточно просто взглянуть на размер и историю репозитория.

Ответ:

- Проверить размер репозитория с помощью команды **git count-objects -vH**
- Удалить все ветки
- Создать новый репозиторий

- Сделать git pull

🔧 Шаг 02:

Миссия 2

Саша заметила, что папка .git занимает аномально много места. Что может быть причиной этого?

Подсказка: Не всё, что видно в рабочей директории, действительно занимает место в истории Git.

Ответ:

- **Большие бинарные файлы или частые их коммиты**
- Слишком много веток
- Много маленьких текстовых файлов
- Отсутствие README.md

🔧 Шаг 03:

Миссия 3

Мария случайно закоммитила огромный архив. Как безопасно удалить его из истории?

Подсказка: Помните, просто удалить файл из последнего коммита недостаточно — он останется в истории.

Ответ:

- **Использовать `git filter-repo --path` для удаления файла из истории**
- Просто удалить файл и сделать новый коммит
- Сделать `git reset --hard`
- Отключить Git LFS

🔧 Шаг 04:

Миссия 4

Разработчик Кирилл хочет уменьшить размер своего репозитория, не меняя историю. Что ему поможет?

Подсказка: Git умеет сжимать старые данные без переписывания истории.

Ответ:

- **Команда `git gc --aggressive --prune=now`**
- Удаление всех тегов
- Создание новой ветки
- Переключение на другой origin

🔧 Шаг 05:

Миссия 5

Лена работает в команде, где часто приходится чистить историю коммитов. Что делает `git reflog expire --expire=now --all` ?

Подсказка: Эта команда не изменяет коммиты, но влияет на то, сколько их помнит Git.

Ответ:

- **Удаляет старые ссылки из reflog, освобождая место**
- Удаляет коммиты из ветки
- Откатывает последний коммит

- Создаёт новую ветку с reflog

🔗 Шаг 06:

Миссия 6

Миша хочет узнать, какие файлы занимают больше всего места в истории. Какой инструмент ему поможет?

Подсказка: Это не встроенная команда Git, но часто используется для аудита.

Ответ:

- Скрипт **git-size** или **git rev-list --objects**
- **git blame**
- **git stash list**
- **git gc**

🔗 Шаг 07:

Миссия 7

Тимур заметил, что после rebase появились конфликты. Что лучше сделать, чтобы сохранить чистую историю?

Подсказка: Иногда правильное разрешение конфликтов важнее, чем скорость коммита.

Ответ:

- Разрешить конфликты вручную и использовать **git rebase --continue**
- Откатить все изменения командой **git reset --hard**
- Удалить ветку
- Сделать **git stash**

🔗 Шаг 08:

Миссия 8

Сопоставьте команды Git с их назначением, чтобы помочь Игорю очистить репозиторий.

Подсказка: Эти команды часто используются вместе при глубокой чистке истории.

Ответ:

- **git gc** — Оптимизация и сжатие репозитория
- **git reflog expire** — Удаление старых записей из журнала ссылок
- **git prune** — Удаление недостижимых объектов
- **git fsck** — Проверка целостности репозитория

🔗 Шаг 09:

Миссия 9

Аня хочет понять, как Git хранит данные. Сопоставьте термин и его описание.

Подсказка: Понимание этих понятий поможет оптимизировать структуру репозитория.

Ответ:

- **Blob** — Хранит содержимое файла
- **Tree** — Структура директорий и файлов
- **Commit** — Указатель на снимок дерева с метаданными
- **Tag** — Ссылка на конкретный коммит

🔧 Шаг 10:

Миссия 10

Разработчик Илья хочет понять, почему его push длится вечно. Сопоставьте возможную причину и решение.

Подсказка: Иногда дело вовсе не в сети, а в истории репозитория.

Ответ:

- **Большие бинарные файлы** — Использовать Git LFS
- **Слишком длинная история коммитов** — Создать shallow clone
- **Много старых веток** — Очистить и удалить устаревшие ветки
- **Много reflog записей** — Применить git reflog expire

🔧 Шаг 11:

Миссия 11

§ Расположите шаги оптимизации репозитория в правильном порядке.

Подсказка: Сначала — анализ, потом — действие, затем — проверка результата.

Ответ:

- Проверить размер репозитория
- Удалить большие ненужные файлы из истории
- Запустить git gc --aggressive
- Проверить размер после оптимизации

🔧 Шаг 12:

Миссия 12

§ Расположите действия при удалении случайно добавленного большого файла в истории.

Подсказка: Сначала нужно идентифицировать файл, потом удалить его из истории, затем переписать репозиторий.

Ответ:

- Найти файл с git rev-list --objects
- Удалить его с помощью git filter-repo
- Запустить git gc
- Проверить размер с git count-objects

🔧 Шаг 13:

Миссия 13

§ Расположите шаги правильной работы с reflog и очисткой старых ссылок.

Подсказка: Не стоит удалять reflog, пока вы не уверены, что коммиты больше не нужны.

Ответ:

- Проверить содержимое reflog
- Применить git reflog expire
- Запустить git gc
- Проверить освобождённое место

🔧 Шаг 14:

Миссия 14

5 Настя хочет объединить несколько веток, удалив лишние. Расположите шаги в правильном порядке.

Подсказка: Перед удалением веток стоит убедиться, что их изменения сохранены.

Ответ:

- Смержить нужные ветки в основную
- Проверить наличие изменений
- Удалить локальные и удалённые лишние ветки
- Запустить `git gc`